

PENTAGON SPACE



PENTAGON SPACE
Mastering The Future

INDEX

❖ Introduction to Software

❖ Introduction to programming

❖ Introduction to Java

- History of Java
- Features of Java
- **Class and objects**
- Java Keywords
- Identifiers
- Compilation and Execution
- Data members
- Variables and Constants
- Datatypes
- Object creation
- Types of variables
- String
- **JVM, JDK and JRE**

❖ Object-Oriented Programming (OOP)

- **Methods**
 - This keyword
 - Method Overloading
- **Inheritance**
 - Method Overriding
 - Types of Inheritance
- **Constructors**
 - Default Constructor
 - Parameterized Constructor
 - this() calling statements
 - Overloading Constructor
 - Super() calling statements
 - Copy Constructor
- **Type casting**
 - Primitive Type Casting
 - Non primitive Type Casting
- **Method binding**
- **Polymorphism**

- **Packages**
 - Defining Packages
 - Importing Packages
 - Access Protection
- **Access Modifiers**
- **Java final keyword**
- **Encapsulation**
- **Java static keyword**
- **Java abstract keyword**
- **Interface**
 - Defining Interfaces
 - Implementing Interfaces
 - Extending Interfaces
- **Abstraction**
- **Class loading and toString() method**
- **Garbage Collection**
 - finalize()
 - System.gc()
- **Design Pattern**
- **Wrapper class**
- **Exception Handling**
 - Basics of Exception Handling
 - Try, Catch, and Finally
 - Types of Exceptions
 - Checked Exceptions
 - Unchecked Exceptions
 - Throw and Throws
 - Custom Exceptions
- **Streams and File I/O**
 - Byte Streams
 - Character Streams
 - File Handling
 - FileReader and FileWriter
 - FileInputStream and FileOutputStream
 - Serialization
- **Java Collections Framework**
 - Introduction to Collections Framework
 - Interfaces and Classes in Collections
 - List

- ArrayList
 - LinkedList
 - Vector
 - Iterator and ListIterator
 - Set
 - HashSet
 - LinkedHashSet
 - TreeSet
 - Comparators and Comparable
 - Map
 - HashMap
 - LinkedHashMap
 - TreeMap
- **Multithreading**
 - Basics of Multithreading
 - Creating Threads
 - Extending Thread Class
 - Implementing Runnable Interface
 - Thread Life Cycle
 - Thread Scheduler
 - Deadlock
 - Synchronization
 - Inter-thread Communication
- **Java 8 Features**
 - Lambda Expressions
 - Functional Interfaces
 - Anonymous class in java
 - Stream API
 - Optional Class
 - Default and Static Methods in Interfaces
 - Date and Time API
- **Java MCQ's based on topic's**
- **Java Mock Questions**
- **Java Important Interview Question and Answers**
- **Problems on Java Programming**

SOFTWARE

“ Software is a collection or set of programs which is used to perform different task based on business requirement ” .

Software helps us in automating tasks, solving problems, processing data, and enhancing productivity. That's why “software is also called as Automated solution for real world problems” and also “ automated version of manual work ”.

Software is responsible for directing all computer-related devices and instructing them regarding what and how the task is to be performed. However, the software is made up of binary language (composed of ones and zeros), and for a programmer writing the binary code would be a slow and tedious task. Therefore, software programmers write the software program in various human-readable languages such as Java, Python, C#, etc. and later use the source code.

Software's are broadly classified into two types, i.e., **System Software and Application Software**.

1. System Software

System software is a computer program that helps the user to run computer hardware or software and manages the interaction between them.

For example : Operating System (Microsoft Windows , Apple's iOS , Android , Linux),
Device Drivers (printers, hard disks, floppy disk, keyboard, mouse) etc...

2. Application Software

Application software is designed to perform a specific task for end-users. It needs system software as platform to run the applications.

For example : Database Software (Oracle, SQLite, Microsoft SQL Server, MySQL),
Web Browsers (Chrome, Mozilla Firefox, Microsoft Internet Explorer,
Microsoft Edge) ,
Multimedia Software (Adobe Photoshop, Picasa, Windows Media Player) etc..

Programming :

It is a set of instructions given to Machine/platform to perform specific task .

Programming language :

The language which is used to give set of instructions to a machine/ platform .

For eg : Java, Python, c, c++, Javascript, Ruby etc..

Syntax :

Syntax refers to set of rules that indicate how code must be written for a system to understand and execute . (or simply a grammar part of programming)

Components of Syntax:

Programming syntax consists of several key components:

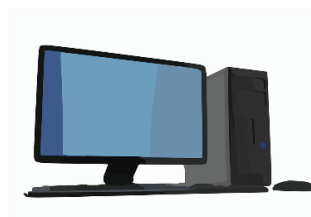
- **Keywords:** Reserved words with predefined meanings in the programming language.
- **Operators:** Symbols or characters used to perform operations on data, such as arithmetic, logical, and relational operations.
- **Punctuation and Delimiters:** Characters used to separate or structure code elements, such as parentheses, braces, commas, and semicolons.
- **Identifiers:** Names given to variables, functions, classes, or other entities in the code.
- **Comments:** Annotations within the code that provide explanations, documentation, or context for developers.

PLATFORM :

- It's a place where we can run all the applications and programs.
- Platform is a combination of Hardware (Processor) and software (Operating software) components

Example : intel (processor) and Windows (OS) .

Note : Without platform is not possible to run any applications or programs .



HARDWARE



SOFTWARE

JAVA

Java is high level , statically typed, object oriented programming language .

Features of Java :

- **Robust**

- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- Java provides automatic garbage collection which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- There are exception handling and the type checking mechanism in Java. All these points make Java robust.

- **Architecture-neutral**

Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.

In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

- **Portable**

Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.

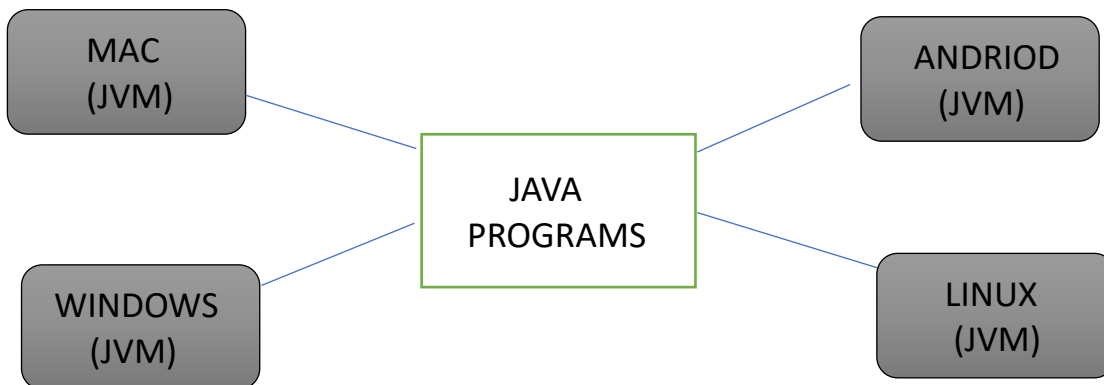
- **High-performance**

Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code. It is still a little bit slower than a compiled language (e.g., C++). Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

Platform Independent :

The java application programs developed in one platform can be able to run on any other

platforms provided with JVM , must be present .



- Java files are always saved with .java extension .
- JAVA was developed by James Gosling at Sun Microsystems Inc in 1995 and later acquired by Oracle Corporation.
- It is intended to let application developers write once, and run anywhere (WORA), meaning that compiled Java code can run on all platforms that support Java without the need for recompilation.
- Java is widely used for developing applications for desktop, web, and mobile devices. Java is known for its simplicity, robustness, and security features, making it a popular choice for enterprise-level applications.

Class

class is logical Entity, class gets a life only when Object is created.

- class is a blueprint / Master copy using which multiple similar objects can be created.
- without class it is not possible to create objects.
- In java if any object is created/destroyed will not affect to other objects not even to another class.
- Class is also known as non-primitive data type.

Objects

- Object is a real world physical Entity .
- Anything which is physically present is called object.
- Each & every object comes with two things i.e,
 - 1) States / properties
 - 2) behaviour

1. States :

It is an information/description/data about an object . technically States are also called as Data Members or properties or fields .

2. behaviour :

It is an task or action performed by an object , technically behaviour is also called as Methods.

Examples :

1. class : Human , object : Human object

states [Data- members]	behaviour [Methods]
○ name = “ Rajesh ”	eat()
○ age = 24	sleep()
○ dob = “ 02/05/2000 ”	read()
○ height = 5.12	write()
○ gender = “ Male ”	run ()

2. class : Mobile , object : Mobile object

states [Data - members]	behaviour [Methods]
○ brand = “ Redmi ”	calling()
○ price = 24,000	browsing ()
○ ram = 8 gb	gaming()
○ rom = 128 gb	selfies()
○ battery = 5000mah	listening()

How to write a class

Syntax

```
class Class_name {  
  
}
```

Class_name.java

example

```
class Student {  
  
}
```

Student.java

➤ What are the contents of class ?

- Data members
- Methods
- Constructors
- Blocks
- Nested class

➤ Coding Standards to write class

1. The class name should be noun [eg : place names , person names]
2. The first letter of the class name should be in uppercase.
3. If there is multiple words then we have to follow pascal case .

Eg : Pentagon_Space

4. Always try to save the filename with the classname .

Keywords

In java many predefined words are already present (in-built) and they have some predefined meaning , which we call it as java keywords or reserved words .

- Java is case sensitive, hence all the keywords is in lower case.

abstract	assert	boolean
break	byte	case
catch	char	class
const	continue	default
do	double	else
enum	extends	final
finally	float	for
goto	if	implements
import	instanceof	int
interface	long	native
new	package	private
protected	public	return
short	static	strictfp
super	switch	synchronized
this	throw	throws
transient	try	void
volatile	while	var
record	yield	

- **const** and **goto** are reserved words but not used.
- **true**, **false** and **null** are literals, not keywords.
- Since Java 8, the **default** keyword is also used to declare default methods in interfaces.
- Since Java 10, the word **var** is used to declare local variables (local variables type inference). For backward compatibility, you can still use **var** as variable names.

So **var** is a reserved word, not keyword.

- Java 14 adds two new keywords **record** and **yield**

Identifiers

- Java identifiers are names given to various elements in a Java program, such as variables, methods, classes, packages, and interfaces.
- All Java variables must be identified with unique names..
- These unique names are called identifiers.

example of identifier:-

```

public class Demo
{
    public static void main(String[] args)
    {
        int x=100;
    }
}
  
```

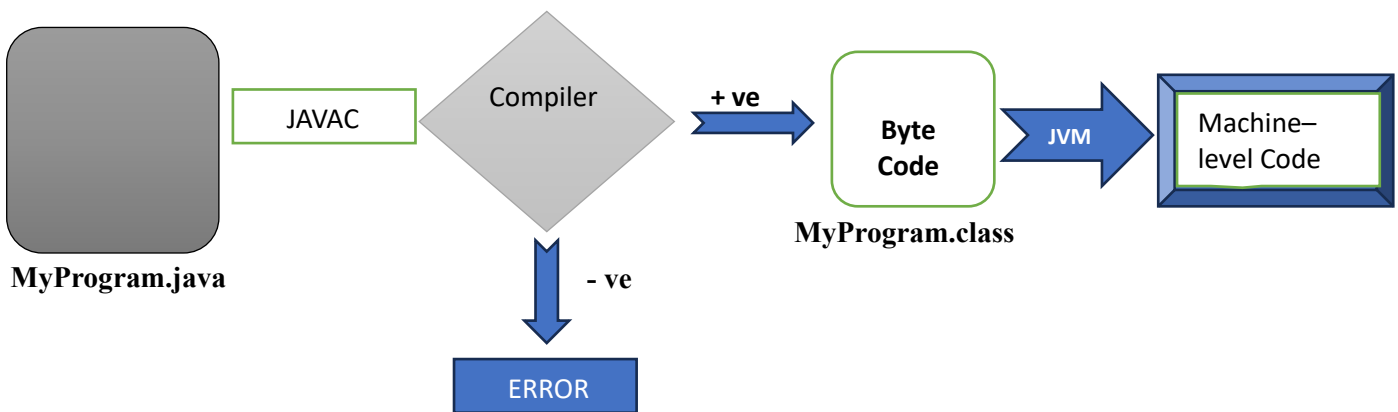
Rules to define Identifiers

- Identifiers should not have any white space .
- Identifiers must begin with a letter (a-z or A-Z), a dollar sign (\$), or an underscore (_).
- Identifiers cannot be Java keywords or reserved words, as these have predefined meanings in the language. Attempting to use a reserved word as an identifier will result in a compilation error.
- Identifiers should not start with number but it can have numbers.

Note :

Java identifiers are case-sensitive, meaning uppercase and lowercase letters are distinct. For example, “myVariable” and “MyVariable” are treated as different identifiers.

Compilation & Execution :



```

public class MyProgram {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
  
```

MyProgram.java

1. Writing the Code:

You start by writing Java code in a text editor or an Integrated Development Environment (IDE) like Eclipse or IntelliJ IDEA. The code is saved with a .java file extension.

2. Compilation:

The Java compiler (javac) takes the .java file and compiles it into bytecode. Bytecode is an intermediate, platform-independent code that can be executed by the Java Virtual Machine (JVM).

Command:

```
javac MyProgram.java
```

After successful compilation, a .class file is generated for each class defined in the .java file. For example, if you have MyProgram.java, the compiler will produce MyProgram.class.

3. Execution:

The JVM interprets or compiles the bytecode into machine code that runs on the host machine. The JVM ensures that Java code is platform-independent, meaning it can run on any machine that has the JVM installed.

Command: java MyProgram

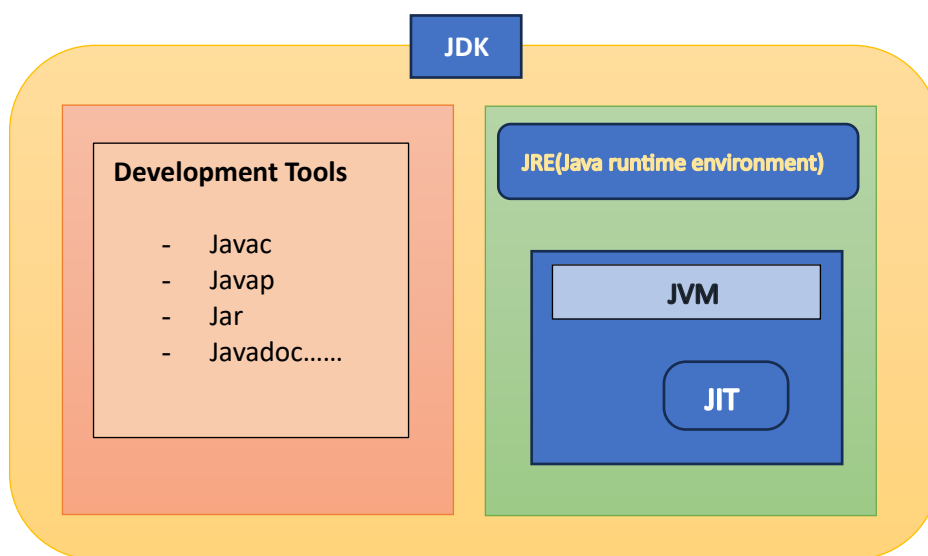
NOTE: Here, MyProgram is the name of the class containing the main method, which serves as the entry point for the program. And if the program is syntactically wrong then it throws error.

JDK (Java Development Kit)

- JDK is a SDK (software development kit)
- It helps us to develop , compile & execute a java program , it includes two sections in it

1. Development Tools

2. Execution engine



1. Development tools:-

It helps us to write a simple program as well as develop a large applications.

It includes tools such as javac, javap, jar, Javadoc, java etc...

2. Execution engine :-

This section JDK helps to execute the application and includes two parts such as JRE and JVM.

JRE

Java Runtime Environment (JRE) is an open-access software distribution that has a Java class library, specific tools, and a separate JVM. In Java, JRE is one of the interrelated components in the Java Development Kit (JDK).

How JRE Works

1. **Compile Once, Run Anywhere:** You write your Java code, and the Java compiler converts it into bytecode. This bytecode can be run on any device that has the JRE installed, making Java a platform-independent language.
2. **Executing Java Programs:** When you run a Java program, the JVM reads the bytecode and translates it into native machine code specific to your operating system and hardware, allowing the program to execute.

Why is JRE Important?

- **Platform Independence:** Ensures that Java applications can run on any operating system without modification.
- **Efficient Resource Management:** Manages memory and system resources efficiently, providing a stable environment for running Java applications.
- **Security:** Offers a secure execution environment by providing built-in security features that help protect against malicious code.

JVM (Java Virtual Machine) Architecture

JVM (Java Virtual Machine) is an abstract machine. It is a specification that provides runtime environment in which java bytecode can be executed.

Just-In-Time(JIT) compiler: It is used to improve the performance. JIT compiles parts of the byte code that have similar functionality at the same time, and hence reduces the amount of time needed for compilation. Here, the term "compiler" refers to a translator from the instruction set of a Java virtual machine (JVM) to the instruction set of a specific CPU.

Data Members

Data Members are nothing but the properties of object .

➤ we Store the Data Using :

1. Variables
2. Constants

1. Variables :-

- It is a place where we store the data (or) It is a name given to a memory location .
 - The Variable data can be varied (Changed).
- object – Properties

example : Human - age, Weight , height ,name etc....

Mobile - brand, price, ram, rom, processor etc....

Syntax : DataType Variable_name [= data/value] ;

2. Constants :-

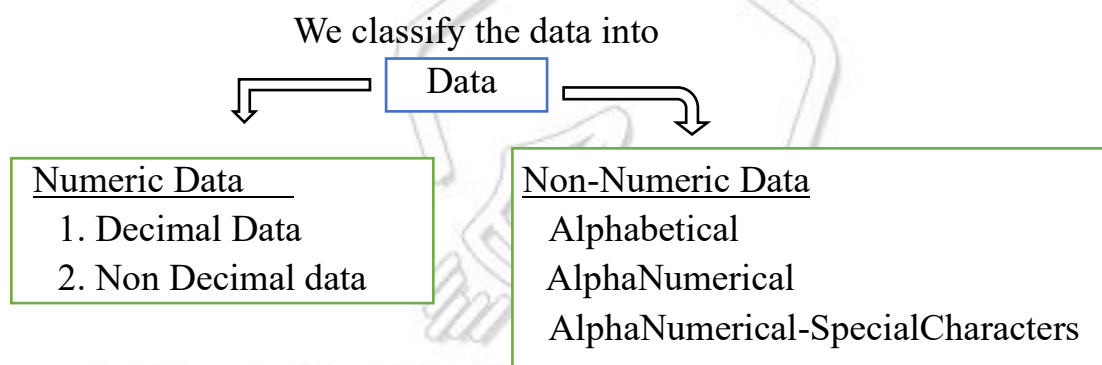
- The Data Which is fixed & it cannot be changed is called Constant.
- Using the Keyword "final" We can make the value as constant.

For Example : pi = 3.14; // fixed value

Syntax : final DataType Variable_name [= data/value] ;

Data

- Data is raw fact / information which describes the attributes of an object.



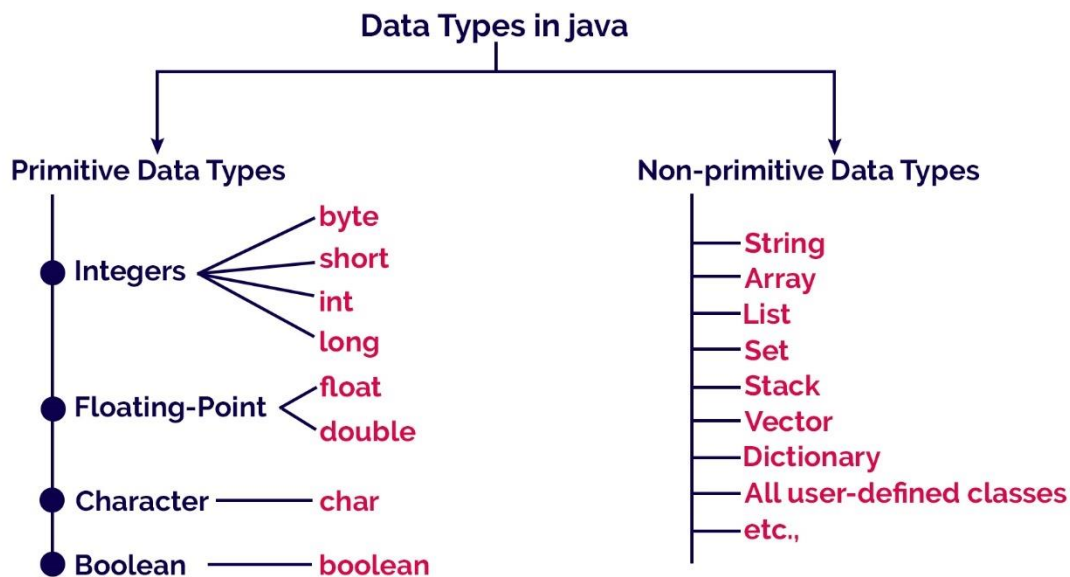
Data Type :

In java or any other programming language data must be mentioned with it's type i.e , what type of data /information it is .

- To specify the type of data we use data type.
- Data types defines or specifies the type & size the data. which is stored in memory location .

In java we have 2 different types of data :

1. Primitive Data Type.
2. Non-Primitive Data Type



www.btechsmartclass.com

Primitive Data Type :-

- These are the basic In-built datatype in java.
- It stores the raw data which holds the numeric values.

Examples : `int a =10;`
`long b=123456789l;`

Non-Primitive Data Type :-

- In java it is possible to declare our own types of data by using class and interfaces , i.e , is called as non- primitive data type.

Eg: for in-built classes :

- String
- Scanner
- System
- Array
- Object etc...

Eg: for inbuilt interfaces :

- Collection
- List
- LinkedList etc...

Eg : `String name="Dinga" ;`

Default values:

Whenever the user doesn't provide any values or data to the variable by default it takes default values based on the data type.

DATA TYPES	SIZE	DEFAULT	EXPLANATION
boolean	1 bit	false	Stores true or false values
byte	1 byte/ 8bits	0	Stores whole numbers from -128 to 127
short	2 bytes/ 16bits	0	Stores whole numbers from -32,768 to 32,767
int	4 bytes/ 32bits	0	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes/ 64bits	0L	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes/ 32bits	0.0f	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes/ 64bits	0.0d	Stores fractional numbers. Sufficient for storing 15 decimal digits
char	2 bytes/ 16bits	'\u0000'	Stores a single character/letter or ASCII values

- Example for initializing properties using Data Types

```
class Student {
    String name = " dinga ";
    int age = 25;
    char sec = 'c';
    long contactNum = 9876543210l;
    double percentage = 75.5;
```

Object Creation :

In java There are Five different ways to create an object .

1. Using new keyword.
2. Using clone().
3. Using newInstance of class.
4. Using newInstance of Constructors.
5. Using deserialization

- In java the most common way of creating an object is using new keyword.
- Each and every object is created at runtime by JVM .
- In java the object is always created & managed by JVM .
- Object will be created inside a Heap Memory .
- Each object has address which is in hexaDecimal format .
- We can create 'n' number of object for a same class.

“new” Keyword

- It instructs the jvm to allocate a memory location for the specified object to be created within the heap area of jvm.
- It also returns the address of the memory location where the object is created .

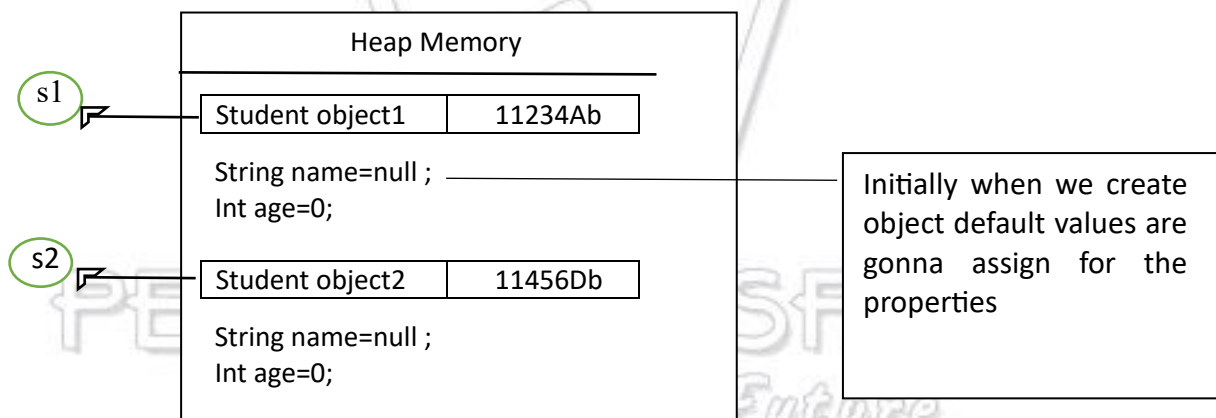
- This address has to be stored by a variable , that variable is called as object reference

How to initialize the object inside the class

```
class Student {
    String name;
    int age;
    public static void main(String[] args)
    {
```

Student	s1	=	new	Student();
Non- Primitive Data type	Variable/ object reference		keyword	Constructor

```
        Student s2 = new Student( );
    }
}
```



How to Access the Data members

- If we want to call data members or fields or states of a class by using the object reference followed by dot (.) operator we can access it.

```
Eg : class Student {
    String name;
    int age;
    public static void main(String[] args) {
        Student s2 = new Student( );
        s2.name = " dinga " ;
        s2.age = " 20 " ;
        System.out.println(s2.name);
    }
}
```

Accessing data members

```

        System.out.println(s2.age);
        System.out.println(s2);    —————> Prints the object Address
    }
}

```

Note : Whenever we print the object reference the address of the object will be displayed.

METHODS

It is a set of instruction or block of code which is used to perform some specific task .

- Methods allows us to **reuse** the code
- A method is like a function i.e. used to expose the behavior of an object.
- It is used to achieve the **reusability** of code. We write a method once and use it many times. We do not require to write code again and again.
- It also provides the **easy modification** and **readability** of code, just by adding or removing a chunk of code. The method is executed only when we call or invoke it.
- **Methods are executed in stack memory.**

Syntax :

```

[Access_modifier] <return_type> <method_name> ( list_of_parameters)
{
    //logic
    return statement ;
}

```

Example Program : *Mastering The Future*

```

class MethodDemo{
    String brand = "Casio" ;
    double Price = 900;

    int add(){
        int a=10;
        int b=20;
        int c=a+b;
        System.out.println(c);
        return c;
    }

    public static void main(String[] args){
        MethodDemo md=new MethodDemo();
        md.add();    // we are accessing the method by object
    }
}

```

reference followed by dot (.) operator .

}

}

Parameters and arguments

- parameters are the input for methods .
- Arguments are the values we are passing to the methods.

Example :

```
class MethodDemo{
    String brand = "Casio" ;
    double Price = 900;

    int add(int a , int b){
        int c=a+b;
        System.out.println(c);
        return c;
    }
    public static void main(String[] args) {
        MethodDemo md=new MethodDemo();
        md.add( 10 , 20 );
        md.add(893, 789);
    }
}
```

There are 4 ways to define a method

- Methods with void as the return type and no parameters
- Methods with void as a return type with parameter
- Methods with different return type with no parameters
- Methods with different return type and with parameters

Rules to define methods

- Methods can have only one return type
- Methods can have only one return statement
- Return type and return value must always match with the same data type.
- Return statement must be the last executable statement inside a method .

Note :

- Return type signifies the output of the method .
- There is no comparison between the return type and parameters but we must ensure that return type and return value are of same type .
- Methods will not execute until we invoke it .
- **Methods are executed in runtime stack memory**
- ****For each method separate thread will be created and it will be executed in runtime**

Example Program :

```
import java.util.Scanner;
public class AtmMachine {
    double balance=1000;

    // void as a return type with passing parameters

    public void insertingThecard(int pin){
        System.out.println("Welcome to Sbi Bank Atm Machine");
    }

    // void as a return type and passing parameters

    public void deposit(int amount){
        balance = balance+amount;
        System.out.println("your amount has been deposited successfully");
    }

    // void as a return type without passing parameters

    public void checkBalance() {
        System.out.println("Your Balance is :"+balance);
    }

    // method with different return types ( String ) with some parameters

    public String withdraw(int amount){
        if(amount<=balance) {
            System.out.println(" withdrawing the money ");
            balance=balance-amount;
            return "success";
        }
        else {
            return "failure";
        }
    }
}
```

//method with different return types (int) without passing parameters

```
public int changePin() {
    System.out.println("Enter the new pin :");
    Scanner sc=new Scanner(System.in);
    int pin =sc.nextInt();
    System.out.println("pin changed");

    return pin;

}
```

```
public static void main(String[] args) {
    AtmMachine atm=new AtmMachine();
    atm.insertingThecard(1234);
    atm.deposit(2000);
    atm.checkBalance();
    String amnt= atm.withdraw(2000);
    if(amnt=="success") {
        atm.dispenseCash();
    }
    else {
        System.out.println("insufficient exception");
    }
    atm.checkBalance();
    // atm.changePin();
}

PENTAGON SPACE
Mastering The Future
```

//method with different return types (String) without parameters

```
public String dispenseCash() {
    System.out.println("Amount has dispensed , please collect your money");
    return "cash";

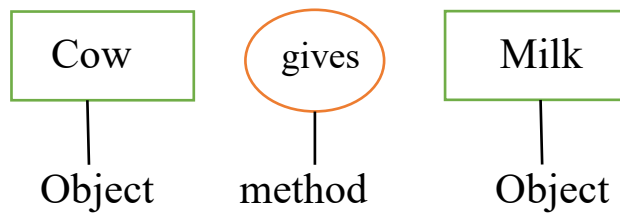
}

}
```

Conceptual Programming (Scenario Based)

Based on scenario we have to build the programs.

Eg 1 : Cow gives milk



// Cow class has been created

```

public class Cow {
    Milk gives(){
        Milk m=new Milk();
        System.out.println("Cow is giving milk ");
        return m;
    }
}
  
```

// Milk class has been created

```

public class Milk {
}
  
```

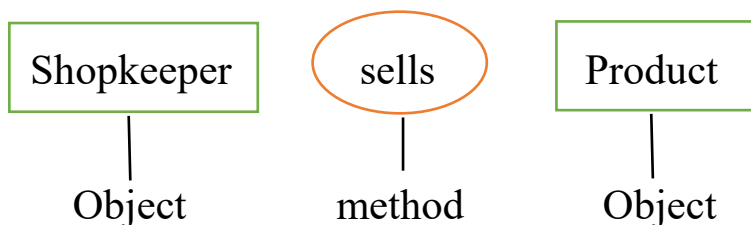
// Test class

```

public class CMtest {
    public static void main(String[] args) {
        Cow c=new Cow();
        c.gives();
    }
}
  
```

Note : An object of one class can be created inside another class

Eg 2 : Shopkeeper sells a product



// Building the shopkeeper class

```

public class Shopkeeper {
    Product sales(){
        Product p=new Product();
        System.out.println("product saled");
    }
}
  
```



```

        return p;
    }
    public static void main(String[] args) {
        Shopkeeper s=new Shopkeeper();
        s.sales();
    }
}
// building the Product class

public class Product {

}

```

Eg3 : A dancer dances and throws the jacket

```

public class Dancer {
    void dance(){
        System.out.println("Dancer danceing");
        Throws();
    }

    Jacket Throws(){
        Jacket j=new Jacket();
        System.out.println(j);
        System.out.println(" and throws the jacket");
        return j;
    }
}

public static void main(String[] args) {
    Dancer d=new Dancer();
    d.dance();
}

}

class Jacket{

}

```

STRING :

- String is a in-built class in java Which is present in java.lang Package .
- String is used to store the sequence of characters.
- Always string data must be stored within the double quotes (“ ”).
- Any value/date in double quotes is consider as a String.
- String is final class in java and also a Non – primitive data type .

Ways of Creating a String

There are two ways to create a string in Java:

- String Literal
- Using new Keyword

1. String literal

To make Java more memory efficient (because no new objects are created if it exists already in the string constant pool).

Eg: String s="Pentagon Space"

2. Using new keyword

- In such a case, JVM will create a new string object in normal (non-pool) heap memory and the literal "Welcome" will be placed in the string constant pool. The variable s will refer to the object in the heap (non-pool)
- Totally two objects will be created when we use new String way to create String object .

Eg: String s = new String("Pentagon");

Note :

String is an **immutable class** which means a constant and cannot be changed once created and if wish to change , we need to create an new object and even the functionality it provides like toupper, tolower, etc all these return a new object , its not modify the original object. It is automatically thread safe.

How Strings get stored

Inside the heap memory there is something called as **String constant pool**. In that String Constant Pool all the strings value get stored .

String Constant Pool

String constant pool is a special memory area that helps optimize the use of strings in your program.

- It's a part of the heap memory where string literals (fixed text directly written in code) are stored.
- When you create a string literal, the Java Virtual Machine (JVM) checks the pool first.

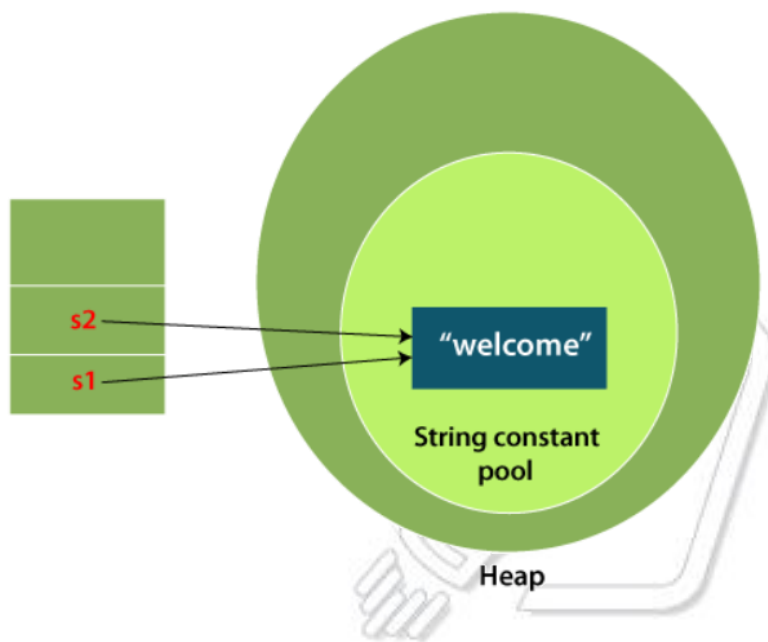
- If the same literal already exists, the JVM reuses the existing one instead of creating a new one. This saves memory and improves performance.

Example:

```
String s1 = "Welcome";
```

```
String s2 = "Welcome";
```

In this case, even though you declare two strings, both str1 and str2 will point to the same object in the string constant pool because they have the same value.



Non-Constant Pool

- This refers to the rest of the heap memory where regular String objects are stored.
- When you create a string using “new” keyword then two objects are going to create one is inside the **non-constant pool** and another inside **the constant pool**.

```
String s1 = "rama";           // Step 1: Adds "rama" to the pool if not present
```

```
String s2 = new String("rama"); // Step 2: Creates a new object on the heap with content "rama"
```

Visual Representation

1. String Constant Pool:

- Contains the string literal "rama".

2. Heap:

- Contains a new String object with the content "rama".

Example program

```
public class Demo {
```

```
    public static void main(String[] args) {
```

```

String s1="dinga";
String s2="dinga";
String s3=new String("guldu");
String s4=new String("dinga");
// for comparing address use == operator
if(s1==s2) {
    System.out.println(" s1 & s2 pointing to same object");
}
else {
    System.out.println("s1 & s2 pointing to different object");
}

if(s1==s4) {
    System.out.println(" s1 & s4 pointing to same object");
}
else {
    System.out.println("s1 & s4 pointing to different object");
}

//for comparing values use equals() method
if(s1.equals(s2)) {
    System.out.println("s1 and s2 are having same value");
}
else {
    System.out.println("s1 and s2 are not having same value");
}
}
}

```

Comparison

```

System.out.println(s1 == s2);    // false: s1 and s2 refer to different objects
System.out.println(s1.equals(s2)); // true: s1 and s2 have the same content

```

- `s1 == s2` is false because `s1` and `s2` are different objects (one in the pool, one on the heap).
- `s1.equals(s2)` is true because the content of both strings is "Hello".

Variables :

It is a place where we store data or value.

Or

It's a name given to memory location .

Types of variables

1. Static variables
2. Non static variables
 - Instance variables
 - Local variables

1. Instance variables :

- Instance variables are the variables which are declared inside a class but outside the methods, blocks and constructors .
- Instance variables gets created inside the heap memory.
- Eg :

```
class Demo {
    int a = 10 ;
    String name = " Pentagon " ;
    // method
    void m( ){
        // statements
    }
    // constructor
    Demo( ){
        //statements
    }

    // block
    {
        // statements
    }
}
```

- Instance variables gets created only when the object is created .
- The number of copies of instance variable depends upon the number of object.
- If the user doesn't provide any value to instance variables then by default it takes default values based on the data type.
- Instance variables life span depends upon the object, if the object gets created

then it is also gets created and if the object is destroyed then variable also get destroyed .

Accessing instance variables

- Instance variable can be accessed within the class and also outside the class by using object reference followed by dot (.) operator and also using this keyword.
- Instance variables can be accessed directly inside non static methods without creating any objects within the same class.

Eg: class Employee {

String name = "Govind" ;

double id = 0023 ;

void work() {

System.out.println(name + " and " + id);

}

public static void main(String[] args){

name = "Gururaj"; // error because ,

/* we cannot access instance variables without creating object so we need to create object */

Student s1 = new Student();

s1.name= "Gururaj" ;

System.out.println(s1.name);

}

}

2. Local variables

- The variable which is declared inside the method, constructor, blocks is called as Local variables.
- Local variables gets stored inside the stack memory

Eg :

public class Main {

int count=15; // Instance variable

//constructor

Main(){

double a = 10; // Local variable

}

//methods

void m() {

int b = 20; // Local variables

}

// blocks

```

    {
        long c = 3000; // Local variables
    }
}

```

- The scope of the local variables is limited i.e, the local variables can be accessed within the local scope only .
- Local variables cannot be accessed outside the scope either by using the object reference or by using this keyword.
- Local variables must be initialized before we use because these variables doesn't have any default values.
- Local variables within the same scope cannot have the same name , but within the different scope can have same names.

Variable shadowing/ clashing

According to the coding standards the instance variables and local variables should be same. But whenever instance and local variable names are same , a name clash would occur. In this clash **the local variables dominates the instance variables** is called variable shadowing.

Eg:

```

class Dog {
    String name="Scooby";
    double height=3.5;

    void sleep() {
        System.out.println(name + " is sleeping and is height is "+height);
    }

    void jump() {
        double height=7.5;
        System.out.println(name+ " dog of height "+ height +" trying to jump a wall of feet "+height);
    }
}
// trying to print
// Scooby dog of height 3.5 trying to jump a wall of feet 7.5 but
observe the output
}

```

```

public class Test{
    public static void main(String[] args) {
        Dog d=new Dog();
        d.sleep();
        d.jump();
    }
}

```

Output :

Scooby is sleeping and is height is 3.5

Scooby dog of height 7.5 trying to jump a wall of feet 7.5

Explanation :

Here instance variable “height” and local variable “height” are getting clash inside the method jump() and giving the output as “Scooby dog of height 7.5 trying to jump a wall of feet 7.5” because as we discussed local variable will always dominate the instance variables, to get rid of this problem we are going to use a keyword called as “ THIS ” .

this keyword

- this keyword is used to access the current invoking object states and behaviours .
- By using this keyword we can avoid variable clashing/ shadowing problems.
- **But we cannot use this keyword inside static methods and not in main method also.

Eg:

```

class Dog {
    String name="Scooby";
    double height=3.5;

    void sleep() {
        System.out.println(name + " is sleeping and is height is "+height);
    }
    void jump() {
        double height=7.5;
        System.out.println(name+ " dog of height "+ this.height +")
    }
}

```



```

        trying to jump a wall of feet "+height);
    // trying to print
    // Scooby dog of height 3.5 trying to jump a wall of feet 7.5 but
    observe the output

```

```

    }
}
public class Test{
    public static void main(String[] args) {
        Dog d=new Dog();
        d.sleep();
        d.jump();
    }
}

```

Eg 2:

```

class Horse
{
    String name;
    double height;
    void run(){
        System.out.println(name+ "horse running");
    }
    void jump(){
        String name="brego";
        System.out.println(this.name+ "horse jumping");
    }
}

class Test123{
    public static void main(String[] args){
        Horse h1=new Horse();
        h1.name="chetak";
        h1.height=5.6;
        h1.run();
    }
}

```

```

        h1.jump();
    }
}

```

Method Overloading

Method overloading is a powerful feature in Java that allows you to define multiple methods within the same class that share the same name but differ in their Signature lists. These methods can vary in terms of

Or

In the same class if methods are having same name but differ in signature is called as method overloading.

- There should be change in the number of parameters.
- There should be change in datatype of the parameter.
- There should be change in sequence of parameters.

Example :

```

class calculator{
    void add() {

    }
    void add(int a) {

    }
    void add(double d) {

    }
    void add(int a, double d) {

    }
    void add(double a, int a) {

    }
}

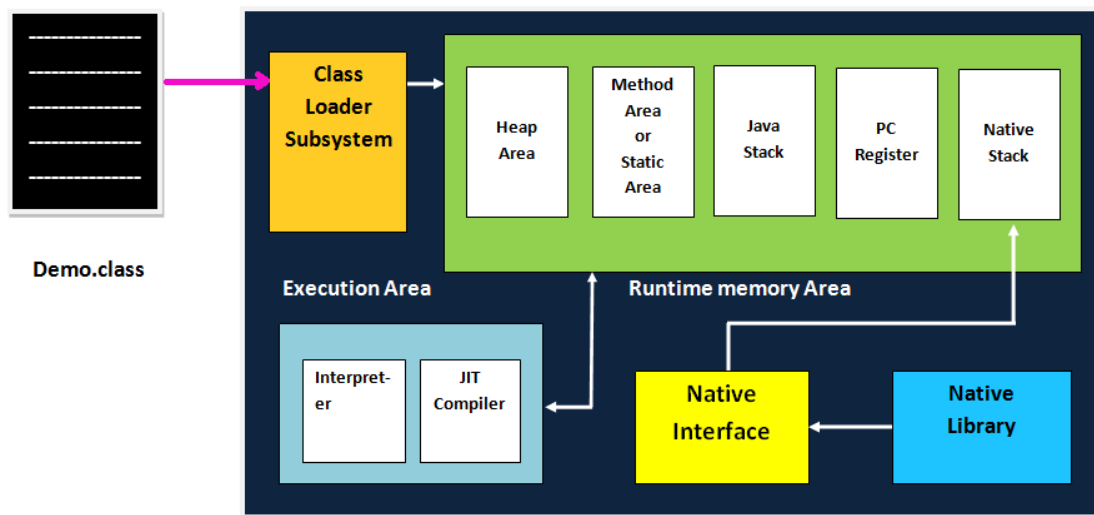
```

Advantage :

- Method overloading is used to achieve compile time polymorphism.
- The return type of a method does not play a role in method overloading. The distinction is solely based on the parameter list (number and data types).
- The main method can be overloaded . In this case the jvm will always execute the main method having string[] as a argument.

JVM (Java Virtual Machine) Architecture

JVM (Java Virtual Machine) is an abstract machine. It is a specification that provides runtime environment in which java bytecode can be executed.



1) Classloader

Classloader is a subsystem of JVM which is used to load class files. Whenever we run the java program, it is loaded first by the classloader.

2) Class(Method) Area

Class(Method) Area stores per-class structures such as the runtime constant pool, field and method data, the code for methods.

3) Heap

It is the runtime data area in which objects are allocated.

4) Stack

- Java Stack stores frames. It holds local variables and partial results, and plays a part in method invocation and return.
- Each thread has a private JVM stack, created at the same time as **thread**.
- A new frame is created each time a method is invoked. A frame is destroyed when its method invocation completes.

5) Program Counter Register

PC (program counter) register contains the address of the Java virtual machine instruction currently being executed.

6) Native Method Stack

It contains all the native methods used in the application.

7) Execution Engine

It contains:

1. **Interpreter:** Read bytecode stream then execute the instructions.
2. **Just-In-Time(JIT) compiler:** It is used to improve the performance. JIT compiles parts of the byte code that have similar functionality at the same time, and hence reduces the amount of time needed for compilation. Here, the term "compiler" refers to a translator from the instruction set of a Java virtual machine (JVM) to the instruction set of a specific CPU.

8) Java Native Interface

Java Native Interface (JNI) is a framework which provides an interface to communicate with another application written in another language like C, C++, Assembly etc. Java uses JNI framework to send output to the Console or interact with OS libraries.

Difference between JVM, JRE, and JDK.

- **JVM:** JVM stands for Java Virtual Machine. JVM is used for running Java bytecode. Java applications are called WORA (Write Once Run Anywhere). This means a programmer can develop Java code on one system and can expect it to run on any other Java-enabled system without any adjustment. This is all possible because of JVM.
- **JRE:** JRE stands for Java Runtime Environment. JRE is made up of class libraries.
- **JDK:** JDK stands for Java Development Kit. JDK contains the JRE with compiler, interpreter, debugger, and other tools. It provides features to run as well as develop Java Programs.

INHERITANCE

- **Inheritance in Java** is a mechanism in which one object acquires all the properties and behaviours of a parent object. It is an important part of [OOPs](#) (Object Oriented programming system).
- Inheritance represents the **IS-A relationship** which is also known as a *parent-child* relationship.
- **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- **Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.

Advantages

- We can avoid code Redundancy.

- We can achieving Code Reusability.
- we can achieve generalization.
- We can achieve polymorphism ***

Example:

```
// Superclass
public class Animal {
    void eat() {
        System.out.println("This animal eats.");
    }
}

// Subclass
public class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");
    }

    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.eat(); // Inherited method
        myDog.bark(); // Subclass method
    }
}
```

Note:

- By using inheritance we can inherit variables methods but not constructors and blocks
- Static data members and methods cannot be inherited .
- Static data members and methods cannot be inherited
- private data members and methods cannot be inherited
- final class cannot have any subclasses so that is also we can't able to inherit.

Interview questions

1. How to avoid inheritance for a class

We can avoid inheritance in two different ways declaring class as a final
declaring a private constructor inside a class

2. Explain immutable classes in Java

String class

Scanner class

System class

Integer class
double class etc....

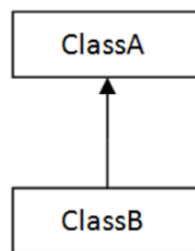
These are some of immutable classes present in Java.

Types of Inheritance

1. Single level inheritance
2. Multi level inheritance
3. Hierarchical inheritance
4. Multiple inheritance
5. Hybrid inheritance

1. Single level inheritance

A class extends to another class we call it as a single level inheritance.



1) Single

Example :

```
// Superclass
class Animal {
    void eat() {
        System.out.println("This animal eats.");
    }
}
```

```
// Subclass
class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");
    }
}
```

```
public static void main(String[] args) {
    Dog myDog = new Dog();
    myDog.eat(); // Inherited method
    myDog.bark(); // Subclass method
}
```

```
}
```

Note:

- In Java each and every class we declare is an example for single level inheritance in Java
- if the class does not extend to any other class then by default automatically it extends to super most class called object class Object class is a root class for all the classes in Java

Object class

- It is a super most class or root class for all the classes in Java
- In Java each and every classes either directly or indirectly extends to super most class called object class.

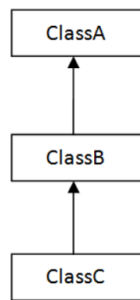
There are 11 methods present in object class namely

- protected object clone()
- public boolean equals (object obj)
- protected final class getClass()
- public int hashCode()
- public string toString()
- public final void notify()
- public final void notifyall()
- public final void wait()
- public final void wait (long timeout)
- public final void wait (long timeout, int nanos)
- protected void finalize() throws throwable()

Note: All these methods inherited to the subclasses in Java by default.

2. Multi level inheritance

A class extends to another class and that class again extends to one more class is called Multi level inheritance



2) Multilevel

Example :

```

// Superclass
class Animal {
    void eat() {
        System.out.println("This animal eats.");
    }
}

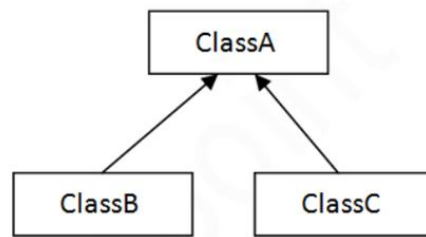
// Intermediate subclass
class Mammal extends Animal {
    void walk() {
        System.out.println("This mammal walks.");
    }
}

// Subclass
class Dog extends Mammal {
    void bark() {
        System.out.println("The dog barks.");
    }
}

public static void main(String[] args) {
    Dog myDog = new Dog();
    myDog.eat(); // Inherited from Animal
    myDog.walk(); // Inherited from Mammal
    myDog.bark(); // Defined in Dog
}
  
```

3. Hierarchical Inheritance

A class which contains multiple subclasses is called hierarchical Inheritance.



3) Hierarchical

Example:

```

// Superclass
class Animal {
    void eat() {
        System.out.println("This animal eats.");
    }
}
  
```

```

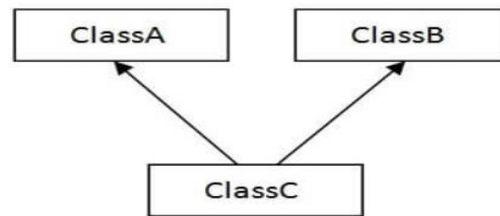
// Subclass 1
class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");
    }
}
  
```

```

// Subclass 2
class Cat extends Animal {
    void meow() {
        System.out.println("The cat meows.");
    }
    public static void main(String[] args) {
        Dog myDog = new Dog();
        Cat myCat = new Cat();
        myDog.eat(); // Inherited from Animal
        myDog.bark(); // Defined in Dog
        myCat.eat(); // Inherited from Animal
        myCat.meow(); // Defined in Cat
    }
}
  
```

4. Multiple Inheritance

- A class extends to more than one super class is called multiple inheritance
- In Java it is not possible to achieve multiple inheritance by using class but we can achieve multiple inheritance by using interface.



4) Multiple

Example :

```
class Father {
}
```

```
class mother {
}
```

```
class son extends Father , Mother{
/   *Not possible
    The following class would create
    ambiguity if multiple inheritance
    were allowed */
}
```

Multiple inheritance not possible in Java using class ?

- We cant able to achieve multiple inheritance by using class because of two reasons:

1. ambiguity while method calling.
2. ambiguity while constructor Chaining.

Example :

```
// Superclass Father
class Father {
    void run() {
        System.out.println("Father is running");
    }
}
```

```
// Superclass Mother
class Mother {
    void run() {
        System.out.println("Mother is running");
    }
}
```

```
// Attempting to create a subclass that extends both Father and Mother
class Child extends Father, Mother {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.run();
    }
}
```

// This code will not compile in Java

Note:

In case of multiple inheritance when child class inherits the properties from more than one parent and also if the parent class is having same method name and parameter then child get confused about which method will be called this problem is called diamond problem.

Even though if we declare different methods in super class we will get an ambiguity due to 11 method which is inherited from the supermost class called object class to the Child class.

5. Hybrid Inheritance

The combination of a hierarchical and multiple inheritance is called hybrid inheritance. Even hybrid inheritance is not possible using class but we can achieve using interface.

Method Overriding

The process of providing sub class specific implementation to the inherited methods is called method overriding.

- Without inheritance it is not possible to override a method.

Rules to override a method

- The method name must be same as in super class
- Method return type must be same as in Superclass
- Method signature will be same as in super class

Advantage : We can achieve runtime polymorphism.

Note: We can use @Override annotation to indicate the compiler about method overriding.

Example :

```
// Superclass
class Vehicle {
    void move() {
```

```

        System.out.println("The vehicle is moving");
    }
}

// Subclass 1
class Car extends Vehicle {
    @Override
    void move() {
        System.out.println("The car is driving");
    }
}

// Subclass 2
class Bicycle extends Vehicle {
    @Override
    void move() {
        System.out.println("The bicycle is pedaling");
    }
}

public class MethodOverridingExample {
    public static void main(String[] args) {
        Vehicle myVehicle = new Vehicle();
        Vehicle myCar = new Car();
        Vehicle myBicycle = new Bicycle();

        myVehicle.move(); // Output: The vehicle is moving
        myCar.move();     // Output: The car is driving
        myBicycle.move(); // Output: The bicycle is pedaling
    }
}

```

1. In how many ways we can initialise instance variable ?

We can initialise instance variable in 5 different ways

They are -

1. At the time of declaration
2. By using object reference
3. By using instance block
4. by using constructors
5. by using setter() method

1. At the time of declaration

Example:

```
class emp{
    String name= " Rama";
}
```

2. By using object reference

Example:

```
class emp{
    String name= " Rama";
    public static void main(String[] args){
        Emp e=new Emp();
        e.name= Rama";
    }
}
```

3. By using instance block

Example:

```
class emp{
    String name;
    {
        name= "Rama";
    }
}
```

- Instance block is also called as object block which is used to initialise instance variable at the time of object creation.
- Instance block get executed at the time of object creation
- Instance block cannot be invoked explicitly but it is invoked by JVM at the time of object creation.

Example :

```
public class emp {
    {
        System.out.println(" Instance block initialization ");
    }
    Public static void main(String[] args){
        Emp e=new Emp();
    }
}
```

- We can have multiple instance block inside a class
- The number of a times the instance block gets executed depends upon number of object created
- When we have multiple instance block then the execution takes place in sequential order.

Drawback:

In case of instance block each object will be initialised to the same values.

CONSTRUCTOR

A constructor in Java is a special method used to initialize objects. It is called when an object of a class is created. A constructor has the same name as the class and does not have a return type, not even void.

The main purpose of the constructor is to initialise the instance variable at the time of object creation.

Key Features of a Constructor:

1. **Same Name as Class:** The constructor name must match the class name.
2. **No Return Type:** Constructors don't have a return type.
3. **Automatic Invocation:** A constructor is automatically called when an object is created.

Types of Constructors:

1. Default Constructor:

- A constructor with no parameters.
- If no constructor is defined in the class, the Java compiler provides a default constructor.

Example :

```
public class Example {
    // Default Constructor
    public Example() {
        System.out.println("Default Constructor called");
    }

    public static void main(String[] args) {
        Example obj = new Example(); // Default Constructor is invoked
    }
}
```

2. User – Defined Constructor

1. Non- parameterized Constructor

A **non-parameterized constructor** (commonly known as a **default constructor**) is a constructor in Java that does not take any parameters. It initializes an object with default values or performs basic setup tasks when no specific information is provided at the time of object creation.

Key Characteristics:

1. It has no parameters.
2. It is typically used to provide user defined values for an object's attributes.
3. If no constructor is explicitly defined in a class, the Java compiler

automatically provides a default constructor. But if user declared a non – parameterized constructor then no default constructor is called.

Example :

```
public class Example {
    int id;
    String name;

    // Non-parameterized constructor
    public Example() {
        id = 10;
        name = "Lambu";
        System.out.println("Non-parameterized constructor called!");
    }

    public void display() {
        System.out.println("ID: " + id + ", Name: " + name);
    }

    public static void main(String[] args) {
        Example obj = new Example(); // Invokes the non-parameterized
        constructor
        obj.display();
    }
}
```

Drawback:

In case of Non – parameterized Constructor each object will be initialised to the same values.

2. Parameterized Constructor:

- A constructor with parameters to initialize the object with specific values.

Example :

```
public class Example {
    int id;
    String name;

    // Parameterized Constructor
    public Example(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

```

public void display() {
    System.out.println("ID: " + id + ", Name: " + name);
}

public static void main(String[] args) {
    Example obj = new Example(101, "John");
    obj.display();
}
}

```

Overloading Constructors

The term overloading refers to the act of using the same constructor name in a class but different signature declarations

- There should be change in the number of parameters.
- There should be change in datatype of the parameter.
- There should be change in sequence of parameters.

// Overloading constructors.

```

public class Student {
    int id;
    String name;
    int age;

```

// Zero - Parameterized Constructor

```

public Student() {
    id = 0;
    name = "Unknown";
    age = 0;
}

```

// Parameterized Constructor with 1 parameter

```

public Student(int id) {
    this.id = id;
    this.name = "Unknown";
    this.age = 0;
}

```



```

// Parameterized Constructor with 2 parameters
public Student(int id, String name) {
    this.id = id;
    this.name = name;
    this.age = 0;
}

// Parameterized Constructor with 3 parameters
public Student(int id, String name, int age) {
    this.id = id;
    this.name = name;
    this.age = age;
}

public void display() {
    System.out.println("ID: " + id + ", Name: " + name + ", Age: " + age);
}

public static void main(String[] args) {
    // Calling different constructors
    Student s1 = new Student(); // Default constructor
    Student s2 = new Student(101); // Constructor with 1 parameter
    Student s3 = new Student(102, "Alice"); // Constructor with 2 parameters
    Student s4 = new Student(103, "Bob", 20); // Constructor with 3 parameters

    // Displaying details
    s1.display();
    s2.display();
    s3.display();
    s4.display();
}
}

```

Note : Constructor can be overloaded but cannot be overridden since constructor cannot be inherited so it cannot be overridden.

Drawbacks

- In the above code we are initialising the same properties again and again which is going to cause code redundancy.
- To overcome this we can use something called as **constructor chaining**.

Constructor chaining

- ❖ A constructor is always called inside another constructor
- ❖ Calling one constructor inside another constructor is called constructor chaining
- ❖ A constructor call only one constructor inside the another constructor
- ❖ Constructor can be called inside another constructor only by using this() or super() calling statement
- ❖ This calling statement is used to call current class overloaded constructors.

Note :

- Either this() or super() must be the first executable statement inside a constructor
- If the user doesn't provide this calling statement then by default it takes the super calling statement.
- The cyclic calling of a constructor is not possible in Java.

Note: It is a mandatory to call immediate super class constructor inside another sub class constructor.

In Java, the this and super keywords are used to call constructors and methods within a class and its superclass, respectively. Here's a brief overview of how they are used in constructor chaining and method invocation.

Example : need of constructor chaining

// constructor overloading

```
public class Student {
```

```
    String name;
```

```
    int age; // 20
```

```
    char Sec; // /u0000
```

```
    Student(){ //s1
```

```
        name="ram";
```

```
    }
```

```
    Student(String name, int age){ //s2
```

```
        this.name=name;
```

```
        this.age=age;
```

```
    }
```

```
    Student(int age , char Sec){ //s3
```

```
        this.age=age;
```

```
        this.Sec=Sec;
```

```
    }
```

```

Student(String name , int age, char Sec){           //s4
    this.name=name;
    this.age=age;
    this.Sec=Sec;
}

```

```

public static void main(String[] args) {
    Student s1=new Student();
    System.out.println(s1.name);
    System.out.println(s1.age);
    System.out.println(s1.Sec);

    Student s2=new Student("Rajesh",20);
    System.out.println(s2.name);
    System.out.println(s2.age);
    System.out.println(s2.Sec);

    Student s3=new Student(20,'c');
    System.out.println(s3.name);
    System.out.println(s3.age);
    System.out.println(s3.Sec);

    Student s4=new Student("Raghu",21,'b');
    System.out.println(s4.name);
    System.out.println(s4.age);
    System.out.println(s4.Sec);
}
}

```

// * . In the above code we are intializing the same properties
 // again and again which causes code redundancy

// * . to overcome this we can use constructor chaining.

this() calling statement

The this() calling statement in Java is used to refer to the current object. It has several uses, including calling another constructor within the same class.

super() calling statement

The super() calling statement in Java is used to refer to the superclass of the current object. It can be used to access superclass methods and constructors.

Example 1: Constructor Chaining within the Same Class using this()

In this example, we demonstrate how constructor chaining works within the same class by using the this() keyword.

```

public class Student {
    String name;        // Raghu
    int age;             // 22
    char Sec;           // c

    Student(String name){
        this.name=name; //Raghu
    }
    Student(String name ,int age){
        this(name); //Raghu
        this.age=age; //22
    }
    Student(String name,int age,char sec){

        this(name,age); // code redundancy , Constructor chaining
        this.Sec=sec;
    }

    public static void main(String[] args) {

        Student s3=new Student("Raghu",22,'c');
        System.out.println(s3.name);
        System.out.println(s3.age);
        System.out.println(s3.Sec);

        Student s4=new Student("Ram",21,'a');
        System.out.println(s4.name);
        System.out.println(s4.age);
        System.out.println(s4.Sec);
    }
}

```

Example 2: Constructor Chaining with Super()

In this example, we demonstrate how constructor chaining works with inheritance using the super() keyword.

```

// Superclass
class Animal {
    String name;

    // Superclass constructor
    Animal(String name) {
        this.name = name;
        System.out.println("Animal constructor called");
    }
}

// Subclass
class Dog extends Animal {
    int age;

    // Subclass constructor
    Dog(String name, int age) {
        super(name); // Calls the superclass constructor
        this.age = age;
        System.out.println("Dog constructor called");
    }

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }

    public static void main(String[] args) {
        Dog myDog = new Dog("Buddy", 3);
        myDog.display();
    }
}

```

Explanation

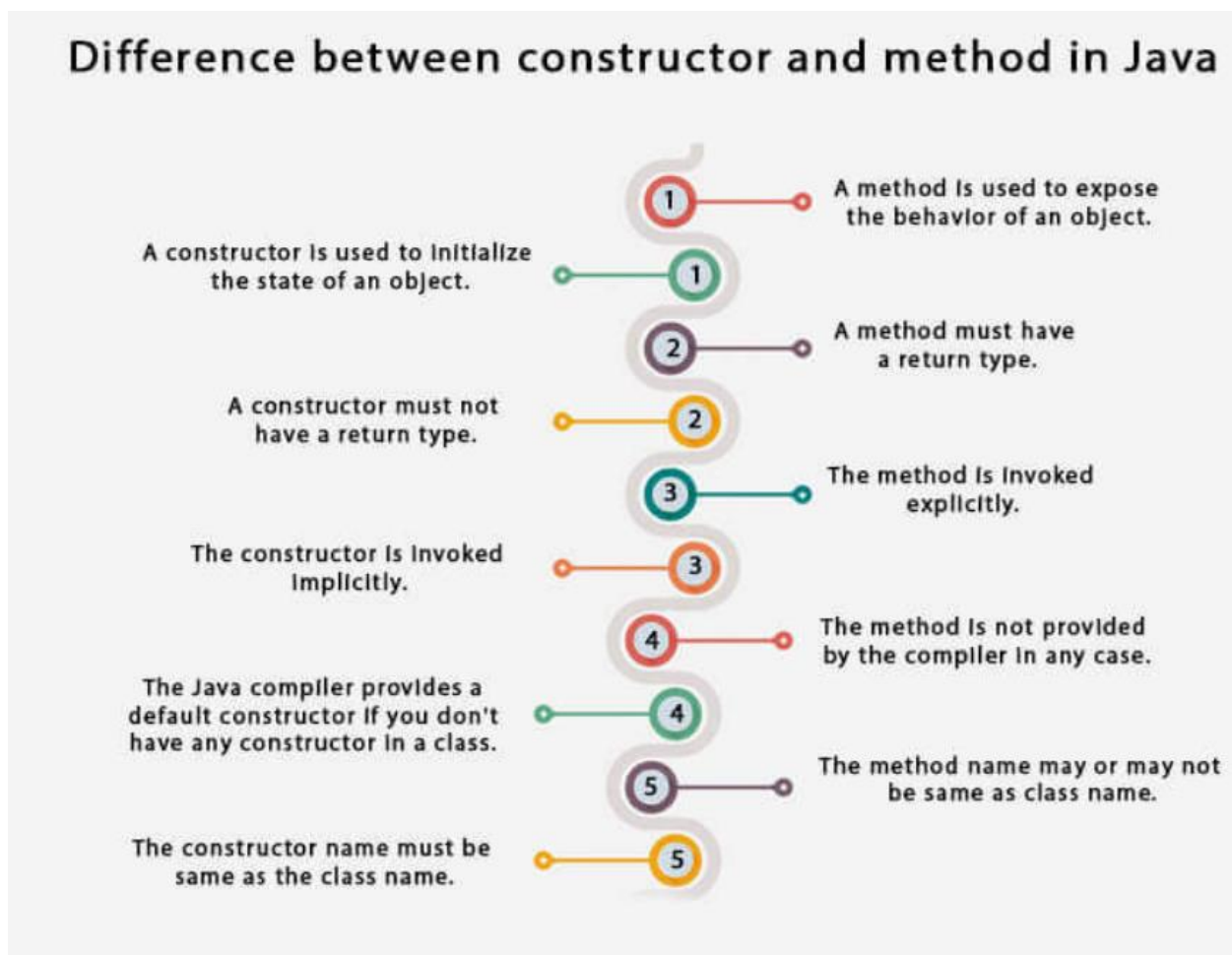
1. Superclass Animal:

- The Animal class has a constructor that initializes the name field and prints a message.

2. Subclass Dog:

- The Dog class extends the Animal class.
- The Dog constructor calls the Animal constructor using `super(name)` to initialize the name field and then initializes the age field.

Difference between methods and constructors



COPY CONSTRUCTORS

A copy constructor is a special type of constructor that initializes a new object using another object of the same class.

While Java does not have a built-in copy constructor like C++, you can define your own to achieve similar functionality.

Example:

```
class Person {
    private String name;
    private int age;

    // Parameterized constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

```

    }

    // Copy constructor
    public Person(Person other) {
        this.name = other.name;
        this.age = other.age;
    }

    // Getters
    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    // Method to display details
    public void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }

    public static void main(String[] args) {
        // Create an original object
        Person person1 = new Person("Alice", 30);
        person1.display(); // Output: Name: Alice, Age: 30

        // Create a copy of the original object using the copy constructor
        Person person2 = new Person(person1);
        person2.display(); // Output: Name: Alice, Age: 30
    }
}

```

Benefits of Copy Constructor

Object Cloning: Creates a new object as a copy of an existing object.

Deep Copy: Helps in creating a deep copy of an object with complex types (e.g., objects containing references to other objects).

Some Important Points Regarding Constructors :

1. Constructors Cannot Be final, static, or abstract

- Reason:
 - A constructor is tied to object creation and is not meant to be inherited, overridden, or shared as a class-level method.
 - final prevents overriding, static associates with the class (not an object), and abstract requires implementation, which conflicts with the nature of constructors.

2. Constructors Are Not Inherited

- Constructors belong to the class they are defined in and are not inherited by subclasses.
- However, a subclass can invoke the parent class constructor using `super()`.

3. Constructors Cannot Be Overridden

- Overriding applies to instance methods, not constructors, as constructors are not inherited.
- However, you can **overload** constructors within the same class.

4. Constructors Can Be Private

- A private constructor prevents object creation from outside the class. It is commonly used in **Singleton Design Pattern**.

```

▪ public class Singleton {
▪     private static Singleton instance;
▪
▪     // Private Constructor
▪     private Singleton() {
▪     }
▪
▪     public static Singleton getInstance() {
▪         if (instance == null) {
▪             instance = new Singleton();
▪         }
▪         return instance;
▪     }
▪ }

```

5. Cannot Call `this()` and `super()` in the Same Constructor

- You cannot use both `this()` and `super()` in the same constructor, as both must be the first statement.

6. Constructor Execution Order in Inheritance

- In an inheritance hierarchy, parent class constructors are called first, followed by subclass constructors.

- **Example:**

```
class A {
    A() {
        System.out.println("A's Constructor");
    }
}

class B extends A {
    B() {
        System.out.println("B's Constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        new B();
    }
}
```

Output:

A's Constructor
B's Constructor

TYPE CASTING

Typecasting in Java refers to the process of converting one data type to another. It can be broadly classified into two categories:

1. primitive typecasting
2. reference typecasting [Non – primitive type casting].

Primitive Typecasting

Primitive typecasting involves converting between the basic data types in Java: byte, short, int, long, float, double, char, and boolean.

Again in primitive type casting we have two branches, they are

- Implicit type casting
- Explicit type casting

1. Implicit (Widening) Typecasting

Implicit typecasting, also known as widening conversion, occurs when a smaller data type is automatically converted to a larger data type. This type of casting is done automatically by the Java compiler and does not require explicit casting.

Example :

```
public class ImplicitTypecasting {
    public static void main(String[] args) {
        int num = 100;
        double doubleNum = num; // Implicit casting: int to double
        System.out.println("Integer value: " + num
        System.out.println("Double value: " + doubleNum
    }
}
```

Output: Integer value: 100

Double value: 100.0

□ Widening Conversion (Implicit Casting):

- Smaller data types are converted to larger data types.
- No data loss, safe, and done automatically by the compiler.
- byte -> short -> int -> long -> float -> double
- char -> int -> long -> float -> double

2. Explicit (Narrowing) Typecasting

Explicit typecasting, also known as narrowing conversion, occurs when a larger data type is converted to a smaller data type. This type of casting must be done manually by the programmer and can potentially lead to data loss.

Example :

```
public class ExplicitTypecasting {
    public static void main(String[] args) {
        double doubleNum = 100.99;
        int num = (int) doubleNum; // Explicit casting: double to int

        System.out.println("Double value: " + doubleNum);
        System.out.println("Integer value: " + num);
    }
}
```

Output: Double value: 100.99

Integer value: 100

□ Narrowing Conversion (Explicit Casting):

- Larger data types are converted to smaller data types.
- Potential data loss, requires explicit casting.
- double -> float -> long -> int -> short -> byte
- double -> float -> long -> int -> char

Reference Typecasting

Reference typecasting involves converting between different types of objects, typically within an inheritance hierarchy.

Again in Non-Primitive type casting we have two branches, they are

- Upcasting
- Downcasting

1. Upcasting

Upcasting is the process of casting a subclass reference to a superclass reference. Upcasting is safe and is done implicitly in Java.

Example:

```
class Animal {
    void makeSound() {
        System.out.println("Animal sound");
    }
}
```

```
class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Bark");
    }
}
```

```
public class UpcastingExample {
    public static void main(String[] args) {
        Dog dog = new Dog();
        Animal animal = dog; // Upcasting: Dog to Animal

        animal.makeSound(); // Output: Bark
    }
}
```

```
}
```

2. Downcasting

Downcasting is the process of casting a superclass reference back to a subclass reference. Downcasting requires explicit casting and can lead to a `ClassCastException` if not done correctly.

Example:

```
class Animal {
    void makeSound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Bark");
    }

    void fetch() {
        System.out.println("Dog fetches the ball");
    }
}

public class DowncastingExample {
    public static void main(String[] args) {
        Animal animal = new Dog(); // Upcasting: Dog to Animal

        if (animal instanceof Dog) {
            Dog dog = (Dog) animal; // Downcasting: Animal to Dog
            dog.makeSound(); // Output: Bark
            dog.fetch();     // Output: Dog fetches the ball
        }
    }
}
```

Note :

- ☐ Upcasting is safe and implicit.
- ☐ Downcasting requires explicit casting and is subject to `ClassCastException`.

Instanceof Operator:

- Use the instanceof operator to check if an object can be safely downcasted.
- Avoids ClassCastException.

Example:

```
class Animal {
    void makeSound() {
        System.out.println("Some generic animal sound");
    }
}

class Dog extends Animal {
    void makeSound() {
        System.out.println("Bark");
    }

    void fetch() {
        System.out.println("Dog fetches the ball");
    }
}

public class InstanceofExample {
    public static void main(String[] args) {
        Animal animal = new Dog(); // Upcasting: Dog to Animal

        // Checking the actual type of the object
        if (animal instanceof Dog) {
            Dog dog = (Dog) animal; // Downcasting: Animal to Dog
            dog.makeSound(); // Output: Bark
            dog.fetch();     // Output: Dog fetches the ball
        } else {
            animal.makeSound();
        }
    }
}
```

Note : We cannot store a superclass object in a subclass.

Characteristics of Upcasting

1. In case of Upcasting using super class reference we can access only inherited properties but not subclass specific properties (methods and variables)

Eg :

```
class Employee {
    // Inherited property
    String name = "Generic Employee";

    // Inherited method
    void work() {
        System.out.println("Employee is working");
    }
}

class Manager extends Employee {
    // Subclass-specific property
    int teamSize = 10;

    // Overridden method
    @Override
    void work() {
        System.out.println("Manager is managing the team");
    }

    // Subclass-specific method
    void conductMeeting() {
        System.out.println("Manager is conducting a meeting");
    }
}

public class EmployeeExample {
    public static void main(String[] args) {
        // Upcasting: Subclass object is assigned to a superclass reference
        Employee employee = new Manager();

        // Accessing inherited property and method
        System.out.println(employee.name);
        employee.work();

        // Trying to access subclass-specific properties and methods (Not allowed)
        // System.out.println(employee.teamSize);
        // employee.conductMeeting();
    }
}
```

Output: Generic Employee
 Manager is managing the team
 Compile-time error
 Compile-time error

2) In case of op pasting when we invoke overridden method then the implementation is always executed from subclass but not from Superclass.

Eg:

```
class Animal {
    // Inherited method
    void makeSound() {
        System.out.println("Some generic animal sound");
    }
}

class Dog extends Animal {
    // Overridden method
    @Override
    void makeSound() {
        System.out.println("Bark");
    }
}

class Cat extends Animal {
    // Overridden method
    @Override
    void makeSound() {
        System.out.println("Meow");
    }
}

public class OverriddenMethodExample {
    public static void main(String[] args) {
        // Upcasting: Subclass objects are assigned to superclass references
        Animal animal1 = new Dog();
        Animal animal2 = new Cat();

        // Invoking overridden method
        animal1.makeSound(); // Output: Bark
        animal2.makeSound(); // Output: Meow
    }
}
```

```
// Even though the reference type is Animal, the actual object's method is called
    }
}
```

Output :

```
Bark
Meow
```

Method binding

The process of attaching method logic or implementation to its method call is called method binding.

Eg: class Mouse {
 void rightClick(){
 //method logic
 }
 public static void main(String[] args){
 Mouse m=new Mouse();
 m.rightClick();
 }
 }

Types of method binding :

- Early binding or static binding
- Late bidding or dynamic binding

1. Early binding

- In case of method overloading when we invoke the overloaded method then the decision about the method binding is taken by compiler based on parameter and arguments
- Since the decision is taken by compiler at a compile time we call it as early binding or static binding

Eg:

```
class Website {
    void login(String your name string pwd){
        // logic
    }

    void login(String your name string pwd){
        // logic
    }
}
```



```

public static void main(String[] args){
    Website w= new Website();
    w.login(123456789011 , 1234);
}
}

```

- Method overloading is the best example of early binding.

2. Late Binding

- In case of Upcasting when we invoke the overridden method then the decision about method binding is taken by jvm at runtime based on object.
- since the decision about method binding is taken by the jvm at runtime we call it as a late binding or dynamic binding.

Eg:

```

// Parent class
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

```

```

// Subclass inheriting from Animal
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Animal animal = new Dog(); // Upcasting
        animal.sound(); // Late binding
    }
}

```

POLYMORPHISM

Polymorphism is considered one of the important features of Object-Oriented

Programming. Polymorphism allows us to perform a single action in different ways.

In other words, polymorphism allows you to define one interface and have multiple implementations. The word “poly” means many and “morphs” means forms, So it means many forms.

Types of Java Polymorphism

In Java Polymorphism is mainly divided into two types:

- Compile-time Polymorphism
- Runtime Polymorphism

1. Compile-Time Polymorphism

- It is also known as static polymorphism. This type of polymorphism is achieved by method overloading.
- Method overloading is an ability of a method to take multiple forms based on input

(Go through the example of method overloading)

2. Runtime Polymorphism

- Runtime polymorphism can be achieved by method overriding. Method overriding is an ability of a method to take multiple forms based on object.
- Runtime polymorphism is also known as late binding

(Go through the example of method Overriding)

Packages

Packages are used to group related classes and interfaces together. They provide a way to manage namespaces in Java and organize code in a modular fashion.

Syntax:

```
package mypackage;
```

```
public class MyClass {
    // class code
}
```

Example:

```
package com.example.utilities;
```

```
public class MathUtils {
    public static int add(int a, int b) {
        return a + b;
    }
}
```

```
}
}
```

Structure

```
com
├── example
│   └── utilities
│       └── MathUtils.java
```

Advantages:

1. **Namespace Management:** Avoids naming conflicts.
2. **Modularity:** Encapsulates related classes.
3. **Access Control:** Provides access protection for classes and interfaces.

Drawbacks:

- Complexity in managing large projects.
- Requires careful planning to avoid package dependencies.

2. Importing Packages

To use a class from a different package, you must import the package.

Syntax:

```
import package.name.ClassName;
```

Example:

```
import com.example.utilities.MathUtils;
```

```
public class Test {
    public static void main(String[] args) {
        int sum = MathUtils.add(5, 10);
        System.out.println("Sum: " + sum);
    }
}
```

Standard Structure

```
Main Package
├── Test.java
│   └── import com.example.utilities.MathUtils
│       └── MathUtils.java
```

Class Name : MathUtils.java

Fully Qualified class name (fqcn) : com.example.utilities.MathUtils

- Package declaration must be the first executable statement .

- Import statement is next executable statement after package declaration.

Some of the inbuilt packages are :

1. java.util package.
 2. java.io package
 3. java.sql package
 4. java.lang package
 5. javax.sql package
- java.lang is the default package .Any class which is present in java.lang package it is directly available to all classes without any import statement.
Eg : String class , StringBuffer class and object class etc..

Access Modifiers

Java provides different access levels to protect data from unauthorized access. Or it is a visibility given to java members.

Types:

- **Default:** Accessible only within the same package.
- **Public:** Accessible from any other class.
- **Protected:** Accessible within the same package and subclasses.
- **Private:** Accessible only within the declared class.

Example:

```
package com.example;  
  
public class Example {  
    private int privateVar;  
    protected int protectedVar;  
    public int publicVar;  
    int defaultVar;  
  
    // Getters and Setters  
}
```

Advantages:

- **Encapsulation:** Protects data integrity.
- **Security:** Limits access to sensitive data.

Access Modifiers In Java

Access Modifier	Within the Class	Other Classes [Within the Package]	In Subclasses [Within the package and other packages]	Any Class [In Other Packages]
public	Y	Y	Y	Y
protected	Y	Y	Y	N
default	Y	Y	Same Package – Y Other Packages – N	N
private	Y	N	N	N

Y – Accessible
N – Not Accessible

Access Modifiers

1. Public

Definition: The public access modifier is accessible from any other class.

Example:

```
public class PublicClass {
    public void display() {
        System.out.println("Public Method");
    }
}
```

Advantages:

- Maximum accessibility.

Drawbacks:

- Can expose sensitive parts of the code.

2. Private

Definition: The private access modifier is accessible only within the declared class.

Example:

```
public class PrivateClass {
    private int data;

    private void display() {
        System.out.println("Private Method");
    }
}
```

Advantages:

- Encapsulation and data hiding.

Drawbacks:

- Limits reuse and accessibility.

3. Protected

Definition: The protected access modifier is accessible within the same package and subclasses.

Example:

```
public class ProtectedClass {
    protected void display() {
        System.out.println("Protected Method");
    }
}
```

Advantages:

- Balances accessibility and encapsulation.

Drawbacks:

- Can be misused in subclasses.

4. Default

Definition: No modifier means default access, which is package-private.

Example:

```
class DefaultClass {
    void display() {
        System.out.println("Default Method");
    }
}
```

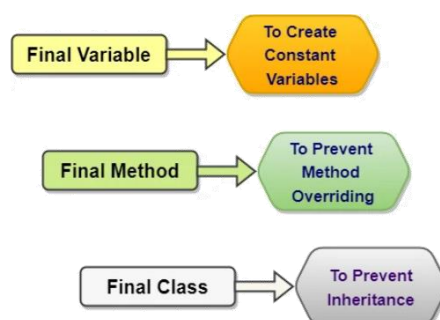
Advantages:

- Package-level access control.

Drawbacks:

- Not suitable for broader accessibility needs.

Java Final Keyword



1. Final Variables

A final variable can be assigned only once.

Example:

```
public class FinalVariableExample {
    final int MAX_VALUE = 100;

    void display() {
        // MAX_VALUE = 200; // This will cause a compilation error
        System.out.println(MAX_VALUE);
    }
}
```

Advantages:

- Ensures constant values.
- Improves code readability.

Drawbacks:

- Inflexibility in changing the value.

2. Final Methods

A final method cannot be overridden by subclasses.

Example:

```
public class FinalMethodExample {
    public final void display() {
        System.out.println("This is a final method.");
    }
}

class SubClass extends FinalMethodExample {
    // void display() { } // This will cause a compilation error
}
```

Advantages:

- Prevents accidental method overriding.

Drawbacks:

- Limits polymorphism.

3. Final Classes

A final class cannot be subclassed.

Example:

```
public final class FinalClassExample {
    public void display() {
        System.out.println("This is a final class.");
    }
}
```


// class SubClass extends FinalClassExample { } // This will cause a compilation error

Advantages:

- Ensures immutability of the class.

Drawbacks:

- Limits inheritance and reuse.

ENCAPSULATION

The process of providing controlled access to the private members of a class using accessors and mutators is referred as Encapsulation. So, the mutators is known as setter() method and the accessors is known as getter() method.

Rules to encapsulate a class

- In case of encapsulation the class should be public non – abstract class.
- The class must contain private data members.
- For each private data member we should have public setter() and getter() methods.

Advantages:

- **Data Hiding:** Protects data from unauthorized access.
- **Modularity:** Easier to manage and understand.
- **Improved Maintenance:** Changes in one part of the code can be made independently.

Example : public class EncapsulationExample {

```
private String name;
private int age;
```

```
public String getName() {
    return name;
}
```

```
public void setName(String name) {
    this.name = name;
}
```

```
public int getAge() {
    return age;
}
```

```
public void setAge(int age) {
    if (age > 0) {
        this.age = age;
    }
}
```



```

    }
  }
}

```

Note :

- Private data member with only getter method without any setter method is called Read – only data.
- Private data member with only setter method without any getter method is called Write – only data.
- **Encapsulation class is also known as JavaBean class / POJO class.**

Java static keyword

The **static keyword** in Java is used for memory management mainly. We can apply static keyword with variables, methods, blocks and nested classes. The static keyword belongs to the class than an instance of the class.

The static can be:

1. Variable (also known as a class variable)
2. Method (also known as a class method)
3. Block

1. Java static variable

If you declare any variable as static, it is known as a static variable.

- The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.
- static variables is also called as class variables
- The static variable gets memory only once in the class area at the time of class loading.
- static variables are related to class but not related to object.
- static variables gets created inside class area which is also known as method area.

Accessibility of static variables

- static variables can be accessed directly inside a static method, constructors and blocks.
- Static variables always store recent values but it wont store initial values once after the modification.
- Static variables always stores recent values but it won't allow us to store values , once after the modification.
- Static variables can be accessed inside a non static methods and outside the class By using class name followed by dot (.) operator.

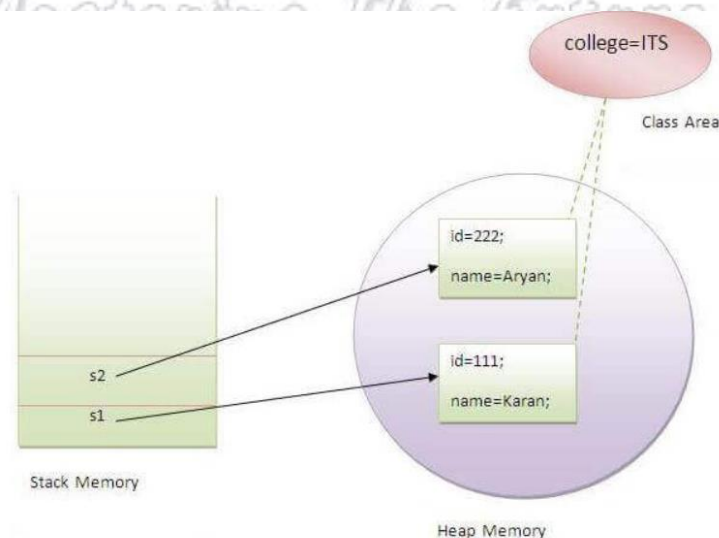
Example:

```

class Student{
    int rollno;//instance variable
    String name;
    static String college ="ITS";//static variable
//constructor
    Student(int r, String n){
        rollno = r;
        name = n;
    }
//method to display the values
    void display (){
        System.out.println(rollno+" "+name+" "+college);
    }
}
//Test class to show the values of objects
public class TestStaticVariable1 {
    public static void main(String args[]){
        Student s1 = new Student(111,"Karan");
        Student s2 = new Student(222,"Aryan");
//we can change the college of all objects by the single line of code
//Student.college="BBDIT";
        s1.display();
        s2.display();
    }
}

```

Memory representation



Note:

- Local variables cannot be static

- Static variables can get the default values based on the data types

2. Java static method

If you apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than the object of a class.
- Static members are loaded into the memory at the time of class loading.
- Static members are loaded and executed in method area.
- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data member and can change the value of it.

Accessibility of static methods

- Static methods can be accessed directly inside a static methods even inside a main method without creating an object.
- Static methods and variables can be accessed directly inside a non - static methods within a same class.
- Static members can be accessed outside the class by using the class name followed by (.) dot operator.

Example :

```
class Calculate{
    static int cube(int x){
        return x*x*x;
    }
    public static void main(String args[]){
        int result=Calculate.cube(5);
        System.out.println(result);
    }
}
```

Restrictions for the static method

There are two main restrictions for the static method. They are:

1. The static method can not use non static data member or call non-static method directly.
2. this and super cannot be used in static context.

Why is the Java main method static?

It is because the object is not required to call a static method. If it were a non-static method, JVM creates an object first then call main() method that will lead the problem of extra memory allocation.

3. Java static block

- Is used to initialize the static data member.
- Static blocks gets executed at the time of class loading even before the execution of

the main method.

- The block declared with a 'static' keyword is called static block.
- It is executed before the main method at the time of classloading.

Example :

```
class A2{
    static{
        System.out.println("static block is invoked");
    }

    public static void main(String args[]){
        System.out.println("Hello main");
    }
}
```

Java abstract keyword

1. abstract methods

A method with declaration without any implementation is called abstract method.

- Abstract method is always declared with abstract keyword.
- Abstract method must be terminated with semicolon (;).

Example : abstract void add();

- Abstract method can be declared only inside abstract 'class' and 'interface' . but we can't able to declare inside normal class.

Example :

// inside abstract class

```
abstract class Bike{
    abstract void run(); // possible
}
```

// inside normal class

```
class Demo{
    abstract void m(); // error not possible
}
```

- Abstract method can be overloaded .

Example :

```
abstract long run();
abstract long run(long meters);
```

- We must override abstract methods in subclass or in implementation classes.

2. abstract class

A class declared with abstract keyword is called abstract class.

- Abstract class is like a normal class which can have constructor, variables, methods but only difference is we can have abstract method and **we can't able to create an object.**
- abstract class and interface act as a non-primitive data type to refer the subclass object.
- Abstract class can have abstract methods and also concrete methods.
- If the class contain abstract method then it is mandatory to declare class as abstract.

Example :

```
abstract class AbstractClassExample {
    abstract void display(); // abstract method

    void show() {           // normal method or concrete method
        System.out.println("Concrete Method");
    }
}
```

```
class SubClass extends AbstractClassExample {
    @Override
    void display() { // we need to override abstract methods
        System.out.println("Abstract Method Implemented");
    }
}
```

- Abstract class And interface is used to achieve abstraction
- Abstract class can also extend to another class
- Abstract methods present in abstract class must be overridden in subclass or we need to declare subclass as abstract
- By using abstract class we can't able to achieve 100% abstraction but by using interface we can achieve 100% abstraction.

INTERFACE

The interface in Java is a mechanism to achieve abstraction. Traditionally, an interface could only have abstract methods (methods without a body) and public, static, and final variables by default. It is used to achieve abstraction and multiple inheritances in Java.

- Java Interface also represents the IS-A relationship.
- To declare an interface, use the interface keyword. It is used to provide total abstraction.
- Syntax :

```
interface interface_name{
    // declare constant fields
    // declare methods that abstract
    // by default.
}
```

- Interface is also called as rules repository or coding contract.
- We can't able to create a object for interface so there is no constructor.
- Interface is used to achieve 100% abstraction.

Real world example :

- Remote is an interface between TV and User.
- Switch is an interface between light and user.

Application based example :

- Payment gateway is an interface between electronic financial transactions.
- JDBC Api is an interface between Java application and database server.

Example 1:

```
interface Switch{
    abstract void son();
    abstract void soff();
}
```

Example 2:

```
interface TimeDate{
    void ChangeTime();
    void ChangeDate();
}
```

- If you don't use abstract keyword while declaring a method by default it takes abstract method.
- By default if any method is terminated with semicolon(;) in interface then automatically it will take public abstract
- A class can inherit interface by using implement keyword.
- A class which inherits the interface is called implementation class.

Example :

```
public interface Switch{
    void son();
    void soff();
}
```

```
class Light implements Switch{
    @Override
    public void son(){
        System.out.println("light switch on");
    }
    @Override
    public void soff(){
```

```

        System.out.println("light switch off");
    }

}

public class Test{
    public static void main(String[] args){
        Switch s = new Switch();
        s.son();
        s.off();
    }
}

```

Note : By using interface we can achieve multiple inheritance and hybrid inheritance also .

Eg : Multiple inheritance.

```

public interface InterfaceA {
    void methodA();
}

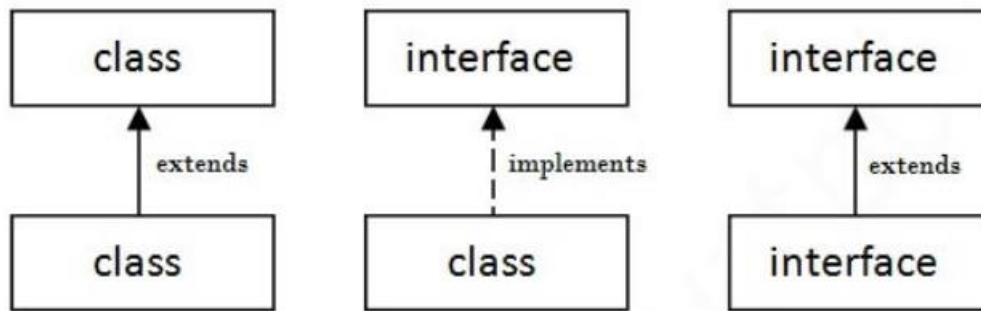
public interface InterfaceB {
    void methodB();
}

public class MyClass implements InterfaceA, InterfaceB {
    @Override
    public void methodA() {
        System.out.println("Method A Implementation");
    }

    @Override
    public void methodB() {
        System.out.println("Method B Implementation");
    }
}

```

- An interface can inherit another interface by using the extends keyword.



- An interface cannot have Instance variables and constructors.

Add on features for interface from jdk 1.8 version

- From jdk 1.8 version we can have static and concrete method inside interface.
- From jdk 1.8 version we can have default concrete methods also.

Examples :

```

public interface MyInterface {
    // abstract method
    void abstractMethod();

    // default method
    default void defaultMethod() {
        System.out.println("Default Method");
    }

    // static method
    static void staticMethod() {
        System.out.println("Static Method");
    }

    // constant variable
    int CONSTANT = 100;
}

```

Types of interface:

There are three types of interfaces.

- Marker interface
- Functional interface
- Regular interface

i. Marker interface :

An interface which has no member is known as a marker or tagged interface, for

example, Serializable, Cloneable, Remote, etc. They are used to provide some essential information to the JVM so that JVM may perform some useful operation.

Eg:

```
public interface Serializable{
}
```

ii. Functional interface :

Functional interfaces are a special type of interface with a single abstract method, used extensively in lambda expressions and method references.

- Functional interface which is used to indicate the main business functionality of an object
- functional interface contains only one abstract method but it can have any number of default or static methods from jdk 1.8 version onwards.

Eg: **Runnable** : It contains only `run()` method.

Comparable : It contains only `compareTo()` method.

Comparator : It contains only `compare()` method.

Functional Interfaces and Lambda Expressions (Java 8+)

A functional interface is an interface with exactly one abstract method. These interfaces are used extensively in **lambda expressions** and **method references**.

- Lambda expression provides implementation of *functional interface*. An interface which has only one abstract method is called functional interface. Java provides an annotation `@FunctionalInterface`, which is used to declare an interface as functional interface.

Eg:

```
public interface FunctionalInterface {
    // Lambda expressions can be implemented only for functional interface

    void abstractTest(int z);
}
```

LAMBDA EXPRESSION

Lambda expression is a new and important feature of Java which was included in Java SE 8. The Lambda expression is used to provide the implementation of an interface which has functional interface. It saves a lot of code. In case of lambda expression, we don't need to define the method again for providing the implementation. Here, we just write the implementation code.

Advantage of Lambda Expression

1. To provide the implementation of Functional interface.

2. Less coding.

Lambda Expression Syntax

(argument-list) -> {body}

Eg 1 :

```
@FunctionalInterface //It is optional
interface Drawable{
    public void draw();
}

public class LambdaExpressionExample2 {
    public static void main(String[] args) {
        int width=10;

        //with lambda
        Drawable d2=()->{
            System.out.println("Drawing "+width);
        };
        d2.draw();
    }
}
```

Eg 2 : Without Lambda Expression if we try to write the same code

```
interface Drawable {
    public void draw();
}

// Create a class that implements the Drawable interface
class DrawableImpl implements Drawable {
    private int width;

    public DrawableImpl(int width) {
        this.width = width;
    }

    @Override
    public void draw() {
        System.out.println("Drawing " + width);
    }
}
```

```

    }

    public class LambdaExpressionExample2 {
        public static void main(String[] args) {
            int width = 10;

            // Without lambda
            Drawable d2 = new DrawableImpl(width);
            d2.draw();
        }
    }

```

ABSTRACTION

Abstraction is a process of hiding the implementation details and showing only the functionality to the user with the help of interface is called abstraction.

Let's understand the abstraction with the help of a real-world example. The best example of abstraction is a car. When we derive a car, we do not know **how is the car moving** or **how internal components are working?** But we know **how to derive a car**. It means it is not necessary to know how the car is working, but it is important how to derive a car. The same is an abstraction.

The same principle (as we have explained in the above example) also applied in Java programming and any OOPs. In the language of programming, the code implementation is hidden from the user and only the necessary functionality is shown or provided to the user. We can achieve abstraction in two ways:

- Using Abstract Class
- Using Interface

By using interface ,

- We can achieve total abstraction.
- We can use multiple interfaces in a class that leads to multiple inheritance.
- It also helps to achieve loose coupling.

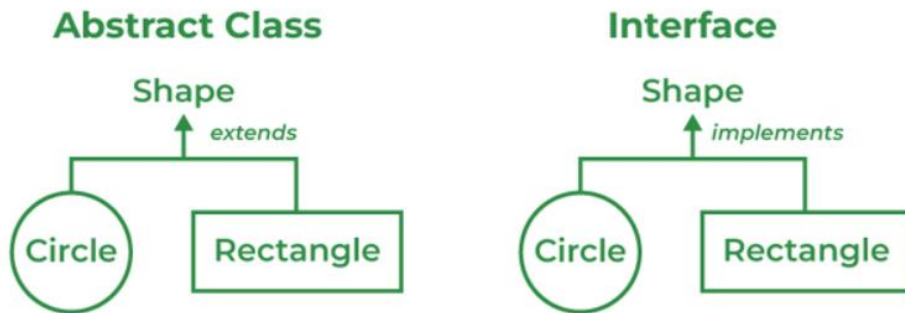
Loose coupling :

The change in implementation which doesn't effect the users is called as loose coupling.

Tight coupling :

The change in implementation which effects the user is called tight coupling.

For example 1 :



Programmatically : w.r.t Abstract class

```
abstract class Shape {
    abstract double area();
}
```

```
class Circle extends Shape {
    double radius;
```

```
    Circle(double radius) {
        this.radius = radius;
    }
```

```
    @Override
    double area() {
        return Math.PI * radius * radius;
    }
}
```

```
class Rectangle extends Shape {
    double width;
    double height;
```

```
    Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }
```

```
    @Override
    double area() {
        return width * height;
    }
}
```

```

    }
    public class Main {
        public static void main(String[] args) {
            Shape circle = new Circle(2);
            Shape rectangle = new Rectangle(3, 4);

            System.out.println("Circle area: " + circle.area());
            System.out.println("Rectangle area: " + rectangle.area());
        }
    }

```

Programmatically : w.r.t interface

```

// Define an interface Shape
interface Shape {
    double area();
}
// Define a Circle class that implements Shape
class Circle implements Shape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double area() {
        return 3.14 * radius * radius;
    }
}

// Define a Rectangle class that implements Shape
class Rectangle implements Shape {
    private double length;
    private double width;
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
}

```

```

@Override
public double area() {
    return length * width;
}
}

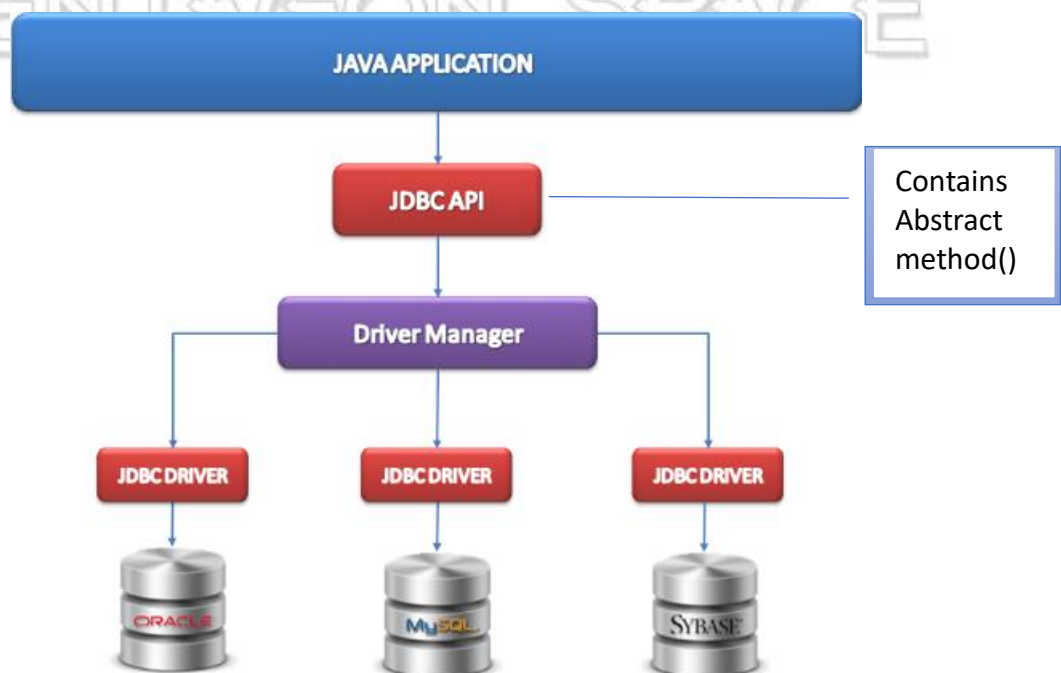
public class Main {
    public static void main(String[] args) {
        Shape circle2 = new Circle(7);
        Shape rectangle2 = new Rectangle(3, 8);

        System.out.println("Circle Area: " + circle2.area());
        System.out.println("Circle Perimeter: " + circle2.perimeter());
        System.out.println("Rectangle Area: " + rectangle2.area());
        System.out.println("RectanglePerimeter: " + rectangle2.perimeter());
    }
}

```

Example 2 : Real time Application

- JDBC Api is an interface which is used to achieve loose coupling between java application and database server .
- In case of JDBC Api we may have many abstract method and the implementation of these methods are provided by respective DB server with the help of driver class in the form of .jar file.



Class Loading:

- When a Java program runs, the Java Virtual Machine (JVM) loads class files on

demand.

- This means the JVM only loads the classes it needs to execute the current part of the program.
- The class loading process involves finding the class file (usually a .class file), reading its bytecode, and defining the class within the JVM.
- There are different class loaders responsible for finding classes from various locations like the system classpath or network locations.

class loading is the process of bringing the bytecode of a class into memory so it can be used by the JVM. There are different ways to load classes in Java: manually and using the `Class.forName` method.

forName() method :

- `forName` is static method which is present in a class by the name **Class**.
- The `Class.forName()` method is used to load a class dynamically at runtime. This method takes a string parameter representing the fully qualified name of the class. When the class is loaded, any static blocks or static initializations in the class are executed.

Example :

```
class Class{
    public static forName() throws ClassNotFoundException{
        // logic
    }
}
```

- Whenever we use `forName()` it throws an exception called `ClassNotFoundException`
- we can handle this exception by using try and catch block or by using throws keyword

To load the class by using forName()

```
package org.ps.staticApp;
public class staticDemo{
    static{
        System.out.println(" Static block loaded.");
    }
}
package org.ps.staticApp;
public class Test
{
    public static void main(String[] args) throws ClassNotFoundException {
        class.forName("org.ps.staticApp.staticDemo");
    }
}
```



```
    }
    toString()
```

- The `toString()` method in Java is a method that is used to return a string representation of an object. This method is defined in the `Object` class, which is the superclass of all classes in Java. Therefore, every class inherits the `toString()` method.
- By default, the `toString()` method returns a string that consists of the class name followed by the `@` character and the hashCode of the object. However, it is common practice to override this method in your classes to provide a more meaningful string representation of an object.

Default toString() Implementation

```
public class DefaultToStringExample {
    public static void main(String[] args) {
        MyClass myObject = new MyClass();
        System.out.println(myObject.toString()); // Calls the default toString()
method
    }
}

class MyClass {
    // No override of toString()
}
```

Output :

MyClass@15db9742

Overriding the toString() Method

To provide a more meaningful string representation of an object, you can override the `toString()` method in your class.

```
public class OverrideToStringExample {
    public static void main(String[] args) {
        Person person = new Person("John", 25);
        System.out.println(person.toString()); // Calls the overridden toString()
method
        System.out.println(person); // toString() is called implicitly
    }
}

class Person {
```



```

private String name;
private int age;

public Person(String name, int age) {
    this.name = name;
    this.age = age;
}

@Override
public String toString() {
    return "Person[name=" + name + ", age=" + age + "]";
}
}

```

Output:

```

Person[name=John, age=25]
Person[name=John, age=25]

```

- **Inheritance:** Every class in Java inherits the `toString()` method from the `Object` class.
- **Default Behavior:** The default implementation returns the class name and the object's hashcode.
- **Override:** It is common to override the `toString()` method to provide a string representation that includes meaningful information about the object's state.
- **Implicit Call:** When you print an object reference, the `toString()` method is called implicitly.

Best Practices for `toString()`

1. **Include Important Information:** Ensure that the overridden `toString()` method provides all the important information about the object.
2. **Readability:** The string representation should be easily readable and provide a clear understanding of the object.
3. **Consistency:** Maintain a consistent format for `toString()` outputs across your classes for better readability.

Garbage collection in Java

Garbage collection in Java is the process by which the Java Virtual Machine (JVM) automatically reclaims memory by destroying objects that are no longer reachable in the program. The `finalize()` method and `System.gc()` method are two mechanisms associated with garbage collection.

Note:

In Java we can't delete the object but we can make an object eligible for garbage

collection by de-referring an object

finalize() Method

protected void finalise(){}

The finalize() method is called by the garbage collector on an object when garbage collection determines that there are no more references to the object. This method allows the object to clean up resources before it is collected.

finalise() method present in java.lang.object class which is used to clean up the process.

System.gc() Method

The System.gc() method is a request to the JVM to perform garbage collection. However, calling System.gc() does not guarantee that the garbage collector will actually run. It's merely a suggestion to the JVM.

Example :

```

    public class GarbageCollectionExample {
    public static void main(String[] args) {
        MyClass obj1 = new MyClass("Object 1");
        MyClass obj2 = new MyClass("Object 2");

        // Nullify the references to make the objects eligible for garbage collection
        obj1 = null;
        obj2 = null;

        // Requesting JVM to run Garbage Collector
        System.gc();

        // Suggesting the JVM to run Garbage Collector
        Runtime.getRuntime().gc();

        System.out.println("Main method execution completed.");
    }
}

class MyClass {
    private String name;

    public MyClass(String name) {

```

```

        this.name = name;
    }

    @Override
    protected void finalize() throws Throwable {
        // Cleanup code or resource releasing code can be added here
        System.out.println(name + " is being garbage collected.");
    }
}

```

Explanation:

- **Nullify References:**

```
obj1 = null;
```

```
obj2 = null;
```

The references obj1 and obj2 are set to null, making the objects eligible for garbage collection.

- **Request Garbage Collection:**

```
System.gc();
```

This line requests the JVM to run the garbage collector. Note that it is not guaranteed that the garbage collector will run immediately or at all.

- **Runtime.gc():**

```
Runtime.getRuntime().gc();
```

This line is another way to request garbage collection, similar to System.gc().

- **finalize() Method:**

```
@Override
```

```
protected void finalize() throws Throwable {
```

```
    System.out.println(this + " is being garbage collected.");
```

```
}
```

The finalize() method is overridden in the MyClass class to print a message when the object is about to be garbage collected. This method will be called by the garbage collector before reclaiming the memory.

- **Deprecation of finalize():** The finalize() method has been deprecated in Java 9 and is not recommended for resource management. Instead, use try-with-resources or explicit resource management methods.
- **No Guarantee with System.gc():** Calling System.gc() does not guarantee that the garbage collector will run. It merely suggests that the JVM should make an effort to reclaim memory.

Design pattern

It is an optimised solution for commonly recurring design problems or design needs

There are two categories of design patterns

1. creational design pattern
2. factory design pattern

I. Creational design pattern

It includes only the object creation logic creation design pattern is example for Singleton classes.

Singleton classes

In case of a Singleton Class we are restricting the user to create only one object for a particular class we can't create multiple classes for Singleton classes.

Example :

```
public class Singleton {
    // Step 1: Create a private static instance of the class
    private static Singleton instance;

    // Step 2: Make the constructor private so that the class cannot be
    instantiated from outside
    private Singleton() {
        // Private constructor
    }

    // Step 3: Provide a public static method to get the instance of the class
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    public void showMessage() {
        System.out.println("Hello from the Singleton!");
    }
}

public class SingletonDemo {
    public static void main(String[] args) {
        // Get the only object available
        Singleton singleton = Singleton.getInstance();
    }
}
```

```

        // Show the message
        singleton.showMessage();
    }
}

```

Real time example in Java

Runtime class present in java.lang package to execute and allocate the memory jvm creates one runtime object for entire execution.

Example :

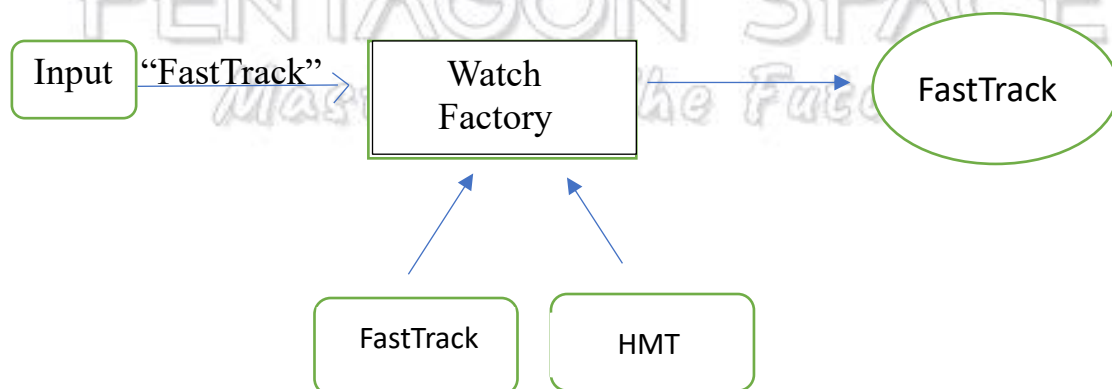
```

Public class test{
    public static Void main(String[] args){
        Runtime rt = Runtime. getRuntime();
        System.out.println(rt);
    }
}

```

II. Factory design pattern :

- Factory is one of the which is used to create and return different object of the same time
- Factory always takes string as a input and return the object based on the objects.



Example :

```

class WatchFactory{
    public static Watch getWatch( String type) {
        if(type.equalsIgnoreCase("FastTrack")) {
            return new FastTrack();
        }
    }
}

```

```

        else if (type.equalsIgnoreCase("HMT")) {
            return new HMT();
        }
        else {
            System.out.println("No such Watch found");
            return null;
        }
    }
}

```

Factory design patterns is always associated with 3 different types of logic They are

- i. Implementation logic
- ii. Object creation logic
- iii. Consumer or utilisation logic

i. Implementation logic

It is a basic fundamental logic which contains only implementation class according to which we can implement object has to be created.

ii. Object creation logic

It is used to create an implementation object according to the implementation logic by using factory or helper class.

iii. Consumer or utilisation logic

It is the most important logic which is used to access the functionalities from the implementation

Note : without writing consumer utilisation logic none of the implementation will work.

Factory or helper method:

It is a method which is used to return and create an implementation of an object

NullPointerException:

Pointing to an object which is not present through an exception called as NullPointerException.

Example :

```

interface Switch{
    void sOn();
    void sOff();
}

```

```

class TubeLight implements Switch{

```

@Override

```

        public void sOn() {
            System.out.println("Tubelight switch on");
        }

        @Override
        public void sOff() {
            System.out.println("Tubelight switch off");
        }
    }
}

```

```

class LEDlight implements Switch{

```

```

    @Override
    public void sOn() {
        System.out.println("LEDlight switch on");
    }

```

```

    @Override
    public void sOff() {
        System.out.println("LEDlight switch off");
    }
}

```

```

// object creational logic

```

```

public class LightFactory{
    //Factory or helper method
    public static Switch getLight(String type) {
        if(type.equalsIgnoreCase("FASTTRACK")) {
            return new TubeLight();
        }
        else if (type.equalsIgnoreCase("HMT")) {
            return new LEDlight();
        }
        else
        {
            System.out.println("No search light found ");
            return null;
        }
    }
}

```



```

    }
}

// Consumer or utilisation logic
public class LightDemo{
    public static void main(String[] args) {
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter the light ");
        String light=sc.next();
        Switch sw=LightFactory.getLight(light);
        if(sw!=null) {
            sw.sOn();
            sw.sOff();
        }
    }
}

```

- Whenever the user provide the implementation then the user has to create the implementation object.
- Also whenever the respective server or vendor is going to provide the implementation then the same vendor has to provide the implementation object.
- In a entire JDBC and advance concept we Need to write consumer or utilisation logic.

Wrapper Classes in Java

Wrapper classes in Java provide a way to use primitive data types (int, boolean, etc.) as objects. Each primitive type has a corresponding wrapper class in the java.lang package:

- byte -> Byte
- short -> Short
- int -> Integer
- long -> Long
- float -> Float
- double -> Double
- boolean -> Boolean
- char -> Character

Why Use Wrapper Classes?

1. **Collections:** Only objects can be added to collections like ArrayList, HashMap, etc.
2. **Utility Methods:** Wrapper classes provide utility methods, like parsing a string to a number.

3. **Null Values:** Objects can be null, whereas primitive types cannot.

Autoboxing and Unboxing

1. **Autoboxing** is the automatic conversion of a primitive type to its corresponding wrapper class

Example of Autoboxing

```
public class AutoboxingExample {
    public static void main(String[] args) {
        // Autoboxing: primitive to wrapper
        int num = 10;
        Integer numObj = num; // Automatically converts int to Integer

        System.out.println("numObj: " + numObj);
    }
}
```

2. **Unboxing** is the automatic conversion of a wrapper class back to its corresponding primitive type.

Example of Unboxing

```
public class UnboxingExample {
    public static void main(String[] args) {
        // Unboxing: wrapper to primitive
        Integer numObj = 20;
        int num = numObj; // Automatically converts Integer to int

        System.out.println("num: " + num);
    }
}
```

Parse Methods

Wrapper classes provide parse methods to convert strings to their respective primitive types.

Example of Parse Methods

```
public class ParseMethodsExample {
    public static void main(String[] args) {
        String intStr = "100";
        int intValue = Integer.parseInt(intStr);
        System.out.println("intValue: " + intValue);

        String doubleStr = "99.99";
        double doubleValue = Double.parseDouble(doubleStr);
    }
}
```

```
System.out.println("doubleValue: " + doubleValue);
```

```
String booleanStr = "true";
```

```
boolean booleanValue = Boolean.parseBoolean(booleanStr);
```

```
System.out.println("booleanValue: " + booleanValue);
```

```
}
```

```
}
```

Note :

- **Wrapper Classes:** Allow use of primitive data types as objects.
- **Autoboxing:** Automatic conversion from primitive type to corresponding wrapper class.
- **Unboxing:** Automatic conversion from wrapper class to corresponding primitive type.
- **Parse Methods:** Convert strings to their respective primitive types.

Examples : Full Example with Autoboxing and Unboxing

```
public class AutoboxingUnboxingExample {
    public static void main(String[] args) {
        // Autoboxing
        char ch = 'a';
        Character chObj = ch; // Autoboxing: char to Character
        System.out.println("chObj: " + chObj);

        // Unboxing
        Character chObj2 = 'b';
        char ch2 = chObj2; // Unboxing: Character to char
        System.out.println("ch2: " + ch2);

        // Autoboxing with Integer
        int num = 50;
        Integer numObj = num; // Autoboxing: int to Integer
        System.out.println("numObj: " + numObj);

        // Unboxing with Integer
        Integer numObj2 = 100;
        int num2 = numObj2; // Unboxing: Integer to int
        System.out.println("num2: " + num2);
    }
}
```

Exception Handling

In Java, Exception is an unwanted or unexpected event, which occurs during the execution of a program, i.e. at run time, that disrupts the normal flow of the program's instructions.

Exceptions can be caught and handled by the program. When an exception occurs within a method, it creates an object. This object is called the exception object.

Exception occurs only during the runtime when it occurs the programme gets terminated immediately And exception never occurs at compile time.

How to handle the exceptions :

- Exception can be handled by using try and catch block
- one or multiple lines of a code which possibly gives an exception must be enclosed in try block followed by immediate catch block.

```
try
{
    // statement(s) that might cause exception
}
```

- Catch block is used to write the exception handling

```
catch
{
    // statement(s) that handle an exception
    // examples, closing a connection, closing
    // file, exiting the process after writing
    // details to a log file.
}
```

- If the exception did not occur in the try block catch block will not be executed
- Catch block gets executed only if the exception happens in the try block and if the exception matches the catch block
- when the exception occurs in the try block then the control immediately comes out of the try block without executing or remaining lines of the code within the try block and executes the matching catch block
- If the matching catch block is not found then the programme gets terminated once the control comes out of the try block it will not go back to the try block again
- there must not be executable code between try and catch block
- handling scenarios must be written in catch block

Example :

```
class ArithmeticException_Demo
{
    public static void main(String args[])
    {
        try {
            int a = 30, b = 0;
            int c = a/b; // cannot divide by zero
            System.out.println ("Result = " + c);
        }
        catch(ArithmeticException e) {
            System.out.println ("Can't divide a number by 0");
        }
    }
}
```

Output

Can't divide a number by 0

Example 2 :

```
class Demo2
{
    public static void main (String[] args)
    {
        // array of size 4.
        int[] arr = new int[4];
        try
        {
            int i = arr[4];

            // this statement will never execute
            // as exception is raised by above statement
            System.out.println("Inside try block");
        }
        catch(ArrayIndexOutOfBoundsException ex)
        {
            System.out.println("Exception caught in Catch block");
        }

        // rest program will be executed
    }
}
```

```

        System.out.println("Outside try-catch clause");
    }
}

```

Output

Exception caught in Catch block
Outside try-catch clause

- Exception means JVM creates object of appropriate exception class and the object is thrown.
- Reference type used in the catch block must belong to the exception object that is it must be directly or indirectly a subclass of a throwable.
- If there is a multiple catch blocks then we have to follow the sequence from subclass to superclass but not superclass to subclass
- Because in case of superclass to subclass whenever we get an exception in the try block all the exception can be handled in the super class and it won't allow for a subclass catch block hence the subclass catch block will become unreachable code which is not supported in Java

Note:

Whenever the handling scenario is same for multiple exception then we can use single catch block with the super class reference.

```

public class UnreachableCatchBlockExample {
    public static void main(String[] args) {
        try {
            // This will cause an ArithmeticException
            int result = 10 / 0;
            System.out.println("Result: " + result);
        } catch (Exception e) {
            System.out.println("Caught Exception");
        } catch (ArithmeticException e) {
            // This block is unreachable because Exception is a superclass of
            // ArithmeticException
            System.out.println("Caught ArithmeticException");
        }
    }
}

```

finally block

- finally block is used to exception handling.

- It is always get executed irrespective of whether the exception occurred or not.
- Single try block can have only one finally block, usually costly resources are enclosed inside the finally block
- we can have try block without the catch block but in this case finally block must be associated with the try block
- Any mandatory code which has to be executed irrespective of exception must be written inside finally block

Example :

```

class Demo
{
    public static void main (String[] args)
    {
        // array of size 4.
        int[] arr = new int[4];
        try
        {
            int i = arr[4];
            // this statement will never execute
            // as exception is raised by above statement
            System.out.println("Inside try block");
        }
        catch(ArrayIndexOutOfBoundsException ex)
        {
            System.out.println("Exception caught in catch block");
        }
        finally
        {
            System.out.println("finally block executed");
        }
        // rest program will be executed
        System.out.println("Outside try-catch-finally clause");
    }
}

```

Re-throwing an exception

When an exception occurs at the time of method calling it is the duty of method to handle the exception and inform the caller about the exception.

To do so , the called method should rethrow the exception object using the throw keyword. The process of handling the exception object and throwing it back to the caller using the throw keyword is referred to as Re – throwing an Exception.

package in.Exceptions;

Example :

```
public class Demo {
    void fun1() {
        System.out.println("fun1 connected with DB");
        try {
            fun2();
        }
        catch (Exception e) {
            System.out.println("Exception handling code");
            throw e;
        }
        finally {
            System.out.println("fun1 closed the connection with DB");
        }
    }

    void fun2() {
        System.out.println("fun2 method connected with DB");
        try {
            int a,b,c;
            a=10;b=0;
            c=a/b;
            System.out.println(c);
        }
        catch (Exception e) {
            System.out.println("Exception handling code");
            throw e;
        }
        finally {
```

```

        System.out.println("closing the connection of fun2");
    }
}
}

package in.Exceptions;

public class FinallyDemo {
    public static void main(String[] args) {
        Demo d1=new Demo();
        System.out.println("main connected to DB");
        try {
            d1.fun1();
        }
        catch (Exception e) {
            System.out.println("Exception handling code");
        }
        finally {
            System.out.println("main method closed connection with DB");
        }
    }
}

```

Methods to print the Exception information:

1. printStackTrace()

This method prints exception information in the format of the Name of the exception: **description of the exception, stack trace.**

Example:

//program to print the exception information using printStackTrace() method

```

class Demo{
    public static void main (String[] args) {
        int a=5;
        int b=0;
        try{

```

```

        System.out.println(a/b);
    }
    catch(ArithmeticException e){
        e.printStackTrace();
    }
}
}

```

Costly Resources

In Java, a costly resource typically refers to any system resource that is limited and expensive to create, maintain, or use. Examples of costly resources include database connections, file handles, network connections, and large memory allocations. Proper management of these resources is crucial to ensure application performance and stability.

Eg : Scanner , file reader , file writer, DataBase Connection etc...

- It is a good practise to close the cost resources otherwise the performance of an application will be reduced.
- In general all the costly resources are closed with the help of finally block if the costly resources are not closed it won't throw any error or exception but the performance will be become slow.

Note :

To handle the exception there are two ways

1. By using try, catch blocks
2. By using throw or throws keyword

throw and throws Keyword

throw : is a keyword which is used to throw an exception object explicitly in the program inside a method or block.

- throw is used to create and return one single exception object.

throws : is a keyword used to indicate the exception and it is used along with the method declaration.

- throws can be used to indicate multiple exception at a time.

Example :

```
public class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }

    // Method that may throw an exception
    public void withdraw(double amount) throws InsufficientFundsException
    {
        if (amount > balance) {
            // Use throw to actually throw the exception
            throw new InsufficientFundsException("Insufficient funds: Current
                balance is " + balance);
        }
        balance -= amount;
        System.out.println("Withdrawal successful. Remaining balance: " +
            balance);
    }
}

public class BankApplication {
    public static void main(String[] args) {
        BankAccount account = new BankAccount(100.0);

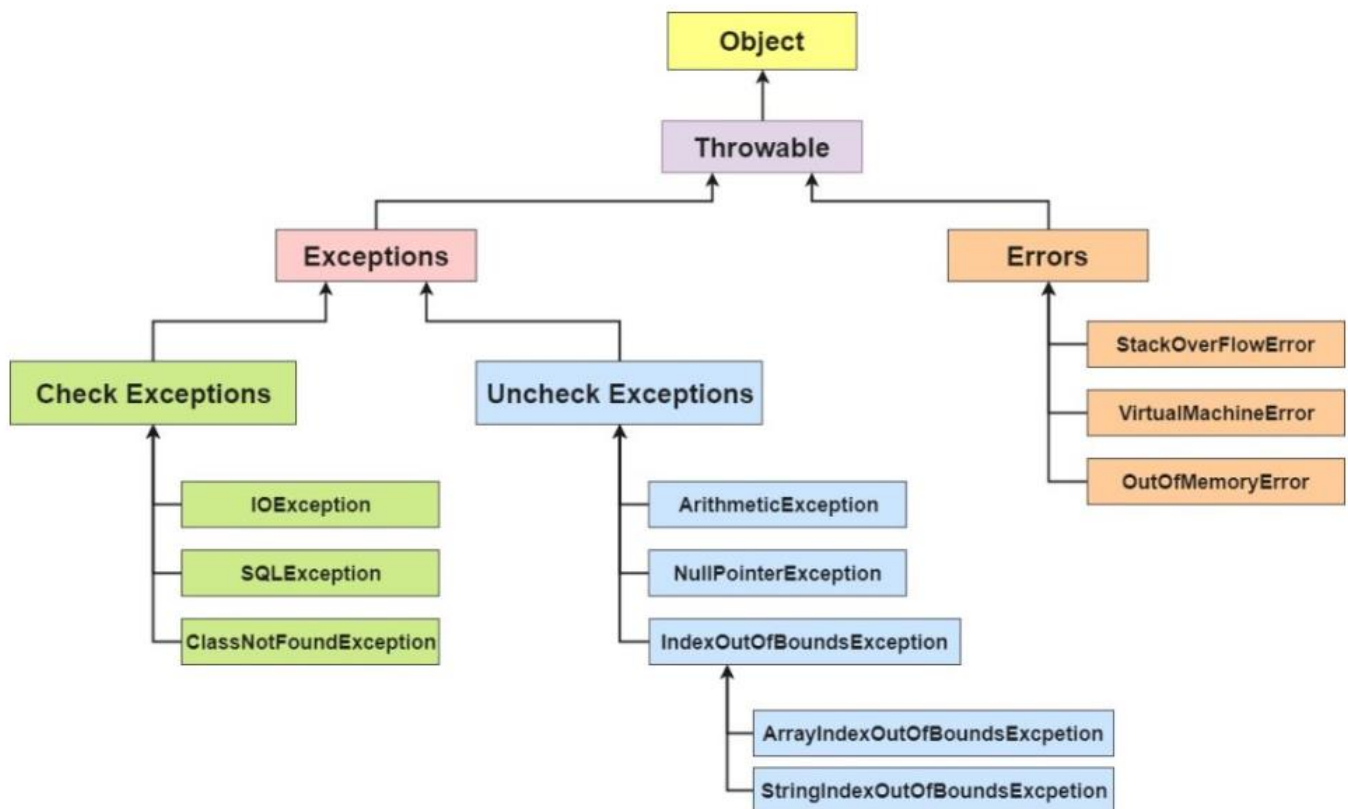
        try {
            account.withdraw(150.0); // Attempting to withdraw more than
                                    // available balance
        } catch (InsufficientFundsException e) {
            // Handling the exception
            System.out.println("Caught exception: " + e.getMessage());
        }
    }
}
```

}

Exception Hierarchy

All exception and error types are subclasses of the class **Throwable**, which is the base class of the hierarchy. One branch is headed by **Exception**. This class is used for exceptional conditions that user programs should catch. `NullPointerException` is an example of such an exception. Another branch, **Error** is used by the Java run-time system([JVM](#)) to indicate errors having to do with the run-time environment itself(JRE). `StackOverflowError` is an example of such an error.

Hierarchy of Java Exception classes



Exceptions can be categorized in two ways:

1. Built - in Exceptions
 - Checked Exception
 - Unchecked Exception
2. User-Defined Exceptions

1. Built-in Exceptions

Built-in exceptions are the exceptions that are available in Java libraries. These

exceptions are suitable to explain certain error situations.

- **Checked Exceptions:** Checked exceptions are called compile-time exceptions because these exceptions are checked at compile-time by the compiler.
- **Unchecked Exceptions:** The unchecked exceptions are just opposite to the checked exceptions. The compiler will not check these exceptions at compile time. In simple words, if a program throws an unchecked exception, and even if we didn't handle or declare it, the program would not give a compilation error.

2. User-Defined Exceptions:

Sometimes, the built-in exceptions in Java are not able to describe a certain situation. In such cases, users can also create exceptions, which are called 'user-defined Exceptions'.

The *advantages of Exception Handling in Java* are as follows:

1. Provision to Complete Program Execution
2. Easy Identification of Program Code and Error-Handling Code
3. Propagation of Errors
4. Meaningful Error Reporting
5. Identifying Error Types

Custom Exception or User defined Exception :

Scenario to construct user defined Exception

Imagine a system for managing student grades where we need to validate that a grade is within a valid range (0 to 100). If a grade is outside this range, we should throw a custom exception.

// Student.java

```
public class Student {
    private String name;
    private int grade;
```

```
    public Student(String name) {
        this.name = name;
    }
}
```

// Method to set the grade that might throw InvalidGradeException

```
public void setGrade(int grade) throws InvalidGradeException {
    if (grade < 0 || grade > 100) {
        // Throwing the custom exception
    }
}
```

```

        throw new InvalidGradeException("Grade must be between 0 and 100.
        Provided grade: " + grade);
    }
    this.grade = grade;
    System.out.println(name + "'s grade set to: " + grade);
}
}

```

// Main.java

```

public class Main {
    public static void main(String[] args) {
        Student student = new Student("John");

        try {
            student.setGrade(105); // This will throw InvalidGradeException
        } catch (InvalidGradeException e) {
            // Handling the custom exception
            System.out.println("Caught exception: " + e.getMessage());
        }

        try {
            student.setGrade(85); // Valid grade
        } catch (InvalidGradeException e) {
            // This block will not be executed
            System.out.println("Caught exception: " + e.getMessage());
        }
    }
}

```

// InvalidGradeException.java

```

public class InvalidGradeException extends Exception {
    public InvalidGradeException(String message) {
        super(message);
    }
}

```

Explanation

1. Custom Exception (InvalidGradeException):

- This class extends Exception and provides a constructor to pass an error message.

2. Student Class:

- The setGrade method declares that it throws InvalidGradeException, indicating that it may throw this custom exception.

- Inside the method, the throw keyword is used to actually throw InvalidGradeException if the grade is not within the valid range (0 to 100).

3. Main Class:

- The setGrade method is called with an invalid grade (105), causing the InvalidGradeException to be thrown.
- The exception is caught and handled in the catch block, printing the error message.
- The second call to setGrade uses a valid grade (85), so no exception is thrown, and the grade is set successfully.

HAS-A relation / Association in java

Association is a relation between two separate classes which is established through their Objects. Association can be one-to-one, one-to-many, many-to-one, many-to-many. In Object-Oriented programming, an Object communicates to another object to use functionality and services provided by that object. **Composition** and **Aggregation** are the two forms of association.

1. Aggregation

It is a special form of Association where:

- It represents Has-A's relationship.
- It is a **unidirectional association** i.e. a one-way relationship. For example, a department can have students but vice versa is not possible and thus unidirectional in nature.
- In Aggregation, **both entries can survive individually** which means ending one entity will not affect the other entity.

Example :

```
class Address {
    private String city;
    private String state;

    public Address(String city, String state) {
        this.city = city;
        this.state = state;
    }

    public String getCity() {
        return city;
    }
}
```

```

    }

    public String getState() {
        return state;
    }
}

class Student {
    private String name;
    private Address address;

    public Student(String name, Address address) {
        this.name = name;
        this.address = address;
    }

    public String getName() {
        return name;
    }

    public Address getAddress() {
        return address;
    }
}

public class Main {
    public static void main(String[] args) {
        Address address = new Address("New York", "NY");
        Student student = new Student("John Doe", address);

        System.out.println(student.getName() + " lives in " +
student.getAddress().getCity() + ", " + student.getAddress().getState());
    }
}

```

2. Composition

Composition is a stronger form of aggregation. It is also a "has-a" relationship, but with a key difference: the lifecycle of the contained object (part) is dependent on the lifecycle of the container object (whole). If the whole object is destroyed, the part is also destroyed.

Example :

```
class Engine {
    private String type;

    public Engine(String type) {
        this.type = type;
    }

    public String getType() {
        return type;
    }
}

class Car {
    private String model;
    private Engine engine;

    public Car(String model, String engineType) {
        this.model = model;
        this.engine = new Engine(engineType);
    }

    public String getModel() {
        return model;
    }

    public Engine getEngine() {
        return engine;
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car("Tesla", "Electric");

        System.out.println(car.getModel() + " has a " +
            car.getEngine().getType() + " engine.");
    }
}
```

Streams and File I/O

In Java, streams act as conduits for handling data flow between the program and external sources like files, devices, and network connections. File I/O (Input/Output) operations specifically deal with reading from and writing to files using these streams.

Byte Streams

- Designed to work with raw, 8-bit binary data.
- Read and write data one byte at a time.

Common Byte Stream Classes:

- **FileInputStream:** Opens an existing file for reading bytes.
- **FileOutputStream:** Creates a new file or overwrites an existing one for writing bytes.
- **BufferedInputStream:** Provides buffering for reading bytes, improving performance.
- **BufferedOutputStream:** Buffers data written as bytes, enhancing efficiency.
- **DataInputStream:** Reads primitive data types (int, float, etc.) from a byte stream.
- **DataOutputStream:** Writes primitive data types to a byte stream.

1. FileInputStream:

- Opens an existing file for reading bytes.
- Useful for reading binary data (images, audio, compressed files).
- Reads data one byte at a time.
- Remember to close the stream after use to release resources.

Example :

```
import java.io.FileInputStream;
```

```
public class ReadTextFile {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            FileInputStream fis = new FileInputStream("mytextfile.txt");
```

```
            int c;
```

```
            while ((c = fis.read()) != -1) {
```

```
                System.out.print((char) c);
```

```
            }
```

```
            fis.close();
```

```
        } catch (Exception e) {
```

```
            System.out.println("Error reading file: " + e.getMessage());
```

```
        }
```

```
    }
```

```
}
```

2. **FileOutputStream:**

- Creates a new file or overwrites an existing one for writing bytes.
- Suitable for writing binary data.
- Data needs to be converted to a byte array before writing.
- Close the stream after use to ensure data is flushed to the file.

Example :

```
import java.io.FileOutputStream;

public class WriteTextFile {

    public static void main(String[] args) {
        try {
            FileOutputStream fos = new FileOutputStream("output.txt");
            String text = "This is some text written to a file.";
            fos.write(text.getBytes()); // Convert String to byte array
            fos.close();
            System.out.println("Text written to output.txt");
        } catch (Exception e) {
            System.out.println("Error writing file: " + e.getMessage());
        }
    }
}
```

3. **BufferedInputStream:**

- Wraps an existing InputStream to provide buffering for reading bytes.
- Improves performance by reducing the number of disk I/O operations.
- Use when dealing with large files or frequent reads.
- Close both the BufferedInputStream and the underlying InputStream.

Example :

```
import java.io.BufferedInputStream;
import java.io.FileInputStream;

public class ReadImageFile {

    public static void main(String[] args) {
        try {
            FileInputStream fis = new FileInputStream("image.jpg");
            BufferedInputStream bis = new BufferedInputStream(fis);
            int b;
            while ((b = bis.read()) != -1) {
```

```

        // Process each byte (e.g., write to another file)
    }
    bis.close();
    fis.close();
} catch (Exception e) {
    System.out.println("Error reading file: " + e.getMessage());
}
}
}

```

4. **BufferedOutputStream:**

- Wraps an existing OutputStream to provide buffering for writing bytes.
- Enhances efficiency by writing data in chunks instead of single bytes.
- Consider using for large files or frequent writes.
- Close both the BufferedOutputStream and the underlying OutputStream.

Example :

```

import java.io.BufferedOutputStream;
import java.io.FileOutputStream;

public class WriteBinaryFile {

    public static void main(String[] args) {
        try {
            byte[] data = { 0xDE, 0xAD, 0xBE, 0xEF }; // Sample byte array
            FileOutputStream fos = new FileOutputStream("newfile.bin");
            BufferedOutputStream bos = new BufferedOutputStream(fos);
            bos.write(data);
            bos.close();
            fos.close();
            System.out.println("Binary data written to newfile.bin");
        } catch (Exception e) {
            System.out.println("Error writing file: " + e.getMessage());
        }
    }
}

```

5. **DataInputStream:**

- Reads primitive data types (int, float, double, etc.) from a byte stream in a platform-independent way.
- Assumes the data was written using DataOutputStream.
- Useful for reading serialized data or binary files containing primitive types.

- Close the stream after use.

Example :

```
import java.io.DataInputStream;
import java.io.FileInputStream;

public class ReadPrimitiveData {

    public static void main(String[] args) {
        try {
            FileInputStream fis = new FileInputStream("data.bin");
            DataInputStream dis = new DataInputStream(fis);
            int age = dis.readInt();
            float salary = dis.readFloat();
            System.out.println("Age: " + age);
            System.out.println("Salary: " + salary);
            dis.close();
            fis.close();
        } catch (Exception e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
    }
}
```

6. DataOutputStream:

- Writes primitive data types (int, float, double, etc.) to a byte stream in a platform-independent way.
- Ensures consistent representation across different systems.
- Often used for creating serialized data or binary files with primitive data.
- Close the stream after use.

Example :

```
import java.io.DataOutputStream;
import java.io.FileOutputStream;

public class WritePrimitiveData {

    public static void main(String[] args) {
        try {
            FileOutputStream fos = new FileOutputStream("data.bin");
            DataOutputStream dos = new DataOutputStream(fos);
            int age = 30;
            float salary = 12345.67f;
```



```
dos.writeInt(age);
dos.writeFloat(salary);
dos.close();
```

Use Cases:

- Reading/writing images, audio, compressed files, or any non-textual data.
- Network communication using sockets or byte arrays.

Advantages:

- Efficient for handling binary data.

Disadvantages:

- Not ideal for text files due to character encoding considerations.
- Can be cumbersome to read/write primitive data types one byte at a time (use `DataInputStream/DataOutputStream`).

Character Streams

- Operate on Unicode characters, making them suitable for text files.
- Internally handle character encoding (e.g., UTF-8) to represent text correctly.

Common Character Stream Classes:

- **FileReader:** Reads characters from a text file.
- **FileWriter:** Writes characters to a text file.
- **BufferedReader:** Provides buffering for reading characters, optimizing performance.
- **BufferedWriter:** Buffers data written as characters, improving efficiency.

1. FileReader:

- **Reads characters from a text file.**
- Handles character encoding (usually UTF-8) transparently.
- More efficient for text files compared to `FileInputStream`.
- Reads data one character at a time.

Example:

```
import java.io.FileReader;

public class ReadTextFile {

    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("mytextfile.txt");
            int c;
```

```

        while ((c = fr.read()) != -1) {
            System.out.print((char) c);
        }
        fr.close();
    } catch (Exception e) {
        System.out.println("Error reading file: " + e.getMessage());
    }
}
}

```

2. FileWriter:

- **Writes characters to a text file.**
- Handles character encoding (usually UTF-8) transparently.
- More efficient for text files compared to FileOutputStream.
- Writes data one character at a time.

Example:

```

import java.io.FileWriter;

public class WriteTextFile {

    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("output.txt");
            String text = "This is some text written to a file.";
            fw.write(text);
            fw.close();
            System.out.println("Text written to output.txt");
        } catch (Exception e) {
            System.out.println("Error writing file: " + e.getMessage());
        }
    }
}

```

3. BufferedReader:

- **Wraps a FileReader to provide buffering for reading characters.**
- Improves performance by reducing the number of disk I/O operations.
- Useful for reading large text files or frequent reads.
- Reads data line by line or character by character.

Example:

```

import java.io.FileReader;
import java.io.BufferedReader;

public class ReadTextFileBuffered {

    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("mytextfile.txt");
            BufferedReader br = new BufferedReader(fr);
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
            br.close();
            fr.close();
        } catch (Exception e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
    }
}

```

4. BufferedWriter:

- **Wraps a FileWriter to provide buffering for writing characters.**
- Enhances efficiency by writing data in chunks instead of single characters.
- Consider using for large text files or frequent writes.

Example:

```

import java.io.FileWriter;
import java.io.BufferedWriter;

public class WriteTextFileBuffered {

    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("output.txt");
            BufferedWriter bw = new BufferedWriter(fw);
            String text = "This is some text written to a file with buffering.";
            bw.write(text);
            bw.newLine(); // Add a newline character
            bw.close();
            fw.close();
        }
    }
}

```

```

        System.out.println("Text written to output.txt with buffering");
    } catch (Exception e) {
        System.out.println("Error writing file: " + e.getMessage());
    }
}
}
}

```

Remember:

- Close all streams (FileReader, FileWriter, BufferedReader, BufferedWriter) after use to release resources.
- BufferedReader and BufferedWriter are generally preferred over their non-buffered counterparts for performance reasons when dealing with large text files.

Use Cases:

- Reading/writing text files (plain text, code, documents).
- User input/output in console applications.

Advantages:

- Streamlined handling of text data.
- Automatic character encoding management.

Disadvantages:

- Less efficient for binary data compared to byte streams (consider converting data if necessary).

Serialization in Java

Serialization is a mechanism in Java for converting the state of an object (its data members) into a byte stream. This byte stream can then be stored in a file, transmitted over a network, or used for any other purpose where you need to persist or transfer the object's data. The reverse process, deserialization, takes a byte stream and recreates the original object from it.

- **Serializable Interface:** To make an object serializable, it needs to implement the `java.io.Serializable` interface. This interface is a marker interface, meaning it doesn't have any methods to implement. It simply indicates that the class supports serialization.
- **ObjectOutputStream and ObjectInputStream:**
 - **ObjectOutputStream:** Used to write an object to a byte stream. It takes the object as input and converts it into a sequence of bytes.
 - **ObjectInputStream:** Used to read an object from a byte stream. It takes the byte stream as input and reconstructs the original object from it.
- **Transient Fields:** You can use the `transient` keyword to mark specific fields in a class as non-serializable. These fields will not be included in the byte stream during serialization.

Use Cases:

- **Persisting Objects:** You can serialize objects to a file and then later deserialize them

to recreate the original objects. This is useful for saving application data or user preferences.

- **Remote Method Invocation (RMI):** RMI allows you to call methods on objects located on different machines. Serialization is used to send the method arguments and return values between the client and server machines.
- **Messaging Systems:** Messages in messaging systems often contain objects. Serialization is used to convert these objects into a format that can be sent over the network.

Advantages:

- Provides a convenient way to store and transfer the state of objects.
- Platform-independent: Objects serialized on one platform can be deserialized on another platform.

Disadvantages:

- Performance overhead: Serialization and deserialization can be slow for large objects.
- Security concerns: Deserialization can be vulnerable to security attacks if you're not careful about the source of the byte stream.

When to Use Serialization:

- When you need to store the state of an object for later use.
- When you need to transfer objects between different parts of your application or across a network.

Alternatives to Serialization:

- **JSON:** JSON (JavaScript Object Notation) is a popular format for data exchange. It is human-readable and can be easily parsed by many programming languages.
- **XML:** XML (Extensible Markup Language) is another format for data exchange. It is more complex than JSON but can be more expressive.

Mastering The Future

Multi-Tasking

Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Multitasking can be achieved in two ways:

- Process-based Multitasking (Multiprocessing)
- Thread-based Multitasking (Multithreading)

1) Process-based Multitasking (Multiprocessing)

- Each process has an address in memory. In other words, each process allocates a separate memory area.
- A process is heavyweight.
- Cost of communication between the process is high.
- Switching from one process to another requires some time for saving and loading [registers](#), memory maps, updating lists, etc.

2) Thread-based Multitasking (Multithreading) / Multithreading in Java

- A thread is lightweight.
- Cost of communication between the thread is low.
- Each Threads are Independent i.e, if on thread got destroyed it will not affect other threads.
- Each threads are having different path of execution.

Multithreading in Java is a process of executing multiple threads simultaneously.

- A thread is a lightweight sub-process, the smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.
- Java Multithreading is mostly used in games, animation, etc.

Advantages of Java Multithreading

- 1) It doesn't block the user because threads are independent and you can perform multiple operations at the same time.
- 2) You can perform many operations together, so it saves time.
- 3) Threads are independent, so it doesn't affect other threads if an exception occurs in a single thread.

Multithreading is a Java feature that allows concurrent execution of two or more parts of a program for maximum utilization of CPU. Each part of such program is called a thread. So, threads are light-weight processes within a process.

Threads can be created by using two mechanisms :

1. Extending the Thread class
2. Implementing the Runnable Interface

1. Thread creation by extending the Thread class

We create a class that extends the **java.lang.Thread** class. This class overrides the `run()` method available in the Thread class. A thread begins its life inside `run()` method. We create an object of our new class and call `start()` method to start the execution of a thread. `Start()` invokes the `run()` method on the Thread object.

```
public class Demo2 extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println("child thread");
        }
    }

    public static void main(String[] args) {
        Demo2 d=new Demo2();
        d.start();
        for (int i = 0; i < 5; i++) {
            System.out.println("main thread");
        }
    }
}
```

Thread Class vs Runnable Interface

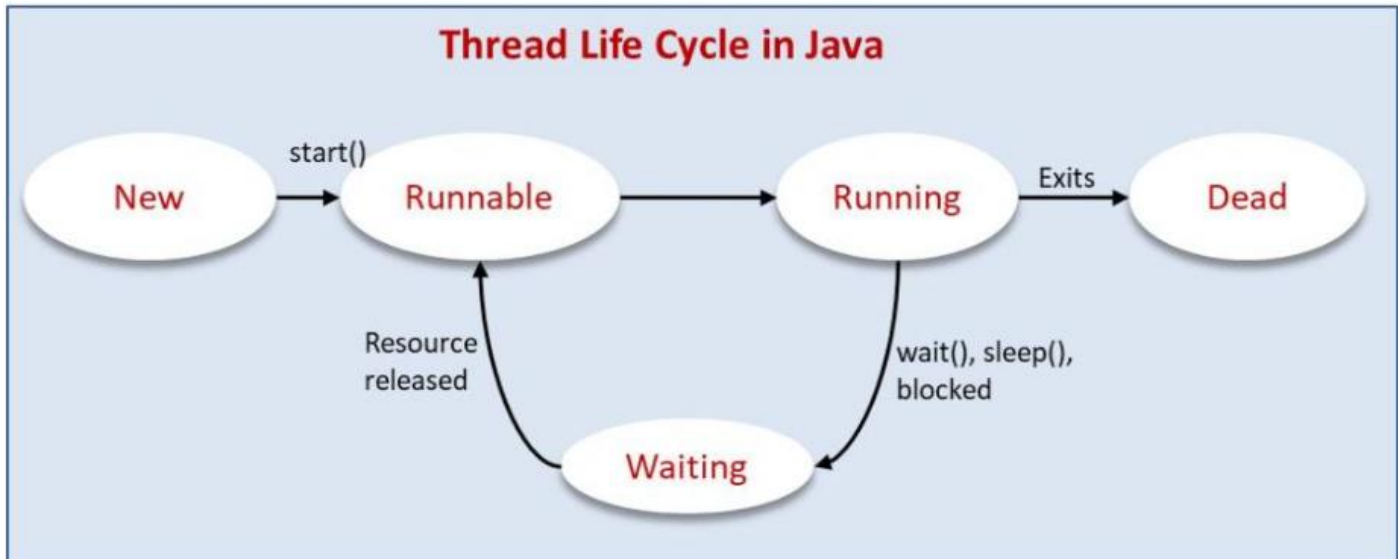
1. If we extend the Thread class, our class cannot extend any other class because Java doesn't support multiple inheritance. But, if we implement the Runnable interface, our class can still extend other base classes.
2. We can achieve basic functionality of a thread by extending Thread class because it provides some inbuilt methods like `yield()`, `interrupt()` etc. that are not available in Runnable interface.
3. Using runnable will give you an object that can be shared amongst multiple threads.

Thread Life Cycle

A thread in Java at any point of time exists in any one of the following states. A thread lies only in one of the shown states at any instant:

1. New State
2. Runnable State

3. Waiting State
4. Running
5. Dead



A thread in java goes through the following life cycle:

New: This is the state of the thread when it is first created. At this state, no program is using it. A thread enters the New state as soon as an instance of the thread is created

Runnable: A thread enters the runnable state as soon as the `start()` method of the thread is called. This is equivalent to the ready state. It is ready to run but the scheduler has not selected it to run

Running: In the running state, the thread has been selected to run and is actually executing a task.

Waiting(Blocked): In this state, the thread cannot run due to some related factor. For example, it's waiting for a resource used by another program to be released.

Terminated(Dead): After completing the task, the thread ends its execution and exits. This is the dead or terminated state.

THREAD SCHEDULER

As we already know java being completely object-oriented works within a multithreading environment in which thread scheduler assigns the processor to a thread based on the priority of thread. Whenever we create a thread in Java, it always has some priority assigned to it. Priority can either be given by JVM while creating the thread or it can be given by the programmer explicitly.

Priorities in threads is a concept where each thread is having a priority which in layman's language one can say every object is having priority here which is represented by numbers

ranging from 1 to 10.

The default priority is set to 5 as excepted.

Minimum priority is set to 1.

Maximum priority is set to 10.

- We will use `currentThread().getName()` method to get the name of the current thread. User can also use `setName()` method to set a name for a thread..

how to get and set priority of a thread in java.

1. **public final int getPriority():** `java.lang.Thread.getPriority()` method returns priority of given thread.
2. **public final void setPriority(int newPriority):** `java.lang.Thread.setPriority()` method changes the priority of thread to the value `newPriority`. This method throws `IllegalArgumentException` if value of parameter `newPriority` goes beyond minimum(1) and maximum(10) limit.

Example :

```
import java.lang.*;

// Main class
class ThreadDemo extends Thread {

    // Method 1
    // run() method for the thread that is called
    // as soon as start() is invoked for thread in main()
    public void run()
    {
        // Print statement
        System.out.println("Inside run method");
    }

    // Main driver method
    public static void main(String[] args)
    {
        // Creating random threads
        // with the help of above class
        ThreadDemo t1 = new ThreadDemo();
        ThreadDemo t2 = new ThreadDemo();
        ThreadDemo t3 = new ThreadDemo();
    }
}
```

```
// Thread 1
// Display the priority of above thread
// using getPriority() method
System.out.println("t1 thread priority : "
                    + t1.getPriority());
```

```
// Thread 1
// Display the priority of above thread
System.out.println("t2 thread priority : "
                    + t2.getPriority());
```

```
// Thread 3
System.out.println("t3 thread priority : "
                    + t3.getPriority());
```

```
// Setting priorities of above threads by
// passing integer arguments
t1.setPriority(2);
t2.setPriority(5);
t3.setPriority(8);
```

```
// t3.setPriority(21); will throw
// IllegalArgumentException
```

```
// 2
System.out.println("t1 thread priority : "
                    + t1.getPriority());
```

```
// 5
System.out.println("t2 thread priority : "
                    + t2.getPriority());
```

```
// 8
System.out.println("t3 thread priority : "
                    + t3.getPriority());
```

```
// Main thread
```

```
// Displays the name of
// currently executing Thread
```

```

System.out.println(
    "Currently Executing Thread : "
    + Thread.currentThread().getName());

System.out.println(
    "Main thread priority : "
    + Thread.currentThread().getPriority());

// Main thread priority is set to 10
Thread.currentThread().setPriority(10);

System.out.println(
    "Main thread priority : "
    + Thread.currentThread().getPriority());
}
}

```

Output

```

t1 thread priority : 5
t2 thread priority : 5
t3 thread priority : 5
t1 thread priority : 2
t2 thread priority : 5
t3 thread priority : 8
Currently Executing Thread : main
Main thread priority : 5
Main thread priority : 10

```

what if we do assign the same priorities to threads then what will happen. All the processing in order to look after threads is carried with help of the thread scheduler.

```

import java.lang.*;
// Main class
// ThreadDemo
// Extending Thread class
class Demo extends Thread {
    public void run()
    {
        System.out.println("Inside run method");
    }
    public static void main(String[] args)

```

```

    {
        // main thread priority is set to 6 now
        Thread.currentThread().setPriority(6);
        System.out.println("main thread priority : "
            + Thread.currentThread().getPriority());

        Demo t1 = new Demo();

        // Print and display priority of current thread
        System.out.println("t1 thread priority : "+ t1.getPriority());
    }
}

```

Output :

```

main thread priority : 6
t1 thread priority : 6

```

- If two threads have the same priority then we can't expect which thread will execute first. It depends on the thread scheduler's algorithm(Round-Robin, First Come First Serve, etc)
- If we are using thread priority for thread scheduling then we should always keep in mind that the underlying platform should provide support for scheduling based on thread priority.

What does start() function do in multithreading in Java?

We have discussed that Java threads are typically created using one of the two methods :
(1) Extending thread class. (2) Implementing Runnable

In both the approaches, we override the run() function, but we start a thread by calling the start() function. So why don't we directly call the overridden run() function? Why always the start function is called to execute a thread?

What happens when a function is called?

When a function is called the following operations take place:

1. The arguments are evaluated.
2. A new stack frame is pushed into the call stack.
3. Parameters are initialized.
4. Method body is executed.
5. Value is returned and current stack frame is popped from the call stack.

The purpose of start() is to create a separate call stack for the thread. A separate call stack is created by it, and then run() is called by JVM.

Example :

```
class ThreadTest extends Thread
{
    public void run()
    {
        try
        {
            // Displaying the thread that is running
            System.out.println ("Thread " +
                Thread.currentThread().getId() +
                " is running");

        }
        catch (Exception e)
        {
            // Throwing an exception
            System.out.println ("Exception is caught");
        }
    }
}
```

```
public class Main
{
    public static void main(String[] args)
    {
        int n = 8;
        for (int i=0; i<n; i++)
        {
            ThreadTest object = new ThreadTest();

            // start() is replaced with run() for
            // seeing the purpose of start
            object.run();
        }
    }
}
```

Output:

```
Thread 1 is running
Thread 1 is running
```

Thread 1 is running
 Thread 1 is running
 Thread 1 is running
 Thread 1 is running
 Thread 1 is running
 Thread 1 is running

Difference between Thread.start() and Thread.run() in Java

start()	run()
Creates a new thread and the run() method is executed on the newly created thread.	No new thread is created and the run() method is executed on the calling thread itself.
Can't be invoked more than one time otherwise throws <i>java.lang.IllegalStateException</i>	Multiple invocation is possible
Defined in <i>java.lang.Thread</i> class.	Defined in <i>java.lang.Runnable</i> interface and must be overridden in the implementing class.

Methods to prevent the execution of thread

- yield()
- join()
- sleep()

1. yield ()

The yield() basically means that the thread is not doing anything particularly important and if any other threads or processes need to be run, they should run. Otherwise, the current thread will continue to run.

- Whenever a thread calls **java.lang.Thread.yield** method gives hint to the thread scheduler that it is ready to pause its execution. The thread scheduler is free to ignore this hint.
- If any thread executes the yield method, the thread scheduler checks if there is any thread with the same or high priority as this thread. If the processor finds any thread with higher or same priority then it will move the current thread to Ready/Runnable state and give the processor to another thread and if not – the current thread will keep executing.

- Once a thread has executed the yield method and there are many threads with the same priority is waiting for the processor, then we can't specify which thread will get the execution chance first.
- The thread which executes the yield method will enter in the Runnable state from Running state.
- Once a thread pauses its execution, we can't specify when it will get a chance again it depends on the thread scheduler.
- The underlying platform must provide support for preemptive scheduling if we are using the yield method.

```
public class Demo2 extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println("child thread");
            Thread.yield();
        }
    }
    public static void main(String[] args) {
        Demo2 d=new Demo2();
        d.setPriority(10);
        d.start();
        for (int i = 0; i < 5; i++) {
            System.out.println("main thread");
            Thread.yield();
        }
    }
}
```

2. sleep() Method

This method causes the currently executing thread to sleep for the specified number of milliseconds.

- Based on the requirement we can make a thread to be in a sleeping state for a specified period of time
- Sleep() causes the thread to definitely stop executing for a given amount of time; if no other thread or process needs to be run, the CPU will be idle (and probably enter a power-saving mode).

Syntax:

```
// sleep for the specified number of milliseconds
public static void sleep(long millis) throws InterruptedException

//sleep for the specified number of milliseconds plus nano seconds
public static void sleep(long millis, int nanos) throws InterruptedException

public static void sleep(Duration duration) throws InterruptedException
```

Example

```
public class Demo2 extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println("child thread");
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    public static void main(String[] args) {
        Demo2 d=new Demo2();
        d.start();
        for (int i = 0; i < 5; i++) {
            System.out.println("main thread");
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

3. join() Method

The join() method of a Thread instance is used to join the start of a thread's execution to the end of another thread's execution such that a thread does not start running until another thread ends. If join() is called on a Thread instance, the currently running thread

will block until the Thread instance has finished executing. The join() method waits at most this many milliseconds for this thread to die. A timeout of 0 means to wait forever

Syntax:

// waits for this thread to die.

public final void join() throws InterruptedException

// waits at most this much milliseconds for this thread to die

public final void join(long millis) throws InterruptedException

// waits at most milliseconds plus nanoseconds for this thread to die.

public final void join (long millis , int nanos) throws InterruptedException

Example

```
public class Demo extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println("Child thread");
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    public static void main(String[] args) {
        Demo d = new Demo();
        d.start();
        try {
            d.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        for (int i = 0; i < 5; i++) {
            System.out.println("main thread");
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Deadlock in Java

Deadlock in Java is a part of multithreading. Deadlock can occur in a situation when a thread is waiting for an object lock, that is acquired by another thread and second thread is waiting for an object lock that is acquired by first thread. Since, both threads are waiting for each other to release the lock, the condition is called deadlock.

Example :

```
class MyThread extends Thread{

    @Override
    public void run(){
        for(int i=0;i<10;i++){
            try{
                currentThread().join();
            }
            catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.println("child thread running");
        }
    }

    public static void main(String[] args)throws InterruptedException
    {
        MyThread t1=new MyThread();
        t1.start();
        t1.join();
        for(int i=0;i<10;i++) {
            System.out.println("main thread running");
            Thread.sleep(1000);
        }
    }
}
```

Output :

No output .

Comparison of yield(), join(), sleep() Methods

Property	yield()	join()	sleep()
Purpose	If a thread wants to pass its execution to give chance to remaining threads of the same priority then we should go for yield()	If a thread wants to wait until completing of some other thread then we should go for join()	If a thread does not want to perform any operation for a particular amount of time, then it goes for sleep()
Is it overloaded?	NO	YES	YES
Is it final?	NO	YES	NO
Is it throws?	NO	YES	YES
Is it native?	YES	NO	sleep(long ms)->native & sleep (long ms, int ns)-> non native
Is it static?	YES	NO	YES

Synchronization

Java Synchronization is used to make sure by some synchronization method that only one thread can access the resource at a given point in time.

- Java provides a way of creating threads and synchronizing their tasks using synchronized blocks.
- A synchronized block in Java is synchronized on some object. All synchronized blocks synchronize on the same object and can only have one thread executed inside them at a time. All other threads attempting to enter the synchronized block are blocked until the thread inside the synchronized block exits the block.
- This synchronization is implemented in Java with a concept called monitors or locks. Only one thread can own a monitor at a given time. When a thread acquires a lock, it is said to have entered the monitor. All other threads attempting to enter the locked monitor will be suspended until the first thread exits the monitor.

Advantages of synchronization

It is used to resolve data inconsistency problem

Drawback :

- It increases the waiting time of a thread
- It decreases the performance of an application

Note :

- Synchronisation internally uses say lock concept
- In Java Each and every object contains a unique lock
- Each and every object comes with only one lock whenever we are executing a synchronised method or a block on a particular object then the lock of object is required.
- Whenever we are executing non synchronised methods or block then the lock is not required.
- If the thread is trying to execute synchronised method or a block then first it occupies the lock of that object once the execution completes then automatically it releases the lock.
- Acquiring and release in the lock is taken care by JVM.
- While a thread executing synchronised method on a given object then the remaining thread are not allowed to execute any synchronised method or a block simultaneously on the same object, but remaining threads can execute non synchronised methods or a block simultaneously.

Example :

```
class Table{
    synchronized void printTable(int n){//synchronized method
        for(int i=1;i<=5;i++){
            System.out.println(n*i);
            try{
                Thread.sleep(400);
            }catch(Exception e){System.out.println(e);}
        }
    }
}
```

```
class MyThread1 extends Thread{
    Table t;
    MyThread1(Table t){
        this.t=t;
    }
    public void run(){
        t.printTable(5);
    }
}
```

```
class MyThread2 extends Thread{
```

```

    Table t;
    MyThread2(Table t){
        this.t=t;
    }
    public void run(){
        t.printTable(10);
    }
}

public class TestSynchronization2 {
    public static void main(String args[]){
        Table obj = new Table();//only one object
        MyThread1 t1=new MyThread1(obj);
        MyThread2 t2=new MyThread2(obj);
        t1.start();
        t2.start();
    }
}

```

Starvation :

A long waiting of a thread where the waiting ends at a certain point is called starvation.

Race condition :

Multiple threads trying to access on the same object which leads to the data inconsistency is called race condition.

Inter thread communication

- Two threads communicate each other by using wait(), notify(), notifyAll() is called interthread communication.
- The thread which requires updation then it has to call wait().
- The thread which is performing updation of object it is responsible to give notification by calling notify().
- After getting the notification the waiting thread will get those updation and continues its execution.
- Wait(), notify(), notify() all methods are present in object class but not in thread class because the thread can call these methods on any common object.
 - public final native void notify();
 - public final native void notifyAll();
 - public final void wait() throws InterruptedException
 - public final void wait(long milliseconds) throws InterruptedException
 - public final void wait (long timeOut, int nanoSec) throws InterruptedException
- To call wait(), notify(), and notifyAll() method compulsory the current thread must be

owner of the object that is current thread should have the lock of that object

- Current thread should be synchronised area, sent to cater some rise then only we call wait(), notify(), notifyAll() methods otherwise we get a runtime exception called IllegalMonitorStateException
- Once the thread calls the wait() method on the given object first it releases the lock of that object immediately and enters into the waiting state
- Once the thread calls notify and notifyAll() methods then it releases the lock of an object but not immediately
- Once the thread call notify and notifyAll() and wait() method on any object then it releases the lock of that particular object but not all the locks it has.

Example on Inter thread communication :

```
public class ATM {
    double initBal = 1000;

    synchronized void withdraw(double amount) throws InsufficientBalance {
        if (amount <= initBal) {
            initBal = initBal - amount;
            System.out.println("Amount has been withdrawn successfully");
        } else {
            System.out.println("Insufficient balance");
            System.out.println("Waiting for deposit...");

            try {
                wait();
                if (amount <= initBal) {
                    initBal = initBal - amount;
                    System.out.println("Withdrawal after deposit successful");
                    System.out.println("Remaining Balance: " + initBal);
                } else {
                    throw new InsufficientBalance();
                }
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    synchronized void deposit(double amount) throws IncorrectAMountException {
        if (amount > 0) {
            initBal = initBal + amount;
            System.out.println("Deposited successfully: " + amount);
            notify();
        }
    }
}
```

```

    } else {
        throw new IncorrectAMountException();
    }
}
}

```

```
package in.dummy;
```

```

public class U1 extends Thread {
    ATM ref;

    public U1(ATM a) {
        ref = a;
    }

    @Override
    public void run() {
        try {
            ref.withdraw(5000);
        } catch (InsuffientBalance e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }
}

```

```
package in.dummy;
```

```

public class U2 extends Thread {
    ATM ref;

    public U2(ATM a) {
        ref = a;
    }

    @Override
    public void run() {
        try {
            ref.deposit(10000);
        } catch (IncorrectAMountException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }
}
}
package in.dummy;

```

```

public class ATMUSER {
    public static void main(String[] args) {
        ATM a = new ATM();
        U1 u1=new U1(a);
        U2 u2=new U2(a);

        u1.start();
        u2.start();
    }
}

```

2. Thread creation by implementing the Runnable Interface

We create a new class which implements java.lang.Runnable interface and override run() method. Then we instantiate a Thread object and call start() method on this object.

```

public class Test implements Runnable{
    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println("Pentagon space");
            try {
                Thread.sleep(300);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

```

Java Collections Framework

Introduction to Collections Framework

- The Java Collections Framework is a unified architecture for representing and manipulating collections, enabling collections to be manipulated independently of implementation details.
- Introduced in JDK 1.2.
- Provides interfaces and classes for storing and manipulating groups of data as a single unit

1. List Interface

- **What is it?**

- An ordered collection (also known as a sequence).
- Introduced in JDK 1.2.
- Extends the Collection interface.

- **Storage Details:**

- Index-based.
- Allows duplicate elements.
- Allows null values.

i. ArrayList Class

- A resizable array implementation of the List interface.
- Introduced in JDK 1.2.
- It implements List, RandomAccess, Cloneable, Serializable.

- **Storage Details:**

- Uses a dynamic array to store elements.
- Allows random access by index.
- Allows duplicate elements and null values.

- **Data Structure:**

- Dynamic array, growable or resizable.

- **Example:**

```
ArrayList<String> list = new ArrayList<>();
list.add("Apple");
list.add("Banana");
list.add("Apple"); // Allows duplicates
list.add(null); // Allows null values
```

- **Methods:**

- boolean add(E e)
- void add(int index, E element)
- boolean addAll(Collection<? extends E> c)
- E get(int index)
- E remove(int index)
- boolean remove(Object o)

- E set(int index, E element)
- int size()
- boolean isEmpty()
- boolean contains(Object o)
- void clear()

- **Constructors:**

- ArrayList()
- ArrayList(Collection<? extends E> c)
- ArrayList(int initialCapacity)

- **Synchronization:**

- Not synchronized.
- Use Collections.synchronizedList(new ArrayList<>()) for thread-safe operations.

- **Use Cases:**

- When you need fast iteration and random access of elements.
- Not suitable for scenarios where a lot of insertions and deletions are done in the middle of the list.

- **Incremental Capacity:**

- Default initial capacity is 10.
- Incremental capacity formula: $\text{newCapacity} = (\text{oldCapacity} * 3/2) + 1$.

ii. **LinkedList Class**

- A doubly-linked list implementation of the List and Deque interfaces.
- Introduced in JDK 1.2.
- implements List, Deque, Cloneable, Serializable.

- **Storage Details:**

- Stores elements as nodes in a doubly-linked list.
- Allows duplicate elements and null values.

- **Data Structure:**

- Doubly-linked list.

- **Example:**

```
LinkedList<String> list = new LinkedList<>();
list.add("Apple");
list.add("Banana");
list.addFirst("Cherry"); // Specific to LinkedList
list.addLast("Date");    // Specific to LinkedList
```

- **Methods:**

- void addFirst(E e)

- void addLast(E e)
- E getFirst()
- E getLast()
- E removeFirst()
- E removeLast()
- **Constructors:**
 - LinkedList()
 - LinkedList(Collection<? extends E> c)
- **Synchronization:**
 - Not synchronized.
 - Use Collections.synchronizedList(new LinkedList<>()) for thread-safe operations.
- **Use Cases:**
 - When you need fast insertions and deletions.
 - Not suitable for scenarios where random access is required frequently.
- **Incremental Capacity:**
 - Not applicable as it is a linked list.

iii. Vector Class

- A synchronized, resizable array implementation of the List interface.
- Introduced in JDK 1.0.
- Extends AbstractList and implements List, RandomAccess, Cloneable, Serializable.
- **Storage Details:**
 - Uses a dynamic array to store elements.
 - Allows random access by index.
 - Allows duplicate elements and null values.
- **Data Structure:**
 - Dynamic array.
- **Example:**

```
Vector<String> vector = new Vector<>();
vector.add("Apple");
vector.add("Banana");
vector.add("Apple"); // Allows duplicates
vector.add(null); // Allows null values
```
- **Methods:**
 - boolean add(E e)
 - void add(int index, E element)

- `boolean addAll(Collection<? extends E> c)`
- `E get(int index)`
- `E remove(int index)`
- `boolean remove(Object o)`
- `E set(int index, E element)`
- `int size()`
- `boolean isEmpty()`
- `boolean contains(Object o)`
- `void clear()`
- **Constructors:**
 - `Vector()`
 - `Vector(Collection<? extends E> c)`
 - `Vector(int initialCapacity)`
 - `Vector(int initialCapacity, int capacityIncrement)`
- **Synchronization:**
 - Synchronized.
- **Use Cases:**
 - When you need thread-safe operations.
 - Not suitable for non-thread-safe operations due to performance overhead.
- **Incremental Capacity:**
 - Default initial capacity is 10.
 - Incremental capacity can be specified in the constructor.
 - Incremental capacity formula: $\text{newCapacity} = \text{oldCapacity} * 2$.

Iterators

- An object that enables traversing through a collection.
- Introduced in JDK 1.2.
- Implements the Iterator interface.
- **Storage Details:**
 - Not applicable as it is a traversal mechanism.
- **Example:**

```
ArrayList<String> list = new ArrayList<>();
list.add("Apple");
list.add("Banana");
```

```
Iterator<String> iterator = list.iterator();
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
```


- **Methods:**
 - boolean hasNext()
 - E next()
 - void remove()
- **Synchronization:**
 - Not synchronized.
- **Use Cases:**
 - When you need to iterate over a collection.

2. Set Interface

- A collection that does not allow duplicate elements.
- Introduced in JDK 1.2.
- Extends the Collection interface.
- **Storage Details:**
 - Not index-based.
 - Does not allow duplicate elements.
 - Allows null values (with exceptions like TreeSet).

i. HashSet Class

- A collection that uses a hash table for storage.
- Introduced in JDK 1.2.
- implements Set, Cloneable, Serializable.

- **Storage Details:**
 - Uses a hash table.
 - Allows only one null value.

- **Data Structure:**
 - Hash table.

- **Example:**

```
HashSet<String> set = new HashSet<>();
set.add("Apple");
set.add("Banana");
set.add("Apple"); // Duplicate element, will not be added
set.add(null); // Allows null values
```

- **Methods:**
 - boolean add(E e)
 - boolean remove(Object o)
 - boolean contains(Object o)
 - void clear()

- **Constructors:**
 - HashSet()
 - HashSet(Collection<? extends E> c)
 - HashSet(int initialCapacity)
 - HashSet(int initialCapacity, float loadFactor)
- **Synchronization:**
 - Not synchronized.
 - Use Collections.synchronizedSet(new HashSet<>()) for thread-safe operations.
- **Use Cases:**
 - When you need a collection with no duplicates and don't care about order.
 - Not suitable for ordered collections.
- **Incremental Capacity:**
 - Default initial capacity is 16.
 - Load factor is 0.75.
 - Incremental capacity formula: $\text{newCapacity} = \text{oldCapacity} * 2$.

ii. **LinkedHashSet Class**

- A hash table and linked list implementation of the Set interface, with predictable iteration order.
- Introduced in JDK 1.4.
- Extends HashSet and implements Set, Cloneable, Serializable.
- **Storage Details:**
 - Uses a hash table and linked list.
 - Maintains insertion order.
 - Allows only one null value.
- **Data Structure:**
 - Hash table and linked list.
- **Example:**

```
LinkedHashSet<String> set = new LinkedHashSet<>();
set.add("Apple");
set.add("Banana");
set.add("Apple"); // Duplicate element, will not be added
set.add(null); // Allows null values
```
- **Methods:**
 - boolean add(E e)
 - boolean remove(Object o)
 - boolean contains(Object o)

- `void clear()`
- **Constructors:**
 - `LinkedHashSet()`
 - `LinkedHashSet(Collection<? extends E> c)`
 - `LinkedHashSet(int initialCapacity)`
 - `LinkedHashSet(int initialCapacity, float loadFactor)`
- **Synchronization:**
 - Not synchronized.
 - Use `Collections.synchronizedSet(new LinkedHashSet<>())` for thread-safe operations.
- **Use Cases:**
 - When you need a collection with no duplicates and predictable iteration order.
 - Not suitable for unordered collections.
- **Incremental Capacity:**
 - Default initial capacity is 16.
 - Load factor is 0.75.
 - Incremental capacity formula: $\text{newCapacity} = \text{oldCapacity} * 2$.

iii. TreeSet Class

- A `NavigableSet` implementation based on a `TreeMap`.
- Introduced in JDK 1.2.
- Extends `AbstractSet` and implements `NavigableSet`, `Cloneable`, `Serializable`.
- **Storage Details:**
 - Stores elements in a red-black tree.
 - Elements are sorted in natural order or by a comparator.
 - Does not allow null values.
- **Data Structure:**
 - Red-black tree.
- **Example:**

```
TreeSet<String> set = new TreeSet<>();
set.add("Apple");
set.add("Banana");
// set.add(null); // Throws NullPointerException
```

- **Methods:**
 - `boolean add(E e)`
 - `boolean remove(Object o)`
 - `boolean contains(Object o)`

- void clear()
 - **Constructors:**
 - TreeSet()
 - TreeSet(Collection<? extends E> c)
 - TreeSet(SortedSet<E> s)
 - TreeSet(Comparator<? super E> comparator)
 - **Synchronization:**
 - Not synchronized.
 - Use Collections.synchronizedSet(new TreeSet<>()) for thread-safe operations.
 - **Use Cases:**
 - When you need a sorted set.
 - Not suitable for unsorted sets or when null values are needed.
 - **Incremental Capacity:**
 - Not applicable as it is a tree-based structure.
-

Comparators and Comparable

- Comparable: An interface used to define the natural ordering of objects.
- Comparator: An interface used to define an external ordering of objects.
- **Methods in Comparable:**

```
int compareTo(T o)
```
- **Methods in Comparator:**

```
int compare(T o1, T o2)
boolean equals(Object obj)
```
- **Example for Comparable:**

```
public class Student implements Comparable<Student> {
    private String name;
    private int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public int compareTo(Student other) {
```

```

        return this.age - other.age; // Compare by age
    }
}

```

- **Example for Comparator:**

```

public class NameComparator implements Comparator<Student> {
    @Override
    public int compare(Student s1, Student s2) {
        return s1.name.compareTo(s2.name); // Compare by name
    }
}

```

- **Synchronization:**

- Not applicable.

- **Use Cases:**

- Comparable: When natural ordering is needed.
- Comparator: When custom ordering is needed.

- **Incremental Capacity:**

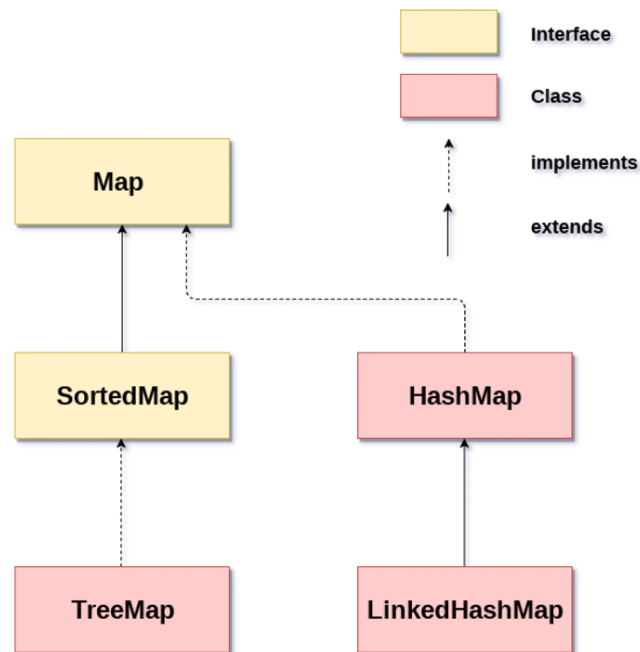
- Not applicable.

Map Interface

- An object that maps keys to values.
- Introduced in JDK 1.2.

- **Storage Details:**

- Not index-based.
- Allows duplicate values but not duplicate keys.
- Allows null values and null keys (with exceptions like TreeMap).



1. HashMap Class

- A hash table-based implementation of the Map interface.
- Introduced in JDK 1.2.
- Extends AbstractMap and implements Map, Cloneable, Serializable.
- **Storage Details:**
 - Uses a hash table.
 - Allows one null key and multiple null values.
- **Data Structure:**
 - Hash table.
- **Example:**

```

HashMap<String, Integer> map = new HashMap<>();
map.put("Apple", 1);
map.put("Banana", 2);
map.put(null, 3); // Allows null key
map.put("Cherry", null); // Allows null value

```
- **Methods:**
 - V put(K key, V value)
 - V get(Object key)
 - V remove(Object key)
 - boolean containsKey(Object key)
 - boolean containsValue(Object value)
 - void clear()
- **Constructors:**
 - HashMap()

- `HashMap(int initialCapacity)`
- `HashMap(int initialCapacity, float loadFactor)`
- `HashMap(Map<? extends K, ? extends V> m)`
- **Synchronization:**
 - Not synchronized.
 - Use `Collections.synchronizedMap(new HashMap<>())` for thread-safe operations.
- **Use Cases:**
 - When you need fast access to elements by key.
 - Not suitable for maintaining order of elements.
- **Incremental Capacity:**
 - Default initial capacity is 16.
 - Load factor is 0.75.
 - Incremental capacity formula: $\text{newCapacity} = \text{oldCapacity} * 2$.

2. LinkedHashMap Class

- A hash table and linked list implementation of the Map interface, with predictable iteration order.
- Introduced in JDK 1.4.
- Extends `HashMap` and implements `Map`, `Cloneable`, `Serializable`.
- **Storage Details:**
 - Uses a hash table and linked list.
 - Maintains insertion order.
 - Allows one null key and multiple null values.
- **Data Structure:**
 - Hash table and linked list.
- **Example:**

```

LinkedHashMap<String, Integer> map = new LinkedHashMap<>();
map.put("Apple", 1);
map.put("Banana", 2);
map.put(null, 3); // Allows null key
map.put("Cherry", null); // Allows null value

```
- **Methods:**
 - `V put(K key, V value)`
 - `V get(Object key)`
 - `V remove(Object key)`
 - `boolean containsKey(Object key)`
 - `boolean containsValue(Object value)`
 - `void clear()`
- **Constructors:**

- `LinkedHashMap()`
- `LinkedHashMap(int initialCapacity)`
- `LinkedHashMap(int initialCapacity, float loadFactor)`
- `LinkedHashMap(Map<? extends K, ? extends V> m)`
- `LinkedHashMap(int initialCapacity, float loadFactor, boolean accessOrder)`
- **Synchronization:**
 - Not synchronized.
 - Use `Collections.synchronizedMap(new LinkedHashMap<>())` for thread-safe operations.
- **Use Cases:**
 - When you need a map with predictable iteration order.
 - Not suitable for unordered collections.
- **Incremental Capacity:**
 - Default initial capacity is 16.
 - Load factor is 0.75.
 - Incremental capacity formula: $\text{newCapacity} = \text{oldCapacity} * 2$.

3. TreeMap Class

- A `NavigableMap` implementation based on a red-black tree.
- Introduced in JDK 1.2.
- Extends `AbstractMap` and implements `NavigableMap`, `Cloneable`, `Serializable`.
- **Storage Details:**
 - Stores elements in a red-black tree.
 - Keys are sorted in natural order or by a comparator.
 - Does not allow null keys.
- **Data Structure:**
 - Red-black tree.
- **Example:**

```
TreeMap<String, Integer> map = new TreeMap<>();
map.put("Apple", 1);
map.put("Banana", 2);
// map.put(null, 3); // Throws NullPointerException
map.put("Cherry", null); // Allows null value
```
- **Methods:**
 - `V put(K key, V value)`
 - `V get(Object key)`
 - `V remove(Object key)`
 - `boolean containsKey(Object key)`
 - `boolean containsValue(Object value)`

- void clear()
- **Constructors:**
 - TreeMap()
 - TreeMap(Comparator<? super K> comparator)
 - TreeMap(Map<? extends K, ? extends V> m)
 - TreeMap(SortedMap<K, ? extends V> m)
- **Synchronization:**
 - Not synchronized.
 - Use Collections.synchronizedMap(new TreeMap<>()) for thread-safe operations.
- **Use Cases:**
 - When you need a sorted map.
 - Not suitable for unsorted maps or when null keys are needed.
- **Incremental Capacity:**
 - Not applicable as it is a tree-based structure.

Java 8 features

It is a new version of Java and was released by Oracle on 18 March 2014. Java provided support for functional programming, new Java 8 APIs, a new JavaScript engine, new Java 8 streaming API, functional interfaces, default methods, date-time API changes, etc.

Major Java 8 Features Introduced

There are a few major Java 8 features mentioned below:

- **Lambda Expressions:** Concise functional code using ->.
- **Functional Interfaces:** Single-method interfaces.
- **Introduced and Improved APIs:**
 1. **Stream API:** Efficient Data Manipulation.
 2. **Date/Time API:** Robust Date and Time Handling.
- **Optional Class:** Handle null values safely.
- **forEach() Method in Iterable Interface:** Executes an action for each element in a Collection.
- **Default Methods:** Evolve interfaces without breaking compatibility.
- **Static Methods:** Allows adding methods with default implementations to interfaces.
- **Method References:** Refer to methods easily.

1. Java Lambda Expressions

Lambda expression is a new and important feature of Java which was included in Java SE 8. It provides a clear and concise way to represent one method

interface using an expression. It is very useful in collection library. It helps to iterate, filter and extract data from collection.

- The Lambda expression is used to provide the implementation of an interface which has functional interface. It saves a lot of code. In case of lambda expression, we don't need to define the method again for providing the implementation. Here, we just write the implementation code.
- Java lambda expression is treated as a function, so compiler does not create .class file.
- Lambda expression provides implementation of *functional interface*. An interface which has only one abstract method is called functional interface. Java provides an annotation `@FunctionalInterface`, which is used to declare an interface as functional interface.

Java Lambda Expression Syntax :

(argument-list) -> {body}

1. No Parameter Syntax

```
() -> {  
    //Body of no parameter lambda  
}
```

2. One Parameter Syntax

```
(p1) -> {  
    //Body of single parameter lambda  
}
```

3. Two Parameter Syntax

```
(p1,p2) -> {  
    //Body of multiple parameter lambda  
}
```

Example :

1. Without Lambda Expression

```
interface Drawable{  
    public void draw();
```

```

    }
    public class LambdaExpressionExample {
        public static void main(String[] args) {
            int width=10;

            //without lambda, Drawable implementation using anonymous class
            Drawable d=new Drawable(){
                public void draw(){System.out.println("Drawing "+width);}
            };
            d.draw();
        }
    }

```

2. Using Lambda Expressions

```

    @FunctionalInterface //It is optional
    interface Drawable{
        public void draw();
    }

    public class LambdaExpressionExample2 {
        public static void main(String[] args) {
            int width=10;

            //with lambda
            Drawable d2=()->{
                System.out.println("Drawing "+width);
            };
            d2.draw();
        }
    }

```

3. Java Lambda Expression Example: Single Parameter

```

    interface Sayable{
        public String say(String name);
    }

    public class LambdaExpressionExample4{
        public static void main(String[] args) {

```

```
// Lambda expression with single parameter.
Sayable s1=(name)->{
    return "Hello, "+name;
};
System.out.println(s1.say("Sonoo"));

// You can omit function parentheses
Sayable s2= name ->{
    return "Hello, "+name;
};
System.out.println(s2.say("Sonoo"));
}
}
```

4. Java Lambda Expression Example: Multiple Parameters

```
interface Addable{
    int add(int a,int b);
}

public class LambdaExpressionExample5{
    public static void main(String[] args) {

        // Multiple parameters in lambda expression
        Addable ad1=(a,b)->(a+b);
        System.out.println(ad1.add(10,20));

        // Multiple parameters with data type in lambda expression
        Addable ad2=(int a,int b)->(a+b);
        System.out.println(ad2.add(100,200));
    }
}
```

Example 5 : Java Lambda Expression Example: Creating Thread

You can use lambda expression to run thread. In the following example, we are implementing run method by using lambda expression.

```
public class LambdaExpressionExample5{
    public static void main(String[] args) {

        //Thread Example without lambda
```

```

Runnable r1=new Runnable(){
    public void run(){
        System.out.println("Thread1 is running...");
    }
};
Thread t1=new Thread(r1);
t1.start();
//Thread Example with lambda
Runnable r2=()->{
    System.out.println("Thread2 is running...");
};
Thread t2=new Thread(r2);
t2.start();
}
}

```

Java Functional Interfaces

An Interface that contains exactly one abstract method is known as functional interface. It can have any number of default, static methods but can contain only one abstract method. It can also declare methods of object class.

Functional Interface is also known as Single Abstract Method Interfaces or SAM Interfaces. It is a new feature in Java, which helps to achieve functional programming approach.

Example 1

```

@FunctionalInterface
interface sayable{
    void say(String msg);
}
public class FunctionalInterfaceExample implements sayable{
    public void say(String msg){
        System.out.println(msg);
    }
    public static void main(String[] args) {
        FunctionalInterfaceExample fie = new FunctionalInterfaceExample();
        fie.say("Hello there");
    }
}

```

Example 2 : Invalid Functional Interface

A functional interface can extend another interface only when it does not have any abstract method.

```
interface sayable{
    void say(String msg); // abstract method
}
@FunctionalInterface
interface Doable extends sayable{
    // Invalid '@FunctionalInterface' annotation; Doable is not a functional interface
    void doIt();
}
```

Example 3 : In the following example, a functional interface is extending to a non-functional interface.

```
interface Doable{
    default void doIt(){
        System.out.println("Do it now");
    }
}
@FunctionalInterface
interface Sayable extends Doable{
    void say(String msg); // abstract method
}
public class FunctionalInterfaceExample3 implements Sayable{
    public void say(String msg){
        System.out.println(msg);
    }
    public static void main(String[] args) {
        FunctionalInterfaceExample3 fie = new FunctionalInterfaceExample3();
        fie.say("Hello there");
        fie.doIt();
    }
}
```

Some Built-in Java Functional Interfaces

Since Java SE 1.8 onwards, there are many interfaces that are converted into functional interfaces. All these interfaces are annotated with `@FunctionalInterface`. These interfaces are

as follows –

- **Runnable** → This interface only contains the run() method.
- **Comparable** → This interface only contains the compareTo() method.
- **ActionListener** → This interface only contains the actionPerformed() method.
- **Callable** → This interface only contains the call() method.

Java Anonymous inner class

Java anonymous inner class is an inner class without a name and for which only a single object is created. An anonymous inner class can be useful when making an instance of an object with certain "extras" such as overloading methods of a class or interface, without having to actually subclass a class.

In simple words, a class that has no name is known as an anonymous inner class in Java. It should be used if you have to override a method of class or interface. Java Anonymous inner class can be created in two ways:

1. Class (may be abstract or concrete).
2. Interface

Example 1: Java anonymous inner class example using class

```
abstract class Person{
    abstract void eat();
}
class TestAnonymousInner{
    public static void main(String args[]){
        Person p=new Person(){
            void eat(){System.out.println("nice fruits");}
        };
        p.eat();
    }
}
```

Example 2 : Java anonymous inner class example using interface

```
interface Eatable{
    void eat();
}
class TestAnonymousInner1 {
    public static void main(String args[]){
        Eatable e=new Eatable(){
```

```

        public void eat(){System.out.println("nice fruits");}
    };
    e.eat();
}
}

```

Example 3 : Anonymous inner class example w.r.t Multi-Threading

```

public class TestDemo4{
    public static void main(String[] args) {
        Runnable r=->{
            for(int i=0;i<10;i++) {
                System.out.println("child thread");
            }
        };
        Thread t=new Thread(r);
        t.start();
        for(int i=0;i<10;i++) {
            System.out.println("Main thread");
        }
    }
}

```

Java Method References

Java provides a new feature called method reference in Java 8. Method reference is used to refer method of functional interface. It is compact and easy form of lambda expression. Each time when you are using lambda expression to just referring a method, you can replace your lambda expression with method reference.

Types of Method References

There are following types of method references in java:

1. Reference to a static method.
2. Reference to an instance method.
3. Reference to a constructor.

1) Reference to a Static Method

You can refer to static method defined in the class. Following is the syntax and example which describe the process of referring static method in Java.

Syntax :

ContainingClass::staticMethodName

Example :

```

interface Sayable{
    void say();
}

public class MethodReference {
    public static void saySomething(){
        System.out.println("Hello, this is static method.");
    }
    public static void main(String[] args) {
        // Referring static method
        Sayable sayable = MethodReference::saySomething;
        // Calling interface method
        sayable.say();
    }
}

```

Example 2

In the following example, we are using predefined functional interface Runnable to refer static method.

```

public class MethodReference2 {
    public static void ThreadStatus(){
        System.out.println("Thread is running...");
    }
    public static void main(String[] args) {
        Thread t2=new Thread(MethodReference2::ThreadStatus);
        t2.start();
    }
}

```

2) Reference to an Instance Method

like static methods, you can refer instance methods also. In the following example, we are describing the process of referring the instance method.

Syntax :

containingObject::instanceMethodName

Example 1 :

In the following example, we are referring non-static methods. You can refer methods by class object and anonymous object.

```

interface Sayable{
    void say();
}

```

```

}
public class InstanceMethodReference {
    public void saySomething(){
        System.out.println("Hello, this is non-static method.");
    }
    public static void main(String[] args) {
        InstanceMethodReference methodReference = new InstanceMethodReference();
// Creating object
        // Referring non-static method using reference
        Sayable sayable = methodReference::saySomething;
        // Calling interface method
        sayable.say();
        // Referring non-static method using anonymous object
        Sayable sayable2 = new InstanceMethodReference()::saySomething; // You can
n use anonymous object also
        // Calling interface method
        sayable2.say();
    }
}

```

3) Reference to a Constructor

You can refer a constructor by using the new keyword. Here, we are referring constructor with the help of functional interface.

Syntax

ClassName::**new**

Example

```

interface Messageable{
    Message getMessage(String msg);
}
class Message{
    Message(String msg){
        System.out.print(msg);
    }
}
public class ConstructorReference {
    public static void main(String[] args) {
        Messageable hello = Message::new;
        hello.getMessage("Hello");
    }
}

```

Java 8 Stream

Java provides a new additional package in Java 8 called `java.util.stream`. This package consists of classes, interfaces and enum to allows functional-style operations on the elements. You can use stream by importing `java.util.stream` package.

Stream provides following features:

- Stream does not store elements. It simply conveys elements from a source such as a data structure, an array, or an I/O channel, through a pipeline of computational operations.
- Stream is functional in nature. Operations performed on a stream does not modify it's source. For example, filtering a Stream obtained from a collection produces a new Stream without the filtered elements, rather than removing elements from the source collection.
- Stream is lazy and evaluates code only when required.
- The elements of a stream are only visited once during the life of a stream. Like an Iterator, a new stream must be generated to revisit the same elements of the source.

How does Stream Work Internally?

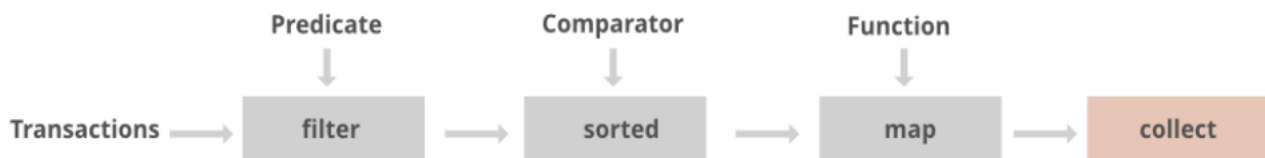
In streams,

- To filter out from the objects we do have a function named **filter()**
- To impose a condition we do have a logic of predicate which is nothing but a functional interface. Here function interface can be replaced by a lambda expression. Hence, we can directly impose the condition check in our predicate.
- To collect elements we will be using **Collectors.toList()** to collect all the required elements.
- Lastly, we will store these elements in a List and display the outputs on the console.

Various Core Operations Over Streams

There are broadly 3 types of operations that are carried over streams namely as follows as depicted from the image shown above:

1. Intermediate operations
2. Terminal operations
3. Short-circuit operations



1. Intermediate Operations:

Intermediate operations transform a stream into another stream. Some common intermediate operations include:

1. **filter()**: Filters elements based on a specified condition.
2. **map()**: Transforms each element in a stream to another value.
3. **sorted()**: Sorts the elements of a stream.

All three of them are discussed below as they go hand in hand in nearly most of the scenarios and to provide better understanding by using them later by implementing in our clean Java programs below. As we already have studied in the above example of which we are trying to filter processed objects can be interpreted as filter() operation operated over streams.

2. Terminal Operations

Terminal Operations are the operations that on execution return a final result as an absolute value.

1. **collect()**: It is used to return the result of the intermediate operations performed on the stream.
2. **forEach()**: It iterates all the elements in a stream.
3. **reduce()**: It is used to reduce the elements of a stream to a single value.

3. Short Circuit Operations

Short-circuit operations provide performance benefits by avoiding unnecessary computations when the desired result can be obtained early. They are particularly useful when working with large or infinite streams.

1. **anyMatch()**: it checks the stream if it satisfies the given condition.
2. **findFirst()**: it checks the element that matches a given condition and stops processing when it finds it.

Example 1:

```

package Java8Features;

import java.util.Arrays;
import java.util.Iterator;
import java.util.List;
import java.util.stream.Collectors;
  
```

```

class Employee{
    int id;
    String name;
public Employee(int id, String name) {
    this.id = id;
    this.name = name;
}
}

```

//Stream is used perform pipelined operation by taking the input from collections or arrays

//Stream is not data structure it only process or express the data into streams

```

public class StreamApi1 {
public static Integer cubes(int a){
return a*a*a;
}
}

```

```

public static void main(String[] args) {
// to store integer values in list
List<Integer> my_num=Arrays.asList(12,24,26,27,67,23,78);
//to store string values in list
List<String> my_name=List.of("vish" , "muthu " , "akash" , "ravi");
//to store objects in a list
List<Employee> my_emp=List.of(new Employee(10,"vish"),new
Employee(20,"akash"));
}

```

// Intermediate operations

// 1. map method

// if we want to perform any operation on list, we can use this map method

```

List<Integer> cubes=my_num.stream()
.map(StreamApi1::cubes)           //using method reference operator or
.collect(Collectors.toList());
System.out.println(cubes);

```

// or we can use lambda expression to perform cube operation like this

/*

```

List<Integer> cubes=my_num.stream()

```



```
.map(i->return i*i*i);
.collect(Collectors.toList());
System.out.println(cubes);
*/
```

// 2. filter method

// if we want to filter any existing data we can use this filter method

```
List<String> Select_candidate=my_name.stream()
.filter(str->str.endsWith("sh"))
.collect(Collectors.toList());
System.out.println(Select_candidate);
```

```
List<Employee> select_emp=my_emp.stream().filter(emp-
>emp.name.startsWith("a")).collect(Collectors.toList());
```

// we can iterate using for each loop

```
for(Employee e:select_emp) {
System.out.println(e.name);
}
```

// or by using regular for loop

```
for(int i=0;i<select_emp.size();i++) {
System.out.println(select_emp.get(i).name);
}
```

// 3. reduce method

// reduce method is an terminal method which takes the input from intermediate operations and

// reduce the elements of a stream to a single value

```
int odd_name=my_num.stream().filter(odd->odd%2!=0).reduce(1,(res,i)->res*i);
System.out.println(odd_name);
```

```
}
```

```
}
```

Example 2 :

```
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;
import java.util.stream.Collectors;

public class StreamApidemo1 {

    public static void main(String[] args) {
        List<Integer> lst=Arrays.asList(12,89,35,46,75,16);

        Integer MaxCount=lst.stream().max((num1,num2)-
        >num1>num2?1:-1).get();
        System.out.println(MaxCount);

        // to find maximum number
        Optional<Integer>
        MaxNum=lst.stream().max(Comparator.naturalOrder());
        System.out.println(MaxNum.get());

        // to find the second maximum number
        Optional<Integer>
        SecondMaxNum=lst.stream().sorted(Comparator.reverseOrder(
        )).skip(1).findFirst();
        System.out.println(SecondMaxNum);

        // to find the minimum number
        Optional<Integer>
        MinNum=lst.stream().sorted(Comparator.naturalOrder()).findF
        irst();

        System.out.println(MinNum);

        // to find the second minimum value
        Optional<Integer>
        SecondMinNum=lst.stream().sorted(Comparator.naturalOrder()
        ).skip(1).findFirst();
```

```
System.out.println(SecondMinNum);
```

```
List<Integer>
```

```
str=lst.stream().sorted().collect(Collectors.toList());
```

```
System.out.println(str);
```

```
Double lst1=lst.stream().collect(Collectors.averagingInt(n->n));
```

```
System.out.println(lst1);
```

```
//          int evenNum=lst.stream().filter(i->i%2==0).collect(Collectors.summingInt(n->n));
//          System.out.println(evenNum);
```

```
Integer evenNum=lst.stream().filter(i->i%2==0).reduce(0,
(num1,num2)-> num1 + num2);
System.out.println(evenNum);
```

```
int oddNum=lst.stream().filter(i->i%2!=0).collect(Collectors.summingInt(n->n));
System.out.println(oddNum);
```

```
List<String> strgs=Arrays.asList("apple", "mango",
"ananas","grape","orange");
long countStr=strgs.stream().filter(s->s.startsWith("a")).count();
System.out.println(countStr);
```

```
List<String>
sortstrAscen=strgs.stream().sorted().collect(Collectors.toList());
;
System.out.println(sortstrAscen);
List<String>
sortstrDecen=strgs.stream().sorted(Comparator.reverseOrder()).
collect(Collectors.toList());
System.out.println(sortstrDecen);
```

```
}
```

```
}
```

Java 8 Optional Class

Every Java Programmer is familiar with [NullPointerException](#). It can crash your code. And it is very hard to avoid it without using too many null checks. So, to overcome this, Java 8 has introduced a new class Optional in **java.util package**. It can help in writing a neat code without using too many null checks. By using Optional, we can specify alternate values to return or alternate code to run. This makes the code more readable because the facts which were hidden are now visible to the developer.

// Java program without Optional Class

```
public class OptionalDemo {
    public static void main(String[] args)
    {
        String[] words = new String[10];
        String word = words[5].toLowerCase();
        System.out.print(word);
    }
}
```

Output:

Exception in thread "main" java.lang.NullPointerException

To avoid abnormal termination, we use the Optional class. In the following example, we are using Optional. So, our program can execute without crashing

Example : Java program with Optional Class

```
import java.util.Optional;
```

// Driver Class

```
public class OptionalDemo {
    // Main Method
    public static void main(String[] args)
    {
        String[] words = new String[10];

        Optional<String> checkNull = Optional.ofNullable(words[5]);

        if (checkNull.isPresent()) {
            String word = words[5].toLowerCase();
        }
    }
}
```

```

        System.out.print(word);
    }
    else
        System.out.println("word is null");
    }
}

```

Output

word is null

Default Methods in Java

Before Java 8, interfaces could have only abstract methods. The implementation of these methods has to be provided in a separate class. So, if a new method is to be added in an interface, then its implementation code has to be provided in the class implementing the same interface. To overcome this issue, Java 8 has introduced the concept of default methods which allow the interfaces to have methods with implementation without affecting the classes that implement the interface.

// A simple program to Test Interface default

// methods in java

```

interface TestInterface
{

```

```

    // abstract method

```

```

    public void square(int a);

```

```

    // default method

```

```

    default void show()
    {

```

```

        System.out.println("Default Method Executed");
    }

```

```

}

```

```

class TestClass implements TestInterface
{

```

```

    // implementation of square abstract method

```

```

    public void square(int a)

```

```

        {
            System.out.println(a*a);
        }

        public static void main(String args[])
        {
            TestClass d = new TestClass();
            d.square(4);

            // default method executed
            d.show();
        }
    }

```

Static Methods:

The interfaces can have static methods as well which is similar to static method of classes.

// A simple Java program to TestClassnstrate static
// methods in java

```

interface TestInterface
{
    // abstract method
    public void square (int a);

    // static method
    static void show()
    {
        System.out.println("Static Method Executed");
    }
}

class TestClass implements TestInterface
{
    // Implementation of square abstract method
    public void square (int a)
    {
        System.out.println(a*a);
    }

    public static void main(String args[])

```

```

    {
        TestClass d = new TestClass();
        d.square(4);

        // Static method executed
        TestInterface.show();
    }
}

```

Data and Time in Java

Example :

```

package Java8Features;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;

public class DateTime {
    public static void main(String[] args) {
        // to print current date
        LocalDate d= LocalDate.now();
        System.out.println(d);

        // to print current time
        LocalTime t=LocalTime.now();
        System.out.println(t);

        // Formatting Date and time
        LocalDateTime dt=LocalDateTime.now();
        DateTimeFormatter
pattern=DateTimeFormatter.ofPattern("dd-MM-yyyy hh:mm:ss");
        String format_date=dt.format(pattern);
        System.out.println(format_date);

        // to get the current day of the week

```



```
        System.out.println(d.getDayOfWeek());

// to get the current day of the month
        System.out.println(d.getDayOfMonth());

// to get the current dayNumber of the year (w.r.t 365)
        System.out.println(d.getDayOfYear());


// to set any day/date. for eg : myBirthday date
        LocalDate mybday=LocalDate.of(2000, 05, 07);
        System.out.println(mybday);
//
        LocalDate myfiancebday=LocalDate.of(2005, 9, 7);
        System.out.println(myfiancebday);

// for comparing dates to check is it occuring befor or after

        // here it display number of days differs
        System.out.println(mybday.compareTo(myfiancebday));
        // it returns boolean value
        System.out.println(mybday.isAfter(myfiancebday));
    }
}
```

Topic wise MCQ'S Questions

❖ Introduction to Java

1. When was the first version of Java officially released?

- A) 1991
- B) 1995
- C) 1998
- D) 2000

○ **Answer: B) 1995**

2. Which company originally developed Java?

- A) Microsoft
- B) IBM
- C) Sun Microsystems
- D) Oracle

○ **Answer: C) Sun Microsystems**

3. Who is considered the father of Java?

- A) James Gosling
- B) Bill Gates
- C) Dennis Ritchie
- D) Bjarne Stroustrup

○ **Answer: A) James Gosling**

4. What was Java originally designed for?

- A) Web applications
- B) Embedded systems
- C) Desktop applications

- D) Game development
- **Answer: B) Embedded systems**

5. Which version of Java introduced Generics?

- A) Java 1.2
- B) Java 1.4
- C) Java 5
- D) Java 6
- **Answer: C) Java 5**

6. Which Java version was known as "Tiger"?

- A) Java 1.2
- B) Java 1.4
- C) Java 5
- D) Java 6
- **Answer: C) Java 5**

7. In which year did Oracle acquire Sun Microsystems?

- A) 2008
- B) 2009
- C) 2010
- D) 2011
- **Answer: C) 2010**

8. What was Java's code name during its development?

- A) Green
- B) Blue
- C) Red
- D) Yellow
- **Answer: A) Green**

9. Which version of Java introduced lambda expressions?

- A) Java 6
- B) Java 7
- C) Java 8
- D) Java 9
- **Answer: C) Java 8**

10. Java was designed to have which key feature that enhances portability?

- A) Platform Independence
- B) Object-Oriented
- C) High Performance
- D) Robust Security
- **Answer: A) Platform Independence**

Features of Java

1. Which of the following is not a feature of Java?

- A) Platform Independence
- B) High Performance
- C) Multiple Inheritance
- D) Object-Oriented
- **Answer: C) Multiple Inheritance**

2. **Java's feature that allows it to run on any platform is known as:**

- A) Just-In-Time Compilation
- B) Platform Independence
- C) Secure Execution
- D) Network-Centric
- **Answer: B) Platform Independence**

3. **What is the default access level for class members in Java?**

- A) Public
- B) Private
- C) Protected
- D) Package-Private
- **Answer: D) Package-Private**

4. **Which feature of Java helps in memory management and prevents memory leaks?**

- A) Garbage Collection
- B) Exception Handling
- C) Encapsulation
- D) Inheritance
- **Answer: A) Garbage Collection**

5. **Java supports which type of memory management mechanism?**

- A) Manual Garbage Collection
- B) Automatic Garbage Collection
- C) Static Allocation
- D) Dynamic Allocation
- **Answer: B) Automatic Garbage Collection**

6. **Java's security feature that restricts unauthorized access is known as:**

- A) Access Modifiers
- B) Security Manager
- C) Encryption
- D) Bytecode Verification
- **Answer: D) Bytecode Verification**

7. **Which of the following is a feature of Java's exception handling mechanism?**

- A) Exceptions must be handled in every method
- B) Exceptions are unchecked by default
- C) Only runtime exceptions are caught
- D) All exceptions are checked by default

- **Answer: B) Exceptions are unchecked by default**
- 8. **Which Java feature allows you to use objects of different types interchangeably?**
 - A) Polymorphism
 - B) Inheritance
 - C) Encapsulation
 - D) Abstraction
 - **Answer: A) Polymorphism**
- 9. **Which keyword in Java is used to create an instance of a class?**
 - A) create
 - B) new
 - C) instantiate
 - D) object
 - **Answer: B) new**
- 10. **Which feature of Java allows methods to have the same name but different parameters?**
 - A) Method Overriding
 - B) Method Overloading
 - C) Method Binding
 - D) Method Resolution
 - **Answer: B) Method Overloading**

Class and Objects

- 1. **Which keyword is used to declare a class in Java?**
 - A) class
 - B) object
 - C) define
 - D) new
 - **Answer: A) class**
- 2. **How do you create an object of a class in Java?**
 - A) `ClassName obj = new ClassName();`
 - B) `ClassName obj = ClassName();`
 - C) `new ClassName obj;`
 - D) `obj = new ClassName();`
 - **Answer: A) `ClassName obj = new ClassName();`**
- 3. **Which method is called when an object is created in Java?**
 - A) `initialize()`
 - B) `construct()`
 - C) `main()`
 - D) constructor
 - **Answer: D) constructor**
- 4. **What is the default value of an instance variable of type int?**

- A) 0
- B) 1
- C) null
- D) -1
- **Answer: A) 0**

5. **In Java, how do you access a class's instance variable?**

- A) Using the class name
- B) Using the instance object
- C) Using the static keyword
- D) Using this keyword
- **Answer: B) Using the instance object**

6. **What is the access level of class members that are declared as private?**

- A) Accessible within the same package
- B) Accessible within the same class
- C) Accessible from anywhere
- D) Accessible from the same package and subclasses
- **Answer: B) Accessible within the same class**

7. **Which of the following is not a part of a class definition in Java?**

- A) Fields
- B) Methods
- C) Constructors
- D) Namespaces
- **Answer: D) Namespaces**

8. **What is the purpose of the static keyword in Java?**

- A) To indicate that a method belongs to an instance
- B) To indicate that a variable or method belongs to the class
- C) To restrict access to a variable or method
- D) To define an abstract method
- **Answer: B) To indicate that a variable or method belongs to the class**

9. **How can you call a static method from within a static method?**

- A) Directly, without creating an object
- B) By creating an instance of the class
- C) Using the new keyword
- D) By using the this keyword
- **Answer: A) Directly, without creating an object**

10. **What does the this keyword refer to in a method?**

- A) The class itself
- B) The current instance of the class
- C) The superclass
- D) The static variables of the class

- **Answer: B) The current instance of the class**

Java Keywords

1. Which of the following is a keyword in Java?

- A) function
- B) interface
- C) data
- D) object
- **Answer: B) interface**

2. Which keyword is used to prevent a class from being subclassed?

- A) final
- B) static
- C) private
- D) protected
- **Answer: A) final**

3. What does the static keyword denote in Java?

- A) The method or variable belongs to an instance of the class
- B) The method or variable belongs to the class itself
- C) The method or variable can only be accessed within the same class
- D) The method or variable cannot be changed
- **Answer: B) The method or variable belongs to the class itself**

4. Which keyword is used to create an interface in Java?

- A) abstract
- B) interface
- C) extends
- D) implements
- **Answer: B) interface**

5. Which of the following keywords is used to handle exceptions?

- A) catch
- B) throw
- C) try
- D) All of the above
- **Answer: D) All of the above**

6. Which keyword is used to define a method that cannot be overridden?

- A) abstract
- B) final
- C) static
- D) private
- **Answer: B) final**

7. What is the role of the super keyword in Java?

- A) To call a constructor of the superclass

- B) To define a new method in a subclass
- C) To access a static method of the superclass
- D) To declare a superclass
- **Answer: A) To call a constructor of the superclass**

8. Which keyword is used to define a constant value in Java?

- A) final
- B) static
- C) const
- D) immutable
- **Answer: A) final**

9. Which keyword is used to indicate that a class cannot be subclassed?

- A) final
- B) abstract
- C) static
- D) private
- **Answer: A) final**

10. What does the volatile keyword do in Java?

- A) It defines a constant variable
- B) It ensures the value of a variable is always read from the main memory
- C) It allows a variable to be accessed by multiple threads
- D) It defines an abstract class
- **Answer: B) It ensures the value of a variable is always read from the main memory**

Identifiers

1. Which of the following is a valid identifier in Java?

- A) 1variable
- B) variable_1
- C) @variable
- D) var iable
- **Answer: B) variable_1**

2. Identifiers in Java cannot start with:

- A) A letter
- B) A dollar sign (\$)
- C) An underscore (_)
- D) A digit
- **Answer: D) A digit**

3. Which of the following is not a valid Java identifier?

- A) myVariable
- B) _myVariable
- C) myVariable1

- D) 1myVariable
- **Answer: D) 1myVariable**

4. Which of the following characters is not allowed in Java identifiers?

- A) _
- B) \$
- C) @
- D) Letters
- **Answer: C) @**

5. Which of the following is a valid identifier for a method name in Java?

- A) myMethod()
- B) my-method()
- C) myMethod@()
- D) my method()
- **Answer: A) myMethod()**

6. Identifiers in Java are case-sensitive. What is the result of the following code? `int a = 5; int A = 10; System.out.println(a);`

- A) 5
- B) 10
- C) Compilation error
- D) 0
- **Answer: A) 5**

7. Which of the following is a reserved keyword in Java?

- A) myVar
- B) public
- C) variable
- D) method
- **Answer: B) public**

8. What will happen if you use a Java keyword as an identifier?

- A) It will compile successfully
- B) It will cause a runtime error
- C) It will cause a compile-time error
- D) It will create a new variable with the keyword's functionality
- **Answer: C) It will cause a compile-time error**

9. Which of the following is a valid Java identifier?

- A) 1_name
- B) name1
- C) @name
- D) name space
- **Answer: B) name1**

10. What is the maximum length of a Java identifier?

- A) 64 characters
- B) 128 characters
- C) No limit
- D) 256 characters
- **Answer: C) No limit**

Compilation and Execution

1. What is the extension of a Java source file?

- A) .java
- B) .class
- C) .byte
- D) .exe
- **Answer: A) .java**

2. Which command is used to compile a Java program?

- A) javac
- B) java
- C) compile
- D) javap
- **Answer: A) javac**

3. What does the Java compiler produce after compilation?

- A) .java files
- B) .class files
- C) .jar files
- D) .exe files
- **Answer: B) .class files**

4. Which command is used to run a Java program?

- A) javac
- B) java
- C) run
- D) execute
- **Answer: B) java**

5. What is the Java bytecode file extension?

- A) .java
- B) .class
- C) .jar
- D) .byte
- **Answer: B) .class**

6. What is the output of the following code snippet if `System.out.println(1 / 2);` is executed?

- A) 0.5
- B) 0

- C) 1
- D) Compilation error
- **Answer: B) 0**

7. **What is the result of compiling and running a Java program with syntax errors?**

- A) The program runs but produces incorrect results
- B) The program runs but produces runtime errors
- C) The compiler produces error messages and the program does not run
- D) The program compiles successfully but produces runtime errors
- **Answer: C) The compiler produces error messages and the program does not run**

8. **Which part of the Java program contains the main method?**

- A) The class
- B) The method
- C) The package
- D) The variable
- **Answer: A) The class**

9. **What is the purpose of the public static void main(String[] args) method in Java?**

- A) It is the entry point of a Java application
- B) It defines a class
- C) It initializes variables
- D) It compiles the Java program
- **Answer: A) It is the entry point of a Java application**

10. **Which tool is used to debug a Java program?**

- A) javac
- B) java
- C) jdb
- D) jar
- **Answer: C) jdb**

Data Members

1. **What are data members of a class in Java?**

- A) Methods of the class
- B) Variables defined inside the class
- C) Classes defined inside other classes
- D) Objects of the class
- **Answer: B) Variables defined inside the class**

2. **What is the default value of a data member of type boolean?**

- A) true
- B) false
- C) 0
- D) null

- **Answer: B) false**

3. Which keyword is used to define a constant data member in Java?

- A) final
- B) static
- C) const
- D) immutable

- **Answer: A) final**

4. Where are instance variables declared?

- A) Inside methods
- B) Inside constructors
- C) Inside the class but outside methods
- D) Inside loops

- **Answer: C) Inside the class but outside methods**

5. What is the default access level of data members in Java if no access modifier is specified?

- A) Public
- B) Private
- C) Protected
- D) Package-private

- **Answer: D) Package-private**

6. How can you access a private data member of a class?

- A) Directly from outside the class
- B) Using a public method of the class
- C) Using the this keyword
- D) By casting

- **Answer: B) Using a public method of the class**

7. What is the access level of a protected data member?

- A) Accessible only within the same class
- B) Accessible within the same package and subclasses
- C) Accessible only within the same package
- D) Accessible from anywhere

- **Answer: B) Accessible within the same package and subclasses**

8. What is the difference between instance variables and class variables in Java?

- A) Instance variables are static, while class variables are non-static
- B) Class variables are static, while instance variables are non-static
- C) Instance variables are defined in methods, while class variables are defined outside methods
- D) Class variables are local to methods, while instance variables are global

- **Answer: B) Class variables are static, while instance variables are non-static**

9. Which of the following statements is true about data members in a class?

- A) Data members can only be of primitive types
- B) Data members can be of any type including user-defined types
- C) Data members are local to the class methods
- D) Data members must be initialized in the class constructor
- **Answer: B) Data members can be of any type including user-defined types**

10.If a data member is declared as static, what does it mean?

- A) The data member is shared among all instances of the class
- B) The data member is local to the instance of the class
- C) The data member is only accessible within the class
- D) The data member cannot be accessed by any method
- **Answer: A) The data member is shared among all instances of the class**

Variables and Constants

1. What is a constant in Java?

- A) A variable whose value can be changed
- B) A variable whose value cannot be changed once initialized
- C) A variable that can be used without initialization
- D) A variable defined inside a method
- **Answer: B) A variable whose value cannot be changed once initialized**

2. Which keyword is used to declare a constant variable in Java?

- A) static
- B) final
- C) const
- D) immutable
- **Answer: B) final**

3. What is the scope of a local variable in Java?

- A) Accessible only within the method or block where it is defined
- B) Accessible throughout the class
- C) Accessible in the entire package
- D) Accessible from all classes
- **Answer: A) Accessible only within the method or block where it is defined**

4. Which of the following is a valid variable name in Java?

- A) 1variable
- B) variable_1
- C) variable-name
- D) variable name
- **Answer: B) variable_1**

5. What is the default value of an instance variable of type double?

- A) 0.0
- B) 0
- C) null

- D) -1.0
- **Answer: A) 0.0**

6. Which of the following is not a primitive data type in Java?

- A) int
- B) char
- C) boolean
- D) String
- **Answer: D) String**

7. How do you declare a variable with an initial value in Java?

- A) int x = 10;
- B) int x; x = 10;
- C) int x := 10;
- D) int x : 10;
- **Answer: A) int x = 10;**

8. Which of the following keywords is used to create a variable that holds a reference to an object?

- A) new
- B) static
- C) final
- D) instance
- **Answer: A) new**

9. What is the difference between final variables and static variables in Java?

- A) final variables cannot be changed after initialization, while static variables are shared among instances
- B) final variables are shared among instances, while static variables cannot be changed
- C) final variables are instance variables, while static variables are local variables
- D) final variables are used for methods, while static variables are used for classes
- **Answer: A) final variables cannot be changed after initialization, while static variables are shared among instances**

10. Which of the following is an example of a variable declaration and initialization?

- A) int x = 5;
- B) int x; x = 5;
- C) x = 5; int x;
- D) int 5 = x;
- **Answer: A) int x = 5;**

Datatypes

1. Which of the following is a valid primitive data type in Java?

- A) Integer
- B) String

- C) boolean
- D) Double
- **Answer: C) boolean**

2. **What is the size of an int data type in Java?**

- A) 8 bits
- B) 16 bits
- C) 32 bits
- D) 64 bits
- **Answer: C) 32 bits**

3. **Which data type is used to store a single character in Java?**

- A) char
- B) String
- C) boolean
- D) int
- **Answer: A) char**

4. **What is the default value of a float data type in Java?**

- A) 0.0f
- B) 0.0
- C) 0
- D) null
- **Answer: A) 0.0f**

5. **Which data type would you use to store large numbers in Java?**

- A) int
- B) float
- C) long
- D) short
- **Answer: C) long**

6. **Which data type in Java has the highest precision for floating-point numbers?**

- A) float
- B) double
- C) long
- D) int
- **Answer: B) double**

7. **What is the range of a byte data type in Java?**

- A) -128 to 127
- B) 0 to 255
- C) -32768 to 32767
- D) 0 to 65535
- **Answer: A) -128 to 127**

8. **Which data type would be suitable for representing a flag with true/false values?**

- A) boolean
- B) int
- C) char
- D) float
- **Answer: A) boolean**

9. **What is the default value of an instance variable of type String?**

- A) ""
- B) "null"
- C) null
- D) "undefined"
- **Answer: C) null**

10. **Which data type is used for decimal values in Java?**

- A) int
- B) float
- C) boolean
- D) char
- **Answer: B) float**

Object Creation

1. **Which keyword is used to create a new object in Java?**

- A) create
- B) new
- C) instantiate
- D) object
- **Answer: B) new**

2. **What is the correct syntax for creating an object of a class named Person?**

- A) Person p = new Person();
- B) new Person p = Person();
- C) Person p = Person.new();
- D) new Person p = new Person;
- **Answer: A) Person p = new Person();**

3. **What happens if you do not use the new keyword when creating an object?**

- A) The object is created and initialized automatically
- B) The program will produce a compile-time error
- C) The object will be null
- D) The object is created but not initialized
- **Answer: B) The program will produce a compile-time error**

4. **Which of the following is a correct way to initialize an object of the Car class with a constructor that takes parameters?**

- A) Car myCar = new Car("Red", 4);
- B) Car myCar = Car("Red", 4);

- C) Car myCar = new Car;
- D) Car myCar = Car.new("Red", 4);
- **Answer: A) Car myCar = new Car("Red", 4);**

5. What does the new keyword in new ClassName() do?

- A) It defines a new class
- B) It creates an instance of a class
- C) It calls a method of the class
- D) It initializes a variable
- **Answer: B) It creates an instance of a class**

6. What is the initial state of an object created using the new keyword if no constructor is defined?

- A) The object will have default values for its fields
- B) The object will be null
- C) The object will throw an exception
- D) The object will have random values for its fields
- **Answer: A) The object will have default values for its fields**

7. Which of the following statements about object creation is true?

- A) Objects are created using the create keyword
- B) Objects are instantiated using a class constructor
- C) Objects are created using the static keyword
- D) Objects are created using the new keyword only in static methods
- **Answer: B) Objects are instantiated using a class constructor**

8. Can you create an object without using a constructor in Java?

- A) Yes, by using default values
- B) No, constructors are mandatory for object creation
- C) Yes, by calling static methods
- D) Yes, by using factory methods
- **Answer: A) Yes, by using default values**

9. How is memory allocated for an object in Java?

- A) At compile-time
- B) At runtime
- C) During the class definition
- D) At load-time
- **Answer: B) At runtime**

10. What is the result of creating an object without calling a constructor explicitly?

- A) The object is created with default values
- B) The object will throw an error
- C) The object will be null
- D) The object will have random values
- **Answer: A) The object is created with default values**

Access Modifiers

1. Which access modifier allows access within the same package and subclasses?
 - A) private
 - B) protected
 - C) public
 - D) Default (package-private)
 - **Answer: B) protected**
2. What is the access level of a private member in Java?
 - A) Accessible only within the same class
 - B) Accessible within the same package and subclasses
 - C) Accessible from anywhere
 - D) Accessible only within the same package
 - **Answer: A) Accessible only within the same class**
3. Which access modifier is used to allow access from any class in any package?
 - A) protected
 - B) private
 - C) public
 - D) Default (package-private)
 - **Answer: C) public**
4. What is the default access level if no access modifier is specified?
 - A) private
 - B) protected
 - C) public
 - D) Package-private
 - **Answer: D) Package-private**
5. Which access modifier can be applied to top-level classes?
 - A) private
 - B) protected
 - C) public
 - D) static
 - **Answer: C) public**
6. Which access modifier is most restrictive?
 - A) protected
 - B) public
 - C) private
 - D) Default (package-private)
 - **Answer: C) private**
7. Which access modifier is used to restrict access to members within the same package and subclasses only?
 - A) private

- B) protected
- C) public
- D) Default (package-private)
- **Answer: B) protected**

8. Can a protected member be accessed from a different package?

- A) Yes, by subclasses only
- B) No, it is package-private
- C) Yes, by any class in the package
- D) Yes, by any class in any package
- **Answer: A) Yes, by subclasses only**

9. What is the access level of a public member?

- A) Accessible only within the same class
- B) Accessible within the same package
- C) Accessible within the same package and subclasses
- D) Accessible from any class in any package
- **Answer: D) Accessible from any class in any package**

10. Can a private member of a class be accessed by its own methods?

- A) Yes, always
- B) No, methods cannot access private members
- C) Yes, but only from within static methods
- D) No, private members are restricted to constructors only
- **Answer: A) Yes, always**

Inheritance

1. Which keyword is used to inherit a class in Java?

- A) extend
- B) implements
- C) extends
- D) inherits
- **Answer: C) extends**

2. What is the base class for all classes in Java?

- A) Object
- B) Class
- C) Base
- D) Super
- **Answer: A) Object**

3. What is method overriding in Java?

- A) Changing the implementation of a method in the subclass that is already defined in the superclass
- B) Changing the name of a method in the subclass

- C) Creating a new method in the subclass
- D) Hiding the superclass method with a new method
- **Answer: A) Changing the implementation of a method in the subclass that is already defined in the superclass**

4. **What is the result if a subclass does not override a method from its superclass?**

- A) The subclass method will use the superclass method implementation
- B) The subclass method will throw an error
- C) The superclass method will be removed
- D) The subclass method will be ignored
- **Answer: A) The subclass method will use the superclass method implementation**

5. **Which keyword is used to call a constructor of the superclass from a subclass?**

- A) this()
- B) super()
- C) base()
- D) parent()
- **Answer: B) super()**

6. **Can a subclass access private members of its superclass?**

- A) Yes, directly
- B) No, private members are not accessible
- C) Yes, but only through public methods of the superclass
- D) Yes, but only if they are static
- **Answer: B) No, private members are not accessible**

7. **What is the main benefit of using inheritance in Java?**

- A) Code reusability
- B) Code security
- C) Faster execution
- D) Enhanced performance
- **Answer: A) Code reusability**

8. **What is multiple inheritance in Java?**

- A) Inheriting from multiple classes
- B) Inheriting from a single class
- C) Implementing multiple interfaces
- D) Using multiple constructors
- **Answer: C) Implementing multiple interfaces**

9. **Which of the following is not a type of inheritance in Java?**

- A) Single inheritance
- B) Multiple inheritance
- C) Multilevel inheritance
- D) Hierarchical inheritance

- **Answer: B) Multiple inheritance** (through classes; Java supports multiple inheritance through interfaces)

10. What is the primary difference between super() and this() in Java?

- A) super() is used to call a superclass constructor, while this() is used to call another constructor in the same class
- B) super() is used to call a method in the same class, while this() is used to call a method in the superclass
- C) super() is used to access private fields, while this() is used for public fields
- D) super() and this() serve the same purpose
- **Answer: A) super() is used to call a superclass constructor, while this() is used to call another constructor in the same class**

Method Overloading

1. What is method overloading in Java?

- A) Defining multiple methods with the same name but different parameters
- B) Defining multiple methods with the same name and same parameters
- C) Overriding a method in the subclass
- D) Hiding a method in the superclass
- **Answer: A) Defining multiple methods with the same name but different parameters**

2. Which of the following is an example of method overloading?

- A) void method(int x) { } void method(int x, int y) { }
- B) void method(int x) { } void method() { }
- C) void method() { } void method(int x) { }
- D) void method(int x) { } void method(int x) { }
- **Answer: A) void method(int x) { } void method(int x, int y) { }**

3. What is required for method overloading to occur?

- A) Methods must have different names
- B) Methods must have different return types
- C) Methods must have different parameter lists
- D) Methods must be in different classes
- **Answer: C) Methods must have different parameter lists**

4. Can constructors be overloaded in Java?

- A) No, constructors cannot be overloaded
- B) Yes, constructors can be overloaded
- C) Yes, but only within the same class
- D) Yes, but only with different return types
- **Answer: B) Yes, constructors can be overloaded**

5. Is it possible to overload a method based on its return type alone?

- A) Yes, overloading is possible based only on return type
- B) No, return type alone is not sufficient for overloading

- C) Yes, but only if the return type is primitive
- D) Yes, but only if the method is static
- **Answer: B) No, return type alone is not sufficient for overloading**

6. **What happens if two overloaded methods have the same name and parameter list but different return types?**

- A) It results in a compile-time error
- B) The method with the different return type will be selected at runtime
- C) The method with the same return type will be selected
- D) The method with the most recent definition will be selected
- **Answer: A) It results in a compile-time error**

7. **What is an example of method overloading in Java?**

- A) `int add(int a, int b) { return a + b; }` `int add(double a, double b) { return a + b; }`
- B) `void display() { }` `void display(String s) { }` `void display(int x) { }`
- C) `void calculate(int x) { }` `void calculate(int x, int y) { }` `void calculate() { }`
- D) `void print() { }` `void print(int x) { }` `void print(int x, int y) { }`
- **Answer: B) `void display() { }` `void display(String s) { }` `void display(int x) { }`**

8. **Can you overload a method in Java by changing only the method's name?**

- A) Yes, it is called method overloading
- B) No, method overloading requires changing parameters
- C) Yes, but only in different classes
- D) No, method overloading is not possible
- **Answer: B) No, method overloading requires changing parameters**

9. **How does Java distinguish between overloaded methods?**

- A) By their return type
- B) By the method name only
- C) By the number and type of parameters
- D) By the access modifier
- **Answer: C) By the number and type of parameters**

10. **Can method overloading be applied to static methods?**

- A) Yes, static methods can be overloaded
- B) No, only instance methods can be overloaded
- C) Yes, but only if they have different return types
- D) No, static methods cannot be overloaded
- **Answer: A) Yes, static methods can be overloaded**

Polymorphism

1. **What is polymorphism in Java?**

- A) The ability of a single method to perform different tasks based on input
- B) The ability of different methods to have the same name but different implementations

- C) The ability of objects to take on multiple forms
- D) The ability to extend a class
- **Answer: C) The ability of objects to take on multiple forms**

2. Which type of polymorphism is achieved through method overriding?

- A) Compile-time polymorphism
- B) Runtime polymorphism
- C) Static polymorphism
- D) Dynamic polymorphism
- **Answer: B) Runtime polymorphism**

3. Which type of polymorphism is achieved through method overloading?

- A) Compile-time polymorphism
- B) Runtime polymorphism
- C) Static polymorphism
- D) Dynamic polymorphism
- **Answer: A) Compile-time polymorphism**

4. What is an example of polymorphism in Java?

- A) Overloading methods
- B) Creating multiple objects from the same class
- C) Using the same method name in different classes
- D) Overriding methods
- **Answer: D) Overriding methods**

5. How does polymorphism benefit object-oriented programming?

- A) It allows methods to be defined multiple times
- B) It enhances code readability and reusability
- C) It restricts access to classes and methods
- D) It optimizes memory usage
- **Answer: B) It enhances code readability and reusability**

6. Can a superclass reference variable hold a subclass object?

- A) Yes, it can
- B) No, it cannot
- C) Yes, but only if the subclass has no additional methods
- D) Yes, but only if the superclass and subclass are in the same package
- **Answer: A) Yes, it can**

7. Which keyword is used to achieve runtime polymorphism in Java?

- A) super
- B) this
- C) static
- D) virtual
- **Answer: A) super**

8. What is the result of calling a method through a superclass reference that points

to a subclass object?

- A) The method from the superclass will be called
- B) The method from the subclass will be called
- C) The program will throw an error
- D) The method call will be ignored
- **Answer: B) The method from the subclass will be called**

9. Which type of polymorphism is implemented using interfaces?

- A) Compile-time polymorphism
- B) Runtime polymorphism
- C) Static polymorphism
- D) Dynamic polymorphism
- **Answer: B) Runtime polymorphism**

10. What is the output of the following code snippet if Base and Derived are classes with overridden methods?

```
Base obj = new Derived();
obj.display();
```

- A) The display() method in Base class will be called
- B) The display() method in Derived class will be called
- C) A compile-time error will occur
- D) The program will throw a runtime exception
- **Answer: B) The display() method in Derived class will be called**

Packages and Interfaces

1. What is the purpose of packages in Java?

- A) To group related classes and interfaces
- B) To create multiple instances of a class
- C) To define a new class
- D) To handle exceptions
- **Answer: A) To group related classes and interfaces**

2. Which keyword is used to define a package in Java?

- A) import
- B) package
- C) class
- D) define
- **Answer: B) package**

3. How do you import a class from another package?

- A) include
- B) import
- C) require

- D) use
 - **Answer: B) import**
4. **What is the default access level for a class in a package without any access modifiers?**
- A) public
 - B) protected
 - C) private
 - D) Package-private (default)
 - **Answer: D) Package-private (default)**
5. **Which keyword is used to define an interface in Java?**
- A) class
 - B) interface
 - C) implements
 - D) extends
 - **Answer: B) interface**
6. **How do you implement an interface in a class?**
- A) class ClassName extends InterfaceName
 - B) class ClassName implements InterfaceName
 - C) class ClassName uses InterfaceName
 - D) class ClassName inherits InterfaceName
 - **Answer: B) class ClassName implements InterfaceName**
7. **What must a class do if it implements an interface with abstract methods?**
- A) It must implement all abstract methods of the interface
 - B) It must extend another class
 - C) It must be declared abstract
 - D) It must include a main method
 - **Answer: A) It must implement all abstract methods of the interface**
8. **Can an interface in Java extend another interface?**
- A) Yes, an interface can extend another interface
 - B) No, interfaces cannot extend other interfaces
 - C) Yes, but only if it is a functional interface
 - D) No, interfaces must be implemented directly
 - **Answer: A) Yes, an interface can extend another interface**
9. **What is the primary use of the import statement in Java?**
- A) To include classes from other packages
 - B) To define a new class
 - C) To create an object
 - D) To handle exceptions
 - **Answer: A) To include classes from other packages**
10. **Which access modifier is not allowed for an interface in Java?**

- A) public
- B) protected
- C) private
- D) default
- **Answer: C) private**

Encapsulation

1. What is encapsulation in Java?

- A) The bundling of data with the methods that operate on that data
- B) The inheritance of properties from a superclass
- C) The ability to override methods in a subclass
- D) The process of creating multiple objects from the same class
- **Answer: A) The bundling of data with the methods that operate on that data**

2. Which access modifier is commonly used to achieve encapsulation?

- A) public
- B) protected
- C) private
- D) static
- **Answer: C) private**

3. What is the purpose of getters and setters in encapsulation?

- A) To provide access to private fields of a class
- B) To create new objects
- C) To handle exceptions
- D) To extend classes
- **Answer: A) To provide access to private fields of a class**

4. How does encapsulation improve code maintainability?

- A) By restricting direct access to fields and methods
- B) By increasing the complexity of code
- C) By allowing multiple inheritance
- D) By providing direct access to all methods
- **Answer: A) By restricting direct access to fields and methods**

5. Which of the following is an example of encapsulation?

- A) Using private fields and providing public getter and setter methods
- B) Extending a class and overriding its methods
- C) Implementing multiple interfaces
- D) Using method overloading
- **Answer: A) Using private fields and providing public getter and setter methods**

6. What is a common practice when using encapsulation in Java?

- A) Keeping fields public and methods private
- B) Keeping fields private and providing public get and set methods

- C) Making all methods public and fields protected
- D) Declaring all members as static
- **Answer: B) Keeping fields private and providing public get and set methods**

7. Why is encapsulation important for object-oriented design?

- A) It enhances data security and integrity
- B) It allows method overloading
- C) It supports multiple inheritance
- D) It simplifies exception handling
- **Answer: A) It enhances data security and integrity**

8. What does the private access modifier do?

- A) Restricts access to the class itself
- B) Allows access only within the same package
- C) Allows access from any class
- D) Allows access from the same package and subclasses
- **Answer: A) Restricts access to the class itself**

9. What is the benefit of using public getter and setter methods for private fields?

- A) It allows controlled access to private fields
- B) It improves performance of the code
- C) It simplifies method overriding
- D) It prevents the use of inheritance
- **Answer: A) It allows controlled access to private fields**

10. Which statement is true about encapsulation?

- A) Encapsulation makes it easier to change the implementation without affecting other parts of the code
- B) Encapsulation requires all methods to be public
- C) Encapsulation is achieved by making all fields and methods static
- D) Encapsulation prevents method overloading
- **Answer: A) Encapsulation makes it easier to change the implementation without affecting other parts of the code**

Java Final Keyword

1. What does the final keyword do when applied to a variable?

- A) It makes the variable immutable
- B) It makes the variable static
- C) It makes the variable volatile
- D) It makes the variable transient
- **Answer: A) It makes the variable immutable**

2. What does the final keyword do when applied to a method?

- A) It prevents the method from being overridden
- B) It makes the method abstract
- C) It allows the method to be static

- D) It makes the method synchronized
- **Answer: A) It prevents the method from being overridden**

3. **What does the final keyword do when applied to a class?**

- A) It prevents the class from being subclassed
- B) It makes the class abstract
- C) It makes the class static
- D) It allows the class to be instantiated only once
- **Answer: A) It prevents the class from being subclassed**

4. **Can a final class be subclassed in Java?**

- A) No, a final class cannot be subclassed
- B) Yes, but only by using the abstract keyword
- C) Yes, but only by using the static keyword
- D) Yes, but only by using reflection
- **Answer: A) No, a final class cannot be subclassed**

5. **What happens if you try to override a final method in a subclass?**

- A) A compile-time error occurs
- B) The method will be overridden successfully
- C) The method will be hidden
- D) The method will be ignored
- **Answer: A) A compile-time error occurs**

6. **Can you assign a new value to a final variable once it has been initialized?**

- A) No, a final variable cannot be reassigned
- B) Yes, but only if it is declared as static
- C) Yes, but only in the same method
- D) Yes, but only if it is a local variable
- **Answer: A) No, a final variable cannot be reassigned**

7. **Can a final method be used as an implementation for an abstract method?**

- A) Yes, a final method can implement an abstract method
- B) No, an abstract method must be overridden
- C) Yes, but only if the method is also static
- D) No, a final method cannot be abstract
- **Answer: A) Yes, a final method can implement an abstract method**

8. **What is the effect of using the final keyword with a parameter in a method?**

- A) The parameter cannot be modified within the method
- B) The parameter becomes static
- C) The parameter becomes transient
- D) The parameter can be reassigned
- **Answer: A) The parameter cannot be modified within the method**

9. **What does the final keyword do when used with a local variable in a method?**

- A) The variable's value cannot be changed after initialization

- B) The variable becomes an instance variable
- C) The variable becomes static
- D) The variable is automatically synchronized
- **Answer: A) The variable's value cannot be changed after initialization**

10. Can you declare a constructor as final in Java?

- A) No, constructors cannot be declared final
- B) Yes, but only if it is a static constructor
- C) Yes, but only if it is an abstract constructor
- D) Yes, but only if it is in an interface
- **Answer: A) No, constructors cannot be declared final**

Java Static Keyword

1. What does the static keyword signify in Java?

- A) A member belongs to the class rather than instances of the class
- B) A member can only be accessed from within the class
- C) A member can only be accessed through instances of the class
- D) A member is not thread-safe
- **Answer: A) A member belongs to the class rather than instances of the class**

2. Which of the following is a valid use of the static keyword?

- A) Defining static methods
- B) Defining static variables
- C) Defining static blocks
- D) All of the above
- **Answer: D) All of the above**

3. What is the purpose of a static block in Java?

- A) To initialize static variables
- B) To initialize instance variables
- C) To define static methods
- D) To override methods
- **Answer: A) To initialize static variables**

4. Can a static method access non-static (instance) variables of a class?

- A) No, static methods cannot access non-static variables directly
- B) Yes, if the non-static variables are declared public
- C) Yes, if the non-static variables are declared protected
- D) Yes, if the non-static variables are declared static
- **Answer: A) No, static methods cannot access non-static variables directly**

5. Can you override a static method in Java?

- A) No, static methods cannot be overridden
- B) Yes, static methods can be overridden
- C) Yes, but only with a different return type

- D) Yes, but only if the static method is abstract
 - **Answer: A) No, static methods cannot be overridden**
6. **What will happen if you access a static variable using an instance of a class?**
- A) It will access the class variable, but using an instance is not recommended
 - B) It will create a new instance variable
 - C) It will cause a compile-time error
 - D) It will not compile
 - **Answer: A) It will access the class variable, but using an instance is not recommended**
7. **When is a static block executed?**
- A) When the class is first loaded into memory
 - B) When an object of the class is created
 - C) When a static method is called
 - D) When an instance method is called
 - **Answer: A) When the class is first loaded into memory**
8. **Can you access a static method from an instance method?**
- A) Yes, static methods can be accessed from instance methods
 - B) No, static methods cannot be accessed from instance methods
 - C) Yes, but only if the instance method is also static
 - D) No, unless the instance method is declared final
 - **Answer: A) Yes, static methods can be accessed from instance methods**
9. **What happens if you try to call a static method using an instance reference?**
- A) The static method will still be called correctly
 - B) A compile-time error will occur
 - C) The method will be ignored
 - D) The method will be overridden
 - **Answer: A) The static method will still be called correctly**
10. **Which of the following can be declared static in Java?**
- A) Methods
 - B) Variables
 - C) Blocks
 - D) All of the above
 - **Answer: D) All of the above**

Java Abstract Keyword

1. **What does the abstract keyword define in Java?**
- A) A class that cannot be instantiated
 - B) A method that must be implemented by subclasses
 - C) A variable that cannot be changed
 - D) A block of code that is executed first

- **Answer: A) A class that cannot be instantiated**

2. Can an abstract class have constructors?

- A) Yes, abstract classes can have constructors
- B) No, abstract classes cannot have constructors
- C) Yes, but only if they are private
- D) Yes, but only if they are static
- **Answer: A) Yes, abstract classes can have constructors**

3. Can an abstract class contain non-abstract methods?

- A) Yes, an abstract class can have both abstract and non-abstract methods
- B) No, an abstract class can only have abstract methods
- C) Yes, but non-abstract methods must be declared final
- D) No, all methods in an abstract class must be abstract
- **Answer: A) Yes, an abstract class can have both abstract and non-abstract methods**

4. Can an abstract class be subclassed?

- A) Yes, an abstract class must be subclassed
- B) No, an abstract class cannot be subclassed
- C) Yes, but only if it is also declared final
- D) Yes, but only if it contains at least one abstract method
- **Answer: A) Yes, an abstract class must be subclassed**

5. What happens if a subclass does not implement all abstract methods of an abstract class?

- A) The subclass must be declared abstract
- B) The subclass will compile successfully
- C) A runtime exception will occur
- D) The code will not compile
- **Answer: A) The subclass must be declared abstract**

6. Can you declare an interface as abstract in Java?

- A) No, interfaces are inherently abstract
- B) Yes, but only if it extends another interface
- C) Yes, but only if it contains abstract methods
- D) No, interfaces cannot be abstract
- **Answer: A) No, interfaces are inherently abstract**

7. What is the effect of declaring a method as abstract in a class?

- A) The method must be implemented by subclasses
- B) The method must be static
- C) The method can be overridden in any subclass
- D) The method must be final
- **Answer: A) The method must be implemented by subclasses**

8. Can an abstract class have instance variables?

- A) Yes, an abstract class can have instance variables
- B) No, abstract classes cannot have instance variables
- C) Yes, but only if the variables are final
- D) No, instance variables must be static
- **Answer: A) Yes, an abstract class can have instance variables**

9. Can an abstract class implement an interface?

- A) Yes, an abstract class can implement an interface
- B) No, abstract classes cannot implement interfaces
- C) Yes, but only if the interface has abstract methods
- D) No, only concrete classes can implement interfaces
- **Answer: A) Yes, an abstract class can implement an interface**

10. Can a method in an abstract class be static?

- A) Yes, an abstract class can have static methods
- B) No, abstract classes cannot have static methods
- C) Yes, but only if the method is also final
- D) No, static methods are not allowed in abstract classes
- **Answer: A) Yes, an abstract class can have static methods**

Interface

1. What is an interface in Java?

- A) A reference type that can contain constants, method signatures, default methods, and static methods
- B) A class that cannot be instantiated
- C) A special type of method that must be overridden
- D) A variable that can be accessed from any class
- **Answer: A) A reference type that can contain constants, method signatures, default methods, and static methods**

2. Can an interface extend multiple interfaces in Java?

- A) Yes, an interface can extend multiple interfaces
- B) No, an interface can only extend one interface
- C) Yes, but only if the interfaces are not abstract
- D) No, extending multiple interfaces is not allowed
- **Answer: A) Yes, an interface can extend multiple interfaces**

3. What is the default access level for methods in an interface?

- A) public
- B) protected
- C) private
- D) Package-private (default)
- **Answer: A) public**

4. Can you instantiate an interface in Java?

- A) No, interfaces cannot be instantiated
- B) Yes, but only if the interface is abstract
- C) Yes, but only if the interface is concrete
- D) Yes, but only using a static factory method
- **Answer: A) No, interfaces cannot be instantiated**

5. Can an interface contain concrete methods?

- A) Yes, interfaces can have concrete methods using default methods
- B) No, interfaces can only have abstract methods
- C) Yes, but only if the interface is abstract
- D) No, concrete methods are not allowed in interfaces
- **Answer: A) Yes, interfaces can have concrete methods using default methods**

6. What is the purpose of default methods in an interface?

- A) To provide default implementations of methods in interfaces
- B) To define abstract methods
- C) To create static variables
- D) To override methods in a class
- **Answer: A) To provide default implementations of methods in interfaces**

7. How do you implement multiple interfaces in a class?

- A) By separating them with commas in the implements clause
- B) By using multiple inheritance
- C) By extending each interface
- D) By creating a separate class for each interface
- **Answer: A) By separating them with commas in the implements clause**

8. Can an interface have constructors in Java?

- A) No, interfaces cannot have constructors
- B) Yes, but only if they are static
- C) Yes, but only if they are abstract
- D) Yes, but only if they are public
- **Answer: A) No, interfaces cannot have constructors**

9. Can a class implement multiple interfaces with the same method signatures?

- A) Yes, a class can implement multiple interfaces with the same method signatures
- B) No, a class cannot implement multiple interfaces with the same method signatures
- C) Yes, but only if the methods are abstract
- D) No, the class must use different method signatures
- **Answer: A) Yes, a class can implement multiple interfaces with the same method signatures**

10. What will happen if a class implements multiple interfaces with conflicting default methods?

- A) The class must override the conflicting methods
- B) The class will compile successfully without any errors
- C) The class can choose any of the conflicting methods to use
- D) The class will use the methods from the interface that is listed first
- **Answer: A) The class must override the conflicting methods**

Abstraction

1. What is abstraction in Java?

- A) The process of hiding implementation details and showing only functionality
- B) The process of creating new classes from existing ones
- C) The process of inheriting methods from a superclass
- D) The process of defining static variables
- **Answer: A) The process of hiding implementation details and showing only functionality**

2. How can you achieve abstraction in Java?

- A) By using abstract classes and interfaces
- B) By using only concrete classes
- C) By declaring all methods as static
- D) By using multiple inheritance
- **Answer: A) By using abstract classes and interfaces**

3. What is an abstract class in Java?

- A) A class that cannot be instantiated and may contain abstract methods
- B) A class that can be instantiated but cannot have methods
- C) A class that contains only static methods
- D) A class that is used only for inheritance
- **Answer: A) A class that cannot be instantiated and may contain abstract methods**

4. Can an abstract class have constructors?

- A) Yes, abstract classes can have constructors
- B) No, abstract classes cannot have constructors
- C) Yes, but only if they are private
- D) Yes, but only if they are static
- **Answer: A) Yes, abstract classes can have constructors**

5. What is an abstract method in Java?

- A) A method that does not have an implementation and must be overridden by subclasses
- B) A method that has a default implementation
- C) A method that is private and cannot be accessed
- D) A method that can be static
- **Answer: A) A method that does not have an implementation and must be overridden by subclasses**

6. Can an abstract class contain non-abstract methods?

- A) Yes, an abstract class can have both abstract and non-abstract methods
- B) No, an abstract class can only contain abstract methods
- C) Yes, but non-abstract methods must be final
- D) No, all methods in an abstract class must be abstract
- **Answer: A) Yes, an abstract class can have both abstract and non-abstract methods**

7. What is the difference between an abstract class and an interface?

- A) Abstract classes can have constructors and instance variables, while interfaces cannot
- B) Interfaces can have constructors, while abstract classes cannot
- C) Abstract classes can be instantiated, while interfaces cannot
- D) Interfaces can contain non-abstract methods, while abstract classes cannot
- **Answer: A) Abstract classes can have constructors and instance variables, while interfaces cannot**

8. Can a class be both abstract and implement an interface?

- A) Yes, a class can be abstract and implement one or more interfaces
- B) No, a class cannot be both abstract and implement an interface
- C) Yes, but only if the interface is also abstract
- D) No, an abstract class cannot implement an interface
- **Answer: A) Yes, a class can be abstract and implement one or more interfaces**

9. What must a concrete subclass do if it extends an abstract class?

- A) It must provide implementations for all abstract methods of the superclass
- B) It must declare itself as abstract
- C) It must include the final keyword in its method implementations
- D) It must override all non-abstract methods of the superclass
- **Answer: A) It must provide implementations for all abstract methods of the superclass**

10. Can an abstract class extend another abstract class?

- A) Yes, an abstract class can extend another abstract class
- B) No, an abstract class cannot extend another abstract class
- C) Yes, but only if the parent abstract class has no abstract methods
- D) No, only concrete classes can extend abstract classes
- **Answer: A) Yes, an abstract class can extend another abstract class**

Class Loading and toString() Method

1. What is class loading in Java?

- A) The process of loading a class into memory during runtime
- B) The process of creating an instance of a class

- C) The process of defining a class
- D) The process of importing a class
- **Answer: A) The process of loading a class into memory during runtime**

2. Which method is used to load a class into memory manually?

- A) Class.forName()
- B) Class.load()
- C) Class.getInstance()
- D) Class.initialize()
- **Answer: A) Class.forName()**

3. What is the purpose of the toString() method in Java?

- A) To provide a string representation of an object
- B) To convert a string to an object
- C) To compare two strings
- D) To concatenate strings
- **Answer: A) To provide a string representation of an object**

4. Which class does the toString() method belong to?

- A) Object
- B) String
- C) Class
- D) System
- **Answer: A) Object**

5. What happens if you do not override the toString() method in a class?

- A) The toString() method of the Object class will be used, which returns a string containing the class name and hash code
- B) A compile-time error will occur
- C) The program will not compile
- D) The object will not be printed
- **Answer: A) The toString() method of the Object class will be used, which returns a string containing the class name and hash code**

6. Can the toString() method be overridden in a subclass?

- A) Yes, toString() can be overridden to provide a custom string representation
- B) No, toString() cannot be overridden
- C) Yes, but only if the method is declared final
- D) No, toString() is an abstract method
- **Answer: A) Yes, toString() can be overridden to provide a custom string representation**

7. What is the output of System.out.println(object) if object is an instance of a class with a custom toString() method?

- A) The custom string representation provided by the toString() method
- B) The default string representation provided by the Object class

- C) The class name and memory address of the object
- D) An empty string
- **Answer: A) The custom string representation provided by the toString() method**

8. What is the default behavior of the toString() method in the Object class?

- A) Returns a string consisting of the class name followed by the @ character and the object's hash code
- B) Returns the string representation of the object's fields
- C) Returns the string representation of the object's methods
- D) Returns an empty string
- **Answer: A) Returns a string consisting of the class name followed by the @ character and the object's hash code**

9. Can the toString() method return null?

- A) No, the toString() method should return a non-null string
- B) Yes, toString() can return null if explicitly set
- C) Yes, but only if the class is abstract
- D) No, toString() will always return a default value
- **Answer: A) No, the toString() method should return a non-null string**

10. When is the toString() method typically called automatically?

- A) When printing an object using System.out.println() or string concatenation
- B) When creating an instance of a class
- C) When loading a class
- D) When calling an instance method
- **Answer: A) When printing an object using System.out.println() or string concatenation**

Garbage Collection

1. What is the purpose of garbage collection in Java?

- A) To automatically reclaim memory used by objects that are no longer referenced
- B) To manually allocate memory to objects
- C) To optimize code execution speed
- D) To provide additional security to the program
- **Answer: A) To automatically reclaim memory used by objects that are no longer referenced**

2. Which method is used to request garbage collection explicitly?

- A) System.gc()
- B) Runtime.gc()
- C) GarbageCollector.run()
- D) MemoryManager.collect()

- **Answer: A) System.gc()**

3. What is the finalize() method used for in Java?

- A) To perform cleanup operations before an object is garbage collected
- B) To define the final implementation of a method
- C) To declare a class as final
- D) To prevent a method from being overridden
- **Answer: A) To perform cleanup operations before an object is garbage collected**

4. When is the finalize() method called?

- A) Before an object is removed from memory by the garbage collector
- B) Immediately after an object is created
- C) During the object instantiation process
- D) When the class is loaded
- **Answer: A) Before an object is removed from memory by the garbage collector**

5. What happens if you do not override the finalize() method in a class?

- A) The default implementation of finalize() in the Object class will be used
- B) The program will throw an exception
- C) The object will not be garbage collected
- D) The finalize() method will be skipped
- **Answer: A) The default implementation of finalize() in the Object class will be used**

6. Can you rely on finalize() to release critical resources such as file handles or network connections?

- A) No, finalize() is not guaranteed to be called promptly or at all
- B) Yes, finalize() is always called for resource cleanup
- C) Yes, but only if finalize() is declared public
- D) No, resources should be released manually
- **Answer: A) No, finalize() is not guaranteed to be called promptly or at all**

7. What is the impact of calling System.gc() on garbage collection?

- A) It suggests to the JVM that garbage collection should occur, but does not guarantee it
- B) It forces immediate garbage collection
- C) It disables garbage collection
- D) It optimizes garbage collection performance
- **Answer: A) It suggests to the JVM that garbage collection should occur, but does not guarantee it**

8. What is a memory leak in the context of Java garbage collection?

- A) When an application retains references to objects that are no longer needed
- B) When the garbage collector fails to reclaim memory

- C) When an object is not instantiated correctly
- D) When memory allocation exceeds the maximum limit
- **Answer: A) When an application retains references to objects that are no longer needed**

9. What type of objects are eligible for garbage collection?

- A) Objects that are no longer reachable from any live thread or static reference
- B) Objects that are referenced by static variables
- C) Objects that are marked as final
- D) Objects that are actively used in the application
- **Answer: A) Objects that are no longer reachable from any live thread or static reference**

10. Which of the following is a common garbage collection algorithm in Java?

- A) Mark-and-Sweep
- B) Quick Sort
- C) Merge Sort
- D) Binary Search
- **Answer: A) Mark-and-Sweep**

Design Patterns

1. What is a design pattern in software engineering?

- A) A general reusable solution to a commonly occurring problem
- B) A specific piece of code that solves a problem
- C) A type of software development methodology
- D) A set of rules for writing code
- **Answer: A) A general reusable solution to a commonly occurring problem**

2. Which design pattern is used to create objects without specifying the exact class of the object that will be created?

- A) Factory Method
- B) Singleton
- C) Observer
- D) Decorator
- **Answer: A) Factory Method**

3. What is the primary purpose of the Singleton design pattern?

- A) To ensure a class has only one instance and provide a global point of access to it
- B) To allow a class to be instantiated multiple times
- C) To enable multiple inheritance
- D) To create objects in a factory method
- **Answer: A) To ensure a class has only one instance and provide a global point of access to it**

4. Which design pattern is used to allow an object to alter its behavior when its internal state changes?
- A) State
 - B) Strategy
 - C) Proxy
 - D) Chain of Responsibility
 - **Answer: A) State**
5. What does the Observer design pattern achieve?
- A) It allows a subject to notify its observers about changes in its state
 - B) It controls the creation of a single instance of a class
 - C) It defines a family of algorithms, encapsulates each one, and makes them interchangeable
 - D) It provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation
 - **Answer: A) It allows a subject to notify its observers about changes in its state**
6. In which design pattern is a new class created by copying an existing object, and modifying it as needed?
- A) Prototype
 - B) Builder
 - C) Adapter
 - D) Composite
 - **Answer: A) Prototype**
7. Which design pattern provides a way to use an existing object as a surrogate or placeholder for another object?
- A) Proxy
 - B) Bridge
 - C) Flyweight
 - D) Mediator
 - **Answer: A) Proxy**
8. What is the primary goal of the Strategy design pattern?
- A) To define a family of algorithms, encapsulate each one, and make them interchangeable
 - B) To allow a class to be extended by modifying its behavior
 - C) To attach additional responsibilities to an object dynamically
 - D) To separate the construction of a complex object from its representation
 - **Answer: A) To define a family of algorithms, encapsulate each one, and make them interchangeable**
9. Which design pattern is used to decouple an abstraction from its implementation so that the two can vary independently?

- A) Bridge
- B) Composite
- C) Decorator
- D) Builder
- **Answer: A) Bridge**

10. What is the purpose of the Composite design pattern?

- A) To allow clients to treat individual objects and compositions of objects uniformly
- B) To encapsulate the construction of complex objects
- C) To provide a way to access the elements of an aggregate object sequentially
- D) To create a single instance of a class

Answer: A) To allow clients to treat individual objects and compositions of objects uniformly

Wrapper Classes

1. What is the purpose of wrapper classes in Java?

- A) To convert primitive types into objects
- B) To create multiple instances of primitive types
- C) To define new primitive types
- D) To enhance performance of primitive types
- **Answer: A) To convert primitive types into objects**

2. Which wrapper class is used for the int primitive type?

- A) Integer
- B) Double
- C) Character
- D) Boolean
- **Answer: A) Integer**

3. How do you obtain the primitive value from a wrapper class object?

- A) By using the xxxValue() method, where xxx is the primitive type
- B) By using the getPrimitive() method
- C) By calling the constructor of the primitive type
- D) By casting the wrapper class object to the primitive type
- **Answer: A) By using the xxxValue() method, where xxx is the primitive type**

4. Which method is used to convert a string to an integer using the Integer wrapper class?

- A) parseInt()
- B) valueOf()
- C) toInteger()
- D) convertToInt()
- **Answer: A) parseInt()**

5. What is the purpose of the valueOf() method in wrapper classes?

- A) To convert a string or primitive value to its corresponding wrapper class object
- B) To convert the wrapper class object to a primitive type
- C) To compare two wrapper class objects
- D) To create a new instance of the primitive type
- **Answer: A) To convert a string or primitive value to its corresponding wrapper class object**

6. Which wrapper class is used for the boolean primitive type?

- A) Boolean
- B) Character
- C) Float
- D) Long
- **Answer: A) Boolean**

7. Can wrapper class objects be null?

- A) Yes, wrapper class objects can be null
- B) No, wrapper class objects cannot be null
- C) Yes, but only for Integer and Double
- D) No, null is not a valid value for any wrapper class
- **Answer: A) Yes, wrapper class objects can be null**

8. How does autoboxing work in Java?

- A) It automatically converts a primitive type to its corresponding wrapper class object
- B) It manually converts a wrapper class object to a primitive type
- C) It allows primitive types to be used as objects
- D) It creates new wrapper class objects for each primitive value
- **Answer: A) It automatically converts a primitive type to its corresponding wrapper class object**

9. Which wrapper class is used for the double primitive type?

- A) Double
- B) Float
- C) Long
- D) Short
- **Answer: A) Double**

10. What method is used to convert a Double object to a String?

- A) toString()
- B) valueOf()
- C) convertToString()
- D) getString()
- **Answer: A) toString()**

Exception Handling

1. What is the purpose of exception handling in Java?

- A) To handle runtime errors and ensure the normal flow of the program
- B) To prevent compilation errors
- C) To improve the performance of the program
- D) To manage memory usage
- **Answer: A) To handle runtime errors and ensure the normal flow of the program**

2. Which keyword is used to handle exceptions in Java?

- A) try
- B) catch
- C) finally
- D) All of the above
- **Answer: D) All of the above**

3. What is the purpose of the try block in exception handling?

- A) To contain code that may throw an exception
- B) To define custom exception classes
- C) To catch exceptions thrown by the code
- D) To finalize the exception handling process
- **Answer: A) To contain code that may throw an exception**

4. Which block is used to handle exceptions thrown by the try block?

- A) catch
- B) finally
- C) throw
- D) default
- **Answer: A) catch**

5. What is the purpose of the finally block in exception handling?

- A) To execute code regardless of whether an exception was thrown or not
- B) To handle exceptions of specific types
- C) To define the exception types to be caught
- D) To throw exceptions explicitly
- **Answer: A) To execute code regardless of whether an exception was thrown or not**

6. Which keyword is used to throw an exception explicitly?

- A) throw
- B) throws
- C) catch
- D) finally
- **Answer: A) throw**

7. What is the difference between throw and throws in Java?

- A) throw is used to explicitly throw an exception, while throws is used to declare exceptions that a method can throw
- B) throws is used to throw an exception, while throw is used to declare exceptions
- C) throw and throws are interchangeable and used for the same purpose
- D) throws is used within the try block, while throw is used outside of it
- **Answer: A) throw is used to explicitly throw an exception, while throws is used to declare exceptions that a method can throw**

8. Which of the following is true about checked exceptions?

- A) They must be either caught or declared in the method signature
- B) They do not need to be handled explicitly
- C) They are derived from RuntimeException
- D) They occur during the compile time
- **Answer: A) They must be either caught or declared in the method signature**

9. What is an unchecked exception?

- A) An exception that is derived from RuntimeException and does not need to be declared in the method signature
- B) An exception that must be handled using a try-catch block
- C) An exception that is only caught at runtime
- D) An exception that occurs during compile time
- **Answer: A) An exception that is derived from RuntimeException and does not need to be declared in the method signature**

10. How can you create a custom exception in Java?

- A) By extending the Exception class or its subclasses
- B) By implementing the Throwable interface
- C) By using the throw keyword
- D) By overriding the toString() method
- **Answer: A) By extending the Exception class or its subclasses**

Streams and File I/O**1. Which class is used to read bytes from a file in Java?**

- A) FileInputStream
- B) FileReader
- C) BufferedReader
- D) PrintWriter
- **Answer: A) FileInputStream**

2. Which class is used to write bytes to a file in Java?

- A) FileOutputStream
- B) FileWriter

- C) BufferedWriter
- D) PrintStream
- **Answer: A) FileOutputStream**

3. What does the BufferedReader class provide that FileReader does not?

- A) Buffered input to improve performance by reducing the number of read operations
- B) Ability to read bytes from a file
- C) The capability to write data to a file
- D) The ability to handle binary data
- **Answer: A) Buffered input to improve performance by reducing the number of read operations**

4. How can you write text to a file using PrintWriter?

- A) By using the println() or print() methods
- B) By using the write() method
- C) By using the append() method
- D) By using the flush() method
- **Answer: A) By using the println() or print() methods**

5. Which class is used to read character data from a file?

- A) FileReader
- B) FileInputStream
- C) BufferedInputStream
- D) DataInputStream
- **Answer: A) FileReader**

6. How can you ensure that a file is properly closed after reading or writing operations?

- A) By using the close() method in a finally block or using the try-with-resources statement
- B) By calling the flush() method
- C) By invoking the reset() method
- D) By using the clear() method
- **Answer: A) By using the close() method in a finally block or using the try-with-resources statement**

7. What is the purpose of the File class in Java I/O?

- A) To represent a file or directory path in the file system
- B) To read and write file contents
- C) To handle buffered input and output streams
- D) To serialize objects to files
- **Answer: A) To represent a file or directory path in the file system**

8. Which method of File is used to check if a file exists?

- A) exists()

- B) isFile()
- C) check()
- D) validate()
- **Answer: A) exists()**

9. What does the Serializable interface enable in Java?

- A) To serialize and deserialize objects for saving to or reading from a file
- B) To manage file I/O operations
- C) To handle text-based data streams
- D) To compress file data
- **Answer: A) To serialize and deserialize objects for saving to or reading from a file**

10. Which of the following is true about ObjectOutputStream and ObjectInputStream?

- A) They are used for serializing and deserializing objects respectively
- B) They handle text-based data streams
- C) They are used for writing and reading primitive data types
- D) They are used for buffering file input and output streams
- **Answer: A) They are used for serializing and deserializing objects respectively**

Java Collections Framework

1. Which interface is the root of the Java Collections Framework?

- A) Collection
- B) List
- C) Set
- D) Map
- **Answer: A) Collection**

2. Which class in the Collections Framework implements the List interface and is dynamically resizable?

- A) ArrayList
- B) LinkedList
- C) Vector
- D) HashSet
- **Answer: A) ArrayList**

3. What is the main difference between ArrayList and LinkedList?

- A) ArrayList provides fast random access, while LinkedList provides faster insertions and deletions
- B) ArrayList supports null values, while LinkedList does not
- C) ArrayList is synchronized, while LinkedList is not
- D) ArrayList uses a linked list for storage, while LinkedList uses an array

- **Answer: A) ArrayList** provides fast random access, while **LinkedList** provides faster insertions and deletions

4. Which interface represents a collection of unique elements?

- A) Set
- B) List
- C) Queue
- D) Map
- **Answer: A) Set**

5. What is the purpose of the HashSet class?

- A) To store unique elements and provide constant-time performance for basic operations
- B) To store elements in a sorted order
- C) To store elements with duplicate values
- D) To implement a stack data structure
- **Answer: A) To store unique elements and provide constant-time performance for basic operations**

6. Which collection class provides a last-in, first-out (LIFO) stack of objects?

- A) Stack
- B) ArrayDeque
- C) LinkedList
- D) PriorityQueue
- **Answer: A) Stack**

7. Which interface should be used to store key-value pairs?

- A) Map
- B) Set
- C) List
- D) Queue
- **Answer: A) Map**

8. What is the main difference between HashMap and TreeMap?

- A) HashMap does not guarantee order, while TreeMap maintains a sorted order based on keys
- B) HashMap is synchronized, while TreeMap is not
- C) HashMap stores keys in a linked list, while TreeMap stores keys in a binary tree
- D) HashMap allows null keys and values, while TreeMap does not
- **Answer: A) HashMap does not guarantee order, while TreeMap maintains a sorted order based on keys**

9. Which interface defines a collection that can be used to store and process a sequence of elements?

- A) Iterable

- B) Collection
- C) Map
- D) Set
- **Answer: B) Collection**

10. What is the purpose of the PriorityQueue class?

- A) To store elements in a priority order based on their natural ordering or a provided comparator
- B) To store elements in a first-in, first-out (FIFO) order
- C) To store unique elements in a hash table
- D) To maintain a stack of elements
- **Answer: A) To store elements in a priority order based on their natural ordering or a provided comparator**

Multithreading and Concurrency

1. Which interface should be implemented to create a thread in Java?

- A) Runnable
- B) Callable
- C) Thread
- D) Executor
- **Answer: A) Runnable**

2. What is the purpose of the Thread class in Java?

- A) To create and control threads
- B) To define task execution policies
- C) To manage thread synchronization
- D) To handle thread communication
- **Answer: A) To create and control threads**

3. How can you start a new thread in Java?

- A) By calling the start() method on an instance of the Thread class or a class implementing Runnable
- B) By calling the run() method directly
- C) By creating a new instance of the Runnable class
- D) By calling the execute() method on an ExecutorService
- **Answer: A) By calling the start() method on an instance of the Thread class or a class implementing Runnable**

4. What is the purpose of the synchronized keyword in Java?

- A) To control access to a block of code or method by multiple threads
- B) To create a new thread
- C) To ensure that a thread waits for a specific condition
- D) To define thread priorities
- **Answer: A) To control access to a block of code or method by multiple**

threads**5. What is a deadlock in multithreading?**

- A) A situation where two or more threads are blocked forever, waiting for each other to release resources
- B) A situation where a thread is not started correctly
- C) A condition where a thread is waiting for I/O operations
- D) A state where threads are executing too slowly
- **Answer: A) A situation where two or more threads are blocked forever, waiting for each other to release resources**

6. Which method is used to pause the execution of the current thread for a specified amount of time?

- A) sleep()
- B) wait()
- C) yield()
- D) suspend()
- **Answer: A) sleep()**

7. What is the purpose of the wait() method in Java?

- A) To make the current thread wait until another thread invokes notify() or notifyAll() on the same object
- B) To pause the current thread for a specified time
- C) To yield the CPU to other threads
- D) To terminate the current thread
- **Answer: A) To make the current thread wait until another thread invokes notify() or notifyAll() on the same object**

8. What is the purpose of the notify() method in Java?

- A) To wake up a single thread that is waiting on the object's monitor
- B) To suspend the current thread's execution
- C) To change the thread's priority
- D) To create a new thread
- **Answer: A) To wake up a single thread that is waiting on the object's monitor**

9. Which class provides a way to manage a pool of threads for executing tasks asynchronously?

- A) ExecutorService
- B) ThreadPoolExecutor
- C) ScheduledExecutorService
- D) ForkJoinPool
- **Answer: A) ExecutorService**

10. What is the purpose of the Callable interface in Java?

- A) To define a task that can be executed by a thread and returns a result

- B) To create a new thread
- C) To handle thread synchronization
- D) To manage thread priorities
- **Answer: A) To define a task that can be executed by a thread and returns a result**

Java 8 Features

1. What is the purpose of lambda expressions in Java 8?

- A) To provide a clear and concise way to implement functional interfaces
- B) To create new classes
- C) To enhance the performance of Java applications
- D) To handle exceptions more effectively
- **Answer: A) To provide a clear and concise way to implement functional interfaces**

2. Which interface is used to represent a function that takes one argument and returns a result?

- A) Function<T, R>
- B) Consumer<T>
- C) Supplier<T>
- D) Predicate<T>
- **Answer: A) Function<T, R>**

3. What does the Stream API in Java 8 provide?

- A) A way to process sequences of elements in a functional style
- B) A mechanism to handle asynchronous tasks
- C) A new way to handle multithreading
- D) A method to serialize objects
- **Answer: A) A way to process sequences of elements in a functional style**

4. What is the purpose of the Optional class in Java 8?

- A) To provide a container that may or may not contain a value and avoid null references
- B) To optimize memory usage in collections
- C) To enhance the performance of loops
- D) To manage thread synchronization
- **Answer: A) To provide a container that may or may not contain a value and avoid null references**

5. Which method in Optional is used to retrieve the value if present, or a default value otherwise?

- A) orElse()
- B) get()
- C) ifPresent()

- D) isPresent()
- **Answer: A) orElse()**

6. What is a functional interface in Java?

- A) An interface with exactly one abstract method
- B) An interface with multiple abstract methods
- C) An interface that extends multiple other interfaces
- D) An interface with no methods
- **Answer: A) An interface with exactly one abstract method**

7. Which annotation is used to indicate that an interface is a functional interface?

- A) @FunctionalInterface
- B) @Interface
- C) @Functional
- D) @AbstractInterface
- **Answer: A) @FunctionalInterface**

8. How can you create a Stream from a collection in Java 8?

- A) By calling the stream() method on the collection
- B) By using the Stream.of() method
- C) By creating a new Stream object directly
- D) By using the Stream.generate() method
- **Answer: A) By calling the stream() method on the collection**

9. Which method in the Stream API is used to filter elements based on a condition?

- A) filter()
- B) map()
- C) reduce()
- D) collect()
- **Answer: A) filter()**

10. What is the purpose of the forEach() method in the Stream API?

- A) To perform an action for each element of the stream
- B) To filter elements based on a predicate
- C) To transform each element in the stream
- D) To sort the elements in the stream
- **Answer: A) To perform an action for each element of the stream**

11. Which method in the Stream API is used to sort the elements of the stream?

- A) sorted()
- B) order()
- C) arrange()
- D) sort()
- **Answer: A) sorted()**

12. What is the purpose of the map() method in the Stream API?

- A) To transform each element in the stream based on a provided function

- B) To filter elements based on a condition
- C) To aggregate elements in the stream
- D) To sort elements in the stream
- **Answer: A) To transform each element in the stream based on a provided function**

13. How can you collect the results of a stream into a List?

- A) By using the collect() method with Collectors.toList()
- B) By using the toList() method
- C) By calling the List() constructor directly on the stream
- D) By using the arrayList() method
- **Answer: A) By using the collect() method with Collectors.toList()**

14. Which method in the Stream API is used to combine elements into a single result?

- A) reduce()
- B) aggregate()
- C) combine()
- D) collect()
- **Answer: A) reduce()**

15. What is the purpose of the flatMap() method in the Stream API?

- A) To flatten nested streams into a single stream
- B) To map each element to a new value
- C) To filter elements based on a predicate
- D) To sort elements in the stream
- **Answer: A) To flatten nested streams into a single stream**

16. Which functional interface is used to represent a function that takes two arguments and returns a result?

- A) BiFunction<T, U, R>
- B) Function<T, R>
- C) Consumer<T>
- D) Supplier<T>
- **Answer: A) BiFunction<T, U, R>**

17. What is the use of the Predicate interface in Java 8?

- A) To represent a function that returns a boolean value based on a condition
- B) To represent a function that produces a result
- C) To represent a function that takes an argument and does not return a result
- D) To represent a supplier of results
- **Answer: A) To represent a function that returns a boolean value based on a condition**

18. Which method of the Optional class is used to execute a block of code if a value is present?

- A) ifPresent()

- B) ifAbsent()
- C) orElse()
- D) map()
- **Answer: A) ifPresent()**

19. What does the Stream.of() method do in Java 8?

- A) Creates a stream from the provided elements
- B) Converts a collection into a stream
- C) Generates a stream of infinite elements
- D) Filters elements from an existing stream
- **Answer: A) Creates a stream from the provided elements**

20. Which method is used to remove duplicates from a stream?

- A) distinct()
- B) unique()
- C) removeDuplicates()
- D) filterDuplicates()
- **Answer: A) distinct()**

21. Which method of Stream is used to check if any elements match a given predicate?

- A) anyMatch()
- B) allMatch()
- C) noneMatch()
- D) filter()

Answer: A) anyMatch()

22. What is the purpose of the allMatch() method in the Stream API?

- A) To check if all elements in the stream match a given predicate
- B) To check if any element in the stream matches a given predicate
- C) To filter elements based on a predicate
- D) To transform elements based on a predicate

Answer: A) To check if all elements in the stream match a given predicate

23. Which functional interface is used to represent a supplier of results?

- A) Supplier<T>
- B) Consumer<T>
- C) Function<T, R>
- D) Predicate<T>

Answer: A) Supplier<T>

24. What does the Stream.concat() method do?

- A) Concatenates two streams into a single stream
- B) Combines two lists into a single list
- C) Merges two sets into a single set
- D) Joins two maps into a single map

Answer: A) Concatenates two streams into a single stream

25. How can you create a stream that represents an infinite sequence of numbers?

- A) By using `Stream.iterate()` or `Stream.generate()`
- B) By using `Stream.of()` with a large number of elements
- C) By converting a large collection into a stream
- D) By calling `Stream.createInfinite()`

Answer: A) By using `Stream.iterate()` or `Stream.generate()`

26. Which method in the Stream API is used to find the first element that matches a given predicate?

- A) `findFirst()`
- B) `findAny()`
- C) `filter()`
- D) `peek()`

Answer: A) `findFirst()`

27. What is the use of the `Collectors.groupingBy()` method?

- A) To group elements of the stream by a classifier function
- B) To collect elements into a single collection
- C) To perform aggregation on stream elements
- D) To transform elements based on a function

Answer: A) To group elements of the stream by a classifier function

28. Which method is used to create a stream of elements from an array?

- A) `Stream.of()`
- B) `Stream.fromArray()`
- C) `Arrays.stream()`
- D) `Stream.array()`

Answer: C) `Arrays.stream()`

29. What does the `Stream.peek()` method do?

- A) Allows you to perform an action on each element of the stream without modifying the stream
- B) Filters elements based on a predicate
- C) Transforms elements in the stream
- D) Collects elements into a collection

Answer: A) Allows you to perform an action on each element of the stream without modifying the stream

30. Which method in Stream can be used to collect elements into a Set?

- A) `collect(Collectors.toSet())`
- B) `collect(Collectors.toList())`
- C) `collect(Collectors.toMap())`
- D) `collect(Collectors.toCollection())`

Answer: A) `collect(Collectors.toSet())`

31. Which functional interface is used to represent a function that takes no arguments and returns a result?

- A) Supplier<T>
- B) Consumer<T>
- C) Function<T, R>
- D) Predicate<T>

Answer: A) Supplier<T>

32. What is the main benefit of using method references in Java 8?

- A) They provide a more concise and readable syntax for lambda expressions.
- B) They allow the creation of anonymous classes.
- C) They provide better performance for functional interfaces.
- D) They simplify the creation of new classes.

Answer: A) They provide a more concise and readable syntax for lambda expressions.

33. Which of the following is a valid syntax for a method reference to a static method?

- A) ClassName::staticMethodName
- B) instance::methodName
- C) ClassName.this::methodName
- D) ClassName::new

Answer: A) ClassName::staticMethodName

34. Which method reference syntax is used to refer to an instance method of a particular object?

- A) instance::methodName
- B) ClassName::methodName
- C) ClassName.this::methodName
- D) ClassName::new

Answer: A) instance::methodName

35. What does the Stream.generate() method do?

- A) Creates a stream that generates elements based on a supplier function
- B) Creates a stream from a collection
- C) Filters elements based on a predicate
- D) Maps elements to new values

Answer: A) Creates a stream that generates elements based on a supplier function

36. What is the purpose of the Stream.reduce() method?

- A) To combine elements of the stream into a single result using an associative accumulation function
- B) To filter elements based on a predicate
- C) To transform each element in the stream

D) To sort the elements in the stream

Answer: A) To combine elements of the stream into a single result using an associative accumulation function

37. Which method can be used to concatenate two Stream instances?

A) Stream.concat()

B) Stream.merge()

C) Stream.join()

D) Stream.append()

Answer: A) Stream.concat()

38. Which method is used to perform an action on each element of a stream, but does not modify the elements?

A) peek()

B) map()

C) filter()

D) collect()

Answer: A) peek()

39. Which method reference syntax is used to refer to a constructor of a class?

A) ClassName::new

B) ClassName::methodName

C) instance::methodName

D) ClassName.this::methodName

Answer: A) ClassName::new

40. How can you transform elements of a stream to their string representations and collect them into a list?

A) stream.map(Object::toString).collect(Collectors.toList())

B) stream.collect(Collectors.toStringList())

C) stream.map(String::valueOf).toList()

D) stream.collect(Collectors.toListOfString())

Answer: A) stream.map(Object::toString).collect(Collectors.toList())

Java Mock Question Bank

Introduction to Java

1. What year was Java first released?
2. Which keyword is used to define a class in Java?
3. Which of the following is a feature of Java?
4. What does JVM stand for in Java?
5. What is the main purpose of the main() method in Java?
6. What does the System.out.println() method do?
7. How do you declare a variable in Java?
8. What is the difference between int and Integer in Java?
9. How do you create an object in Java?

10. What is the purpose of the static keyword in Java?

Object-Oriented Programming (OOP)

- 11. What is encapsulation in Java?
- 12. How does inheritance work in Java?
- 13. What is method overriding?
- 14. What is method overloading?
- 15. How does polymorphism work in Java?
- 16. What is the role of the super keyword in Java?
- 17. What is the purpose of the this keyword?
- 18. How is abstract class different from an interface?
- 19. What is the difference between == and .equals() method in Java?
- 20. What is the use of the final keyword in Java?

Class and Objects

- 21. What is the difference between a class and an object in Java?
- 22. How do you define a method within a class?
- 23. What is the purpose of constructors in a class?
- 24. How can you create an instance of a class?
- 25. What is the significance of this keyword in a class?
- 26. How do you access the members of an object?
- 27. What is method hiding in Java?
- 28. How can you invoke a method of an object?
- 29. What is the use of static variables and methods in a class?
- 30. How do you differentiate between instance variables and local variables?
- 31. What is an inner class in Java?
- 32. How does a static nested class differ from a non-static nested class?
- 33. What is the purpose of the toString() method in a class?
- 34. How do you initialize a class with class-level initialization blocks?
- 35. What is the purpose of the instanceof operator?
- 36. How do you override the equals() method in a class?
- 37. What is the difference between shallow copy and deep copy?
- 38. How do you create a copy of an object using cloning?
- 39. What is the use of Class.forName()?
- 40. How do you make a class immutable in Java?

Object-Oriented Programming (OOP)

- 41. What is the concept of encapsulation in Java?
- 42. How does inheritance support code reusability?
- 43. What is the difference between abstract class and an interface?
- 44. How do you achieve polymorphism through method overriding?
- 45. What is method hiding and how is it different from method overriding?
- 46. How does Java handle multiple inheritance?
- 47. What is the significance of the super keyword in Java?
- 48. How does Java handle multiple inheritance?

49. What is method hiding, and how does it differ from method overriding?
50. What is the significance of abstract methods in abstract classes?
51. How do interfaces contribute to multiple inheritance in Java?
52. What is a concrete class, and how is it different from an abstract class?
53. How do you achieve abstraction in Java?
54. What is the role of the default keyword in an interface?
55. How do you use inheritance to extend functionality in Java?
56. What is the impact of using final with classes, methods, and variables?
57. How does Java handle method resolution in the presence of multiple inheritance?
58. How can you make a class immutable?
59. What is the role of constructors in inheritance?

Constructors

60. What is a constructor in Java?
61. What is a default constructor?
62. What is a parameterized constructor?
63. What is the purpose of this() in constructors?
64. How does constructor overloading work?
65. What is the purpose of super() in constructors?
66. What is a copy constructor?
67. Can constructors be inherited in Java?
68. What happens if you don't define any constructor in a class?
69. How is a constructor different from a method?

Type Casting

70. What is type casting in Java?
71. What is the difference between implicit and explicit type casting?
72. How do you cast a double to an int in Java?
73. What is autoboxing and unboxing?
74. How does type casting work with objects?
75. What is the purpose of the instanceof operator?
76. How do you cast an object to a subclass type?
77. What happens during a ClassCastException?
78. Can you cast between unrelated classes in Java?
79. What is the significance of the Object class in type casting?

Method Binding

80. What is method binding in Java?
81. What is early binding (static binding)?
82. What is late binding (dynamic binding)?
83. How does method overriding affect binding?
84. What role does the super keyword play in method binding?
85. What is the difference between method overloading and method overriding?
86. How does Java handle method resolution in case of ambiguity?
87. What is the purpose of final methods in method binding?
88. How are private methods bound in Java?

89. Can constructors be dynamically bound?

Polymorphism

- 90. What is polymorphism in Java?
- 91. How does polymorphism achieve runtime flexibility?
- 92. What are the types of polymorphism in Java?
- 93. How does method overriding contribute to polymorphism?
- 94. How does method overloading contribute to polymorphism?
- 95. What is the use of polymorphism in object-oriented design?
- 96. Can polymorphism be achieved through interfaces?
- 97. How does Java implement polymorphism through inheritance?
- 98. What is the role of the super keyword in polymorphism?
- 99. Can you achieve polymorphism without inheritance?

Packages and Interfaces

- 100. How do you define a package in Java?
- 101. How do you import a package in Java?
- 102. What is the purpose of import statements in Java?
- 103. How do you define an interface in Java?
- 104. What are the differences between abstract classes and interfaces?
- 105. How do you implement an interface in a class?
- 106. Can interfaces extend other interfaces?
- 107. What are default methods in interfaces?
- 108. How do you use the package keyword in Java?
- 109. What is the purpose of access modifiers in packages?

Access Modifiers

- 110. What are access modifiers in Java?
- 111. What is the default access level in Java?
- 112. How does the public access modifier work?
- 113. How does the protected access modifier work?
- 114. How does the private access modifier work?
- 115. Can access modifiers be applied to local variables?
- 116. What is the significance of package-private access level?
- 117. How do access modifiers affect inheritance?
- 118. Can a private method be accessed from outside the class?
- 119. How does the protected access modifier differ from public?

Java Final Keyword

- 120. What is the purpose of the final keyword in Java?
- 121. Can a final variable be changed after initialization?
- 122. Can a final method be overridden?
- 123. Can a final class be subclassed?
- 124. How does the final keyword affect method parameters?
- 125. Can you declare a final local variable?

- 126. How does final affect variable initialization?
- 127. What are the restrictions imposed by the final keyword?
- 128. How does final impact performance?
- 129. Can a final method be used in an abstract class?

Exception Handling

- 130. What is the difference between throw and throws in Java?
- 131. How do you use the try-with-resources statement?
- 132. What is the purpose of the Throwable class?
- 133. How do you handle exceptions that occur in multiple methods?
- 134. What is a StackTrace in exception handling?
- 135. How do you rethrow an exception in Java?
- 136. What is an exception in Java?
- 137. What is the difference between a checked and an unchecked exception?
- 138. How does Java differentiate between runtime exceptions and compile-time exceptions?
- 139. What is the purpose of the throw keyword in Java?
- 140. How does the throws keyword differ from throw?
- 141. Can a finally block be skipped if no exception is thrown?
- 142. What happens if you don't catch an exception in a method?
- 143. How do you catch multiple exceptions in a single catch block?
- 144. What is the impact of exception handling on performance?
- 145. How do you create a custom exception in Java?
- 146. What is the purpose of the Exception class in the Java exception hierarchy?
- 147. How can you suppress exceptions using try-with-resources?
- 148. What is the role of the Throwable class in Java?
- 149. How do you retrieve the stack trace of an exception?
- 150. What is the purpose of getCause() method in an exception?
- 151. Can a catch block handle exceptions of multiple types?
- 152. How do you use printStackTrace() method in exception handling?
- 153. What is a chained exception and how is it created?
- 154. How can you handle exceptions that occur during exception handling?
- 155. What are some best practices for exception handling in Java?
- 156. What is the significance of the NoClassDefFoundError and ClassNotFoundException?
- 157. How does the try-with-resources statement handle resource management?
- 158. What is the difference between IllegalArgumentException and IllegalStateException?
- 159. What is the difference between RuntimeException and IOException?
- 160. How do you use the assert statement for error handling?
- 161. What is the role of ExceptionInInitializerError?
- 162. How do you throw multiple exceptions from a method?
- 163. What is a NullPointerException, and how can you avoid it?
- 164. How does Java handle exceptions thrown by constructors?
- 165. What is the significance of the try block in exception handling?
- 166.

167. What is the role of the catch block?
168. How do you create a custom unchecked exception?
169. What is the significance of the Exception class in Java?
170. How does Java handle runtime exceptions?

Multithreading

1. What is the purpose of multithreading in Java?
2. How do you create a thread by extending the Thread class?
3. What is the difference between Thread and Runnable?
4. How do you start a thread in Java?
5. What is the role of the run() method in a thread?
6. What is thread synchronization?
7. How do you avoid deadlock in Java?
8. What is the purpose of synchronized keyword?
9. How does wait() and notify() work in inter-thread communication?
10. What is the thread life cycle in Java?
11. What is the difference between synchronized method and synchronized block?
12. How does Thread.sleep() affect a thread's execution?
13. What is the purpose of the volatile keyword in multithreading?
14. How do you handle thread interruption?

Java Collections Framework

15. What is the Java Collections Framework?
16. Which interface in the Java Collections Framework represents a collection of elements with unique values?
17. What is the primary implementation of the List interface that allows dynamic resizing?
18. How do you iterate over elements in a HashSet?
19. What method is used to remove a specific element from a HashMap?
20. What is the difference between ArrayList and LinkedList?
21. Which class in the Collections Framework provides a key-value mapping?
22. What does the Collections.sort() method do?
23. How do you check if a TreeSet contains a specific element?
24. What is the role of Comparator and Comparable interfaces in sorting collections?
25. What is the difference between HashSet and LinkedHashSet?
26. How do you iterate over elements in a Vector?
27. What is the purpose of the HashMap class?
28. How do you access elements in a TreeMap?
29. What method in the List interface returns the number of elements?
30. What is the difference between ArrayList and Vector?

Streams and File I/O

31. What is the purpose of the File class in Java?
32. Which class is used to read binary data from a file?
33. What does the Files.write() method do?
34. How can you read a file line by line using BufferedReader?
35. Which class provides methods to write formatted text to a file?

36. What is the purpose of the `FileInputStream` class?
37. How does the `Files.copy()` method work?
38. What is the function of `ObjectOutputStream`?
39. How do you append data to an existing file using `FileWriter`?
40. What is the difference between `FileReader` and `FileWriter`?

Java 8 Features

41. What is the purpose of Lambda expressions in Java 8?
42. What is a functional interface?
43. How does the Stream API enhance data manipulation in Java 8?
44. What are default methods in interfaces?
45. How do you use the `Optional` class to handle null values?
46. What are method references and how are they used?
47. How does the `forEach` method work with streams?
48. What is the role of Collectors in the Stream API?
49. How do you create a Stream from a collection?
50. What improvements were introduced in the Date and Time API in Java 8?
51. What is a lambda expression and how is it used in Java 8?
52. What are the main benefits of using Stream API in Java 8?
53. How do you filter data using streams?
54. What is the use of `reduce()` method in the Stream API?
55. How do you use the `map()` function in a stream?
56. What is the purpose of the `flatMap()` function in streams?
57. How does the `Optional` class improve handling null values?
58. What is the significance of default methods in interfaces introduced in Java 8?
59. How do you use Collectors to aggregate results in streams?
60. What are method references and how are they used in Java 8?

Mastering The Future



Core Java: Basics of Java Interview Questions

1) What is Java?

Java is the high-level, object-oriented, robust, secure programming language, platform-independent, high performance, Multithreaded, and portable programming language. It was developed by **James Gosling** in June 1991. It can also be known as the platform as it provides its own JRE and API.

2) What are the differences between C++ and Java?

The differences between C++ and Java are given in the following table.

Comparison Index	C++	Java
Platform-independent	C++ is platform-dependent.	Java is platform-independent.
Mainly used for	C++ is mainly used for system programming.	Java is mainly used for application programming. It is widely used in window, web-based, enterprise and mobile applications.
Design Goal	C++ was designed for systems and applications programming. It was an extension of <u>C programming language</u> .	Java was designed and created as an interpreter for printing systems but later extended as a support network computing. It was designed with a goal of being easy to use and accessible to a broader audience.
Goto	C++ supports the <u>goto</u> statement.	Java doesn't support the goto statement.
Multiple inheritance	C++ supports multiple inheritance.	Java doesn't support multiple inheritance through class. It can be achieved by <u>interfaces in java</u> .
Operator Overloading	C++ supports <u>operator overloading</u> .	Java doesn't support operator overloading.
Pointers	C++ supports <u>pointers</u> . You can write pointer program in C++.	Java supports pointer internally. However, you can't write the pointer program in java. It means java has restricted pointer support in Java.

Compiler and Interpreter	C++ uses compiler only. C++ is compiled and run using the compiler which converts source code into machine code so, C++ is platform dependent.	Java uses compiler and interpreter both. Java source code is converted into bytecode at compilation time. The interpreter executes this bytecode at runtime and produces output. Java is interpreted that is why it is platform independent.
Call by Value and Call by reference	C++ supports both call by value and call by reference.	Java supports call by value only. There is no call by reference in java.
Structure and Union	C++ supports structures and unions.	Java doesn't support structures and unions.
Thread Support	C++ doesn't have built-in support for threads. It relies on third-party libraries for thread support.	Java has built-in <u>thread</u> support.
Documentation comment	C++ doesn't support documentation comment.	Java supports documentation comment (<code>/** ... */</code>) to create documentation for java source code.
Virtual Keyword	C++ supports virtual keyword so that we can decide whether or not override a function.	Java has no virtual keyword. We can override all non-static methods by default. In other words, non-static methods are virtual by default.
unsigned right shift >>>	C++ doesn't support >>> operator.	Java supports unsigned right shift >>> operator that fills zero at the top for the negative numbers. For positive numbers, it works same like >>

		operator.
Inheritance Tree	C++ creates a new inheritance tree always.	Java uses a single inheritance tree always because all classes are the child of Object class in java. The object class is the root of the <u>inheritance</u> tree in java.
Hardware	C++ is nearer to hardware.	Java is not so interactive with hardware.
Object-oriented	C++ is an object-oriented language. However, in C language, single root hierarchy is not possible.	Java is also an <u>object-oriented</u> language. However, everything (except fundamental types) is an object in Java. It is a single root hierarchy as everything gets derived from java.lang.Object.

3) List the features of Java Programming language.

There are the following features in Java Programming Language.

- **Simple:** Java is easy to learn. The syntax of Java is based on C++ which makes easier to write the program in it.
- **Object-Oriented:** Java follows the object-oriented paradigm which allows us to maintain our code as the combination of different type of objects that incorporates both data and behavior.
- **Portable:** Java supports read-once-write-anywhere approach. We can execute the Java program on every machine. Java program (.java) is converted to bytecode (.class) which can be easily run on every machine.
- **Platform Independent:** Java is a platform independent programming language. It is

different from other programming languages like C and C++ which needs a platform to be executed. Java comes with its platform on which its code is executed. Java doesn't depend upon the operating system to be executed.

- **Secured:** Java is secured because it doesn't use explicit pointers. Java also provides the concept of ByteCode and Exception handling which makes it more secured.
- **Robust:** Java is a strong programming language as it uses strong memory management. The concepts like Automatic garbage collection, Exception handling, etc. make it more robust.
- **Architecture Neutral:** Java is architectural neutral as it is not dependent on the architecture. In C, the size of data types may vary according to the architecture (32 bit or 64 bit) which doesn't exist in Java.
- **Interpreted:** Java uses the Just-in-time (JIT) interpreter along with the compiler for the program execution.
- **High Performance:** Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code. It is still a little bit slower than a compiled language (e.g., C++).
- **Multithreaded:** We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc.
- **Distributed:** Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.

- **Dynamic:** Java is a dynamic language. It supports dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.

4) What do you understand by Java virtual machine?

Java Virtual Machine is a virtual machine that enables the computer to run the Java program. JVM acts like a run-time engine which calls the main method present in the Java code. JVM is the specification which must be implemented in the computer system. The Java code is compiled by JVM to be a Bytecode which is machine independent and close to the native code.

5) What is the difference between JDK, JRE, and JVM?

JVM

JVM is an acronym for Java Virtual Machine; it is an abstract machine which provides the runtime environment in which Java bytecode can be executed. It is a specification which specifies the working of Java Virtual Machine. Its implementation has been provided by Oracle and other companies. Its implementation is known as JRE.

JVMs are available for many hardware and software platforms (so JVM is platform dependent). It is a runtime instance which is created when we run the Java class. There are three notions of the JVM: specification, implementation, and instance.

JRE

JRE stands for Java Runtime Environment. It is the implementation of JVM. The Java Runtime Environment is a set of software tools which are used for developing Java applications. It is used to provide the runtime environment. It is the implementation of JVM. It physically exists. It contains a set of libraries + other files that JVM uses at runtime.

JDK

JDK is an acronym for Java Development Kit. It is a software development environment which is used to develop Java applications and applets. It physically exists. It contains JRE + development tools. JDK is an implementation of any one of the below given Java Platforms

released by Oracle Corporation:

- Standard Edition Java Platform
 - Enterprise Edition Java Platform
 - Micro Edition Java Platform
-

6) How many types of memory areas are allocated by JVM?

Many types:

1. **Class(Method) Area:** Class Area stores per-class structures such as the runtime constant pool, field, method data, and the code for methods.
 2. **Heap:** It is the runtime data area in which the memory is allocated to the objects
 3. **Stack:** Java Stack stores frames. It holds local variables and partial results, and plays a part in method invocation and return. Each thread has a private JVM stack, created at the same time as the thread. A new frame is created each time a method is invoked. A frame is destroyed when its method invocation completes.
 4. **Program Counter Register:** PC (program counter) register contains the address of the Java virtual machine instruction currently being executed.
 5. **Native Method Stack:** It contains all the native methods used in the application.
-

7) What is JIT compiler?

Just-In-Time(JIT) compiler: It is used to improve the performance. JIT compiles parts of the bytecode that have similar functionality at the same time, and hence reduces the amount of time needed for compilation. Here the term “compiler” refers to a translator from the instruction set of a Java virtual machine (JVM) to the instruction set of a specific CPU.

8) What is the platform?

A platform is the hardware or software environment in which a piece of software is executed. There are two types of platforms, software-based and hardware-based. Java provides the software-based platform.

9) What are the main differences between the Java platform and other platforms?

There are the following differences between the Java platform and other platforms.

- Java is the software-based platform whereas other platforms may be the hardware platforms or software-based platforms.
- Java is executed on the top of other hardware platforms whereas other platforms can only have the hardware components.

10) What gives Java its 'write once and run anywhere' nature?

The bytecode. Java compiler converts the Java programs into the class file (Byte Code) which is the intermediate language between source code and machine code. This bytecode is not platform specific and can be executed on any computer.

11) What is classloader?

ClassLoader is a subsystem of JVM which is used to load class files. Whenever we run the java program, it is loaded first by the classloader. There are three built-in classloaders in Java.

1. **Bootstrap ClassLoader:** This is the first classloader which is the superclass of Extension classloader. It loads the *rt.jar* file which contains all class files of Java Standard Edition like java.lang package classes, java.net package classes, java.util package classes, java.io package classes, java.sql package classes, etc.
2. **Extension ClassLoader:** This is the child classloader of Bootstrap and parent classloader of System classloader. It loads the jar files located inside `$JAVA_HOME/jre/lib/ext` directory.
3. **System/Application ClassLoader:** This is the child classloader of Extension classloader. It loads the class files from the classpath. By default, the classpath is set to the current directory. You can change the classpath using "-cp" or "-classpath" switch. It is also known as Application classloader.

12) Is Empty .java file name a valid source file name?

Yes, Java allows to save our java file by **.java** only, we need to compile it by **javac .java** and run by **java classname** Let's take a simple example:

//save by .java only

```
class A{
    public static void main(String args[]){
```



```
System.out.println("Hello java");  
}  
}
```

//compile by javac .java

//run by java A

compile it by **javac .java**

run it by **java A**

13) Is delete, next, main, exit or null keyword in java?

No.

14) If I don't provide any arguments on the command line, then what will the value stored in the String array passed into the main() method, empty or NULL?

It is empty, but not null.

15) What if I write static public void instead of public static void?

The program compiles and runs correctly because the order of specifiers doesn't matter in Java.

16) What is the default value of the local variables?

The local variables are not initialized to any default value, neither primitives nor object references.

17) What are the various access specifiers in Java?

In Java, access specifiers are the keywords which are used to define the access scope of the method, class, or a variable. In Java, there are four access specifiers given below.

- **Public** The classes, methods, or variables which are defined as public, can be accessed by any class or method.
- **Protected** Protected can be accessed by the class of the same package, or by the subclass of this class, or within the same class.

- **Default** Default are accessible within the package only. By default, all the classes, methods, and variables are of default scope.
 - **Private** The private class, methods, or variables defined as private can be accessed within the class only.
-

18) What is the purpose of static methods and variables?

The methods or variables defined as static are shared among all the objects of the class. The static is the part of the class and not of the object. The static variables are stored in the class area, and we do not need to create the object to access such variables. Therefore, static is used in the case, where we need to define variables or methods which are common to all the objects of the class.

For example, In the class simulating the collection of the students in a college, the name of the college is the common attribute to all the students. Therefore, the college name will be defined as **static**.

19) What are the advantages of Packages in Java?

There are various advantages of defining packages in Java.

- Packages avoid the name clashes.
 - The Package provides easier access control.
 - We can also have the hidden classes that are not visible outside and used by the package.
 - It is easier to locate the related classes.
-

20) What is the output of the following Java program?

```
class Test
{
    public static void main (String args[])
    {
        System.out.println(10 + 20 + "pentagon");
        System.out.println("pentagon" + 10 + 20);
    }
}
```

The output of the above code will be

30Pentagon

Pentagon1020

Explanation

In the first case, 10 and 20 are treated as numbers and added to be 30. Now, their sum 30 is treated as the string and concatenated with the string **pentagon**. Therefore, the output will be **30pentagon**.

In the second case, the string Pentagon is concatenated with 10 to be the string **pentagon10** which will then be concatenated with 20 to be **pentagon1020**.

21) What is the output of the following Java program?

```
class Test
{
    public static void main (String args[])
    {
        System.out.println(10 * 20 + "Pentagon");
        System.out.println("Pentagon" + 10 * 20);
    }
}
```

The output of the above code will be

200Pentagon

Pentagon200

Explanation

In the first case, The numbers 10 and 20 will be multiplied first and then the result 200 is treated as the string and concatenated with the string **Pentagon** to produce the output **200Pentagon**.

In the second case, The numbers 10 and 20 will be multiplied first to be 200 because the precedence of the multiplication is higher than addition. The result 200 will be treated as the string and concatenated with the string **Pentagon** to produce the output as **Pentagon200**.

22) What is the output of the following Java program?

```
class Test
{
    public static void main (String args[])
    {
        for(int i=0; 0; i++)
        {
            System.out.println("Hello Pentagon");
        }
    }
}
```

The above code will give the compile-time error because the for loop demands a boolean value in the second part and we are providing an integer value, i.e., 0.

Core Java - OOPs Concepts: Initial OOPs Interview Questions

23) What is object-oriented paradigm?

It is a programming paradigm based on objects having data and methods defined in the class to which it belongs. Object-oriented paradigm aims to incorporate the advantages of modularity and reusability. Objects are the instances of classes which interacts with one another to design applications and programs. There are the following features of the object-oriented paradigm.

- Follows the bottom-up approach in program design.
- Focus on data with methods to operate upon the object's data
- Includes the concept like Encapsulation and abstraction which hides the complexities from the user and show only functionality.
- Implements the real-time approach like inheritance, abstraction, etc.
- The examples of the object-oriented paradigm are C++, Simula, Smalltalk, Python, C#, etc.

24) What is an object?

The Object is the real-time entity having some state and behavior. In Java, Object is an instance of the class having the instance variables as the state of the object and the methods as the behavior of the object. The object of a class can be created by using the **new** keyword.

25) What is the difference between an object-oriented programming language and object-based programming language?

There are the following basic differences between the object-oriented language and object-based language.

- Object-oriented languages follow all the concepts of OOPs whereas, the object-based language doesn't follow all the concepts of OOPs like inheritance and polymorphism.
 - Object-oriented languages do not have the inbuilt objects whereas Object-based languages have the inbuilt objects, for example, JavaScript has window object.
 - Examples of object-oriented programming are Java, C#, Smalltalk, etc. whereas the examples of object-based languages are JavaScript, VBScript, etc.
-

26) What will be the initial value of an object reference which is defined as an instance variable?

All object references are initialized to null in Java.

Core Java - OOPs Concepts: Constructor Interview Questions

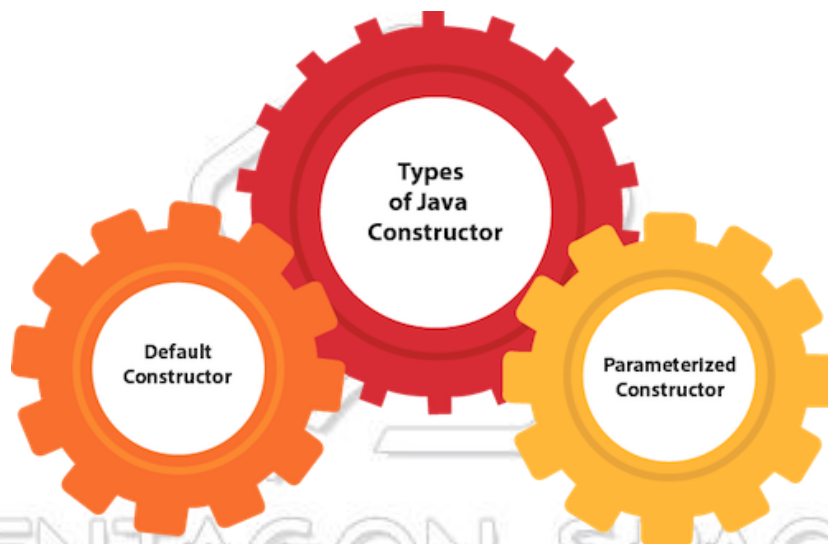
27) What is the constructor?

The constructor can be defined as the special type of method that is used to initialize the state of an object. It is invoked when the class is instantiated, and the memory is allocated for the object. Every time, an object is created using the **new** keyword, the default constructor of the class is called. The name of the constructor must be similar to the class name. The constructor must not have an explicit return type.

28) How many types of constructors are used in Java?

Based on the parameters passed in the constructors, there are two types of constructors in Java.

- **Default Constructor:** default constructor is the one which does not accept any value. The default constructor is mainly used to initialize the instance variable with the default values. It can also be used for performing some useful task on object creation. A default constructor is invoked implicitly by the compiler if there is no constructor defined in the class.
- **Parameterized Constructor:** The parameterized constructor is the one which can initialize the instance variables with the given values. In other words, we can say that the constructors which can accept the arguments are called parameterized constructors.



29) What is the purpose of a default constructor?

The purpose of the default constructor is to assign the default value to the objects. The java compiler creates a default constructor implicitly if there is no constructor in the class.

```
class Student3 {
    int id;
    String name;

    void display(){System.out.println(id+" "+name);}

    public static void main(String args[]){
        Student3 s1=new Student3();
    }
}
```

```

Student3 s2=new Student3();
s1.display();
s2.display();
}
}

```

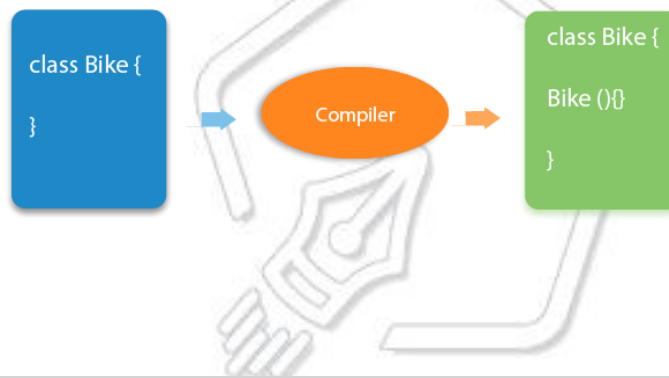
Test it Now

Output:

0 null

0 null

Explanation: In the above class, you are not creating any constructor, so compiler provides you a default constructor. Here 0 and null values are provided by default constructor.



30) Does constructor return any value?

Ans: yes, The constructor implicitly returns the current instance of the class (You can't use an explicit return type with the constructor)._

31)Is constructor inherited?

No, The constructor is not inherited.

32) Can you make a constructor final?

No, the constructor can't be final.

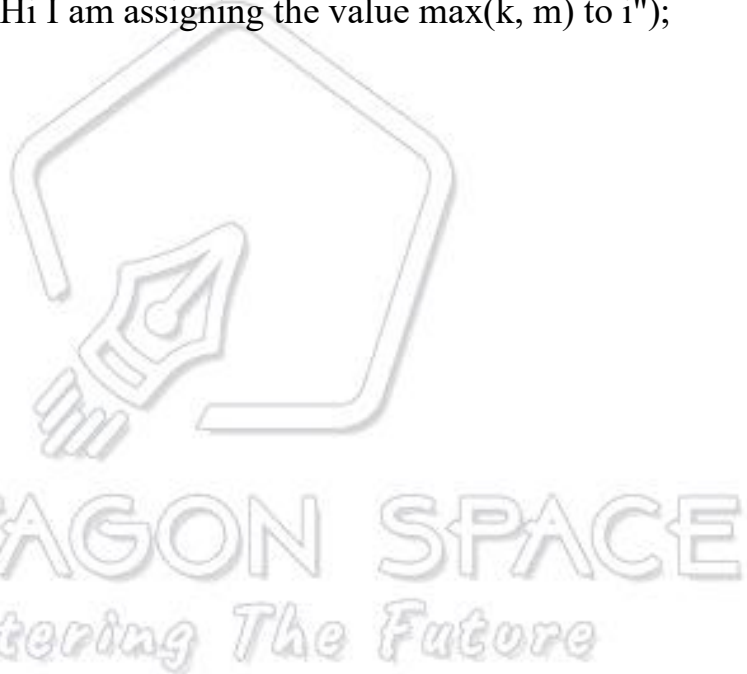
33) Can we overload the constructors?

Yes, the constructors can be overloaded by changing the number of arguments accepted by the constructor or by changing the data type of the parameters. Consider the following

example.

```
class Test
{
    int i;
    public Test(int k)
    {
        i=k;
    }
    public Test(int k, int m)
    {
        System.out.println("Hi I am assigning the value max(k, m) to i");
        if(k>m)
        {
            i=k;
        }
        else
        {
            i=m;
        }
    }
}

public class Main
{
    public static void main (String args[])
    {
        Test test1 = new Test(10);
        Test test2 = new Test(12, 15);
        System.out.println(test1.i);
        System.out.println(test2.i);
    }
}
```



In the above program, The constructor Test is overloaded with another constructor. In the first call to the constructor, The constructor with one argument is called, and i will be initialized with the value 10. However, In the second call to the constructor, The constructor with the 2 arguments is called, and i will be initialized with the value 15.

34) What do you understand by copy constructor in Java?

There is no copy constructor in java. However, we can copy the values from one object to another like copy constructor in C++.

There are many ways to copy the values of one object into another in java. They are:

- By constructor
- By assigning the values of one object into another
- By clone() method of Object class

In this example, we are going to copy the values of one object into another using java constructor.

//Java program to initialize the values from one object to another

```
class Student6{
    int id;
    String name;
    //constructor to initialize integer and string
    Student6(int i,String n){
        id = i;
        name = n;
    }
    //constructor to initialize another object
    Student6(Student6 s){
        id = s.id;
        name =s.name;
    }
    void display(){System.out.println(id+" "+name);}
```

```

public static void main(String args[]){
    Student6 s1 = new Student6(111,"Karan");
    Student6 s2 = new Student6(s1);
    s1.display();
    s2.display();
}
}

```

Test it Now

Output:

111 Karan

111 Karan

35) What are the differences between the constructors and methods?

There are many differences between constructors and methods. They are given below.

Java Constructor	Java Method
A constructor is used to initialize the state of an object.	A method is used to expose the behavior of an object.
A constructor must not have a return type.	A method must have a return type.
The constructor is invoked implicitly.	The method is invoked explicitly.
The Java compiler provides a default constructor if you don't have any constructor in a class.	The method is not provided by the compiler in any case.
The constructor name must be same as the class name.	The method name may or may not be same as class name.

36) What is the output of the following Java program?

```
public class Test
{
    Test(int a, int b)
    {
        System.out.println("a = "+a+" b = "+b);
    }
    Test(int a, float b)
    {
        System.out.println("a = "+a+" b = "+b);
    }
    public static void main (String args[])
    {
        byte a = 10;
        byte b = 15;
        Test test = new Test(a,b);
    }
}
```

The output of the following program is:

a = 10 b = 15

Here, the data type of the variables a and b, i.e., byte gets promoted to int, and the first parameterized constructor with the two integer parameters is called.

37) What is the output of the following Java program?

```
class Test
{
    int i;
}
public class Main
{
    public static void main (String args[])
```

```

    {
        Test test = new Test();
        System.out.println(test.i);
    }
}

```

The output of the program is 0 because the variable i is initialized to 0 internally. As we know that a default constructor is invoked implicitly if there is no constructor in the class, the variable i is initialized to 0 since there is no constructor in the class.

38) What is the output of the following Java program?

```

class Test
{
    int test_a, test_b;
    Test(int a, int b)
    {
        test_a = a;
        test_b = b;
    }
    public static void main (String args[])
    {
        Test test = new Test();
        System.out.println(test.test_a+" "+test.test_b);
    }
}

```

There is a **compiler error** in the program because there is a call to the default constructor in the main method which is not present in the class. However, there is only one parameterized constructor in the class Test. Therefore, no default constructor is invoked by the constructor implicitly.

39) What is the static variable?

The static variable is used to refer to the common property of all objects (that is not unique for each object), e.g., The company name of employees, college name of students, etc. Static variable gets memory only once in the class area at the time of class loading. Using a static variable makes your program more memory efficient (it saves memory). Static variable belongs to the class rather than the object.

//Program of static variable

```

class Student8{
    int rollno;
    String name;
    static String college ="ITS";

    Student8(int r,String n){
        rollno = r;
        name = n;
    }

    void display (){System.out.println(rollno+" "+name+" "+college);}

    public static void main(String args[]){
        Student8 s1 = new Student8(111,"Karan");
        Student8 s2 = new Student8(222,"Aryan");

        s1.display();
        s2.display();
    }
}

```

Test it Now

Output:111 Karan ITS

222 Aryan ITS

40) What is the static method?

- A static method belongs to the class rather than the object.
 - There is no need to create the object to call the static methods.
 - A static method can access and change the value of the static variable.
-

41) What are the restrictions that are applied to the Java static methods?

Two main restrictions are applied to the static methods.

- The static method can not use non-static data member or call the non-static method directly.
 - this and super cannot be used in static context as they are non-static.
-

42) Why is the main method static?

Because the object is not required to call the static method. If we make the main method non-static, JVM will have to create its object first and then call main() method which will lead to the extra memory allocation._

43) Can we override the static methods?

No, we can't override static methods.

44) What is the static block?

Static block is used to initialize the static data member. It is executed before the main method, at the time of classloading.

```
class A2{  
    static{System.out.println("static block is invoked");}  
    public static void main(String args[]){  
        System.out.println("Hello main");  
    }  
}
```

Output: static block is invoked

Hello main

45) Can we execute a program without main() method?

Ans) No, It was possible before JDK 1.7 using the static block. Since JDK 1.7, it is not possible. [More Details](#).

46) What if the static modifier is removed from the signature of the main method?

Program compiles. However, at runtime, It throws an error "NoSuchMethodError."

47) What is the difference between static (class) method and instance method?

static or class method	instance method
1) A method that is declared as static is known as the static method.	A method that is not declared as static is known as the instance method.
2) We don't need to create the objects to call the static methods.	The object is required to call the instance methods.
3) Non-static (instance) members cannot be accessed in the static context (static method, static block, and static nested class) directly.	Static and non-static variables both can be accessed in instance methods.
4) For example: <pre>public static int cube(int n){ return n*n*n;} </pre>	For example: <pre>public void msg(){...}. </pre>

48) Can we make constructors static?

As we know that the static context (method, block, or variable) belongs to the class, not the object. Since Constructors are invoked only when the object is created, there is no sense to make the constructors static. However, if you try to do so, the compiler will show the compiler error.

49) Can we make the abstract methods static in Java?

In Java, if we make the abstract methods static, It will become the part of the class, and we can directly call it which is unnecessary. Calling an undefined method is completely useless

therefore it is not allowed.

50) Can we declare the static variables and methods in an abstract class?

Yes, we can declare static variables and methods in an abstract method. As we know that there is no requirement to make the object to access the static context, therefore, we can access the static context declared inside the abstract class by using the name of the abstract class.

Consider the following example.

```
abstract class Test
```

```
{  
    static int i = 102;  
    static void TestMethod()  
    {  
        System.out.println("hi !! I am good !!");  
    }  
}
```

```
public class TestClass extends Test
```

```
{  
    public static void main (String args[])  
    {  
        Test.TestMethod();  
        System.out.println("i = "+Test.i);  
    }  
}
```

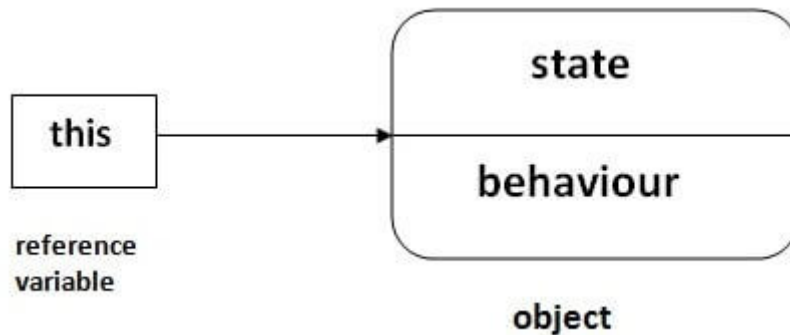
Output

```
hi !! I am good !!
```

```
i = 102
```

51) What is this keyword in java?

The **this** keyword is a reference variable that refers to the current object. There are the various uses of this keyword in Java. It can be used to refer to current class properties such as instance methods, variable, constructors, etc. It can also be passed as an argument into the methods or constructors. It can also be returned from the method as the current class instance.



52) What are the main uses of this keyword?

There are the following uses of **this** keyword.

- **this** can be used to refer to the current class instance variable.
- **this** can be used to invoke current class method (implicitly)
- **this()** can be used to invoke the current class constructor.
- **this** can be passed as an argument in the method call.
- **this** can be passed as an argument in the constructor call.
- **this** can be used to return the current class instance from the method.

53) Can we assign the reference to this variable?

No, this cannot be assigned to any value because it always points to the current class object and this is the final reference in Java. However, if we try to do so, the compiler error will be shown. Consider the following example.

```

public class Test
{
    public Test()
    {
        this = null;
    }
  
```

```

        System.out.println("Test class constructor called");
    }
    public static void main (String args[])
    {
        Test t = new Test();
    }
}

```

Output

Test.java:5: error: cannot assign a value to final variable this

```
this = null;
```

^

1 error

54) Can this keyword be used to refer static members?

Yes, It is possible to use this keyword to refer static members because this is just a reference variable which refers to the current class object. However, as we know that, it is unnecessary to access static variables through objects, therefore, it is not the best practice to use this to refer static members. Consider the following example.

```

public class Test
{
    static int i = 10;
    public Test ()
    {
        System.out.println(this.i);
    }
    public static void main (String args[])
    {
        Test t = new Test();
    }
}

```

Output

55) How can constructor chaining be done using this keyword?

Constructor chaining enables us to call one constructor from another constructor of the class with respect to the current class object. We can use this keyword to perform constructor chaining within the same class. Consider the following example which illustrates how can we use this keyword to achieve constructor chaining.

```
public class Employee
{
    int id,age;
    String name, address;
    public Employee (int age)
    {
        this.age = age;
    }
    public Employee(int id, int age)
    {
        this(age);
        this.id = id;
    }
    public Employee(int id, int age, String name, String address)
    {
        this(id, age);
        this.name = name;
        this.address = address;
    }
    public static void main (String args[])
    {
        Employee emp = new Employee(105, 22, "Vikas", "Delhi");
        System.out.println("ID: "+emp.id+" Name:"+emp.name+" age:"+emp.age
+" address: "+emp.address);
    }
}
```

```
}
```

```
}
```

Output

ID: 105 Name:Vikas age:22 address: Delhi

56) What are the advantages of passing this into a method instead of the current class object itself?

As we know, that this refers to the current class object, therefore, it must be similar to the current class object. However, there can be two main advantages of passing this into a method instead of the current class object.

- this is a final variable. Therefore, this cannot be assigned to any new value whereas the current class object might not be final and can be changed.
 - this can be used in the synchronized block.
-

57) What is the Inheritance?

Inheritance is a mechanism by which one object acquires all the properties and behavior of another object of another class. It is used for Code Reusability and Method Overriding. The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also. Inheritance represents the IS-A relationship which is also known as a parent-child relationship.

There are five types of inheritance in Java.

- Single-level inheritance
- Multi-level inheritance
- Multiple Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

Multiple inheritance is not supported in Java through class.

58) Why is Inheritance used in Java?

There are various advantages of using inheritance in Java that is given below.

- Inheritance provides code reusability. The derived class does not need to redefine the method of base class unless it needs to provide the specific implementation of the method.
- Runtime polymorphism cannot be achieved without using inheritance.
- We can simulate the inheritance of classes with the real-time objects which makes OOPs more realistic.
- Inheritance provides data hiding. The base class can hide some data from the derived class by making it private.
- Method overriding cannot be achieved without inheritance. By method overriding, we can give a specific implementation of some basic method contained by the base class.

59) Which class is the superclass for all the classes?

The object class is the superclass of all other classes in Java.

60) Why is multiple inheritance not supported in java?

To reduce the complexity and simplify the language, multiple inheritance is not supported in java. Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class.

Since the compile-time errors are better than runtime errors, Java renders compile-time error if you inherit 2 classes. So whether you have the same method or different, there will be a compile time error.

```
class A{
    void msg(){System.out.println("Hello");}
}
class B{
    void msg(){System.out.println("Welcome");}
}
class C extends A,B{//suppose if it were
```



```

Public Static void main(String args[]){
    C obj=new C();
    obj.msg();//Now which msg() method would be invoked?
}
}

```

Compile Time Error

61) What is aggregation?

Aggregation can be defined as the relationship between two classes where the aggregate class contains a reference to the class it owns. Aggregation is best described as a **has-a** relationship. For example, The aggregate class Employee having various fields such as age, name, and salary also contains an object of Address class having various fields such as Address-Line 1, City, State, and pin-code. In other words, we can say that Employee (class) has an object of Address class. Consider the following example.

Address.java

```

public class Address {
    String city,state,country;

    public Address(String city, String state, String country) {
        this.city = city;
        this.state = state;
        this.country = country;
    }
}

```

Employee.java

```

public class Emp {
    int id;
    String name;
    Address address;
}

```

```
public Emp(int id, String name,Address address) {  
    this.id = id;  
    this.name = name;  
    this.address=address;  
}  
  
void display(){  
    System.out.println(id+" "+name);  
    System.out.println(address.city+" "+address.state+" "+address.country);  
}  
  
public static void main(String[] args) {  
    Address address1=new Address("gzb","UP","india");  
    Address address2=new Address("gno","UP","india");  
  
    Emp e=new Emp(111,"varun",address1);  
    Emp e2=new Emp(112,"arun",address2);  
  
    e.display();  
    e2.display();  
  
}  
}
```

Output

```
111 varun  
gzb UP india  
112 arun  
gno UP india
```

Holding the reference of a class within some other class is known as composition. When an object contains the other object, if the contained object cannot exist without the existence of container object, then it is called composition. In other words, we can say that composition is the particular case of aggregation which represents a stronger relationship between two objects. Example: A class contains students. A student cannot exist without a class. There exists composition between class and students.

63) What is the difference between aggregation and composition?

Aggregation represents the weak relationship whereas composition represents the strong relationship. For example, the bike has an indicator (aggregation), but the bike has an engine (composition).

64) Why does Java not support pointers?

The pointer is a variable that refers to the memory address. They are not used in Java because they are unsafe(unsecured) and complex to understand.

65) What is super in java?

The **super** keyword in Java is a reference variable that is used to refer to the immediate parent class object. Whenever you create the instance of the subclass, an instance of the parent class is created implicitly which is referred by super reference variable. The `super()` is called in the class constructor implicitly by the compiler if there is no `super` or `this`.

```
class Animal{
    Animal(){System.out.println("animal is created");}
}
class Dog extends Animal{
    Dog(){
        System.out.println("dog is created");
    }
}
class TestSuper4{
    public static void main(String args[]){
```

```
Dog d=new Dog();  
}  
}
```

Output:

```
animal is created  
dog is created
```

66) How can constructor chaining be done by using the super keyword?

```
class Person  
{  
    String name,address;  
    int age;  
    public Person(int age, String name, String address)  
    {  
        this.age = age;  
        this.name = name;  
        this.address = address;  
    }  
}  
class Employee extends Person  
{  
    float salary;  
    public Employee(int age, String name, String address, float salary)  
    {  
        super(age,name,address);  
        this.salary = salary;  
    }  
}  
public class Test  
{  
    public static void main (String args[])
```

```

    {
        Employee e = new Employee(22, "Mukesh", "Delhi", 90000);
        System.out.println("Name: "+e.name+" Salary: "+e.salary+" Age: "+e.age
+" Address: "+e.address);
    }
}

```

Output

Name: Mukesh Salary: 90000.0 Age: 22 Address: Delhi

67) What are the main uses of the super keyword?

There are the following uses of super keyword.

- super can be used to refer to the immediate parent class instance variable.
 - super can be used to invoke the immediate parent class method.
 - super() can be used to invoke immediate parent class constructor.
-

68) What are the differences between this and super keyword?

There are the following differences between this and super keyword.

- The super keyword always points to the parent class contexts whereas this keyword always points to the current class context.
 - The super keyword is primarily used for initializing the base class variables within the derived class constructor whereas this keyword primarily used to differentiate between local and instance variables when passed in the class constructor.
 - The super and this must be the first statement inside constructor otherwise the compiler will throw an error.
-

69) What is the output of the following Java program?

```

class Person
{
    public Person()
    {
        System.out.println("Person class constructor called");
    }
}

```

```

    }
}
public class Employee extends Person
{
    public Employee()
    {
        System.out.println("Employee class constructor called");
    }
    public static void main (String args[])
    {
        Employee e = new Employee();
    }
}

```

Output

Person class constructor called
Employee class constructor called

Explanation

The `super()` is implicitly invoked by the compiler if no `super()` or `this()` is included explicitly within the derived class constructor. Therefore, in this case, The Person class constructor is called first and then the Employee class constructor is called.

70) Can you use `this()` and `super()` both in a constructor?

No, because `this()` and `super()` must be the first statement in the class constructor.

Example:

1. **public class** Test{
2. Test()
3. {
4. **super**();
5. **this**();
6. System.out.println("Test class object is created");

```

7.    }
8.    public static void main(String []args){
9.        Test t = new Test();
10.   }
11.}

```

Output:

Test.java:5: error: call to this must be first statement in constructor

71)What is object cloning?

The object cloning is used to create the exact copy of an object. The clone() method of the Object class is used to clone an object. The **java.lang.Cloneable** interface must be implemented by the class whose object clone we want to create. If we don't implement Cloneable interface, clone() method generates CloneNotSupportedException.

1. **protected** Object clone() **throws** CloneNotSupportedException
-

Core Java - OOPs Concepts: Method Overloading Interview Questions

72) What is method overloading?

Method overloading is the polymorphism technique which allows us to create multiple methods with the same name but different signature. We can achieve method overloading in two ways.

- By Changing the number of arguments
- By Changing the data type of arguments

Method overloading increases the readability of the program. Method overloading is performed to figure out the program quickly.

73) Why is method overloading not possible by changing the return type in java?

In Java, method overloading is not possible by changing the return type of the program due to avoid the ambiguity.

```

class Adder{

```



```

static int add(int a,int b){return a+b;}
static double add(int a,int b){return a+b;}
}
class TestOverloading3 {
public static void main(String[] args){
System.out.println(Adder.add(11,11));//ambiguity
}}

```

Output:

Compile Time Error: method add(int, int) is already defined in class Adder

74) Can we overload the methods by making them static?

No, We cannot overload the methods by just applying the static keyword to them(number of parameters and types are the same). Consider the following example.

```

public class Animal
{
    void consume(int a)
    {
        System.out.println(a+" consumed!!");
    }
    static void consume(int a)
    {
        System.out.println("consumed static "+a);
    }
    public static void main (String args[])
    {
        Animal a = new Animal();
        a.consume(10);
        Animal.consume(20);
    }
}

```

Output

Animal.java:7: error: method consume(int) is already defined in class Animal

```
static void consume(int a)
```

^

Animal.java:15: error: non-static method consume(int) cannot be referenced from a static context

```
Animal.consume(20);
```

^

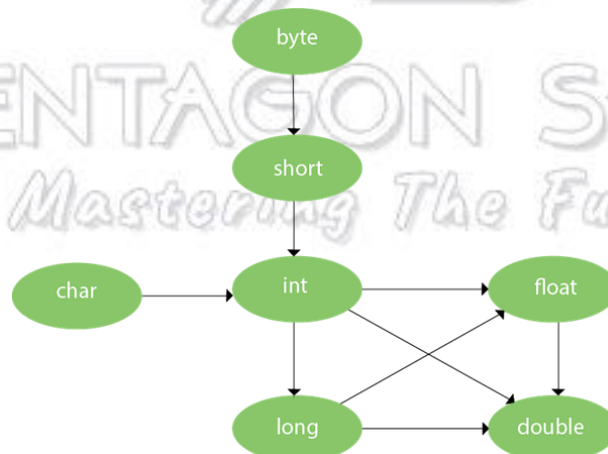
2 errors

75) Can we overload the main() method?

Yes, we can have any number of main methods in a Java program by using method overloading.

76) What is method overloading with type promotion?

By Type promotion is method overloading, we mean that one data type can be promoted to another implicitly if no exact matching is found.



As displayed in the above diagram, the byte can be promoted to short, int, long, float or double. The short datatype can be promoted to int, long, float or double. The char datatype can be promoted to int, long, float or double and so on. Consider the following example.

```
class OverloadingCalculation1 {
    void sum(int a,long b){System.out.println(a+b);}
    void sum(int a,int b,int c){System.out.println(a+b+c);}
}
```

```

public static void main(String args[]){
    OverloadingCalculation1 obj=new OverloadingCalculation1();
    obj.sum(20,20);//now second int literal will be promoted to long
    obj.sum(20,20,20);
}
}

```

Output

40

60

77) What is the output of the following Java program?

```

class OverloadingCalculation3 {
    void sum(int a,long b){System.out.println("a method invoked");}
    void sum(long a,int b){System.out.println("b method invoked");}

    public static void main(String args[]){
        OverloadingCalculation3 obj=new OverloadingCalculation3();
        obj.sum(20,20);//now ambiguity
    }
}

```

Output

OverloadingCalculation3.java:7: error: reference to sum is ambiguous

obj.sum(20,20);*//now ambiguity*

^

both method sum(int,long) in OverloadingCalculation3

and method sum(long,int) in OverloadingCalculation3 match

1 error

Explanation

There are two methods defined with the same name, i.e., sum. The first method accepts the integer and long type whereas the second method accepts long and the integer type. The

parameter passed that are $a = 20$, $b = 20$. We can not tell that which method will be called as there is no clear differentiation mentioned between integer literal and long literal. This is the case of ambiguity. Therefore, the compiler will throw an error.

Core Java - OOPs Concepts: Method Overriding Interview Questions

78) What is method overriding

If a subclass provides a specific implementation of a method that is already provided by its parent class, it is known as Method Overriding. It is used for runtime polymorphism and to implement the interface methods.

Rules for Method overriding

- The method must have the same name as in the parent class.
- The method must have the same signature as in the parent class.
- Two classes must have an IS-A relationship between them.

79) Can we override the static method?

No, you can't override the static method because they are the part of the class, not the object.

80) Why can we not override static method?

It is because the static method is the part of the class, and it is bound with class whereas instance method is bound with the object, and static gets memory in class area, and instance gets memory in a heap.

81) Can we override the overloaded method?

Yes.

82) Difference between method Overloading and Overriding.

Method Overloading	Method Overriding
--------------------	-------------------

1) Method overloading increases the readability of the program.	Method overriding provides the specific implementation of the method that is already provided by its superclass.
2) Method overloading occurs within the class.	Method overriding occurs in two classes that have IS-A relationship between them.
3) In this case, the parameters must be different.	In this case, the parameters must be the same.

83) Can we override the private methods?

No, we cannot override the private methods because the scope of private methods is limited to the class and we cannot access them outside of the class.

84) Can we change the scope of the overridden method in the subclass?

Yes, we can change the scope of the overridden method in the subclass. However, we must notice that we cannot decrease the accessibility of the method. The following point must be taken care of while changing the accessibility of the method.

- The private can be changed to protected, public, or default.
- The protected can be changed to public or default.
- The default can be changed to public.
- The public will always remain public.

85) Can we modify the throws clause of the superclass method while overriding it in the subclass?

Yes, we can modify the throws clause of the superclass method while overriding it in the subclass. However, there are some rules which are to be followed while overriding in case of exception handling.

- If the superclass method does not declare an exception, subclass overridden method cannot declare the checked exception, but it can declare the unchecked exception.
- If the superclass method declares an exception, subclass overridden method can declare

same, subclass exception or no exception but cannot declare parent exception.

86) What is the output of the following Java program?

```
class Base
{
    void method(int a)
    {
        System.out.println("Base class method called with integer a = "+a);
    }

    void method(double d)
    {
        System.out.println("Base class method called with double d =" +d);
    }
}

class Derived extends Base
{
    @Override
    void method(double d)
    {
        System.out.println("Derived class method called with double d =" +d);
    }
}

public class Main
{
    public static void main(String[] args)
    {
        new Derived().method(10);
    }
}
```

```
}
```

Output

Base class method called with integer a = 10

Explanation

The method() is overloaded in class Base whereas it is derived in class Derived with the double type as the parameter. In the method call, the integer is passed.

87) Can you have virtual functions in Java?

Yes, all functions in Java are virtual by default.

88) What is covariant return type?

Now, since java5, it is possible to override any method by changing the return type if the return type of the subclass overriding method is subclass type. It is known as covariant return type. The covariant return type specifies that the return type may vary in the same direction as the subclass.

```
class A{
    A get(){return this;}
}

class B1 extends A{
    B1 get(){return this;}
    void message(){System.out.println("welcome to covariant return type");}

    public static void main(String args[]){
        new B1().get().message();
    }
}
```

Output: welcome to covariant return type

89) What is the output of the following Java program?

```
class Base
```



```

{
    public void baseMethod()
    {
        System.out.println("BaseMethod called ...");
    }
}
class Derived extends Base
{
    public void baseMethod()
    {
        System.out.println("Derived method called ...");
    }
}
public class Test
{
    public static void main (String args[])
    {
        Base b = new Derived();
        b.baseMethod();
    }
}

```

Output

Derived method called ...

Explanation

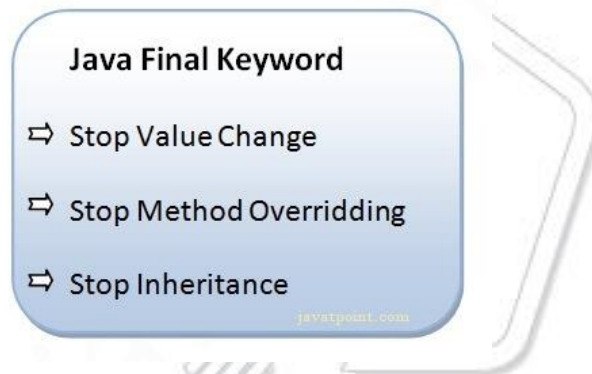
The method of Base class, i.e., baseMethod() is overridden in Derived class. In Test class, the reference variable b (of type Base class) refers to the instance of the Derived class. Here, Runtime polymorphism is achieved between class Base and Derived. At compile time, the presence of method baseMethod checked in Base class, If it presence then the program compiled otherwise the compiler error will be shown. In this case, baseMethod is present in Base class; therefore, it is compiled successfully. However, at runtime, It checks whether the

baseMethod has been overridden by Derived class, if so then the Derived class method is called otherwise Base class method is called. In this case, the Derived class overrides the baseMethod; therefore, the Derived class method is called.

Core Java - OOPs Concepts: final keyword Interview Questions

90) What is the final variable?

In Java, the final variable is used to restrict the user from updating it. If we initialize the final variable, we can't change its value. In other words, we can say that the final variable once assigned to a value, can never be changed after that. The final variable which is not assigned to any value can only be assigned through the class constructor.



```
class Bike9{
    final int speedlimit=90;//final variable
    void run(){
        speedlimit=400;
    }
    public static void main(String args[]){
        Bike9 obj=new Bike9();
        obj.run();
    }
} //end of class
```

Output:Compile Time Error

91) What is the final method?

If we change any method to a final method, we can't override it. [More Details.](#)

```
class Bike{
    final void run(){System.out.println("running");}
}

class Honda extends Bike{
    void run(){System.out.println("running safely with 100kmph");}

    public static void main(String args[]){
        Honda honda= new Honda();
        honda.run();
    }
}
```

Output:Compile Time Error

92) What is the final class?

If we make any class final, we can't inherit it into any of the subclasses.

```
final class Bike{}

class Honda1 extends Bike{
    void run(){System.out.println("running safely with 100kmph");}

    public static void main(String args[]){
        Honda1 honda= new Honda1();
        honda.run();
    }
}
```

Output:Compile Time Error

93) What is the final blank variable?

A final variable, not initialized at the time of declaration, is known as the final blank variable. We can't initialize the final blank variable directly. Instead, we have to initialize it by using the class constructor. It is useful in the case when the user has some data which must not be changed by others, for example, PAN Number. Consider the following example:

```
class Student{
    int id;
    String name;
    final String PAN_CARD_NUMBER;
    ...
}
```

94) Can we initialize the final blank variable?

Yes, if it is not static, we can initialize it in the constructor. If it is static blank final variable, it can be initialized only in the static block.

95) Can you declare the main method as final?

Yes, We can declare the main method as public static final void main(String[] args){}.

96) What is the output of the following Java program?

```
class Main {
    public static void main(String args[]){
        final int i;
        i = 20;
        System.out.println(i);
    }
}
```

Output

20

Explanation

Since i is the blank final variable. It can be initialized only once. We have initialized it to 20. Therefore, 20 will be printed.

97) What is the output of the following Java program?

```
class Base
{
    protected final void getInfo()
    {
        System.out.println("method of Base class");
    }
}

public class Derived extends Base
{
    protected final void getInfo()
    {
        System.out.println("method of Derived class");
    }

    public static void main(String[] args)
    {
        Base obj = new Base();
        obj.getInfo();
    }
}
```

Output

Derived.java:11: error: getInfo() in Derived cannot override getInfo() in Base

protected final void getInfo()

^

overridden method is final

1 error

Explanation

The `getDetails()` method is final; therefore it can not be overridden in the subclass.

98) Can we declare a constructor as final?

The constructor can never be declared as final because it is never inherited. Constructors are not ordinary methods; therefore, there is no sense to declare constructors as final. However, if you try to do so, The compiler will throw an error.

99) Can we declare an interface as final?

No, we cannot declare an interface as final because the interface must be implemented by some class to provide its definition. Therefore, there is no sense to make an interface final. However, if you try to do so, the compiler will show an error.

100) What is the difference between the final method and abstract method?

The main difference between the final method and abstract method is that the abstract method cannot be final as we need to override them in the subclass to give its definition.

101) What is the difference between compile-time polymorphism and runtime polymorphism?

There are the following differences between compile-time polymorphism and runtime polymorphism.

SN	compile-time polymorphism	Runtime polymorphism
1	In compile-time polymorphism, call to a method is resolved at compile-time.	In runtime polymorphism, call to an overridden method is resolved at runtime.
2	It is also known as static binding, early binding, or overloading.	It is also known as dynamic binding, late binding, overriding, or dynamic method dispatch.
3	Overloading is a way to achieve compile-time polymorphism in which, we can define multiple	Overriding is a way to achieve runtime polymorphism in which, we can redefine some particular method or variable in the derived class.

	methods or constructors with different signatures.	By using overriding, we can give some specific implementation to the base class properties in the derived class.
4	It provides fast execution because the type of an object is determined at compile-time.	It provides slower execution as compare to compile-time because the type of an object is determined at run-time.
5	Compile-time polymorphism provides less flexibility because all the things are resolved at compile-time.	Run-time polymorphism provides more flexibility because all the things are resolved at runtime.

102) What is Runtime Polymorphism?

Runtime polymorphism or dynamic method dispatch is a process in which a call to an overridden method is resolved at runtime rather than at compile-time. In this process, an overridden method is called through the reference variable of a superclass. The determination of the method to be called is based on the object being referred to by the reference variable.

```

class Bike{
    void run(){System.out.println("running");}
}
class Splendor extends Bike{
    void run(){System.out.println("running safely with 60km");}
    public static void main(String args[]){
        Bike b = new Splendor();//upcasting
        b.run();
    }
}

```

Output:

running safely with 60km.

In this process, an overridden method is called through the reference variable of a superclass. The determination of the method to be called is based on the object being referred to by the reference variable.

103) Can you achieve Runtime Polymorphism by data members?

No, because method overriding is used to achieve runtime polymorphism and data members cannot be overridden. We can override the member functions but not the data members. Consider the example given below.

```
class Bike{
    int speedlimit=90;
}
class Honda3 extends Bike{
    int speedlimit=150;
    public static void main(String args[]){
        Bike obj=new Honda3();
        System.out.println(obj.speedlimit);//90
    }
}
```

Output:

90

104) What is the difference between static binding and dynamic binding?

In case of the static binding, the type of the object is determined at compile-time whereas, in the dynamic binding, the type of the object is determined at runtime.

Static Binding

```
class Dog{
    private void eat(){System.out.println("dog is eating...");}

    public static void main(String args[]){
        Dog d1=new Dog();
        d1.eat();
    }
}
```

Dynamic Binding

```
class Animal{
```

```
void eat(){System.out.println("animal is eating...");}
}

class Dog extends Animal{
void eat(){System.out.println("dog is eating...");}

public static void main(String args[]){
Animal a=new Dog();
a.eat();
}
}
```

105) What is the output of the following Java program?

```
class BaseTest
{
void print()
{
System.out.println("BaseTest:print() called");
}
}

public class Test extends BaseTest
{
void print()
{
System.out.println("Test:print() called");
}

public static void main (String args[])
{
BaseTest b = new Test();
b.print();
}
}
```

Output

Test:print() called

Explanation

It is an example of Dynamic method dispatch. The type of reference variable `b` is determined at runtime. At compile-time, it is checked whether that method is present in the Base class. In this case, it is overridden in the child class, therefore, at runtime the derived class method is called.

106) What is Java instanceof operator?

The instanceof in Java is also known as type comparison operator because it compares the instance with type. It returns either true or false. If we apply the instanceof operator with any variable that has a null value, it returns false. Consider the following example.

```
class Simple1 {  
    public static void main(String args[]){  
        Simple1 s=new Simple1();  
        System.out.println(s instanceof Simple1);//true  
    }  
}
```

Output

true

An object of subclass type is also a type of parent class. For example, if Dog extends Animal then object of Dog can be referred by either Dog or Animal class.

Mastering The Future

Core Java - OOPs Concepts: Abstraction Interview Questions

107) What is the abstraction?

Abstraction is a process of hiding the implementation details and showing only functionality to the user. It displays just the essential things to the user and hides the internal information, for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery. Abstraction enables you to focus on what the object does instead of how it does it. Abstraction lets you focus on what the object does instead of how it does it.

In Java, there are two ways to achieve the abstraction.

Abstract Class

Interface

108) What is the difference between abstraction and encapsulation?

Abstraction hides the implementation details whereas encapsulation wraps code and data into a single unit.

109) What is the abstract class?

A class that is declared as abstract is known as an abstract class. It needs to be extended and its method implemented. It cannot be instantiated. It can have abstract methods, non-abstract methods, constructors, and static methods. It can also have the final methods which will force the subclass not to change the body of the method. Consider the following example.

```
abstract class Bike{  
    abstract void run();  
}  
class Honda4 extends Bike{  
    void run(){System.out.println("running safely");}  
    public static void main(String args[]){  
        Bike obj = new Honda4();  
        obj.run();  
    }  
}
```

Output

running safely

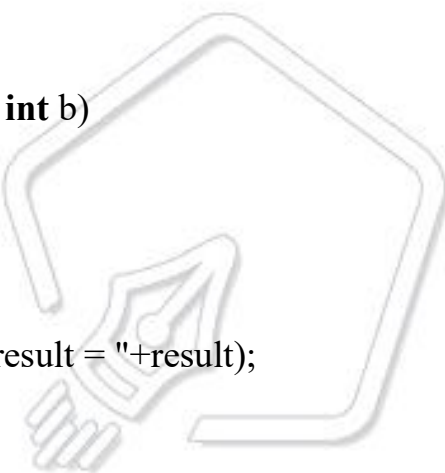
110) Can there be an abstract method without an abstract class?

No, if there is an abstract method in a class, that class must be abstract.

111) Is the following program written correctly? If yes then what will be the output of the program?

```
abstract class Calculate
{
    abstract int multiply(int a, int b);
}

public class Main
{
    public static void main(String[] args)
    {
        int result = new Calculate()
        {
            @Override
            int multiply(int a, int b)
            {
                return a*b;
            }
        }.multiply(12,32);
        System.out.println("result = "+result);
    }
}
```



Yes, the program is written correctly. The Main class provides the definition of abstract method multiply declared in abstract class Calculation. The output of the program will be:

Output

384

112) Can you use abstract and final both with a method?

No, because we need to override the abstract method to provide its implementation, whereas we can't override the final method.

113) Is it possible to instantiate the abstract class?

No, the abstract class can never be instantiated even if it contains a constructor and all of its methods are implemented.

114) What is the interface?

The interface is a blueprint for a class that has static constants and abstract methods. It can be used to achieve full abstraction and multiple inheritance. It is a mechanism to achieve abstraction. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance in Java. In other words, you can say that interfaces can have abstract methods and variables. Java Interface also represents the IS-A relationship. It cannot be instantiated just like the abstract class. However, we need to implement it to define its methods. Since Java 8, we can have the default, static, and private methods in an interface.

115) Can you declare an interface method static?

No, because methods of an interface are abstract by default, and we can not use static and abstract together.

116) Can the Interface be final?

No, because an interface needs to be implemented by the other class and if it is final, it can't be implemented by any class.

117) What is a marker interface?

A Marker interface can be defined as the interface which has no data member and member functions. For example, Serializable, Cloneable are marker interfaces. The marker interface can be declared as follows.

```
public interface Serializable{  
}
```

118) What are the differences between abstract class and interface?

Abstract class	Interface
An abstract class can have a method body (non-abstract methods).	The interface has only abstract methods.
An abstract class can have instance variables.	An interface cannot have instance variables.
An abstract class can have the constructor.	The interface cannot have the constructor.
An abstract class can have static methods.	The interface cannot have static methods.
You can extend one abstract class.	You can implement multiple interfaces.
The abstract class can provide the implementation of the interface.	The Interface can't provide the implementation of the abstract class.
The abstract keyword is used to declare an abstract class.	The interface keyword is used to declare an interface.
An abstract class can extend another Java class and implement multiple Java interfaces.	An interface can extend another Java interface only.
An abstract class can be extended using keyword extends	An interface class can be implemented using keyword implements
A Java abstract class can have class members like private, protected, etc.	Members of a Java interface are public by default.
Example: <pre>public abstract class Shape{ public abstract void draw(); }</pre>	Example: <pre>public interface Drawable{ void draw(); }</pre>

119) Can we define private and protected modifiers for the members in interfaces?

No, they are implicitly public.

120) When can an object reference be cast to an interface reference?

An object reference can be cast to an interface reference when the object implements the referenced interface.

121) How to make a read-only class in Java?

A class can be made read-only by making all of the fields private. The read-only class will have only getter methods which return the private property of the class to the main method. We cannot modify this property because there is no setter method available in the class. Consider the following example.

//A Java class which has only getter methods.

```
public class Student{  
    //private data member  
    private String college="AKG";  
    //getter method for college  
    public String getCollege(){  
        return college;  
    }  
}
```



122) How to make a write-only class in Java?

A class can be made write-only by making all of the fields private. The write-only class will have only setter methods which set the value passed from the main method to the private fields. We cannot read the properties of the class because there is no getter method in this class. Consider the following example.

//A Java class which has only setter methods.

```
public class Student{  
    //private data member  
    private String college;  
    //getter method for college  
    public void setCollege(String college){  
        this.college=college;  
    }  
}
```

123) What are the advantages of Encapsulation in Java?

- By providing only the setter or getter method, you can make the class read-only or write-only. In other words, you can skip the getter or setter methods.
- It provides you the control over the data. Suppose you want to set the value of id which should be greater than 100 only, you can write the logic inside the setter method. You can write the logic not to store the negative numbers in the setter methods.
- It is a way to achieve data hiding in Java because other class will not be able to access the data through the private data members.
- The encapsulate class is easy to test. So, it is better for unit testing.
- The standard IDE's are providing the facility to generate the getters and setters. So, it is easy and fast to create an encapsulated class in Java.

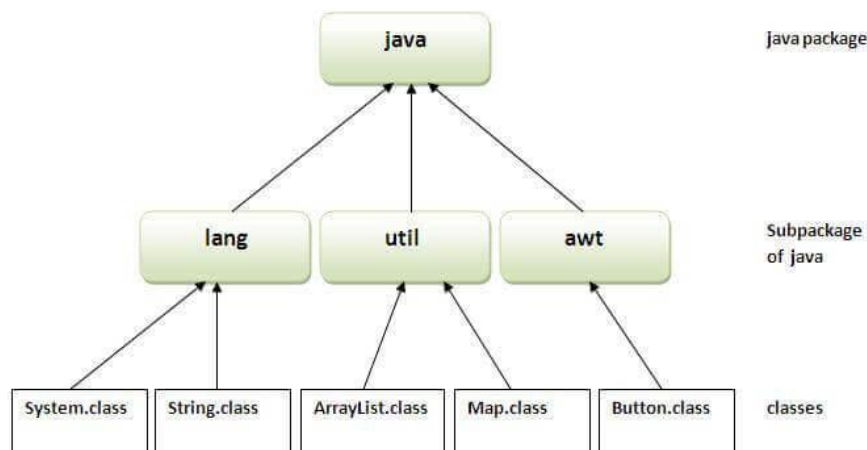
Core Java - OOPs Concepts: Package Interview Questions

124) What is the package?

A package is a group of similar type of classes, interfaces, and sub-packages. It provides access protection and removes naming collision. The packages in Java can be categorized into two forms, inbuilt package, and user-defined package. There are many built-in packages such as Java, lang, awt, javax, swing, net, io, util, sql, etc. Consider the following example to create a package in Java.

```
//save as Simple.java
package mypack;
public class Simple{
    public static void main(String args[]){
        System.out.println("Welcome to package");
    }
}
```

}



125) What are the advantages of defining packages in Java?

By defining packages, we can avoid the name conflicts between the same class names defined in different packages. Packages also enable the developer to organize the similar classes more effectively. For example, one can clearly understand that the classes present in java.io package are used to perform io related operations.

126) How to create packages in Java?

If you are using the programming IDEs like Eclipse, NetBeans, MyEclipse, etc. click on **file->new->project** and eclipse will ask you to enter the name of the package. It will create the project package containing various directories such as src, etc. If you are using an editor like notepad for java programming, use the following steps to create the package.

- Define a package **package_name**. Create the class with the name **class_name** and save this file with **your_class_name.java**.
- Now compile the file by running the following command on the terminal.
- `javac -d . your_class_name.java`
- The above command creates the package with the name **package_name** in the present working directory.

- Now, run the class file by using the absolute class file name, like following.
 - `java package_name.class_name`
-

127) How can we access some class in another class in Java?

There are two ways to access a class in another class.

By using the fully qualified name: To access a class in a different package, either we must use the fully qualified name of that class, or we must import the package containing that class.

By using the relative path, We can use the path of the class that is related to the package that contains our class. It can be the same or subpackage.

128) Do I need to import java.lang package any time? Why?

No. It is by default loaded internally by the JVM.

129) Can I import same package/class twice? Will the JVM load the package twice at runtime?

One can import the same package or the same class multiple times. Neither compiler nor JVM complains about it. However, the JVM will internally load the class only once no matter how many times you import the same class.

Mastering The Future

Java: Exception Handling Interview Questions

131) How many types of exception can occur in a Java program?

There are mainly two types of exceptions: checked and unchecked. Here, an error is considered as the unchecked exception. According to Oracle, there are three types of exceptions:

Checked Exception: Checked exceptions are the one which are checked at compile-time. For example, `SQLException`, `ClassNotFoundException`, etc.

Unchecked Exception: Unchecked exceptions are the one which are handled at runtime because they can not be checked at compile-time. For example, `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException`, etc.

Error: Error cause the program to exit since they are not recoverable. For Example, `OutOfMemoryError`, `AssertionError`, etc.

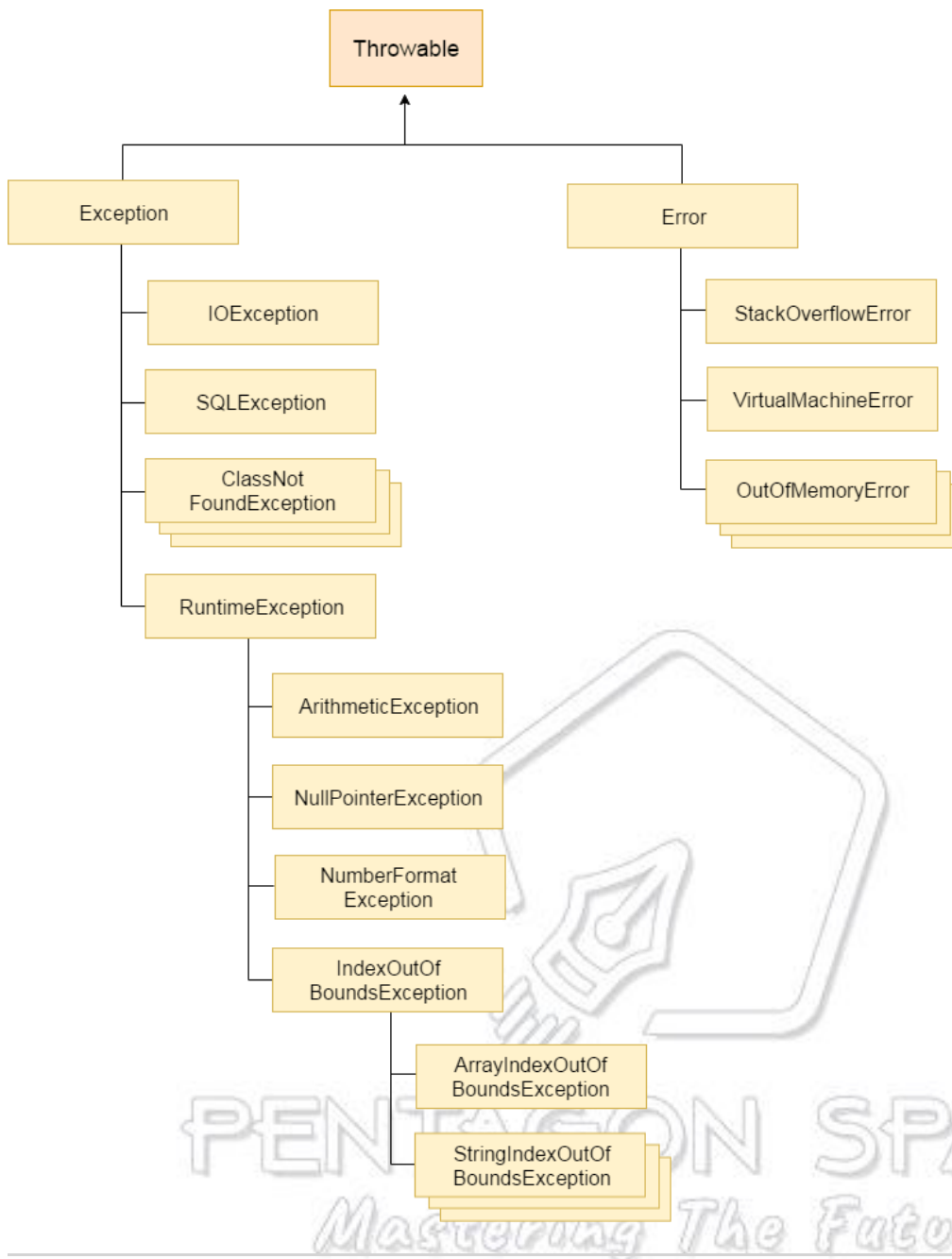
132) What is Exception Handling?

Exception Handling is a mechanism that is used to handle runtime errors. It is used primarily to handle checked exceptions. Exception handling maintains the normal flow of the program. There are mainly two types of exceptions: checked and unchecked. Here, the error is considered as the unchecked exception.

133) Explain the hierarchy of Java Exception classes?

The `java.lang.Throwable` class is the root class of Java Exception hierarchy which is inherited by two subclasses: `Exception` and `Error`. A hierarchy of Java Exception classes are given below:

PENTAGON SPACE
Mastering The Future



134) What is the difference between Checked Exception and Unchecked Exception?

Checked Exception

The classes that extend Throwable class except RuntimeException and Error are known as checked exceptions, e.g., IOException, SQLException, etc. Checked exceptions are checked **at compile-time**.

Unchecked Exception

The classes that extend RuntimeException are known as unchecked exceptions, e.g., ArithmeticException, NullPointerException, etc. Unchecked exceptions are not checked at compile-time.

135) What is the base class for Error and Exception?

The Throwable class is the base class for Error and Exception.

136) Is it necessary that each try block must be followed by a catch block?

It is not necessary that each try block must be followed by a catch block. It should be followed by either a catch block OR a finally block. So whatever exceptions are likely to be thrown should be declared in the throws clause of the method. Consider the following example.

```
public class Main{  
    public static void main(String []args){  
        try{  
            int a = 1;  
            System.out.println(a/0);  
        }  
        finally  
        {  
            System.out.println("rest of the code...");  
        }  
    }  
}
```

Output:

```
Exception in thread main java.lang.ArithmeticException:/ by zero  
rest of the code...
```

137) What is the output of the following Java program?

```
public class ExceptionHandlingExample {  
    public static void main(String args[])  
    {  
        try  
        {  
            int a = 1/0;  
            System.out.println("a = "+a);  
        }  
        catch(Exception e){System.out.println(e);}  
    }  
}
```



```
        catch(ArithmeticException ex){System.out.println(ex);}
    }
}
```

Output

```
ExceptionHandlingExample.java:10: error: exception ArithmeticException has already been caught
```

```
catch(ArithmeticException ex){System.out.println(ex);}
^
```

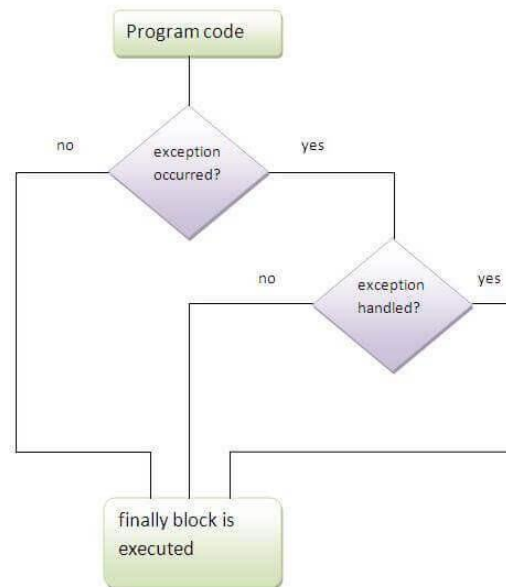
```
error
```

Explanation

ArithmeticException is the subclass of Exception. Therefore, it can not be used after Exception. Since Exception is the base class for all the exceptions, therefore, it must be used at last to handle the exception. No class can be used after this.

138) What is finally block?

The "finally" block is used to execute the important code of the program. It is executed whether an exception is handled or not. In other words, we can say that finally block is the block which is always executed. Finally block follows try or catch block. If you don't handle the exception, before terminating the program, JVM runs finally block, (if any). The finally block is mainly used to place the cleanup code such as closing a file or closing a connection. Here, we must know that for each try block there can be zero or more catch blocks, but only one finally block. The finally block will not be executed if program exits(either by calling System.exit() or by causing a fatal error that causes the process to abort).



139) Can finally block be used without a catch?

Yes, According to the definition of finally block, it must be followed by a try or catch block, therefore, we can use try block instead of catch. [More details.](#)

140) Is there any case when finally will not be executed?

Finally block will not be executed if program exits(either by calling System.exit() or by causing a fatal error that causes the process to abort). [More details.](#)

141) What is the difference between throw and throws?

throw keyword	throws keyword
1) The throw keyword is used to throw an exception explicitly.	The throws keyword is used to declare an exception.
2) The checked exceptions cannot be propagated with throw only.	The checked exception can be propagated with throws

3) The throw keyword is followed by an instance.	The throws keyword is followed by class.
4) The throw keyword is used within the method.	The throws keyword is used with the method signature.
5) You cannot throw multiple exceptions.	You can declare multiple exceptions, e.g., public void method()throws IOException, SQLException.

142) What is the output of the following Java program?

```

public class Main{
    public static void main(String []args){
        try
        {
            throw 90;
        }
        catch(int e){
            System.out.println("Caught the exception "+e);
        }
    }
}

```

Output

```

Main.java:6: error: incompatible types: int cannot be converted to Throwable
throw 90;
^

```

```

Main.java:8: error: unexpected type
catch(int e){
^

```

```

required: class

```

```

found:    int

```

errors

Explanation

In Java, the throwable objects can only be thrown. If we try to throw an integer object, The compiler will show an error since we can not throw basic data type from a block of code.

143) What is the output of the following Java program?

```
class Calculation extends Exception
{
    public Calculation()
    {
        System.out.println("Calculation class is instantiated");
    }
    public void add(int a, int b)
    {
        System.out.println("The sum is "+(a+b));
    }
}
public class Main{
    public static void main(String []args){
        try
        {
            throw new Calculation();
        }
        catch(Calculation c){
            c.add(10,20);
        }
    }
}
```

Output

```
Calculation class is instantiated
The sum is 30
```

Explanation

The object of Calculation is thrown from the try block which is caught in the catch block. The add() of Calculation class is called with the integer values 10 and 20 by using the object of this class. Therefore there sum 30 is printed. The object of the Main class can only be thrown in the case when the type of the object is throwable. To do so, we need to extend the throwable class.

144) Can an exception be rethrown?

Yes.

145) Can subclass overriding method declare an exception if parent class method doesn't throw an exception?

Yes but only unchecked exception not checked.

146) What is exception propagation?

An exception is first thrown from the top of the stack and if it is not caught, it drops down the call stack to the previous method, If not caught there, the exception again drops down to the previous method, and so on until they are caught or until they reach the very bottom of the call stack. This procedure is called exception propagation. By default, checked exceptions are not propagated.

```
class TestExceptionPropagation1 {
    void m(){
        int data=50/0;
    }
    void n(){
        m();
    }
    void p(){
        try{
            n();
        }catch(Exception e){System.out.println("exception handled");}
    }
    public static void main(String args[]){
```

```

        TestExceptionPropagation1 obj=new TestExceptionPropagation1();
        obj.p();
        System.out.println("normal flow...");
    }
}

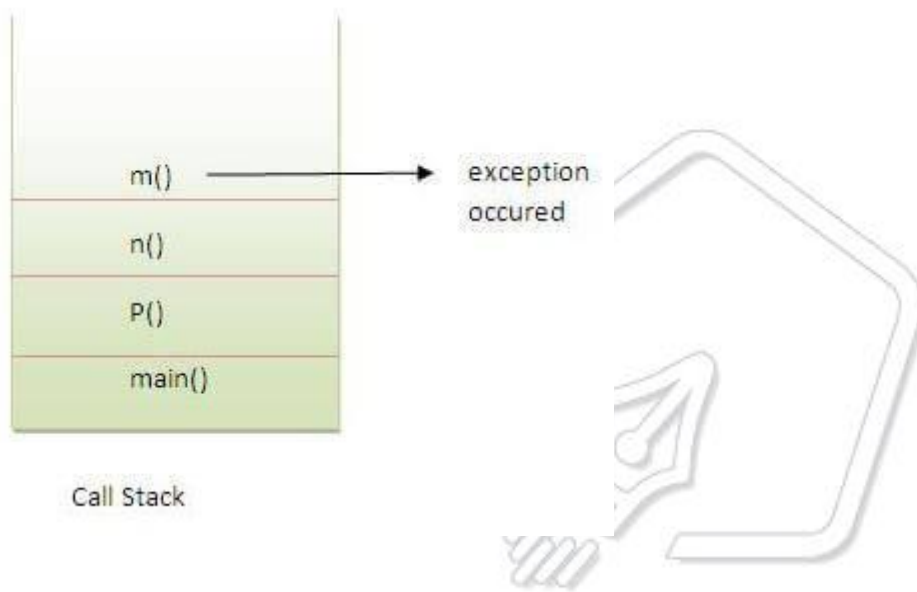
```

Output:

```

exception handled
normal flow...

```



PENTAGON SPACE
Mastering The Fundamentals

147) What is the output of the following Java program?

```

public class Main
{
    void a()
    {
        try{
            System.out.println("a(): Main called");
            b();
        }catch(Exception e)
        {
            System.out.println("Exception is caught");
        }
    }
}

```

```
void b() throws Exception
{
    try{
        System.out.println("b(): Main called");
        c();
    } catch(Exception e){
        throw new Exception();
    }
    finally
    {
        System.out.println("finally block is called");
    }
}

void c() throws Exception
{
    throw new Exception();
}

public static void main (String args[])
{
    Main m = new Main();
    m.a();
}
}
```

Output

```
a(): Main called
b(): Main called
finally block is called
Exception is caught
```

Explanation

In the main method, a() of Main is called which prints a message and call b(). The method b() prints some message and then call c(). The method c() throws an exception which is handled by the catch block of method b. However, It propagates this exception by using **throw Exception()** to be handled by the method a(). As we know, finally block is always executed therefore the finally block in the method b() is executed first and prints a message. At last, the exception is handled by the catch block of the method a().

148) What is the output of the following Java program?

```
public class Calculation
{
    int a;
    public Calculation(int a)
    {
        this.a = a;
    }
    public int add()
    {
        a = a+10;
        try
        {
            a = a+10;
            try
            {
                a = a*10;
                throw new Exception();
            } catch (Exception e) {
                a = a - 10;
            }
        } catch (Exception e) {
            a = a - 10;
        }
        return a;
    }

    public static void main (String args[])
    {
        Calculation c = new Calculation(10);
        int result = c.add();
        System.out.println("result = "+result);
    }
}
```

Output

```
result = 290
```

Explanation

The instance variable `a` of class `Calculation` is initialized to 10 using the class constructor which is called while instantiating the class. The `add` method is called which returns an integer value `result`. In `add()` method, `a` is incremented by 10 to be 20. Then, in the first try block, 10 is again incremented by 10 to be 30. In the second try block, `a` is multiplied by 10 to be 300. The second try block throws the exception which is caught by the catch block associated with this try block. The catch block again alters the value of `a` by decrementing it by 10 to make it 290. Thus the `add()` method returns 290 which is assigned to `result`. However, the catch block associated with the outermost try block will never be executed since there is no exception which can be handled by this catch block.

Java: String Handling Interview Questions

There is given a list of string handling interview questions with short and pointed answers. If you know any string handling interview question, kindly post it in the comment section.

149) What is String Pool?

String pool is the space reserved in the heap memory that can be used to store the strings. The main advantage of using the String pool is whenever we create a string literal; the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new string instance is created and placed in the pool. Therefore, it saves the memory by avoiding the duplicacy.

150) What is the meaning of immutable regarding String?

The simple meaning of immutable is unmodifiable or unchangeable. In Java, String is immutable, i.e., once string object has been created, its value can't be changed. Consider the following example for better understanding.

```
class Testimmutablestring{
    public static void main(String args[]){
        String s="Sachin";
```

```
s.concat(" Tendulkar");//concat() method appends the string at the end
System.out.println(s);//will print Sachin because strings are immutable objects
}
}
```

Test it Now

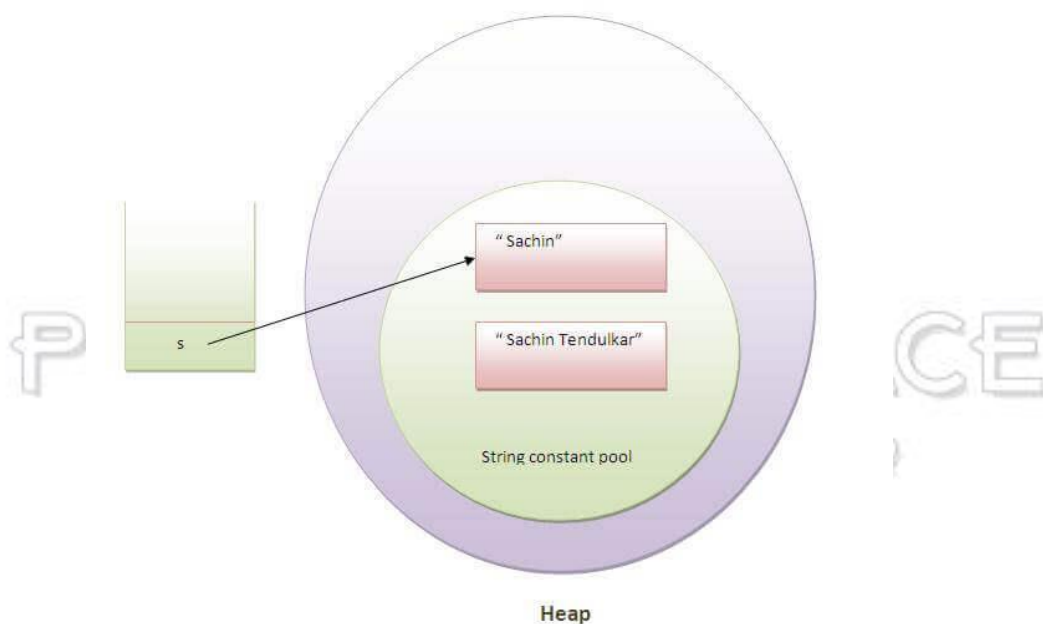
Output:

Sachin

[More details.](#)

151) Why are the objects immutable in java?

Because Java uses the concept of the string literal. Suppose there are five reference variables, all refer to one object "sachin". If one reference variable changes the value of the object, it will be affected by all the reference variables. That is why string objects are immutable in java.



152) How many ways can we create the string object?

1. String Literal

Java String literal is created by using double quotes. For Example:

```
String s="welcome";
```

Each time you create a string literal, the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new string instance is created and placed in the pool. String objects are stored in a special memory area known as the **string constant pool**. For example:

```
String s1="Welcome";  
String s2="Welcome";//It doesn't create a new instance
```

2. By new keyword

```
String s=new String("Welcome");//creates two objects and one reference variable
```

In such case, JVM will create a new string object in normal (non-pool) heap memory, and the literal "Welcome" will be placed in the constant string pool. The variable s will refer to the object in a heap (non-pool).

153) How many objects will be created in the following code?

```
String s1="Welcome";  
String s2="Welcome";  
String s3="Welcome";
```

Only one object will be created using the above code because strings in Java are immutable.

154) Why java uses the concept of the string literal?

To make Java more memory efficient (because no new objects are created if it exists already in the string constant pool).

155) How many objects will be created in the following code?

```
String s = new String("Welcome");
```

Two objects, one in string constant pool and other in non-pool(heap).

156) What is the output of the following Java program?

```
public class Test

    public static void main (String args[])
    {
        String a = new String("Sharma is a good player");
        String b = "Sharma is a good player";
        if(a == b)
        {
            System.out.println("a == b");
        }
        if(a.equals(b))
        {
            System.out.println("a equals b");
        }
    }
}
```

Output

a equals b

Explanation

The operator `==` also check whether the references of the two string objects are equal or not. Although both of the strings contain the same content, their references are not equal because both are created by different ways(Constructor and String literal) therefore, **a == b** is unequal. On the other hand, the `equal()` method always check for the content. Since their content is equal hence, **a equals b** is printed.

157) What is the output of the following Java program?

```
public class Test
{
    public static void main (String args[])
    {
        String s1 = "Sharma is a good player";
        String s2 = new String("Sharma is a good player");
    }
}
```

```

        s2 = s2.intern();
        System.out.println(s1 ==s2);
    }
}

```

Output

```
true
```

Explanation

The intern method returns the String object reference from the string pool. In this case, s1 is created by using string literal whereas, s2 is created by using the String pool. However, s2 is changed to the reference of s1, and the operator == returns true.

158) What are the differences between String and StringBuffer?

The differences between the String and StringBuffer is given in the table below.

No.	String	StringBuffer
1)	The String class is immutable.	The StringBuffer class is mutable.
2)	The String is slow and consumes more memory when you concat too many strings because every time it creates a new instance.	The StringBuffer is fast and consumes less memory when you concat strings.
3)	The String class overrides the equals() method of Object class. So you can compare the contents of two strings by equals() method.	The StringBuffer class doesn't override the equals() method of Object class.

159) What are the differences between StringBuffer and StringBuilder?

The differences between the StringBuffer and StringBuilder is given below.

No.	StringBuffer	StringBuilder
1)	StringBuffer is <i>synchronized</i> , i.e., thread safe. It means two threads can't call the methods of StringBuffer simultaneously.	StringBuilder is <i>non-synchronized</i> , i.e., not thread safe. It means two threads can call the methods of StringBuilder simultaneously.
2)	StringBuffer is <i>less efficient</i> than StringBuilder.	StringBuilder is <i>more efficient</i> than StringBuffer.

160) How can we create an immutable class in Java?

We can create an immutable class by defining a final class having all of its members as final. Consider the following example.

```

public final class Employee{
    final String pancardNumber;

    public Employee(String pancardNumber){
        this.pancardNumber=pancardNumber;
    }

    public String getPancardNumber(){
        return pancardNumber;
    }
}

```

161) What is the purpose of toString() method in Java?

The toString() method returns the string representation of an object. If you print any object, java compiler internally invokes the toString() method on the object. So overriding the toString() method, returns the desired output, it can be the state of an object, etc. depending upon your implementation. By overriding the toString() method of the Object class, we can return the values of the object, so we don't need to write much code. Consider the following example.

```

class Student{

```



```
int rollno;
String name;
String city;

Student(int rollno, String name, String city){
    this.rollno=rollno;
    this.name=name;
    this.city=city;
}

public String toString(){//overriding the toString() method
    return rollno+" "+name+" "+city;
}

public static void main(String args[]){
    Student s1=new Student(101,"Raj","lucknow");
    Student s2=new Student(102,"Vijay","ghaziabad");

    System.out.println(s1);//compiler writes here s1.toString()
    System.out.println(s2);//compiler writes here s2.toString()
}
}
```

Output:

```
101 Raj lucknow
102 Vijay ghaziabad
```

Mastering The Future

162) Why CharArray() is preferred over String to store the password?

String stays in the string pool until the garbage is collected. If we store the password into a string, it stays in the memory for a longer period, and anyone having the memory-dump can extract the password as clear text. On the other hand, Using CharArray allows us to set it to blank whenever we are done with the password. It avoids the security threat with the string by enabling us to control the memory.

163) Write a Java program to count the number of words present in a string?**Program:**

```
public class Test
{
    public static void main (String args[])
    {
        String s = "Sharma is a good player and he is so punctual";
        String words[] = s.split(" ");
        System.out.println("The Number of words present in the string are : "+words.length);
    }
}
```

Output

The Number of words present in the string are : 10

164) Name some classes present in java.util.regex package.

There are the following classes and interfaces present in java.util.regex package.

- MatchResult Interface
- Matcher class
- Pattern class
- PatternSyntaxException class

165) How the metacharacters are different from the ordinary characters?

Metacharacters have the special meaning to the regular expression engine. The metacharacters are ^, \$, ., *, +, etc. The regular expression engine does not consider them as the regular characters. To enable the regular expression engine treating the metacharacters as ordinary characters, we need to escape the metacharacters with the backslash.

166) Write a regular expression to validate a password. A password must start with an alphabet and followed by alphanumeric characters; Its length must be in between 8 to 20.

The regular expression for the above criteria will be: `^[a-zA-Z][a-zA-Z0-9]{8,19}` where ^ represents the start of the regex, [a-zA-Z] represents that the first character must be an alphabet,

[a-zA-Z0-9] represents the alphanumeric character, {8,19} represents that the length of the password must be in between 8 and 20.

167) What is the output of the following Java program?

```
import java.util.regex.*;  
class RegexExample2{  
    public static void main(String args[]){  
        System.out.println(Pattern.matches(".s", "as")); //line 4  
        System.out.println(Pattern.matches(".s", "mk")); //line 5  
        System.out.println(Pattern.matches(".s", "mst")); //line 6  
        System.out.println(Pattern.matches(".s", "amms")); //line 7  
        System.out.println(Pattern.matches("..s", "mas")); //line 8  
    }  
}
```

Output

```
true  
false  
false  
false  
true
```

Explanation

line 4 prints true since the second character of string is s, line 5 prints false since the second character is not s, line 6 prints false since there are more than 3 characters in the string, line 7 prints false since there are more than 2 characters in the string, and it contains more than 2 characters as well, line 8 prints true since the third character of the string is s.

Garbage Collection Interview Questions

179) What is Garbage Collection?

Garbage collection is a process of reclaiming the unused runtime objects. It is performed for memory management. In other words, we can say that It is the process of removing unused

objects from the memory to free up space and make this space available for Java Virtual Machine. Due to garbage collection java gives 0 as output to a variable whose value is not set, i.e., the variable has been defined but not initialized. For this purpose, we were using free() function in the C language and delete() in C++. In Java, it is performed automatically. So, java provides better memory management.

180) What is gc()?

The gc() method is used to invoke the garbage collector for cleanup processing. This method is found in System and Runtime classes. This function explicitly makes the Java Virtual Machine free up the space occupied by the unused objects so that it can be utilized or reused. Consider the following example for the better understanding of how the gc() method invoke the garbage collector.

```
public class TestGarbage1 {
    public void finalize(){System.out.println("object is garbage collected");}
    public static void main(String args[]){
        TestGarbage1 s1=new TestGarbage1();
        TestGarbage1 s2=new TestGarbage1();
        s1=null;
        s2=null;
        System.gc();
    }
}
```

```
object is garbage collected
object is garbage collected
```

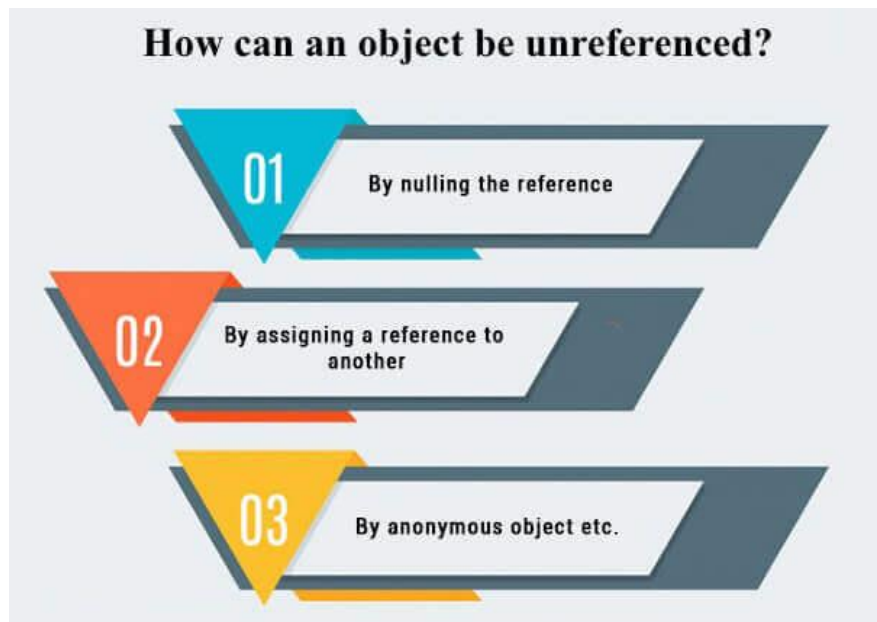
181) How is garbage collection controlled?

Garbage collection is managed by JVM. It is performed when there is not enough space in the memory and memory is running low. We can externally call the System.gc() for the garbage collection. However, it depends upon the JVM whether to perform it or not.

182) How can an object be unreferenced?

There are many ways:

- By nulling the reference
- By assigning a reference to another
- By anonymous object etc.



By nulling a reference:

```
Employee e=new Employee();
e=null;
```

By assigning a reference to another:

```
Employee e1=new Employee();
Employee e2=new Employee();
e1=e2;//now the first object referred by e1 is available for garbage collection
```

By anonymous object:

```
new Employee();
```

183) What is the purpose of the finalize() method?

The finalize() method is invoked just before the object is garbage collected. It is used to perform cleanup processing. The Garbage collector of JVM collects only those objects that are created by new keyword. So if you have created an object without new, you can use the finalize method to perform cleanup processing (destroying remaining objects). The cleanup processing is the process to free up all the resources, network which was previously used and no longer needed. It is essential to remember that it is not a reserved keyword, finalize method is present

in the object class hence it is available in every class as object class is the superclass of every class in java. Here, we must note that neither finalization nor garbage collection is guaranteed. Consider the following example.

```
public class FinalizeTest {
    int j=12;
    void add()
    {
        j=j+12;
        System.out.println("J="+j);
    }
    public void finalize()
    {
        System.out.println("Object is garbage collected");
    }
    public static void main(String[] args) {
        new FinalizeTest().add();
        System.gc();
        new FinalizeTest().add();
    }
}
```

184) Can an unreferenced object be referenced again?

Yes,

185) What kind of thread is the Garbage collector thread?

Daemon thread.

186) What is the difference between final, finally and finalize?

No.	final	finally	finalize
-----	-------	---------	----------

1)	Final is used to apply restrictions on class, method, and variable. The final class can't be inherited, final method can't be overridden, and final variable value can't be changed.	Finally is used to place important code, it will be executed whether an exception is handled or not.	Finalize is used to perform clean up processing just before an object is garbage collected.
2)	Final is a keyword.	Finally is a block.	Finalize is a method.

187) What is the purpose of the Runtime class?

Java Runtime class is used to interact with a java runtime environment. Java Runtime class provides methods to execute a process, invoke GC, get total and free memory, etc. There is only one instance of java.lang.Runtime class is available for one java application. The Runtime.getRuntime() method returns the singleton instance of Runtime class.

188) How will you invoke any external process in Java?

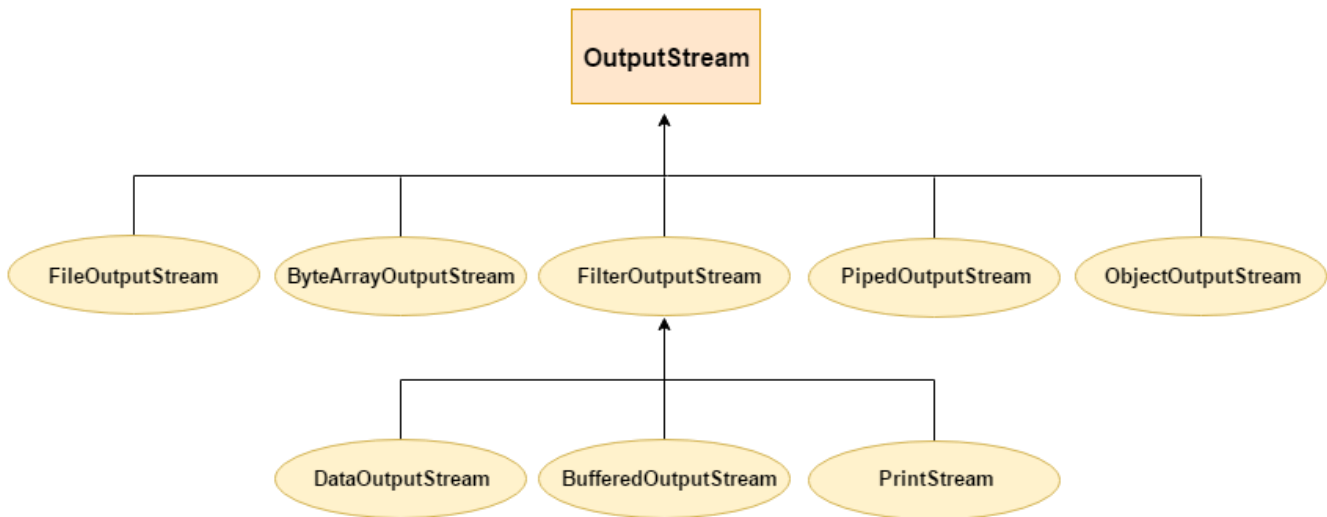
By Runtime.getRuntime().exec(?) method. Consider the following example.

```
public class Runtime1 {
    public static void main(String args[]) throws Exception {
        Runtime.getRuntime().exec("notepad");//will open a new notepad
    }
}
```

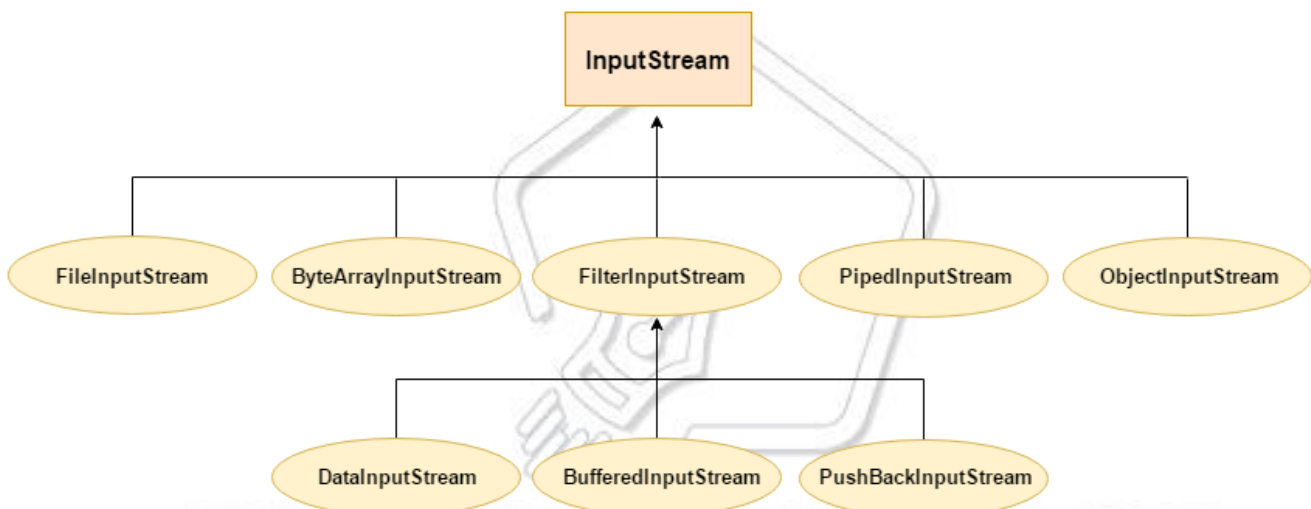
I/O Interview Questions

189) Give the hierarchy of InputStream and OutputStream classes.

OutputStream Hierarchy



InputStream Hierarchy



190) What do you understand by an IO stream?

The stream is a sequence of data that flows from source to destination. It is composed of bytes. In Java, three streams are created for us automatically.

`System.out`: standard output stream

`System.in`: standard input stream

`System.err`: standard error stream

191) What is the difference between the Reader/Writer class hierarchy and the InputStream/OutputStream class hierarchy?

The Reader/Writer class hierarchy is character-oriented, and the InputStream/OutputStream class hierarchy is byte-oriented. The ByteStream classes are used to perform input-output of 8-bit bytes whereas the CharacterStream classes are used to perform the input/output for the 16-bit Unicode system. There are many classes in the ByteStream class hierarchy, but the most frequently used classes are FileInputStream and FileOutputStream. The most frequently used classes CharacterStream class hierarchy is FileReader and FileWriter.

192) What are the super most classes for all the streams?

All the stream classes can be divided into two types of classes that are ByteStream classes and CharacterStream Classes. The ByteStream classes are further divided into InputStream classes and OutputStream classes. CharacterStream classes are also divided into Reader classes and Writer classes. The SuperMost classes for all the InputStream classes is java.io.InputStream and for all the output stream classes is java.io.OutPutStream. Similarly, for all the reader classes, the super-most class is java.io.Reader, and for all the writer classes, it is java.io.Writer.

193) What are the FileInputStream and FileOutputStream?

Java FileOutputStream is an output stream used for writing data to a file. If you have some primitive values to write into a file, use FileOutputStream class. You can write byte-oriented as well as character-oriented data through the FileOutputStream class. However, for character-oriented data, it is preferred to use FileWriter than FileOutputStream. Consider the following example of writing a byte into a file.

```
import java.io.FileOutputStream;
public class FileOutputStreamExample {
    public static void main(String args[]){
        try{
            FileOutputStream fout=new FileOutputStream("D:\\testout.txt");
            fout.write(65);
            fout.close();
            System.out.println("success...");
        } catch(Exception e){System.out.println(e);}
    }
}
```

Java FileInputStream class obtains input bytes from a file. It is used for reading byte-oriented data (streams of raw bytes) such as image data, audio, video, etc. You can also read

character-stream data. However, for reading streams of characters, it is recommended to use `FileReader` class. Consider the following example for reading bytes from a file.

```
import java.io.FileInputStream;
public class DataStreamExample {
    public static void main(String args[]){
        try{
            FileInputStream fin=new FileInputStream("D:\\testout.txt");
            int i=fin.read();
            System.out.print((char)i);

            fin.close();
        } catch(Exception e){System.out.println(e);}
    }
}
```

194) What is the purpose of using `BufferedInputStream` and `BufferedOutputStream` classes?

Java `BufferedOutputStream` class is used for buffering an output stream. It internally uses a buffer to store data. It adds more efficiency than to write data directly into a stream. So, it makes the performance fast. Whereas, Java `BufferedInputStream` class is used to read information from the stream. It internally uses the buffer mechanism to make the performance fast.

Mastering The Future

195) How to set the Permissions to a file in Java?

In Java, `FilePermission` class is used to alter the permissions set on a file. Java `FilePermission` class contains the permission related to a directory or file. All the permissions are related to the path. The path can be of two types:

D:\\IO\\-: It indicates that the permission is associated with all subdirectories and files recursively.

D:\\IO*: It indicates that the permission is associated with all directory and files within this directory excluding subdirectories.

Let's see the simple example in which permission of a directory path is granted with read permission and a file of this directory is granted for write permission.

```
import java.io.*;
import java.security.PermissionCollection;
public class FilePermissionExample{
    public static void main(String[] args) throws IOException {
        String srg = "D:\\IO Package\\java.txt";
        FilePermission file1 = new FilePermission("D:\\IO Package\\-", "read");
        PermissionCollection permission = file1.newPermissionCollection();
        permission.add(file1);
        FilePermission file2 = new FilePermission(srg, "write");
        permission.add(file2);
        if(permission.implies(new FilePermission(srg, "read,write"))) {
            System.out.println("Read, Write permission is granted for the path "+srg );
        } else {
            System.out.println("No Read, Write permission is granted for the path "+srg);
        }
    }
}
```

Output

Read, Write permission is granted for the path D:\IO Package\java.txt

196) What are FilterStreams?

FilterStream classes are used to add additional functionalities to the other stream classes. FilterStream classes act like an interface which read the data from a stream, filters it, and pass the filtered data to the caller. The FilterStream classes provide extra functionalities like adding line numbers to the destination file, etc.

197) What is an I/O filter?

An I/O filter is an object that reads from one stream and writes to another, usually altering the data in some way as it is passed from one stream to another. Many Filter classes that allow a user to make a chain using multiple input streams. It generates a combined effect on several filters.

198) In Java, How many ways you can take input from the console?

In Java, there are three ways by using which, we can take input from the console.

Using BufferedReader class: we can take input from the console by wrapping System.in into an InputStreamReader and passing it into the BufferedReader. It provides an efficient reading as the input gets buffered. Consider the following example.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
public class Person
{
    public static void main(String[] args) throws IOException
    {
        System.out.println("Enter the name of the person");
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
        String name = reader.readLine();
        System.out.println(name);
    }
}
```

Using Scanner class: The Java Scanner class breaks the input into tokens using a delimiter that is whitespace by default. It provides many methods to read and parse various primitive values. Java Scanner class is widely used to parse text for string and primitive types using a regular expression. Java Scanner class extends Object class and implements Iterator and Closeable interfaces. Consider the following example.

```
import java.util.*;
public class ScannerClassExample2 {
    public static void main(String args[]){
        String str = "Hello/This is Pentagon/My name is Abhishek.";
        //Create scanner with the specified String Object
        Scanner scanner = new Scanner(str);
        System.out.println("Boolean Result: "+scanner.hasNextBoolean());
        //Change the delimiter of this scanner
        scanner.useDelimiter("/");
        //Printing the tokenized Strings
        System.out.println("---Tokenizes String---");
        while(scanner.hasNext()){
            System.out.println(scanner.next());
        }
    }
}
```

```

    }
    //Display the new delimiter
    System.out.println("Delimiter used: " +scanner.delimiter());
    scanner.close();
    }
}

```

Using Console class: The Java Console class is used to get input from the console. It provides methods to read texts and passwords. If you read the password using the Console class, it will not be displayed to the user. The java.io.Console class is attached to the system console internally. The Console class is introduced since 1.5. Consider the following example.

```

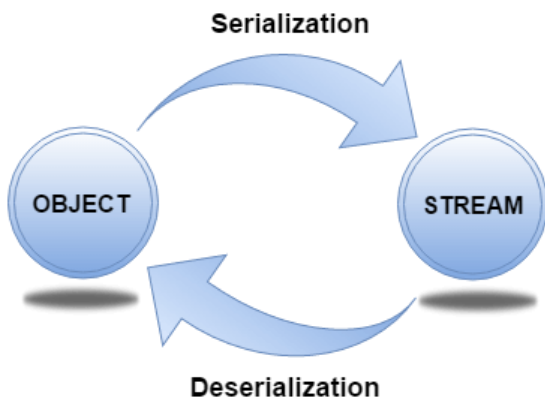
import java.io.Console;
class ReadStringTest{
public static void main(String args[]){
Console c=System.console();
System.out.println("Enter your name: ");
String n=c.readLine();
System.out.println("Welcome "+n);
}
}

```

Serialization Interview Questions

199) What is serialization?

Serialization in Java is a mechanism of writing the state of an object into a byte stream. It is used primarily in Hibernate, RMI, JPA, EJB and JMS technologies. It is mainly used to travel object's state on the network (which is known as marshaling). Serializable interface is used to perform serialization. It is helpful when you require to save the state of a program to storage such as the file. At a later point of time, the content of this file can be restored using deserialization. It is also required to implement RMI(Remote Method Invocation). With the help of RMI, it is possible to invoke the method of a Java object on one machine to another machine.



200) How can you make a class serializable in Java?

A class can become serializable by implementing the Serializable interface.

201) How can you avoid serialization in child class if the base class is implementing the Serializable interface?

It is very tricky to prevent serialization of child class if the base class is intended to implement the Serializable interface. However, we cannot do it directly, but the serialization can be avoided by implementing the `writeObject()` or `readObject()` methods in the subclass and throw `NotSerializableException` from these methods. Consider the following example.

```

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.NotSerializableException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
class Person implements Serializable
{
    String name = " ";
    public Person(String name)
    {
        this.name = name;
    }
}
  
```



```
class Employee extends Person
{
    float salary;
    public Employee(String name, float salary)
    {
        super(name);
        this.salary = salary;
    }
    private void writeObject(ObjectOutputStream out) throws IOException
    {
        throw new NotSerializableException();
    }
    private void readObject(ObjectInputStream in) throws IOException
    {
        throw new NotSerializableException();
    }
}

public class Test
{
    public static void main(String[] args)
        throws Exception
    {
        Employee emp = new Employee("Sharma", 10000);
        System.out.println("name = " + emp.name);
        System.out.println("salary = " + emp.salary);

        FileOutputStream fos = new FileOutputStream("abc.ser");
        ObjectOutputStream oos = new ObjectOutputStream(fos);

        oos.writeObject(emp);

        oos.close();
        fos.close();

        System.out.println("Object has been serialized");

        FileInputStream f = new FileInputStream("ab.txt");
        ObjectInputStream o = new ObjectInputStream(f);
    }
}
```

```

Employee emp1 = (Employee)o.readObject();

o.close();
f.close();

System.out.println("Object has been deserialized");

System.out.println("name = " + emp1.name);
System.out.println("salary = " + emp1.salary);
}
}

```

202) Can a Serialized object be transferred via network?

Yes, we can transfer a serialized object via network because the serialized object is stored in the memory in the form of bytes and can be transmitted over the network. We can also write the serialized object to the disk or the database.

203) What is Deserialization?

Deserialization is the process of reconstructing the object from the serialized state. It is the reverse operation of serialization. An `ObjectInputStream` deserializes objects and primitive data written using an `ObjectOutputStream`.

```

import java.io.*;
class Depersist{
    public static void main(String args[])throws Exception{

        ObjectInputStream in=new ObjectInputStream(new FileInputStream("f.txt"));
        Student s=(Student)in.readObject();
        System.out.println(s.id+" "+s.name);

        in.close();
    }
}

```

204) What is the transient keyword?

If you define any data member as transient, it will not be serialized. By determining transient keyword, the value of variable need not persist when it is restored.

205) What is Externalizable?

The Externalizable interface is used to write the state of an object into a byte stream in a compressed format. It is not a marker interface.

206) What is the difference between Serializable and Externalizable interface?

No.	Serializable	Externalizable
1)	The Serializable interface does not have any method, i.e., it is a marker interface.	The Externalizable interface contains is not a marker interface, It contains two methods, i.e., writeExternal() and readExternal().
2)	It is used to "mark" Java classes so that objects of these classes may get the certain capability.	The Externalizable interface provides control of the serialization logic to the programmer.
3)	It is easy to implement but has the higher performance cost.	It is used to perform the serialization and often result in better performance.
4)	No class constructor is called in serialization.	We must call a public default constructor while using this interface.

207) What is the purpose of using java.lang.Class class?

The java.lang.Class class performs mainly two tasks:

- Provides methods to get the metadata of a class at runtime.
- Provides methods to examine and change the runtime behavior of a class.

208) What are the ways to instantiate the Class class?

There are three ways to instantiate the Class class.

forName() method of Class class: The forName() method is used to load the class dynamically. It returns the instance of Class class. It should be used if you know the fully qualified name of the class. This cannot be used for primitive types.

getClass() method of Object class: It returns the instance of Class class. It should be used if you know the type. Moreover, it can be used with primitives.

the .class syntax: If a type is available, but there is no instance then it is possible to obtain a Class by appending ".class" to the name of the type. It can be used for primitive data type also.

209) What is the output of the following Java program?

```
class Simple{  
    public Simple()  
    {  
        System.out.println("Constructor of Simple class is invoked");  
    }  
    void message(){System.out.println("Hello Java");}  
}
```

```
class Test1 {  
    public static void main(String args[]){  
        try{  
            Class c=Class.forName("Simple");  
            Simple s=(Simple)c.newInstance();  
            s.message();  
        }catch(Exception e){System.out.println(e);}  
    }  
}
```

Output

Constructor of Simple class is invoked
Hello Java

Explanation

The newInstance() method of the Class class is used to invoke the constructor at runtime. In this program, the instance of the Simple class is created.

210) What is the purpose of using javap?

The javap command disassembles a class file. The javap command displays information about the fields, constructors and methods present in a class file.

Syntax

```
javap fully_class_name
```

211) Can you access the private method from outside the class?

Yes, by changing the runtime behavior of a class if the class is not secured.

Miscellaneous Interview Questions

212) What are wrapper classes?

Wrapper classes are classes that allow primitive types to be accessed as objects. In other words, we can say that wrapper classes are built-in java classes which allow the conversion of objects to primitives and primitives to objects. The process of converting primitives to objects is called autoboxing, and the process of converting objects to primitives is called unboxing. There are eight wrapper classes present in **java.lang** package is given below.

Primitive Type	Wrapper class
boolean	Boolean
char	Character

byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

213) What are autoboxing and unboxing? When does it occur?

The autoboxing is the process of converting primitive data type to the corresponding wrapper class object, eg., int to Integer. The unboxing is the process of converting wrapper class object to primitive data type. For eg., Integer to int. Unboxing and autoboxing occur automatically in Java. However, we can externally convert one into another by using the methods like `valueOf()` or `xxxValue()`.

- It can occur whenever a wrapper class object is expected, and primitive data type is provided or vice versa.
- Adding primitive types into Collection like ArrayList in Java.
- Creating an instance of parameterized classes ,e.g., ThreadLocal which expect Type.
- Java automatically converts primitive to object whenever one is required and another is provided in the method calling.
- When a primitive type is assigned to an object type.

214) What is the output of the below Java program?

```
public class Test1
{
    public static void main(String[] args) {
        Integer i = new Integer(201);
        Integer j = new Integer(201);
        if(i == j)
        {
            System.out.println("hello");
        }
    }
}
```

```

else
{
    System.out.println("bye");
}
}
}

```

Output

```
bye
```

Explanation

The Integer class caches integer values from -127 to 127. Therefore, the Integer objects can only be created in the range -128 to 127. The operator == will not work for the value greater than 127; thus **bye** is printed.

215) What is object cloning?

The object cloning is a way to create an exact copy of an object. The clone() method of the Object class is used to clone an object. The java.lang.Cloneable interface must be implemented by the class whose object clone we want to create. If we don't implement Cloneable interface, clone() method generates CloneNotSupportedException. The clone() method is defined in the Object class. The syntax of the clone() method is as follows:

protected Object clone() throws CloneNotSupportedException

216) What are the advantages and disadvantages of object cloning?

Advantage of Object Cloning

- You don't need to write lengthy and repetitive codes. Just use an abstract class with a 4- or 5-line long clone() method.
- It is the easiest and most efficient way of copying objects, especially if we are applying it to an already developed or an old project. Just define a parent class, implement Cloneable in it, provide the definition of the clone() method and the task will be done.
- Clone() is the fastest way to copy the array.

Disadvantage of Object Cloning

- To use the `Object.clone()` method, we have to change many syntaxes to our code, like implementing a `Cloneable` interface, defining the `clone()` method and handling `CloneNotSupportedException`, and finally, calling `Object.clone()`, etc.
- We have to implement the `Cloneable` interface while it does not have any methods in it. We have to use it to tell the JVM that we can perform a `clone()` on our object.
- `Object.clone()` is protected, so we have to provide our own `clone()` and indirectly call `Object.clone()` from it.
- `Object.clone()` does not invoke any constructor, so we do not have any control over object construction.
- If you want to write a clone method in a child class, then all of its superclasses should define the `clone()` method in them or inherit it from another parent class. Otherwise, the `super.clone()` chain will fail.
- `Object.clone()` supports only shallow copying, but we will need to override it if we need deep cloning.

217) What is a native method?

A native method is a method that is implemented in a language other than Java. Native methods are sometimes also referred to as foreign methods.

218) What is the purpose of the `strictfp` keyword?

Java `strictfp` keyword ensures that you will get the same result on every platform if you perform operations in the floating-point variable. The precision may differ from platform to platform that is why java programming language has provided the `strictfp` keyword so that you get the same result on every platform. So, now you have better control over the floating-point arithmetic.

219) What is the purpose of the `System` class?

The purpose of the `System` class is to provide access to system resources such as standard input and output. It cannot be instantiated. Facilities provided by `System` class are given below.

- Standard input
- Error output streams
- Standard output

- utility method to copy the portion of an array
 - utilities to load files and libraries
 - There are the three fields of Java System class, i.e., static printstream err, static inputstream in, and standard output stream.
-

220) What comes to mind when someone mentions a shallow copy in Java?

Object cloning.

221) What is a singleton class?

Singleton class is the class which can not be instantiated more than once. To make a class singleton, we either make its constructor private or use the static getInstance method. Consider the following example.

```
class Singleton{
    private static Singleton single_instance = null;
    int i;
    private Singleton ()
    {
        i=90;
    }
    public static Singleton getInstance()
    {
        if(single_instance == null)
        {
            single_instance = new Singleton();
        }
        return single_instance;
    }
}

public class Main
{
    public static void main (String args[])
    {
        Singleton first = Singleton.getInstance();
        System.out.println("First instance integer value:"+first.i);
        first.i=first.i+90;
    }
}
```

```

Singleton second = Singleton.getInstance();
System.out.println("Second instance integer value:"+second.i);
}
}

```

Java Bean Interview Questions

222) What is a JavaBean?

JavaBean is a reusable software component written in the Java programming language, designed to be manipulated visually by a software development environment, like JBuilder or VisualAge for Java. t. A JavaBean encapsulates many objects into one object so that we can access this object from multiple places. Moreover, it provides the easy maintenance. Consider the following example to create a JavaBean class.

```

//Employee.java
package mypack;
public class Employee implements java.io.Serializable{
private int id;
private String name;
public Employee(){}
public void setId(int id){this.id=id;}
public int getId(){return id;}
public void setName(String name){this.name=name;}
public String getName(){return name;}
}

```

223) What is the purpose of using the Java bean?

According to Java white paper, it is a reusable software component. A bean encapsulates many objects into one object so that we can access this object from multiple places. Moreover, it provides the easy maintenance.

224) What do you understand by the bean persistent property?

The persistence property of Java bean comes into the act when the properties, fields, and state information are saved to or retrieve from the storage.

Multithreading Interview Questions

1) What is multithreading?

Multithreading is a process of executing multiple threads simultaneously. Multithreading is used to obtain the multitasking. It consumes less memory and gives the fast and efficient performance. Its main advantages are:

- Threads share the same address space.
 - The thread is lightweight.
 - The cost of communication between the processes is low.
-

2) What is the thread?

A thread is a lightweight subprocess. It is a separate path of execution because each thread runs in a different stack frame. A process may contain multiple threads. Threads share the process resources, but still, they execute independently.

3) Differentiate between process and thread?

There are the following differences between the process and thread.

- A Program in the execution is called the process whereas; A thread is a subset of the process
 - Processes are independent whereas threads are the subset of process.
 - Process have different address space in memory, while threads contain a shared address space.
 - Context switching is faster between the threads as compared to processes.
 - Inter-process communication is slower and expensive than inter-thread communication.
 - Any change in Parent process doesn't affect the child process whereas changes in parent thread can affect the child thread.
-

4) What do you understand by inter-thread communication?

- The process of communication between synchronized threads is termed as inter-thread communication.
- Inter-thread communication is used to avoid thread polling in Java.

- The thread is paused running in its critical section, and another thread is allowed to enter (or lock) in the same critical section to be executed.
 - It can be obtained by `wait()`, `notify()`, and `notifyAll()` methods.
-

5) What is the purpose of `wait()` method in Java?

The `wait()` method is provided by the `Object` class in Java. This method is used for inter-thread communication in Java. The `java.lang.Object.wait()` is used to pause the current thread, and wait until another thread does not call the `notify()` or `notifyAll()` method. Its syntax is given below.

```
public final void wait()
```

6) Why must `wait()` method be called from the synchronized block?

We must call the `wait` method otherwise it will throw **`java.lang.IllegalMonitorStateException`** exception. Moreover, we need `wait()` method for inter-thread communication with `notify()` and `notifyAll()`. Therefore It must be present in the synchronized block for the proper and correct communication.

7) What are the advantages of multithreading?

Multithreading programming has the following advantages:

- Multithreading allows an application/program to be always reactive for input, even already running with some background tasks
 - Multithreading allows the faster execution of tasks, as threads execute independently.
 - Multithreading provides better utilization of cache memory as threads share the common memory resources.
 - Multithreading reduces the number of the required server as one server can execute multiple threads at a time.
-

8) What are the states in the lifecycle of a Thread?

A thread can have one of the following states during its lifetime:

1. **New:** In this state, a Thread class object is created using a new operator, but the thread is not alive. Thread doesn't start until we call the start() method.
2. **Runnable:** In this state, the thread is ready to run after calling the start() method. However, the thread is not yet selected by the thread scheduler.
3. **Running:** In this state, the thread scheduler picks the thread from the ready state, and the thread is running.
4. **Waiting/Blocked:** In this state, a thread is not running but still alive, or it is waiting for the other thread to finish.
5. **Dead/Terminated:** A thread is in terminated or dead state when the run() method exits.

09) Differentiate between the Thread class and Runnable interface for creating a Thread?

The Thread can be created by using two ways.

- By extending the Thread class
- By implementing the Runnable interface

However, the primary differences between both the ways are given below:

- By extending the Thread class, we cannot extend any other class, as Java does not allow multiple inheritances while implementing the Runnable interface; we can also extend other base class(if required).
- By extending the Thread class, each of thread creates the unique object and associates with it while implementing the Runnable interface; multiple threads share the same object
- Thread class provides various inbuilt methods such as getPriority(), isAlive and many more while the Runnable interface provides a single method, i.e., run().

10) What does join() method?

The join() method waits for a thread to die. In other words, it causes the currently running threads to stop executing until the thread it joins with completes its task. Join method is overloaded in Thread class in the following ways.

- public void join()throws InterruptedException
- public void join(long milliseconds)throws InterruptedException

11) Describe the purpose and working of sleep() method.

The sleep() method in java is used to block a thread for a particular time, which means it pause the execution of a thread for a specific time. There are two methods of doing so.

Syntax:

- public static void sleep(long milliseconds)throws InterruptedException
- public static void sleep(long milliseconds, int nanos)throws InterruptedException

Working of sleep() method

When we call the sleep() method, it pauses the execution of the current thread for the given time and gives priority to another thread(if available). Moreover, when the waiting time completed then again previous thread changes its state from waiting to runnable and comes in running state, and the whole process works so on till the execution doesn't complete.

12) What is the difference between wait() and sleep() method?

wait()	sleep()
1) The wait() method is defined in Object class.	The sleep() method is defined in Thread class.
2) The wait() method releases the lock.	The sleep() method doesn't release the lock.

13) Is it possible to start a thread twice?

No, we cannot restart the thread, as once a thread started and executed, it goes to the Dead state. Therefore, if we try to start a thread twice, it will give a runtimeException "java.lang.IllegalThreadStateException". Consider the following example.

```
public class Multithread1 extends Thread
{
    public void run()
    {
```



```

    try {

        System.out.println("thread is executing now.....");

    } catch(Exception e) {

    }

}

public static void main (String[] args) {

    Multithread1 m1= new Multithread1();

    m1.start();

    m1.start();

}

}

```

Output

thread is executing now.....

Exception in thread "main" java.lang.IllegalThreadStateException

at java.lang.Thread.start(Thread.java:708)

at Multithread1.main(Multithread1.java:13)

14) Can we call the run() method instead of start()?

Yes, calling run() method directly is valid, but it will not work as a thread instead it will work as a normal object. There will not be context-switching between the threads. When we call the start() method, it internally calls the run() method, which creates a new stack for a thread while directly calling the run() will not create a new stack.

15) What is the synchronization?

Synchronization is the capability to control the access of multiple threads to any shared resource. It is used:

1. To prevent thread interference.
2. To prevent consistency problem.

When the multiple threads try to do the same task, there is a possibility of an erroneous result, hence to remove this issue, Java uses the process of synchronization which allows only one thread to be executed at a time. Synchronization can be achieved in three ways:

- by the synchronized method
- by synchronized block
- by static synchronization

Syntax for synchronized block

```
synchronized(object reference expression)
{
    //code block
}
```

16) What is the purpose of the Synchronized block?

The Synchronized block can be used to perform synchronization on any specific resource of the method. Only one thread at a time can execute on a particular resource, and all other threads which attempt to enter the synchronized block are blocked.

- Synchronized block is used to lock an object for any shared resource.
- The scope of the synchronized block is limited to the block on which, it is applied. Its scope is smaller than a method.

17) What is the difference between notify() and notifyAll()?

The notify() is used to unblock one waiting thread whereas notifyAll() method is used to unblock all the threads in waiting state.

18) What is the deadlock?

Deadlock is a situation in which every thread is waiting for a resource which is held by some other waiting thread. In this situation, Neither of the thread executes nor it gets the chance to be executed. Instead, there exists a universal waiting state among all the threads. Deadlock is a very complicated situation which can break our code at runtime.

19) How to detect a deadlock condition? How can it be avoided?

We can detect the deadlock condition by running the code on cmd and collecting the Thread Dump, and if any deadlock is present in the code, then a message will appear on cmd.

Ways to avoid the deadlock condition in Java:

- **Avoid Nested lock:** Nested lock is the common reason for deadlock as deadlock occurs when we provide locks to various threads so we should give one lock to only one thread at some particular time.
- **Avoid unnecessary locks:** we must avoid the locks which are not required.
- **Using thread join:** Thread join helps to wait for a thread until another thread doesn't finish its execution so we can avoid deadlock by maximum use of join method.

20) What is Thread Scheduler in java?

In Java, when we create the threads, they are supervised with the help of a Thread Scheduler, which is the part of JVM. Thread scheduler is only responsible for deciding which thread should be executed. Thread scheduler uses two mechanisms for scheduling the threads: Preemptive and Time Slicing.

Java thread scheduler also works for deciding the following for a thread:

- It selects the priority of the thread.
- It determines the waiting time for a thread
- It checks the Nature of thread

21) Does each thread have its stack in multithreaded programming?

Yes, in multithreaded programming every thread maintains its own or separate stack area in memory due to which every thread is independent of each other.

22) How is the safety of a thread achieved?

If a method or class object can be used by multiple threads at a time without any race condition, then the class is thread-safe. Thread safety is used to make a program safe to use in multithreaded programming. It can be achieved by the following ways:

- Synchronization
 - Using Volatile keyword
 - Using a lock based mechanism
 - Use of atomic wrapper classes
-

23) What is race-condition?

A Race condition is a problem which occurs in the multithreaded programming when various threads execute simultaneously accessing a shared resource at the same time. The proper use of synchronization can avoid the Race condition.

24) What is the volatile keyword in java?

Volatile keyword is used in multithreaded programming to achieve the thread safety, as a change in one volatile variable is visible to all other threads so one variable can be used by one thread at a time.

Collection framework interview questions

25) What is the Collection framework in Java?

Collection Framework is a combination of classes and interface, which is used to store and manipulate the data in the form of objects. It provides various classes such as ArrayList, Vector, Stack, and HashSet, etc. and interfaces such as List, Queue, Set, etc. for this purpose.

26) What are the main differences between array and collection?

Array and Collection are somewhat similar regarding storing the references of objects and manipulating the data, but they differ in many ways. The main differences between the array and Collection are defined below:

- Arrays are always of fixed size, i.e., a user can not increase or decrease the length of the array according to their requirement or at runtime, but In Collection, size can be changed dynamically as per need.
- Arrays can only store homogeneous or similar type objects, but in Collection, heterogeneous objects can be stored.
- Arrays cannot provide the ?ready-made? methods for user requirements as sorting, searching, etc. but Collection includes readymade methods to use.

27) Explain various interfaces used in Collection framework?

Collection framework implements various interfaces, Collection interface and Map interface (java.util.Map) are the mainly used interfaces of Java Collection Framework. List of interfaces of Collection Framework is given below:

1. Collection interface: Collection (java.util.Collection) is the primary interface, and every collection must implement this interface.

Syntax:

1. **public interface** Collection<E>**extends** Iterable

Where <E> represents that this interface is of Generic type

2. List interface: List interface extends the Collection interface, and it is an ordered collection of objects. It contains duplicate elements. It also allows random access of elements.

Syntax:

1. **public interface** List<E> **extends** Collection<E>

3. Set interface: Set (java.util.Set) interface is a collection which cannot contain duplicate elements. It can only include inherited methods of Collection interface

Syntax:

1. **public interface** Set<E> **extends** Collection<E>

Queue interface: Queue (java.util.Queue) interface defines queue data structure, which stores the elements in the form FIFO (first in first out).

Syntax:

1. **public interface Queue<E> extends Collection<E>**

4. Dequeue interface: it is a double-ended-queue. It allows the insertion and removal of elements from both ends. It implants the properties of both Stack and queue so it can perform LIFO (Last in first out) stack and FIFO (first in first out) queue, operations.

Syntax:

1. **public interface Dequeue<E> extends Queue<E>**

5. Map interface: A Map (java.util.Map) represents a key, value pair storage of elements. Map interface does not implement the Collection interface. It can only contain a unique key but can have duplicate elements. There are two interfaces which implement Map in java that are Map interface and Sorted Map.

28) What is the difference between ArrayList and Vector?

No.	ArrayList	Vector
1)	ArrayList is not synchronized.	Vector is synchronized.
2)	ArrayList is not a legacy class.	Vector is a legacy class.
3)	ArrayList increases its size by 50% of the array size.	Vector increases its size by doubling the array size.
4)	ArrayList is not ?thread-safe? as it is not synchronized.	Vector list is ?thread-safe? as it?s every method is synchronized.

29) What is the difference between ArrayList and LinkedList?

No.	ArrayList	LinkedList
1)	ArrayList uses a dynamic array.	LinkedList uses a doubly linked list.
2)	ArrayList is not efficient for manipulation because too much is required.	LinkedList is efficient for manipulation.

3)	ArrayList is better to store and fetch data.	LinkedList is better to manipulate data.
4)	ArrayList provides random access.	LinkedList does not provide random access.
5)	ArrayList takes less memory overhead as it stores only object	LinkedList takes more memory overhead, as it stores the object as well as the address of that object.

30) What is the difference between Iterator and ListIterator?

Iterator traverses the elements in the forward direction only whereas ListIterator traverses the elements into forward and backward direction.

No.	Iterator	ListIterator
1)	The Iterator traverses the elements in the forward direction only.	ListIterator traverses the elements in backward and forward directions both.
2)	The Iterator can be used in List, Set, and Queue.	ListIterator can be used in List only.
3)	The Iterator can only perform remove operation while traversing the collection.	ListIterator can perform ?add,? ?remove,? and ?set? operation while traversing the collection.

31) What is the difference between Iterator and Enumeration?

No.	Iterator	Enumeration
1)	The Iterator can traverse legacy and non-legacy elements.	Enumeration can traverse only legacy elements.
2)	The Iterator is fail-fast.	Enumeration is not fail-fast.
3)	The Iterator is slower than Enumeration.	Enumeration is faster than Iterator.

4)	The Iterator can perform remove operation while traversing the collection.	The Enumeration can perform only traverse operation on the collection.
----	--	--

32) What is the difference between List and Set?

The List and Set both extend the collection interface. However, there are some differences between the both which are listed below.

- The List can contain duplicate elements whereas Set includes unique items.
- The List is an ordered collection which maintains the insertion order whereas Set is an unordered collection which does not preserve the insertion order.
- The List interface contains a single legacy class which is Vector class whereas Set interface does not have any legacy class.
- The List interface can allow n number of null values whereas Set interface only allows a single null value.

33) What is the difference between HashSet and TreeSet?

The HashSet and TreeSet, both classes, implement Set interface. The differences between the both are listed below.

- HashSet maintains no order whereas TreeSet maintains ascending order.
- HashSet implemented by hash table whereas TreeSet implemented by a Tree structure.
- HashSet performs faster than TreeSet.
- HashSet is backed by HashMap whereas TreeSet is backed by TreeMap.

34) What is the difference between Set and Map?

The differences between the Set and Map are given below.

- Set contains values only whereas Map contains key and values both.
- Set contains unique values whereas Map can contain unique Keys with duplicate values.
- Set holds a single number of null value whereas Map can include a single null key with n number of null values.

35) What is the difference between HashSet and HashMap?

The differences between the HashSet and HashMap are listed below.

- HashSet contains only values whereas HashMap includes the entry (key, value). HashSet can be iterated, but HashMap needs to convert into Set to be iterated.
- HashSet implements Set interface whereas HashMap implements the Map interface
- HashSet cannot have any duplicate value whereas HashMap can contain duplicate values with unique keys.
- HashSet contains the only single number of null value whereas HashMap can hold a single null key with n number of null values.

36) What is the difference between HashMap and TreeMap?

The differences between the HashMap and TreeMap are given below.

- HashMap maintains no order, but TreeMap maintains ascending order.
- HashMap is implemented by hash table whereas TreeMap is implemented by a Tree structure.
- HashMap can be sorted by Key or value whereas TreeMap can be sorted by Key.
- HashMap may contain a null key with multiple null values whereas TreeMap cannot hold a null key but can have multiple null values.

37) What is the difference between HashMap and Hashtable?

No.	HashMap	Hashtable
1)	HashMap is not synchronized.	Hashtable is synchronized.
2)	HashMap can contain one null key and multiple null values.	Hashtable cannot contain any null key or null value.
3)	HashMap is not thread-safe, so it is useful for non-threaded applications.	Hashtable is thread-safe, and it can be shared between various threads.
4)	4) HashMap inherits the AbstractMap class	Hashtable inherits the Dictionary class.

38) What is the difference between Collection and Collections?

The differences between the Collection and Collections are given below.

- The Collection is an interface whereas Collections is a class.
- The Collection interface provides the standard functionality of data structure to List, Set, and Queue. However, Collections class is to sort and synchronize the collection elements.
- The Collection interface provides the methods that can be used for data structure whereas Collections class provides the static methods which can be used for various operation on a collection.

39) What is the difference between Comparable and Comparator?

No.	Comparable	Comparator
1)	Comparable provides only one sort of sequence.	The Comparator provides multiple sorts of sequences.
2)	It provides one method named compareTo().	It provides one method named compare().
3)	It is found in java.lang package.	It is located in java.util package.
4)	If we implement the Comparable interface, The actual class is modified.	The actual class is not changed.

40) What does the hashCode() method?

- The hashCode() method returns a hash code value (an integer number).
- The hashCode() method returns the same integer number if two keys (by calling equals() method) are identical.
- However, it is possible that two hash code numbers can have different or the same keys.
- If two objects do not produce an equal result by using the equals() method, then the hashCode() method will provide the different integer result for both the objects.

41) Why we override equals() method?

The equals method is used to check whether two objects are the same or not. It needs to be overridden if we want to check the objects based on the property.

For example, Employee is a class that has 3 data members: id, name, and salary. However, we want to check the equality of employee object by the salary. Then, we need to override the equals() method.

42) How to synchronize List, Set and Map elements?

Yes, Collections class provides methods to make List, Set or Map elements as synchronized:

```
public static List synchronizedList(List l){}
```

```
public static Set synchronizedSet(Set s){}
```

```
public static SortedSet synchronizedSortedSet(SortedSet s){}
```

```
public static Map synchronizedMap(Map m){}
```

```
public static SortedMap synchronizedSortedMap(SortedMap m){}
```

43) What is the advantage of the generic collection?

There are three main advantages of using the generic collection.

- If we use the generic class, we don't need typecasting.
 - It is type-safe and checked at compile time.
 - Generic confirms the stability of the code by making it bug detectable at compile time.
-

44) What is the Dictionary class?

The Dictionary class provides the capability to store key-value pairs.

45) What is the default size of load factor in hashing based collection?

The default size of load factor is **0.75**. The default capacity is computed as initial capacity * load factor. For example, $16 * 0.75 = 12$. So, 12 is the default capacity of Map.

46) What is the difference between Array and ArrayList?

The main differences between the Array and ArrayList are given below.

SN	Array	ArrayList
1	The Array is of fixed size, means we cannot resize the array as per need.	ArrayList is not of the fixed size we can change the size dynamically.
2	Arrays are of the static type.	ArrayList is of dynamic size.
3	Arrays can store primitive data types as well as objects.	ArrayList cannot store the primitive data types it can only store the objects.

47) What is the difference between the length of an Array and size of ArrayList?

The length of an array can be obtained using the property of length whereas ArrayList does not support length property, but we can use size() method to get the number of objects in the list.

Finding the length of the array

```
int [] array = new int[4];
```

```
System.out.println("The size of the array is " + array.length);
```

Finding the size of the ArrayList

```
ArrayList<String> list=new ArrayList<String>();
```

```
list.add("ankit");
```

```
list.add("nippun");
```

```
System.out.println(list.size());
```

48) When to use ArrayList and LinkedList?

LinkedLists are better to use for the update operations whereas ArrayLists are better to use for the search operations.

Theory based programming questions

1. Question: Demonstrate automatic garbage collection in Java with a simple example.

```
public class GarbageCollectionDemo {

    public static void main(String[] args) {

        GarbageCollectionDemo obj = new GarbageCollectionDemo();

        obj = null;

        System.gc();

        System.out.println("Garbage collection requested.");

    }

    @Override

    protected void finalize() throws Throwable {

        System.out.println("Garbage collector called.");

    }

}
```

2. Question: Demonstrate automatic garbage collection in Java with a simple example.

```
public class GarbageCollectionDemo {

    public static void main(String[] args) {

        GarbageCollectionDemo obj = new GarbageCollectionDemo();

        obj = null;

        System.gc();

        System.out.println("Garbage collection requested.");

    }

}
```

```

    }

    @Override
    protected void finalize() throws Throwable {
        System.out.println("Garbage collector called.");
    }
}

```

Class and Objects

3. Question: Create a simple class and object example in Java.

```

class Car {
    String model;
    int year;

    void display() {
        System.out.println("Model: " + model);
        System.out.println("Year: " + year);
    }
}

public class CarDemo {
    public static void main(String[] args) {
        Car car = new Car();
        car.model = "Toyota";
        car.year = 2020;
        car.display();
    }
}

```

4. Question: Write a Java program to create an object and call a method to display object properties.

```

class Student {
    String name;
    int age;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

```



```

    }

    public class StudentDemo {
        public static void main(String[] args) {
            Student student = new Student();
            student.name = "John";
            student.age = 22;
            student.display();
        }
    }

```

5. Question: Demonstrate the concept of constructors in Java.

```

class Book {
    String title;
    double price;

    // Constructor
    Book(String title, double price) {
        this.title = title;
        this.price = price;
    }

    void display() {
        System.out.println("Title: " + title);
        System.out.println("Price: $" + price);
    }
}

public class BookDemo {
    public static void main(String[] args) {
        Book book = new Book("Java Programming", 45.99);
        book.display();
    }
}

```

Java Keywords

6. Question: Demonstrate the use of static keyword in Java.

```

class StaticDemo {
    static int count = 0;

    StaticDemo() {

```

```

        count++;
    }

    static void displayCount() {
        System.out.println("Count: " + count);
    }
}

public class TestStatic {
    public static void main(String[] args) {
        StaticDemo obj1 = new StaticDemo();
        StaticDemo obj2 = new StaticDemo();
        StaticDemo.displayCount(); // Output: Count: 2
    }
}

```

7. Question: Demonstrate the use of final keyword in Java.

```

final class FinalClass {
    final void display() {
        System.out.println("This is a final method in a final class.");
    }
}

public class FinalDemo {
    public static void main(String[] args) {
        FinalClass obj = new FinalClass();
        obj.display();
    }
}

```

8. Question: Demonstrate the use of this keyword in Java.

```

class ThisDemo {
    int num;

    ThisDemo(int num) {
        this.num = num;
    }

    void display() {
        System.out.println("Number: " + this.num);
    }
}

```

```

public class TestThis {
    public static void main(String[] args) {
        ThisDemo obj = new ThisDemo(5);
        obj.display(); // Output: Number: 5
    }
}

```

Identifiers

9. Question: Write a Java program to demonstrate valid identifiers.

```

public class IdentifierDemo {
    public static void main(String[] args) {
        int $amount = 100;
        int _balance = 200;
        int age1 = 30;

        System.out.println("$amount: " + $amount);
        System.out.println("_balance: " + _balance);
        System.out.println("age1: " + age1);
    }
}

```

10. Question: Write a Java program that uses both upper and lower case identifiers.

```

public class CaseSensitiveDemo {
    public static void main(String[] args) {
        int Age = 25;
        int age = 30;

        System.out.println("Age: " + Age);
        System.out.println("age: " + age);
    }
}

```

11. Question: Write a Java program demonstrating the use of class and method identifiers.

```

class Person {
    String name;

    void setName(String name) {
        this.name = name;
    }
}

```

```

    }

    void display() {
        System.out.println("Name: " + name);
    }
}

public class IdentifierExample {
    public static void main(String[] args) {
        Person person = new Person();
        person.setName("Alice");
        person.display();
    }
}

```

Compilation and Execution

12.Question: Write a Java program to demonstrate the steps of compilation and execution.

```

public class CompileExecuteDemo {
    public static void main(String[] args) {
        System.out.println("This program demonstrates compilation and
        execution.");
    }
}

```

// Steps:

// 1. Save the file as CompileExecuteDemo.java

// 2. Open the terminal and navigate to the directory containing the file.

// 3. Compile the program: javac CompileExecuteDemo.java

// 4. Execute the program: java CompileExecuteDemo

13.Question: Create a simple Java program and explain how to compile and run it.

```

public class SimpleProgram {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}

```

// Steps:

// 1. Save the file as SimpleProgram.java

// 2. Open the terminal and navigate to the directory containing the file.

```
// 3. Compile the program: javac SimpleProgram.java
// 4. Run the program: java SimpleProgram
```

14.Question: Write a Java program to print your name and explain how to execute it.

```
public class PrintName {
    public static void main(String[] args) {
        System.out.println("My name is John Doe.");
    }
}
```

// Steps:

```
// 1. Save the file as PrintName.java
// 2. Open the terminal and navigate to the directory containing the file.
// 3. Compile the program: javac PrintName.java
// 4. Run the program: java PrintName
```

Data Members

15.Question: Create a class with data members and a method to display their values.

```
class Employee {
    String name;
    int id;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
    }
}
```

```
public class DataMembersDemo {
    public static void main(String[] args) {
        Employee emp = new Employee();
        emp.name = "Alice";
        emp.id = 101;
        emp.display();
    }
}
```

16.Question: Write a Java program with private data members and public methods.

```
class Account {
    private double balance;
```

```

    public void setBalance(double balance) {
        this.balance = balance;
    }

    public double getBalance() {
        return balance;
    }
}

public class PrivateDataMembers {
    public static void main(String[] args) {
        Account acc = new Account();
        acc.setBalance(5000);
        System.out.println("Balance: " + acc.getBalance());
    }
}

```

17.Question: Demonstrate the use of static data members in Java.

```

class Counter {
    static int count = 0;

    Counter() {
        count++;
    }

    static void displayCount() {
        System.out.println("Count: " + count);
    }
}

```

```

public class StaticDataMembers {
    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        Counter.displayCount(); // Output: Count: 2
    }
}

```

Variables and Constants

18.Question: Write a Java program to demonstrate the use of variables.

```
public class VariableDemo {
    public static void main(String[] args) {
        int age = 25;
        String name = "John";
        double salary = 50000.50;

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Salary: $" + salary);
    }
}
```

19.Question: Create a Java program to demonstrate the use of constants.

```
public class ConstantDemo {
    public static final double PI = 3.14159;

    public static void main(String[] args) {
        System.out.println("Value of PI: " + PI);
    }
}
```

20.Question: Write a Java program to swap two variables.

```
public class SwapVariables {
    public static void main(String[] args) {
        int a = 5, b = 10;
        System.out.println("Before Swap: a = " + a + ", b = " + b);

        // Swapping
        int temp = a;
        a = b;
        b = temp;

        System.out.println("After Swap: a = " + a + ", b = " + b);
    }
}
```


21.Question: Write a Java program to demonstrate different data types.

```
public class DataTypesDemo {
    public static void main(String[] args) {
        int intVar = 10;
        double doubleVar = 20.5;
        char charVar = 'A';
        boolean boolVar = true;
        String strVar = "Hello";

        System.out.println("Integer: " + intVar);
        System.out.println("Double: " + doubleVar);
        System.out.println("Char: " + charVar);
        System.out.println("Boolean: " + boolVar);
        System.out.println("String: " + strVar);
    }
}
```

22.Question: Create a program to perform arithmetic operations on different data types.

```
public class ArithmeticOperations {
    public static void main(String[] args) {
        int a = 10, b = 5;
        double x = 10.5, y = 5.5;

        System.out.println("Addition: " + (a + b));
        System.out.println("Subtraction: " + (x - y));
        System.out.println("Multiplication: " + (a * y));
        System.out.println("Division: " + (x / b));
    }
}
```

23.Question: Write a program to demonstrate type casting in Java.

```
public class TypeCastingDemo {
    public static void main(String[] args) {
        // Implicit casting
        int intVar = 100;
        double doubleVar = intVar;

        System.out.println("Integer: " + intVar);
        System.out.println("Double (after implicit casting): " + doubleVar);

        // Explicit casting
```

```

doubleVar = 100.25;
intVar = (int) doubleVar;

System.out.println("Double: " + doubleVar);
System.out.println("Integer (after explicit casting): " + intVar);
}
}

```

Object Creation

24.Question: Write a Java program to create an object and call its method.

```

class Dog {
    void bark() {
        System.out.println("Woof Woof");
    }
}

public class ObjectCreation {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.bark();
    }
}

```

25.Question: Create a Java program to demonstrate object creation using constructors.

```

class Laptop {
    String brand;

    Laptop(String brand) {
        this.brand = brand;
    }

    void displayBrand() {
        System.out.println("Brand: " + brand);
    }
}

public class ConstructorDemo {
    public static void main(String[] args) {
        Laptop laptop = new Laptop("Dell");
    }
}

```

```

        laptop.displayBrand();
    }
}

```

26.Question: Demonstrate the use of multiple objects in a Java program.

```

class Person {
    String name;

    Person(String name) {
        this.name = name;
    }

    void display() {
        System.out.println("Name: " + name);
    }
}

public class MultipleObjects {
    public static void main(String[] args) {
        Person person1 = new Person("Alice");
        Person person2 = new Person("Bob");

        person1.display();
        person2.display();
    }
}

```

Types of Variables

27.Question: Write a Java program to demonstrate local and global variables.

```

class VariableTypes {
    int globalVar = 10; // Global variable

    void display() {
        int localVar = 20; // Local variable
        System.out.println("Global Variable: " + globalVar);
        System.out.println("Local Variable: " + localVar);
    }
}

public class VariableTypesDemo {

```

```

        public static void main(String[] args) {
            VariableTypes obj = new VariableTypes();
            obj.display();
        }
    }
}

```

28.Question: Create a Java program to demonstrate static and instance variables.

```

class Employee {
    static String company = "TechCorp";
    String name;

    Employee(String name) {
        this.name = name;
    }

    void display() {
        System.out.println("Company: " + company);
        System.out.println("Name: " + name);
    }
}

public class StaticInstanceDemo {
    public static void main(String[] args) {
        Employee emp1 = new Employee("Alice");
        Employee emp2 = new Employee("Bob");
        emp1.display();
        emp2.display();
    }
}

```

29.Question: Demonstrate the use of final variables in Java.

```

public class FinalVariableDemo {
    final int MAX_VALUE = 100;

    void display() {
        System.out.println("Max Value: " + MAX_VALUE);
    }

    public static void main(String[] args) {
        FinalVariableDemo obj = new FinalVariableDemo();
        obj.display();
    }
}

```

```

    }
}

```

String

30.Question: Write a Java program to demonstrate basic string operations.

```

public class StringOperations {
    public static void main(String[] args) {
        String str = "Hello, World!";

        System.out.println("Original String: " + str);
        System.out.println("Length: " + str.length());
        System.out.println("Uppercase: " + str.toUpperCase());
        System.out.println("Substring: " + str.substring(7));
    }
}

```

31.Question: Create a Java program to compare two strings.

```

public class StringComparison {
    public static void main(String[] args) {
        String str1 = "Java";
        String str2 = "Java";
        String str3 = "Python";

        System.out.println("str1 equals str2: " + str1.equals(str2));
        System.out.println("str1 equals str3: " + str1.equals(str3));
    }
}

```

32.Question: Write a program to concatenate two strings in Java.

```

public class StringConcatenation {
    public static void main(String[] args) {
        String str1 = "Hello";
        String str2 = "World";

        String result = str1 + " " + str2;
        System.out.println("Concatenated String: " + result);
    }
}

```

Object-Oriented Programming (OOP)

Methods

33.Question: Write a Java program to demonstrate a simple method with no parameters.

```
class SimpleMethod {
    void displayMessage() {
        System.out.println("Hello from the method!");
    }
}

public class MethodDemo {
    public static void main(String[] args) {
        SimpleMethod obj = new SimpleMethod();
        obj.displayMessage();
    }
}
```

34.Question: Create a Java program to demonstrate method overloading.

```
class OverloadingDemo {
    void display(int a) {
        System.out.println("Value: " + a);
    }

    void display(String str) {
        System.out.println("String: " + str);
    }
}
```

```
public class MethodOverloading {
    public static void main(String[] args) {
        OverloadingDemo obj = new OverloadingDemo();
        obj.display(5);
        obj.display("Hello");
    }
}
```

35.Question: Write a Java program to demonstrate the use of this keyword.

```
class ThisKeywordDemo {
    int num;

    ThisKeywordDemo(int num) {
        this.num = num;
    }
}
```

```

    }

    void display() {
        System.out.println("Number: " + this.num);
    }
}

public class ThisDemo {
    public static void main(String[] args) {
        ThisKeywordDemo obj = new ThisKeywordDemo(10);
        obj.display();
    }
}

```

JVM, JDK, and JRE

36.Question: Write a Java program to print JVM specifications.

```

public class JvmSpecs {
    public static void main(String[] args) {
        System.out.println("JVM Name: " +
System.getProperty("java.vm.name"));
        System.out.println("JVM Version: " +
System.getProperty("java.vm.version"));
        System.out.println("JVM Vendor: " +
System.getProperty("java.vm.vendor"));
    }
}

```

37.Question: Create a Java program to print JDK and JRE paths.

```

public class JdkJrePaths {
    public static void main(String[] args) {
        System.out.println("JDK Path: " + System.getProperty("java.home"));
        System.out.println("JRE Path: " +
System.getProperty("java.runtime.version"));
    }
}

```

38.Question: Write a Java program to display JRE and JDK versions.

```

public class JavaVersions {
    public static void main(String[] args) {
        System.out.println("JRE Version: " +
System.getProperty("java.runtime.version"));
    }
}

```



```
        System.out.println("JDK Version: " + System.getProperty("java.version"));
    }
}
```

Inheritance

39.Question: Write a Java program to demonstrate single inheritance.

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}
```

```
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
```

```
public class SingleInheritance {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat();
        dog.bark();
    }
}
```

40.Question: Create a Java program to demonstrate method overriding.

```
class Parent {
    void display() {
        System.out.println("Parent class display method");
    }
}
```

```
class Child extends Parent {
    @Override
    void display() {
        System.out.println("Child class display method");
    }
}
```

```

public class MethodOverriding {
    public static void main(String[] args) {
        Child child = new Child();
        child.display();
    }
}

```

41.Question: Write a Java program to demonstrate multiple inheritance using interfaces.

```

interface A {
    void methodA();
}

```

```

interface B {
    void methodB();
}

```

```

class C implements A, B {
    public void methodA() {
        System.out.println("Method A");
    }

```

```

        public void methodB() {
            System.out.println("Method B");
        }
    }

```

```

public class MultipleInheritance {
    public static void main(String[] args) {
        C obj = new C();
        obj.methodA();
        obj.methodB();
    }
}

```

Constructors

42.Question: Write a Java program to demonstrate a default constructor.

```

class DefaultConstructor {
    DefaultConstructor() {
        System.out.println("Default constructor called");
    }
}

```

```

    }
}

```

```

public class DefaultConstructorDemo {
    public static void main(String[] args) {
        DefaultConstructor obj = new DefaultConstructor();
    }
}

```

43.Question: Create a Java program to demonstrate a parameterized constructor.

```

class ParameterizedConstructor {
    int x;

    ParameterizedConstructor(int x) {
        this.x = x;
    }

    void display() {
        System.out.println("Value: " + x);
    }
}

```

```

public class ParameterizedConstructorDemo {
    public static void main(String[] args) {
        ParameterizedConstructor obj = new ParameterizedConstructor(10);
        obj.display();
    }
}

```

44.Question: Write a Java program to demonstrate constructor overloading.

```

class ConstructorOverloading {
    int x;
    String str;

    ConstructorOverloading() {
        this.x = 0;
        this.str = "Default";
    }

    ConstructorOverloading(int x) {
        this.x = x;
        this.str = "Default";
    }
}

```

```

    }

    ConstructorOverloading(int x, String str) {
        this.x = x;
        this.str = str;
    }

    void display() {
        System.out.println("Value: " + x);
        System.out.println("String: " + str);
    }
}

public class ConstructorOverloadingDemo {
    public static void main(String[] args) {
        ConstructorOverloading obj1 = new ConstructorOverloading();
        ConstructorOverloading obj2 = new ConstructorOverloading(10);
        ConstructorOverloading obj3 = new ConstructorOverloading(20, "Hello");

        obj1.display();
        obj2.display();
        obj3.display();
    }
}

```

Type Casting

45.Question: Write a Java program to demonstrate primitive type casting.

```

public class PrimitiveTypeCasting {
    public static void main(String[] args) {
        int intVar = 100;
        double doubleVar = intVar; // Implicit casting

        System.out.println("Integer: " + intVar);
        System.out.println("Double (after implicit casting): " + doubleVar);

        doubleVar = 100.25;
        intVar = (int) doubleVar; // Explicit casting

        System.out.println("Double: " + doubleVar);
    }
}

```

```

        System.out.println("Integer (after explicit casting): " + intVar);
    }
}

```

46.Question: Create a Java program to demonstrate non-primitive type casting.

```

class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

public class NonPrimitiveCasting {
    public static void main(String[] args) {
        Animal animal = new Dog(); // Upcasting
        animal.makeSound();

        Dog dog = (Dog) animal; // Downcasting
        dog.bark();
    }
}

```

47.Question: Write a Java program to demonstrate casting between wrapper classes and primitives.

```

public class WrapperCasting {
    public static void main(String[] args) {
        int intVar = 10;
        Integer intObj = Integer.valueOf(intVar); // Boxing
        int newIntVar = intObj.intValue(); // Unboxing

        System.out.println("Primitive int: " + intVar);
        System.out.println("Boxed Integer: " + intObj);
        System.out.println("Unboxed int: " + newIntVar);

        double doubleVar = 20.5;
        Double doubleObj = doubleVar; // Autoboxing
    }
}

```

```

        double newDoubleVar = doubleObj; // Auto-unboxing

        System.out.println("Primitive double: " + doubleVar);
        System.out.println("Autoboxed Double: " + doubleObj);
        System.out.println("Auto-unboxed double: " + newDoubleVar);
    }
}

```

Method Binding

48.Question: Write a Java program to demonstrate static binding.

```

class StaticBinding {
    static void display() {
        System.out.println("Static method display");
    }
}

public class StaticBindingDemo {
    public static void main(String[] args) {
        StaticBinding.display();
    }
}

```

49.Question: Create a Java program to demonstrate dynamic binding.

```

class DynamicBinding {
    void display() {
        System.out.println("Dynamic method display");
    }
}

class SubClass extends DynamicBinding {
    void display() {
        System.out.println("Overridden dynamic method display");
    }
}

public class DynamicBindingDemo {
    public static void main(String[] args) {
        DynamicBinding obj = new SubClass();
    }
}

```

```

        obj.display(); // Overridden dynamic method display
    }
}

```

50.Question: Write a Java program to demonstrate method overriding and dynamic binding.

```

class SuperClass {
    void show() {
        System.out.println("Superclass method");
    }
}

class SubClass extends SuperClass {
    @Override
    void show() {
        System.out.println("Subclass method");
    }
}

public class MethodBindingDemo {
    public static void main(String[] args) {
        SuperClass obj = new SubClass();
        obj.show(); // Subclass method
    }
}

```

Polymorphism

51.Question: Write a Java program to demonstrate compile-time polymorphism.

```

class CompileTimePolymorphism {
    void display(int a) {
        System.out.println("Integer: " + a);
    }

    void display(String str) {
        System.out.println("String: " + str);
    }
}

public class CompileTimeDemo {
    public static void main(String[] args) {
        CompileTimePolymorphism obj = new CompileTimePolymorphism();
    }
}

```



```

        obj.display(5);
        obj.display("Hello");
    }
}

```

52.Question: Create a Java program to demonstrate run-time polymorphism.

```

class RunTimePolymorphism {
    void display() {
        System.out.println("Parent class display method");
    }
}

class SubClass extends RunTimePolymorphism {
    @Override
    void display() {
        System.out.println("Child class display method");
    }
}

public class RunTimeDemo {
    public static void main(String[] args) {
        RunTimePolymorphism obj = new SubClass();
        obj.display(); // Child class display method
    }
}

```

53.Question: Write a Java program to demonstrate polymorphism using interfaces.

```

interface Shape {
    void draw();
}

class Circle implements Shape {
    public void draw() {
        System.out.println("Drawing Circle");
    }
}

class Square implements Shape {
    public void draw() {
        System.out.println("Drawing Square");
    }
}

```

```

public class InterfacePolymorphism {
    public static void main(String[] args) {
        Shape shape1 = new Circle();
        Shape shape2 = new Square();

        shape1.draw();
        shape2.draw();
    }
}

```

Abstraction

54.Question: Write a Java program to demonstrate abstract classes.

```

abstract class Animal {
    abstract void makeSound();

    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void makeSound() {
        System.out.println("Dog barks");
    }
}

```

```

public class AbstractClassDemo {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.makeSound();
        dog.eat();
    }
}

```

55.Question: Create a Java program to demonstrate interfaces.

```

interface Animal {
    void makeSound();

    default void eat() {

```

```

        System.out.println("Animal eats");
    }
}

class Dog implements Animal {
    public void makeSound() {
        System.out.println("Dog barks");
    }
}

public class InterfaceDemo {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.makeSound();
        dog.eat();
    }
}

```

56.Question: Write a Java program to demonstrate the use of multiple interfaces.

```

interface A {
    void methodA();
}

interface B {
    void methodB();
}

class C implements A, B {
    public void methodA() {
        System.out.println("Method A");
    }

    public void methodB() {
        System.out.println("Method B");
    }
}

public class MultipleInterfacesDemo {
    public static void main(String[] args) {
        C obj = new C();
        obj.methodA();
    }
}

```

```

        obj.methodB();
    }
}

```

Encapsulation

57.Question: Write a Java program to demonstrate encapsulation.

```

class EncapsulationDemo {
    private String name;
    private int age;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age > 0) {
            this.age = age;
        }
    }
}

public class Main {
    public static void main(String[] args) {
        EncapsulationDemo obj = new EncapsulationDemo();
        obj.setName("Alice");
        obj.setAge(25);

        System.out.println("Name: " + obj.getName());
        System.out.println("Age: " + obj.getAge());
    }
}

```

58.Question: Create a Java program to demonstrate the benefits of encapsulation.

```
class BankAccount {
    private double balance;

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
        }
    }
}

public class EncapsulationBenefits {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.deposit(500);
        account.withdraw(100);

        System.out.println("Balance: " + account.getBalance());
    }
}
```

59.Question: Write a Java program to demonstrate encapsulation with validation.

```
class Employee {
    private String name;
    private int age;

    public String getName() {
        return name;
    }

    public void setName(String name) {
```

```

        if (name != null && !name.isEmpty()) {
            this.name = name;
        }
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age > 18) {
            this.age = age;
        }
    }
}

public class EncapsulationWithValidation {
    public static void main(String[] args) {
        Employee emp = new Employee();
        emp.setName("John");
        emp.setAge(30);

        System.out.println("Name: " + emp.getName());
        System.out.println("Age: " + emp.getAge());
    }
}

```

Packages

60.Question: Write a Java program to demonstrate creating and using a package.

```

// In file: mypackage/Example.java
package mypackage;

public class Example {
    public void display() {
        System.out.println("This is an example from mypackage");
    }
}

// In file: Main.java

```

```
import mypackage.Example;

public class Main {
    public static void main(String[] args) {
        Example example = new Example();
        example.display();
    }
}
```

61.Question: Create a Java program to demonstrate using built-in packages.

```
import java.util.ArrayList;

public class BuiltInPackageDemo {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Alice");
        list.add("Bob");

        for (String name : list) {
            System.out.println(name);
        }
    }
}
```

62.Question: Write a Java program to demonstrate importing classes from multiple packages.

```
import java.util.Scanner;
import java.util.ArrayList;

public class MultiplePackages {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ArrayList<String> list = new ArrayList<>();

        System.out.print("Enter a name: ");
        String name = scanner.nextLine();
        list.add(name);

        System.out.println("Names: " + list);
    }
}
```


Keywords

63.Question: Write a Java program to demonstrate the use of the final keyword with variables, methods, and classes.

```
class FinalDemo {  
    final int MAX_VALUE = 100;  
  
    final void display() {  
        System.out.println("This is a final method");  
    }  
}  
  
final class FinalClass {  
    void show() {  
        System.out.println("This is a final class");  
    }  
}  
  
public class FinalKeywordDemo {  
    public static void main(String[] args) {  
        FinalDemo obj = new FinalDemo();  
        obj.display();  
  
        FinalClass finalObj = new FinalClass();  
        finalObj.show();  
    }  
}
```

64.Question: Create a Java program to demonstrate the use of the static keyword.

```
class StaticDemo {  
    static int count = 0;  
  
    StaticDemo() {  
        count++;  
    }  
  
    static void displayCount() {  
        System.out.println("Count: " + count);  
    }  
}
```

```

public class StaticKeywordDemo {
    public static void main(String[] args) {
        StaticDemo obj1 = new StaticDemo();
        StaticDemo obj2 = new StaticDemo();
        StaticDemo.displayCount(); // Output: Count: 2
    }
}

```

65.Question: Write a Java program to demonstrate the use of the this keyword.

```

class ThisDemo {
    int num;

    ThisDemo(int num) {
        this.num = num;
    }

    void display() {
        System.out.println("Number: " + this.num);
    }
}

public class ThisKeywordDemo {
    public static void main(String[] args) {
        ThisDemo obj = new ThisDemo(10);
        obj.display();
    }
}

```

Interfaces

66.Question: Write a Java program to demonstrate creating and using an interface.

```

interface Animal {
    void makeSound();
}

class Dog implements Animal {
    public void makeSound() {
        System.out.println("Dog barks");
    }
}

public class InterfaceDemo {

```

```

        public static void main(String[] args) {
            Dog dog = new Dog();
            dog.makeSound();
        }
    }

```

67.Question: Create a Java program to demonstrate implementing multiple interfaces.

```

interface A {
    void methodA();
}

interface B {
    void methodB();
}

class C implements A, B {
    public void methodA() {
        System.out.println("Method A");
    }

    public void methodB() {
        System.out.println("Method B");
    }
}

public class MultipleInterfacesDemo {
    public static void main(String[] args) {
        C obj = new C();
        obj.methodA();
        obj.methodB();
    }
}

```

68.Question: Write a Java program to demonstrate using default methods in interfaces.

```

interface Animal {
    void makeSound();

    default void eat() {
        System.out.println("Animal eats");
    }
}

```

```

class Dog implements Animal {
    public void makeSound() {
        System.out.println("Dog barks");
    }
}

public class DefaultMethodDemo {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.makeSound();
        dog.eat();
    }
}

```

Collections

69.Question: Write a Java program to demonstrate the use of ArrayList.

```

import java.util.ArrayList;

public class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Alice");
        list.add("Bob");

        for (String name : list) {
            System.out.println(name);
        }
    }
}

```

70.Question: Create a Java program to demonstrate the use of HashSet.

```

import java.util.HashSet;

public class HashSetDemo {
    public static void main(String[] args) {
        HashSet<String> set = new HashSet<>();
        set.add("Alice");
        set.add("Bob");
        set.add("Alice"); // Duplicate, will not be added

        for (String name : set) {

```

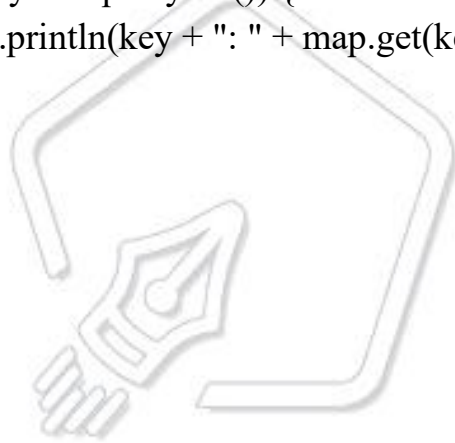
```
        System.out.println(name);
    }
}
}
```

71.Question: Write a Java program to demonstrate the use of HashMap.

```
import java.util.HashMap;

public class HashMapDemo {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();
        map.put(1, "Alice");
        map.put(2, "Bob");

        for (Integer key : map.keySet()) {
            System.out.println(key + ": " + map.get(key));
        }
    }
}
```



PENTAGON SPACE
Mastering The Future